

时频冲突检测与消解

摘 要

本文研究有限带宽下大量用频计划的时频冲突检测与消解。把每个用频计划展开为它在“频段 $\times$ 时隙”栅格上占用的时频单元集合，则“冲突”等价于两集合相交、“无冲突”等价于诸集合两两不交。这一刻画把四个问题统一为同一语言，并使每个结果都能被一个与求解器代码完全分离的校验器复核。

问题 1 由半开区间判据给出精确检测：成对枚举（双指针归并）与按频段分桶的扫描线两种算法独立实现，输出完全一致。150 个计划中检出 297 对冲突，涉及 148 个计划、587 个使用次（构成 431 对相交的使用次）；卷入冲突的 148 个计划连成唯一一个连通分量（另有 2 个计划与任何计划都不冲突），但图很稀疏（平均度 3.96、密度 0.0266）；B-C 类冲突占六成以上。时频单元占用率仅 22.9%，表明冲突源于局部拥塞而非资源总量不足。

问题 2 建立 0-1 规划：每个计划恰选一个动作（保持 / 频移 / 时移 / 撤销），以“覆盖同一时频单元的变量至多取一”作为无冲突的充要编码，并按题目优先级构造七级词典序目标（撤销 A/B/C $\rightarrow$ 调整 A/B/C $\rightarrow$ 归一化幅度）。求解采用现任解保持的逐级搜索：每级加入由可行现任解导出的上界割，该割不删去任何最优解，却保证报告值永不劣于任何已知可行解。结果为保留 14、调整 124、撤销 12，且撤销的全部是 C 类，A、B 类一个未撤。撤销三级 0/0/12 经 CP-SAT 闭合对偶间隙，为已证明的全局最优；由此直接得到两个推论：任何无冲突方案都至少要撤销一个计划；且在坚持不撤销任何 A、B 类计划的前提下，被撤销的 C 类计划数恰好最少为 12。调整各级未闭合间隙，本文只报告“值 / 证明下界 / 状态”，不作最优性声明。

问题 3 归结为二维集合装填。指出题干“不限制平移幅度”存在实质歧义：既有计划固定（解释 A）与既有计划亦可重排（解释 B）。以 A 为主解释（与结果模板一一对应、可独立复核），得最多再安排 109 个 C 类装备，且 CP-SAT 对偶界、HiGHS 线性松弛界与 MILP 界三者同时等于 109，间隙为 0，利用率由 24.6% 升至 36.8%；对解释 B 则由面积守恒给上界、由首次适配重排给构造下界，得区间 [572, 673]。

问题 4 为 C 类增加“改间隔”动作后重解同一模型，并按问题 3 的同一尺度加入“不增加时频资源”的硬约束 $e_i \leq 643$ ：撤销数由 12 降至 8，代价是多调整 1 个计划，而时间范围未被延长。去掉该硬约束的变体一并报告（撤销 8、最晚结束时刻 686），供对照。

检验：四个独立校验器全部通过（零剩余冲突）；24 个随机微型实例与穷举最优 24/24 一致（其中 7 个实例的撤销层非零）；两个求解器交叉验证，分离权重标量化在问题 2 复现词典序解、在问题 4 未复现（词典序解严格更优，命题 5.4 保证不会出现反向矛盾）。灵敏度分析给出一条有实用价值的结论：把最大频段平移幅度由 $10\Delta f$ 放宽到 $15\Delta f$ ，撤销数即由 12 降为 0；按单位增量折算，每放宽 $1\Delta f$ 平均少撤销 6.0 个计划，每放宽 $1\Delta t$ 少撤销 4.0 个，频域的单位效果约为时域的 1.50 倍。

关键词：时频冲突检测；冲突图；词典序整数规划；约束规划（CP-SAT）；二维集合装填；最优性界

1 问题重述

1.1 背景与资源模型

在同一空域、同一时段内工作的用频装备越来越多，若两部装备在重叠的频段上同时发射，就会互相干扰。为在计划阶段（而非事后）消除干扰，需要把“谁在什么时间用哪一段频率”的全部申请汇总起来，先检测其中潜在的时频冲突，再通过调整少量计划参数或撤销个别计划把冲突消解掉。

题目将时频资源离散化：时域以 Δt 为最小不可拆分单位，频域把有限带宽等分为 100 个宽度为 Δf 的频段，编号 $0, 1, \dots, 99$ 。于是“时频资源”就是一张以频段编号为行、以时刻为列的二维栅格，每个栅格单元 (b, t) 表示“第 b 个频段在第 t 个时隙”这一份最小资源。冲突即两个用频计划占用了同一个单元。

1.2 用频计划的参数与冲突的判定

附件 1 给出 150 个用频装备提报的用频计划，按编号首字母分为 A、B、C 三类（各类装备数与参数见表 2）。每个计划由四个参数确定：

- (i) **频段区间**：连续占用的频段 $[f, f + w)$ ，占用的频段数 w 在任何调整下保持不变；
- (ii) **首次使用的时间区间**： $[s, s + d)$ ， d 为每次使用的时长；
- (iii) **间隔时长** g ：相邻两次使用之间的空闲时长；
- (iv) **使用次数** n ：按上述规则重复使用 n 次。

题目以装备 A001 为例规定了推进规则：首次使用 $[35, 40)$ 、间隔 60，则第二次使用为 $[40 + 60, 45 + 60) = [100, 105)$ 。即第 k 次 ($k = 0, 1, \dots, n - 1$) 使用的时间区间为 $[s + k(d + g), s + d + k(d + g))$ ，所有区间均为半开区间。**时频冲突**的判定是：两个用频计划的频段区间存在交叠，且它们的某一次使用在时间上存在交叠。

1.3 消解规则

题目对消解给出了如下规则，本文将其区分为硬约束与优先目标两类：

- **硬约束**：一个用频计划只能调整其中一个参数或撤销；一般不调整使用次数和使用间隔时长（问题 4 对 C 类装备的间隔时长解除该限制）；频段、时间调整的最大平移幅度分别为 $10\Delta f$ 、 $5\Delta t$ ；消解后的全部计划之间不得存在任何时频冲突。
- **优先目标**（按题目给出的先后次序）：尽可能减少撤销的用频计划；尽量减少被调整的用频计划数量；A 类装备优先级最高、B 类其次、C 类最低，高优先级装备的计划应尽量保持；尽量减少调整的幅度。

1.4 四个问题与交付物

问题 1（检测） 对附件 1 的全部计划进行时频冲突检测，把每一对冲突写入 `result1.xlsx`，并在论文中给出冲突的统计结果。

问题 2 (消解) 对检测出的冲突进行消解, 把需要调整或撤销的装备写入 `result2.xlsx` (B 列填调整后频段区间、C 列填调整后时间区间、D 列填“是”表示撤销), 并按题目表 1 的格式给出各类别的保留 / 调整 / 撤销数量。

问题 3 (增容) 基于问题 2 得到的无冲突方案, 在不限频段和时间平移幅度、不增加时频资源的前提下, 最多还能安排多少个 C 类用频装备? 给出相应计划并写入 `result3.xlsx`。

问题 4 (放宽间隔) 允许部分 C 类装备调整用频间隔时长 (与原间隔的差异不超过 $10\Delta t$), 重新消解问题 1 检测出的冲突, 写入 `result4.xlsx` 并给出表 1 格式的统计结果。

问题之间是串行依赖的: 问题 1 的冲突集合是问题 2、4 的输入, 问题 2 的无冲突方案是问题 3 的输入。本文把四个问题分别形式化为区间相交判定、带类别优先级的词典序整数规划、二维集合装填与动作集合扩张后的同一词典序规划, 并对每个结果给出与求解器分离的独立校验。

2 问题分析

2.1 四个问题的共同结构

四个问题看似分属检测、调度与装填三类, 但它们共享同一个底层对象: **时频单元格**。把计划 P 展开为它所占用的单元集合

$$U(P) = \{(b, t) : f \leq b < f+w, \exists k \in \{0, \dots, n-1\}, s+k(d+g) \leq t < s+d+k(d+g)\},$$

则“两计划冲突”就等价于 $U(P_i) \cap U(P_j) \neq \emptyset$, “一组计划无冲突”等价于它们的单元集合两两不交。这一观察把四个问题统一成同一语言:

- **问题 1** 是判定 $\binom{150}{2}$ 对单元集合是否相交。直接求交代价高, 但由于 $U(P)$ 是“频段区间 $\times$ 周期性时间区间”的乘积结构, 相交性可用区间算术在常数时间内判定 (见命题 4.1)。
- **问题 2、4** 是: 每个计划从一个有限动作集合中恰选一个动作 (保持、频移、时移、问题 4 另加改间隔、撤销), 使得所选动作对应的单元集合两两不交, 并在题目给出的优先级下最优。这是一个列表着色 / 带选项的装填型问题, 其特例 (只许保持或撤销) 正是冲突图的最大独立集, 故为 NP-难 [1-2]。本题属于频率指配问题族 [3-4], 但与经典频率指配 (给每个发射机指定频点、相邻发射机须相隔足够频距) 相比多出时间这一维, 且调整方式受“只改一个参数”的强限制。
- **问题 3** 是: 在问题 2 方案留下的自由单元中, 放入尽可能多的同形状“砖块” (C 类计划的单元集合), 这是二维集合装填 (set packing) 问题 [5-6]。

统一的语言带来一个直接的好处: 无论求解器给出什么, 都可以用同一个与求解器代码完全分离的校验器——把每个计划重新展开成单元集合、用集合求交重新找冲突——来复核。本文的全部结论都经过这一关。

2.2 建模的三个关键抉择

(一) 冲突的精确语义。 题目只说“时间区间和频段区间都存在交叠”，未言明区间端点开闭、是否逐次比较。附件的示例 ($[35, 40)$ 、间隔 60、第二次 $[100, 105)$) 给出了半开区间与推进规则，本文据此把冲突定义在整个使用序列上 (MDR-0001)：只要存在一对使用次在时间上交叠且频段交叠，两计划即冲突。若改用闭区间，相邻不重叠的两次使用会被误判为冲突；若只比较首次使用，“间隔时长”与“使用次数”两个参数将失去意义。

(二) 优先目标如何形式化。 题目按“撤销 $\succ$ 调整数量 $\succ$ 类别优先级 $\succ$ 调整幅度”的次序给出四条目标，却没有给出权重。把“类别优先级”嵌进前两条，得到本文的主方案 P：七级词典序

$$L = (\text{撤销}_A, \text{撤销}_B, \text{撤销}_C, \text{调整}_A, \text{调整}_B, \text{调整}_C, \text{归一化幅度}),$$

逐级最小化。这不是唯一的读法，因此本文同时求解另外三种方案（总量优先 T、不含优先级 S、纯加权 W）并给出对照，用数据回答“优先级条款到底起了多大作用”（第 5.7 节）。这是一个建模决策而非计算细节，故全部备选与理由记录在 MDR-0003 中。

(三) 问题 3 中“不限制平移幅度”指谁。 “平移”一词只对已有位置的计划有意义，而新增装备本无“原位置”可平移，故该句存在两种读法：解释 A（新增计划的放置不受 $10\Delta f/5\Delta t$ 约束，既有计划固定）与解释 B（既有计划也可不受幅度限制地再次平移以腾出空间）。两种读法给出的答案相差约五倍。本文以 A 为主解释（它与 result3.xlsx 模板的三列一一对应，评阅者可仅凭结果文件独立复核），同时为 B 给出可核查的上下界（第 6.5 节、MDR-0010）。

2.3 技术路线

图 1 给出本文的技术路线。论文中报告的全部数值都产生于同一次云端运行，每个阶段的输入输出、参数、代码版本与 SHA-256 散列都写入清单文件，数字由该次运行的阶段产物自动注入（附录 C）。唯一的例外是问题 2 与问题 4 求解时使用的现任解（热启动，第 5.4 节）：它来自更早的一次运行，作为文件存放在仓库中，其来源与目标向量记录在文件内。本文对这一依赖如实报告，并另跑一次不加载现任解的冷启动作对照。

3 模型假设与符号说明

3.1 模型假设

下列假设分为两类：(题) 表示题目已明确给定、本文仅作形式化；(本文) 表示题目未言明、由本文补充的解释或约定。每条给出依据，并说明假设被违反时结论会如何变化。

假设 1 (时频资源的离散性与半开性 (题)). 时间以 Δt 、频率以 Δf 为不可拆分的最小单位；频率资源共 $B = 100$ 个频段，编号 $0, \dots, 99$ 。一切区间均为半开整数区间 $[\cdot, \cdot)$ ，两区间相交当且仅当 $lo_1 < hi_2$ 且 $lo_2 < hi_1$ 。依据：题目规定“用频计划所占用的时间

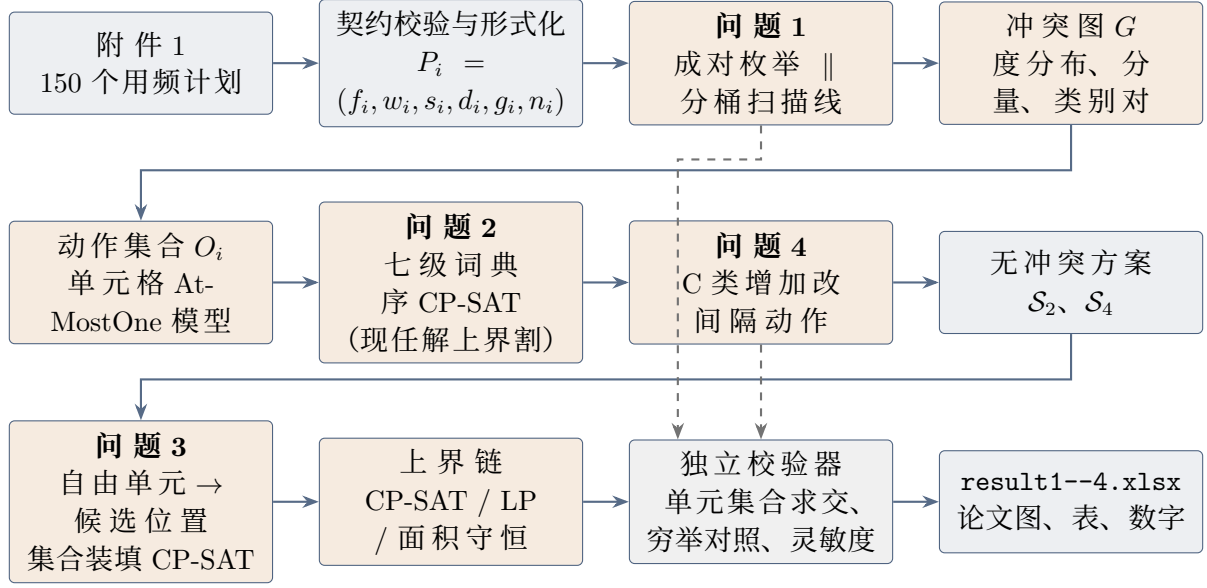


图 1 技术路线。实线为数据流，虚线为独立校验（校验器不复用检测与求解代码，把每个计划重新展开为时频单元集合后用集合求交复核）。灰蓝色块为数据与产物，橙色块为模型与算法。

长度必须是 Δt 的整数倍”，且附件示例以 $[35, 40)$ 、 $[100, 105)$ 的半开形式书写。违反的后果：若改为闭区间，首尾相接的两次使用（如 $[35, 40)$ 与 $[40, 45)$ ）会被判为冲突，冲突对数将被高估，消解代价随之虚增。

假设 2 (使用序列的周期性 (题)). 计划的第 k 次 ($k = 0, \dots, n-1$) 使用占用时间区间 $[s + k(d + g), s + d + k(d + g))$ ，频段区间在全部 n 次使用中保持为 $[f, f + w)$ 。依据：题目对 A001 的示例明确给出第二次使用为 $[40 + 60, 45 + 60)$ 。

假设 3 (冲突定义在整个使用序列上 (本文)). 两计划冲突当且仅当频段区间相交且至少存在一对使用次在时间上相交。依据：题目给出“间隔时长”与“使用次数”两个参数，若只比较首次使用，这两个参数将不影响任何结论。违反的后果：只比较首次使用会漏检大量冲突（本文实例中 297 对冲突共涉及 431 对相交的使用次，其中许多并非首次）。

假设 4 (平移的整数性与同步性 (本文)). 频段平移量 δ 与时间平移量 τ 均为整数（以 Δf 、 Δt 计）；时间平移把整个使用序列同步平移（即只改变 s ），间隔 g 与次数 n 不变。依据：资源已离散化，非整数平移无意义；“只能调整其中一个参数”要求平移不得同时改变 g 或 n 。

假设 5 (平移的可行域 (本文)). 频段平移后必须仍落在带宽内，即 $0 \leq f + \delta$ 且 $f + \delta + w \leq 100$ ；时间平移后首次使用不得早于零时刻，即 $s + \tau \geq 0$ ；平移方向不受限制（可正可负）。依据：题目设定的是“有限带宽”的 100 个频段，越界无物理意义；题目未限制平移方向。违反的后果：若只允许向后平移，可行域缩小，撤销数量会上升。

假设 6 (动作的互斥性 (题)). 每个计划恰好选择一个动作：保持、频段平移、时间平移、(问题 4 中 C 类的) 间隔调整、撤销。不允许同时平移频段与时间。依据：题目“一个用频计划只能调整其中一个参数或撤销”。

假设 7 (间隔调整的下界 (本文)). 问题 4 中 C 类计划的新间隔 g' 满足 $|g' - g| \leq 10$ 且 $g' \geq 1$ 。依据：题目只给出“与原间隔时长的差异不超过 $10\Delta t$ ”。本文补充 $g' \geq 1$ ：

若 $g' = 0$ ，相邻两次使用将首尾相连合并为一次更长的使用，等价于改变了“使用次数”的语义，与假设 6 冲突。违反的后果：允许 $g' = 0$ 会给 C 类多出一个动作，撤销数量可能略降，但所得方案不再满足“只改一个参数”。

假设 8 (目标的词典序解释 (本文)). 题目的四条优先目标按“撤销 $\succ$ 调整数量 $\succ$ 调整幅度”的次序构成词典序，类别优先级 $A \succ B \succ C$ 嵌入前两条之内，得到七级目标 L (式 (7))。幅度按各自允许的最大幅度归一化： $|\delta|/10 + |\tau|/5 + |\Delta g|/10$ 。依据：题目对四条目标给出了明确的先后次序，却未给权重；词典序是“次序已知、权重未知”情形下最保守的形式化 [7]。违反的后果：改用别的形式化会得到不同的方案；本文对三种备选方案 (T、S、W) 全部求解并给出对照 (第 5.7 节)，以量化这一假设的影响。

假设 9 (问题 3 的资源区域与新增计划参数 (本文)). “不增加时频资源”指新增计划必须落在 $100 \text{ 个频段} \times [0, T_{\text{end}})$ 的区域内，其中 T_{end} 为问题 2 方案的最晚结束时刻；新增 C 类装备的参数与附件中 C 类完全相同，即 $w = 3$ 、 $d = 2$ 、 $g = 8$ 、 $n = 12$ ；新增计划之间以及与既有计划之间均不得冲突。依据：题目要求“安排 C 类用频装备”，C 类的参数在附件中是同质的 (表 2)；“不增加时频资源”最自然的度量就是不扩大频段数与时间范围。违反的后果：若允许时间范围延长，可安排的数量将随时间线性增长，问题失去意义。

假设 10 (计划之间无其他耦合 (本文)). 除时频占用外，计划之间无其他约束：不存在装备间的最小频率保护间隔、地理隔离、互调干扰、先后次序或资源配额等要求；同一装备的多次使用之间也不因平移而产生新的约束。依据：题目只给出时频两维的占用信息，未提供任何几何、功率或体制参数。违反的后果：真实电磁兼容分析中需要保护带与空间隔离，届时“冲突”的判据应放宽为“频段间隔小于保护带宽”，模型结构不变 (只需把 $U(P)$ 沿频率方向膨胀)，但冲突对数会显著上升。

3.2 符号说明

全文使用的符号汇总于表 1。

表 1 全文符号说明。

符号	含义	单位
$\Delta f, \Delta t$	频域与时域的基本资源单位 (题目给定，未赋数值)	—
$B = 100$	频段总数，编号 $0, \dots, B - 1$ ；总带宽 $B\Delta f$	个
$N = 150$	用频计划 (装备) 总数	个
$c(i) \in \{A, B, C\}$	计划 i 所属的装备类别，优先级 $A \succ B \succ C$	—
P_i	计划 i ：频段起点、频宽、首次起始时刻、单次时长、	—
$(f_i, w_i, s_i, d_i, g_i, n_i)$	间隔、次数	—
$[f_i, f_i + w_i)$	计划 i 占用的频段区间	Δf
$I_i^{(k)}$	计划 i 第 k 次使用的时间区间 $[s_i + k\pi_i, s_i + d_i + k\pi_i)$ ， 其中 $\pi_i = d_i + g_i$ 为周期	Δt
$U(P)$	计划 P 占用的时频单元集合， $ U(P) = wdn$	单元

符号	含义	单位
$e_i = s_i + d_i + (n_i - 1)\pi_i$	计划 i 的结束时刻（不含）	Δt
T_{end}	一组计划的最晚结束时刻 $\max_i e_i$ （时间范围）	Δt
$G = (V, E)$	冲突图：顶点为计划，边为冲突对	—
O_i	计划 i 的动作集合（保持 / 频移 / 时移 / 改间隔 / 撤销）	—
$o = (\text{kind}, \delta)$	一个动作：类型与幅度	—
$y_{i,o} \in \{0, 1\}$	计划 i 采取动作 o 时取 1	—
P_i^o	计划 i 采取动作 o 后的计划（撤销时为空）	—
$\mathcal{C}$	原始计划占用范围内的时频单元全集 $\{0, \dots, B - 1\} \times \{0, \dots, T_{\text{end}} - 1\}$	单元
$\mathcal{C}_2$	问题 2/4 模型的单元全集，取 $T_{\text{max}} = \max_{i,o} e(P_i^o)$ （动作可把计划推出 T_{end} ）	单元
$\mathcal{R}$	问题 3 的资源区域 $\{0, \dots, B - 1\} \times \{0, \dots, T_{\text{end}} - 1\}$ （与 $\mathcal{C}$ 同，但作为约束而非索引集）	单元
$R_{b,t}$	覆盖单元 (b, t) 的全部 (i, o) 变量下标集合（“单元行”）	—
Φ	最大频段平移幅度（题目为 10）	Δf
$\mathcal{T}$	最大时间平移幅度（题目为 5）	Δt
Γ	最大间隔调整幅度（问题 4 为 10）	Δt
$L = (L_1, \dots, L_7)$	七级词典序目标向量（式 (7)）	—
$A(o)$	动作 o 的归一化幅度 $ \delta /\Phi + \tau /\mathcal{T} + \Delta g /\Gamma$	—
$x_{f,s} \in \{0, 1\}$	问题 3 中在频段起点 f 、首次时刻 s 放置一个新增 C 类计划	—
$\mathcal{F}$	问题 2 方案留下的自由单元集合	单元
$\mathcal{S}_2, \mathcal{S}_4$	问题 2 / 问题 4 方案中未被撤销（存留）的计划下标集合	—
σ	新增 C 类计划的时间跨度 $d_C + (n_C - 1)\pi_C$	Δt
$\mathcal{P}$	问题 3 的候选放置位置集合 $\{(f, s)\}$	—
m	词典序目标的层数（本文 $m = 7$ ）	—
$v_k, \underline{v}_k$	第 k 级的取值与求解器给出的证明下界	—

附件 1 给定的三类装备参数完全同质，汇总如表 2；该表由数据本身生成，生成时在程序中断言“同类计划的 (w, d, g, n) 完全相同”，断言不成立则阶段失败。

4 问题一：时频冲突的精确检测

本节回答：附件 1 的 150 个用频计划中，哪些计划两两之间存在时频冲突？共有多少对？冲突的结构如何？

表 2 附件 1 给定的三类用频装备参数（由数据生成，同类同参已在程序中断言）。“时间跨度”为 $d + (n-1)(d+g)$ ，即单个计划从首次使用开始到最后一次使用结束所横跨的时长；“单计划单元数”为 wdn 。

类别	装备数/个	频宽 $w/\Delta f$	时长 $d/\Delta t$	间隔 $g/\Delta t$	次数 $n/\text{次}$	周期 $(d+g)/\Delta t$	时间跨度/ Δt	单计划单元数/单元	该类单元合计/单元
A 类	20	10	5	60	3	65	135	150	3000
B 类	40	15	3	40	4	43	132	180	7200
C 类	90	3	2	8	12	10	112	72	6480

4.1 冲突的形式化与单元格刻画

按假设 1–3，计划 $P = (f, w, s, d, g, n)$ 占用的时频单元集合为

$$U(P) = [f, f+w) \times \bigcup_{k=0}^{n-1} [s+k\pi, s+d+k\pi), \quad \pi = d+g, \quad (1)$$

其中乘积表示“频段编号 $\times$ 时刻”的笛卡尔积， $|U(P)| = wdn$ 。

命题 4.1 (冲突的单元格刻画). 两个用频计划 P_i, P_j 存在时频冲突，当且仅当 $U(P_i) \cap U(P_j) \neq \emptyset$ ，当且仅当式 (2) 成立：

$$\underbrace{f_i < f_j + w_j \quad \wedge \quad f_j < f_i + w_i}_{\text{频段区间相交}} \quad (2)$$

$$\wedge \quad \exists k, l: \underbrace{s_i + k\pi_i < s_j + d_j + l\pi_j \quad \wedge \quad s_j + l\pi_j < s_i + d_i + k\pi_i}_{\text{第 } k \text{ 次使用与第 } l \text{ 次使用在时间上相交}}.$$

证明. $U(P)$ 是频段集合与时间集合的笛卡尔积，故 $U(P_i) \cap U(P_j) = (F_i \cap F_j) \times (T_i \cap T_j)$ ，它非空当且仅当两个因子都非空。频段因子是两个区间，非空即半开区间相交判据的第一项；时间因子是两族区间的并， $T_i \cap T_j \neq \emptyset$ 当且仅当存在一对区间 $I_i^{(k)}, I_j^{(l)}$ 相交，即第二项。□

命题 4.1 把一个集合论判定化为常数时间的区间算术，是第 4.2 节两种算法的共同基础；同时它给出了一个与区间算术无关的第三种判定方式——直接展开单元集合求交——本文的独立校验器正是用后者复核前者（第 8.1 节）。

4.2 两种检测算法

本文实现两种互相独立的检测算法并要求其输出完全一致。

算法 1：成对枚举 + 双指针。 枚举全部 $\binom{N}{2}$ 对计划，先做代价为 $O(1)$ 的频段区间相交测试，仅当频段相交时再比较时间。由于每个计划的使用区间本身按时间有序，两条使用序列的相交性可用双指针归并在 $O(n_i + n_j)$ 内判定，而不必枚举 $n_i n_j$ 对。

算法 1: 成对枚举检测（双指针归并）

Input: 计划集合 $\{P_1, \dots, P_N\}$

Output: 冲突对集合 E

```

1  $E \leftarrow \emptyset$ ;
2 for  $i \leftarrow 1$  to  $N - 1$  do
3   for  $j \leftarrow i + 1$  to  $N$  do
4     if  $\neg(f_i < f_j + w_j \wedge f_j < f_i + w_i)$  then continue // 频段不相交, 必不冲突;
5      $k \leftarrow 0, l \leftarrow 0$ ;
6     while  $k < n_i$  and  $l < n_j$  do
7       if  $I_i^{(k)} \cap I_j^{(l)} \neq \emptyset$  then  $E \leftarrow E \cup \{(i, j)\}$ ; break;
8       if  $\sup I_i^{(k)} \leq \sup I_j^{(l)}$  then  $k \leftarrow k + 1$ ;
9       else  $l \leftarrow l + 1$ ;
10 return  $E$ ;

```

算法 2: 按频段分桶的扫描线。 把每个计划的每一次使用投影到它占用的每一个频段上, 得到 $N_{\text{item}} = \sum_i w_i n_i$ 个三元组 (起, 止, i); 在每个频段内按起始时刻排序后做扫描线 [8-9], 维护“当前仍活跃”的使用集合, 新到达的使用与全部活跃使用构成冲突对。该算法不枚举计划对, 因而在计划数增大时更具可扩展性。固定频段上的使用区间族构成一个区间图, 其相交结构正是区间图的边集 [10]; 本题的冲突图则是这些区间图沿频段方向的并。

命题 4.2 (检测复杂度). 算法 1 的最坏复杂度为 $O(N^2 + \sum_{i < j} (n_i + n_j)) = O(N^2 \bar{n})$; 算法 2 的复杂度为 $O(N_{\text{item}} \log N_{\text{item}} + K)$, 其中 K 为输出的冲突 (使用次) 对数。对附件实例 $N = 150$ 、 $N_{\text{item}} = 20 \cdot 10 \cdot 3 + 40 \cdot 15 \cdot 4 + 90 \cdot 3 \cdot 12$ 项。

两种算法在本实例上的实测用时分别为 11 ms 与 6 ms, 且给出完全相同的冲突对集合; 在 $N = 600$ 的合成实例上二者仍一致 (第 8.1 节)。

4.3 检测结果与冲突结构

表 3 问题 1 的冲突统计摘要。

统计量	数值
用频计划数 (A/B/C)	20/40/90
冲突对数	297
相交的使用次对数 / 参与冲突的使用次数 / 使用次总数	431 / 587 / 1300
卷入冲突的计划数 (A/B/C)	148 (20/40/88)
与任何计划都不冲突的孤立计划数	2
冲突图连通分量数 (不少于 2 个节点)	1
最大连通分量规模	148
最大度 / 平均度	8 / 3.96
冲突图密度	0.0266
时频单元总数 / 被占用 / 重叠	64300 / 14698 / 1842
时间范围 (最晚结束时刻, Δt)	643
冲突图的最大独立集规模 (状态)	76 (OPTIMAL)
由此得到的被触动计划数下界 (计划总数减最大独立集规模)	74

表 4 按装备类别对统计的冲突对数 (对称矩阵, 对角线为同类内部的冲突对)。

类别	A 类	B 类	C 类
A 类	0	21	66
B 类	21	10	181
C 类	66	181	19

检测共发现 **297 对时频冲突**（表 3），涉及 148 个计划——即 150 个计划中只有 2 个完全不与任何其他计划冲突。全部结果按装备编号字典序写入 `result1.xlsx`，共 297 行，明细见附录表 17。若把“冲突对”细化到使用次，则共有 431 对使用次在时频平面上重叠：多数冲突对只在一次使用上相撞，但附录表 15 的末行显示，存在计划对在其全部 12 次使用上相撞。

类别对矩阵（表 4）揭示了冲突的来源：B-C 之间的冲突多达 181 对，占全部冲突的六成以上；A-A 之间一对也没有（0 对）。这与三类装备的参数（表 2）直接吻合：A 类只有 20 个且使用次数最少（3 次），时间上稀疏；B 类频宽最大（15 个频段，为三类之最）因而在频域上“挡住”了最多的对手；C 类数量最多（90 个）且使用次数最多（12 次），在时域上最密。换言之，**冲突主要产生在“频域宽”的 B 类与“时域密”的 C 类之间**——这一判断在第 8.5 节的灵敏度分析中得到独立印证（按单位增量折算，放宽频段平移上限的收益约为放宽时间平移上限的 1.50 倍，第 8.5 节）。

从图论看，冲突图 G 有 1 个规模不小于 2 的连通分量，其规模为 148：**几乎所有卷入冲突的计划连成一片**。这一点对建模至关重要——它意味着不能把问题拆成若干独立的小块分别求解，必须在全局上一次性消解。同时 G 很稀疏：密度 0.0266、平均度 3.96、最大度 8，且该最大度由 6 个装备同时达到（B009、B016、B024、B027、B030、C076，附录表 16）。这提示“局部让路”的空间是存在的：每个计划平均只需避开 3.96 个对手。图 2 与图 3 给出时频平面的全局图景与冲突结构。

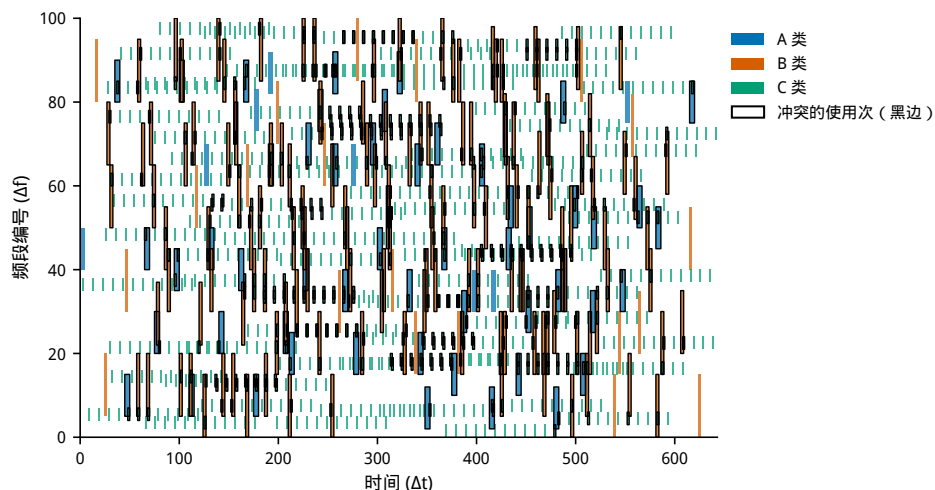


图 2 附件 1 中 150 个用频计划在时频平面上的甘特图。横轴为时间 (Δt)，纵轴为频段编号 (Δf)；每个矩形为一个计划的一次使用，颜色区分 A（蓝）、B（橙）、C（绿）三类装备；黑色描边标出参与冲突的 587 个使用次（它们构成 431 对相交的使用次）。全区域 64300 个时频单元中 14698 个被占用（22.9%），其中 1842 个被两个及以上计划同时占用。

值得注意的是占用率：22.9% 的单元被占用，而被占用的 14698 个单元中，发生重叠的只有 1842 个。**冲突并非源于资源总量不足，而是源于局部拥塞**——这个判断决定了问题 2 的基调：绝大多数计划应当可以通过小幅平移避开彼此，而不必撤销。

5 问题二：冲突消解的词典序整数规划

本节回答：如何以最小代价消除第 4 节检出的全部冲突？结论是保留 14、调整 124、撤销 12，且撤销的全部是 C 类；撤销三级已闭合对偶间隙（定理 5.5），调整各级只报“值 / 证明下界 / 状态”。

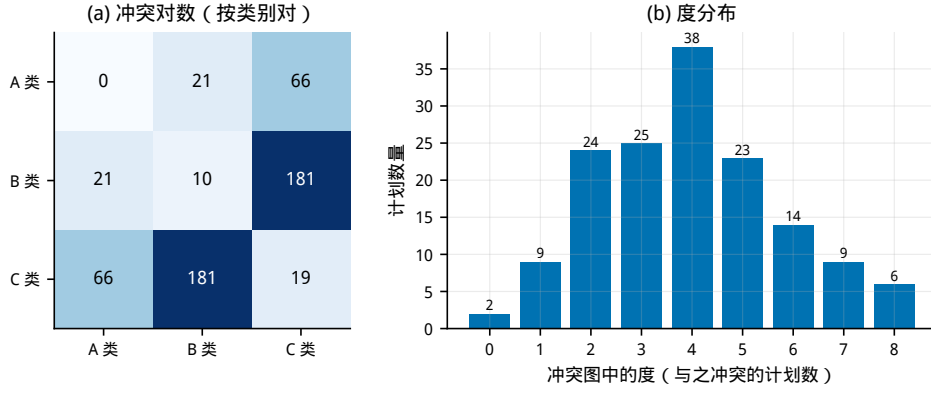


图 3 冲突的结构。(a) 按装备类别对统计的冲突对数矩阵 (共 297 对, B-C 之间最多, 达 181 对); (b) 冲突图的度分布, 最大度 8、平均度 3.96, 148 个计划至少与一个其他计划冲突。

5.1 决策变量与动作集合

按假设 4-6, 计划 i 的动作集合为

$$O_i = \{\text{keep}\} \cup \{\text{freq}_\delta : 0 < |\delta| \leq \Phi, 0 \leq f_i + \delta, f_i + \delta + w_i \leq B\} \cup \{\text{time}_\tau : 0 < |\tau| \leq \mathcal{T}, s_i + \tau \geq 0\} \cup \{\text{cancel}\}, \quad (3)$$

其中 $\Phi = 10$ 、 $\mathcal{T} = 5$ 。动作 o 把计划 P_i 变为 P_i^o (撤销时 $P_i^o = \emptyset$)。引入 0-1 变量 $y_{i,o}$ 表示“计划 i 采取动作 o ”, 则恰选一个动作即

$$\sum_{o \in O_i} y_{i,o} = 1, \quad i = 1, \dots, N. \quad (4)$$

5.2 无冲突约束的单元格编码

朴素的做法是对每一对可能冲突的 (i, o) 、 (j, o') 写一条 $y_{i,o} + y_{j,o'} \leq 1$ 。按表 6 的规模, 需要枚举约 $1843 \times 32 \times 32 \approx 2 \times 10^6$ 个四元组并生成同量级的约束, 比下面单元格编码的 35406 行高出近两个数量级, 建模与传播的代价都不划算。本文改用单元格约束: 模型所涉及的单元全集要比第 4 节的 $\mathcal{C}$ 大一些: 动作会把计划推到原时间范围之外 (时移最多到 $T_{\text{end}} + \mathcal{T}$, 问题 4 的间隔调整更远), 因此令 $T_{\text{max}} = \max_{i,o \in O_i} e(P_i^o)$ 并取 $\mathcal{C}_2 = \{0, \dots, B-1\} \times \{0, \dots, T_{\text{max}}-1\}$, 以保证任意两个被选动作共享的单元一定落在 $\mathcal{C}_2$ 内。

命题 5.1 (单元格编码的精确性). 设 $R_{b,t} = \{(i, o) : (b, t) \in U(P_i^o)\}$ 为覆盖单元 (b, t) 的全部 (计划, 动作) 对。在约束 (4) 之下, “被选中的动作两两不冲突” 等价于

$$\sum_{(i,o) \in R_{b,t}} y_{i,o} \leq 1, \quad \forall (b, t) \in \mathcal{C}_2. \quad (5)$$

约束条数不超过单元总数, 其中字面量总数为 $\sum_i \sum_{o \in O_i} |U(P_i^o)|$ 。

证明. 若某两个被选动作 (i, o) 、 (j, o') ($i \neq j$) 冲突, 由命题 4.1 它们共享某单元 (b, t) , 该单元的行 (5) 左端至少为 2, 约束被违反。反之若所有行都满足, 则任意两个被选动作不共享单元, 故由命题 4.1 互不冲突。同一计划的两个动作不可能同时被选 (式 (4)), 故行内的多个同计划变量不会造成误判。□

实现上只保留至少含两个变量的行并做去重，模型规模由 53046 行降至 35406 行（决策变量 4589 个、涉及 54768 个单元、变量-单元隶属 491904 项，表 6）。

加强约束。 整数规划的求解效率在很大程度上取决于线性松弛的强度与传播能力 [11-12]。为此另加入两族冗余约束（它们由式 (5) 蕴含，因而不改变可行域）：(i) **成对蕴含团**（式 (6)）：对变量 (i, o) 与另一计划 j ，令 $C_j(i, o)$ 为 j 中与 (i, o) 共享单元的动作集合，由于 j 恰选一个动作， $\{(i, o)\} \cup C_j(i, o)$ 构成一个团，故

$$y_{i,o} + \sum_{o' \in C_j(i,o)} y_{j,o'} \leq 1; \quad (6)$$

(ii) **强制撤销割**：若计划对 (i, j) 的任意一对非撤销动作都冲突，则 $y_{i,\text{cancel}} + y_{j,\text{cancel}} \geq 1$ 。本实例中共生成 58587 条团约束，强制撤销割 0 条（即不存在“无论如何平移都必须撤销其一”的计划对，这与第 4 节“局部拥塞而非总量不足”的判断一致）。

5.3 目标：类别内嵌的七级词典序

按假设 8，令 $\mathcal{X}$ 为满足式 (4)–(5) 的可行域，定义

$$\begin{aligned} L_1, L_2, L_3 &= \sum_{i: c(i)=A} y_{i,\text{cancel}}, \quad \sum_{i: c(i)=B} y_{i,\text{cancel}}, \quad \sum_{i: c(i)=C} y_{i,\text{cancel}}, \\ L_4, L_5, L_6 &= \sum_{i: c(i)=A} \sum_{o \in O_i^{\text{adj}}} y_{i,o}, \quad \sum_{i: c(i)=B} \sum_{o \in O_i^{\text{adj}}} y_{i,o}, \quad \sum_{i: c(i)=C} \sum_{o \in O_i^{\text{adj}}} y_{i,o}, \\ L_7 &= 10 \sum_i \sum_{o \in O_i} A(o) y_{i,o}, \quad A(o) = \frac{|\delta|}{\Phi} + \frac{|\tau|}{\mathcal{T}} + \frac{|\Delta g|}{\Gamma}, \end{aligned} \quad (7)$$

其中 O_i^{adj} 为非保持、非撤销的动作。问题 2 即

$$\text{lexmin}_{y \in \mathcal{X}} (L_1, L_2, L_3, L_4, L_5, L_6, L_7), \quad (8)$$

L_7 乘 10 是为了使幅度在 $\Phi = \Gamma = 10$ 、 $\mathcal{T} = 5$ 下取整数值 $(|\delta| + 2|\tau| + |\Delta g|)$ 。

命题 5.2 (词典序与分离权重的等价). 设第 k 级的取值上界为 U_k ，令 $W_k = \prod_{j>k} (U_j + 1)$ ，则单目标 $\min \sum_k W_k L_k$ 的最优解恰为 (8) 的词典序最优解。

证明. 对任意 k ， $W_k = \prod_{j>k} (U_j + 1) > \sum_{j>k} W_j U_j$ （对 k 逆序归纳：当 $k = m - 1$ 时 $W_{m-1} = U_m + 1 > U_m = W_m U_m$ ；设 $W_{k+1} > \sum_{j>k+1} W_j U_j$ ，则 $\sum_{j>k} W_j U_j = W_{k+1} U_{k+1} + \sum_{j>k+1} W_j U_j < W_{k+1} (U_{k+1} + 1) = W_k$ ）。于是第 k 级减少 1 所带来的收益 W_k 严格大于其后各级取值之和的最大可能值，故最优解必先最小化 L_1 ，其次 L_2 ，依此类推。□

命题 5.2 给出一条独立的求解路径：用一次单目标求解复核逐级求解的结果。本文两条路径都走，并把二者是否一致作为交叉验证证据（第 8.1 节）。

命题 5.3 (受限族的计算复杂性). 考察本问题的如下受限族：取 $\Phi = \mathcal{T} = 0$ （动作集合 (3) 只剩 $\{\text{keep}, \text{cancel}\}$ ）、 $n_i = 1$ （使用序列退化为单个矩形），并允许频段总数 B

随实例规模增长。此时冲突图 G 恰是 轴平行矩形相交图，而以**总撤销数**为目标的最小化等价于在该图上求最大独立集，这已知是 NP -完全的 [13]（一般图上的对应结论见 [1-2]）。本文式 (8) 的第一层是它的按类别加权版本（只计 A 类），允许平移则进一步推广为每个顶点带一张候选位置表的列表装填型问题。

需要说明的是：本文并不把该命题用作任何数值结论的依据，它只回答“为什么不去寻找多项式精确算法”。固定 $B = 100$ 的单个实例当然谈不上渐近复杂性——上面的受限族让 B 增长，正是为了使这一陈述有意义。

这解释了为什么本文必须依靠成熟的组合优化求解器而非解析方法，也解释了为什么“证明最优”与“求得好解”需要分别报告：前者要求求解器闭合对偶间隙，后者只要求找到可行解。

5.4 求解算法：现任解保持的逐级搜索

词典序的标准解法是逐级求解：求出 L_1 的最小值 v_1 ，把 $L_1 = v_1$ 作为等式加入模型，再求 L_2 ，依此类推。本实例的困难在于中间各级无法在可承受的预算内闭合间隙，一旦某级到时返回一个较差的可行解，后续各级都被锁死在这个较差的分支上。本文采用**现任解保持**的改进（MDR-0009）：

命题 5.4 (现任解上界割的有效性). 设逐级求解进行到第 k 级，前 $k - 1$ 级的最优值 $v_1, \dots, v_{k-1}$ 已作为等式固定。若 $x^* \in \mathcal{X}$ 满足 $L_j(x^*) = v_j$ ($j < k$)，则第 k 级的受限最优值满足 $v_k \leq L_k(x^*)$ ；因此把割 $L_k \leq L_k(x^*)$ 加入模型不会删去任何第 k 级最优解，各级最优值不变，而求解器返回的值恒不超过 $L_k(x^*)$ 。

证明. x^* 是受限问题（可行域 $\mathcal{X} \cap \{L_j = v_j, j < k\}$ ）的一个可行点，故该问题的最小值不超过 $L_k(x^*)$ ，即 $v_k \leq L_k(x^*)$ 。于是满足 $L_k = v_k$ 的解自动满足 $L_k \leq L_k(x^*)$ ，割不删去任何最优解。 $\square$

算法 2: 现任解保持的词典序求解

Input: 可行域 $\mathcal{X}$ ，目标层级 $L_1, \dots, L_m$ ，各级时限 $T_1, \dots, T_m$ ，热启动解 x_0 (可选)

Output: 目标向量 $(v_1, \dots, v_m)$ 、解 x 、各级证明下界与状态

```

1  $x^* \leftarrow x_0$  若  $x_0$  可行 否则  $\emptyset$  // 现任解;
2 for  $k \leftarrow 1$  to  $m$  do
3   if  $x^* \neq \emptyset$  then 向模型加入割  $L_k \leq L_k(x^*)$  // 命题 5.4: 不删去最优解;
4    $(v_k, \hat{x}, \underline{v}_k, \text{状态}) \leftarrow$  在时限  $T_k$  内极小化  $L_k$ ;
5   if 时限内未返回任何解 then  $v_k \leftarrow L_k(x^*)$ ,  $\hat{x} \leftarrow x^*$ , 状态  $\leftarrow$  INCUMBENT;
6   if  $v_k = L_k(x^*)$  then
7      $x^* \leftarrow \{x^*, \hat{x}\}$  中在剩余各级  $(L_{k+1}, \dots, L_m)$  上词典序较小者;
8   else
9      $x^* \leftarrow \hat{x}$  // 求解器严格改进了现任解;
10  向模型加入等式  $L_k = v_k$ ，并以  $x^*$  重设热启动提示;
11 return  $(v_1, \dots, v_m)$ 、 $x^*$ 、各级  $\underline{v}_k$  与状态;
```

算法 2 的两条性质对本文的诚实性至关重要：(i) 由命题 5.4，各级最优值不受割影响，故凡状态为 OPTIMAL 的层级，其值仍是真正的全局最优；(ii) 返回向量在词典序上永

不劣于任何已知可行解，因而“交叉验证的解比主解更好”这种自相矛盾不可能出现。热启动解取自仓库内经独立校验的现任解文件（其目标向量 (0, 0, 12, 16, 37, 71, 750)，可行性检查 通过），使本文结果可由仓库直接复现（MDR-0008）。

关于热启动的一个必要交代。 该现任解文件是更早一次运行的解，其来源 run、代码版本与目标向量都记录在文件内；本轮不会用本轮的结果去覆盖它，否则“可复现”就成了循环论证。本轮的逐级搜索是否严格改进了它，各问分别如实报告：问题 2 为“未改进”、问题 4 为“严格改进”。这一交代不影响任何最优性结论——由命题 5.4，割不改变各级最优值，凡状态为 OPTIMAL 的层级，其值是该级的全局最优，与搜索从哪里出发无关；只有未证明最优的层级才依赖起点，本文对这些层级本就只报“值 / 证明下界 / 状态”。

为把这一点做实，本文另设一个冷启动阶段：同一模型、同一目标、同一时限，但**不加载任何现任解**，从零开始求解问题 2。它得到的向量为 (0, 0, 13, 14, 37, 69, 716)，与热启动结果 (0, 0, 12, 16, 37, 71, 750) 的关系是“冷启动更差”（撤销总数相差 1、调整总数相差 -4）。该阶段不参与任何交付工作簿的生成，只回答一个方法层面的问题：仓库中存放的现任解，是搜索的输出还是输入。

答案必须说清楚：在本文的时限下，它是输入—— 单次冷启动没有达到 (0, 0, 12, 16, 37, 71, 750)。这并不削弱结论：报告解是经独立校验器复核过的真实可行解，可行性与它从哪里来无关；撤销三级状态为 OPTIMAL，由命题 5.4 与起点无关；未闭合的各级本就只报“值 / 证明下界 / 状态”，冷启动的对照恰好量化了起点的影响。把这一点写出，比声称“完全可从零复现”更接近事实。

求解器为 OR-Tools CP-SAT[14]（24 线程、固定随机种子、显式加入核心引导子求解器），各级时限 [20, 20, 600, 300, 300, 300, 240] 秒，总用时 1612.8 s；另用 HiGHS[15] 对同一模型独立建模，逐级给出线性松弛下界与限时 MILP 结果（总用时 291.7 s）；命题 5.2 的分离权重单目标独立求解一次（182.3 s）。

5.5 求解结果

表 5 问题 2：时频冲突消解的统计结果（题目表 1 格式）。

类别	保留数量	调整数量	撤销数量
A 类	4	16	0
B 类	3	37	0
C 类	7	71	12
合计	14	124	12

表 6 问题 2 模型规模与求解设置。

项目	数值
决策变量数（计划-动作对）	4589
时频单元数（含动作扩展）	54768
变量-单元隶属数	491904
单元格 AtMostOne 约束数（去重前 / 去重后）	53046 / 35406
成对蕴含团约束数 / 强制撤销割数	58587 / 0
可能相互冲突的计划对数	1843
每计划最多动作数（A/B/C）	32/32/32
CP-SAT 词典序总用时/s	1612.8
热启动现任解目标向量（MDR-0009）	(0, 0, 12, 16, 37, 71, 750)
词典序求解起点	仓库内现任解
各级均证明最优	否
HiGHS 交叉验证总用时/s	291.7
HiGHS 证明最优的各级与 CP-SAT 一致	是
分离权重单目标解与词典序向量一致	是

表 5 是题目要求的表 1 格式统计结果：共保留 14 个计划、调整 124 个、撤销 12 个，且

被撤销的 12 个全部是 C 类——A、B 两类计划一个也没有撤销 (0, 0)，完全满足“高优先级用频装备的用频计划应尽量保持”。目标向量为 (0, 0, 12, 16, 37, 71, 750)。调整的具体构成是：85 个计划平移了频段（平均 $|\delta| = 6.24$ 、最大 10、合计 530 个 Δf ），39 个计划平移了时间（平均 $|\tau| = 2.82$ 、最大 5、合计 110 个 Δt ），归一化总幅度 75.0。把它分摊到 124 个被调整的计划上可见，平均每个计划只用掉了约六成的可用平移幅度，说明消解的余量仍相当充裕。消解后方案的最晚结束时刻为 643，与原始计划的 643 相同——本方案没有占用任何额外的时间资源。逐计划的动作清单见附录表 19，写入 result2.xlsx 的共 136 行（保持不变的计划按模板不填写）。

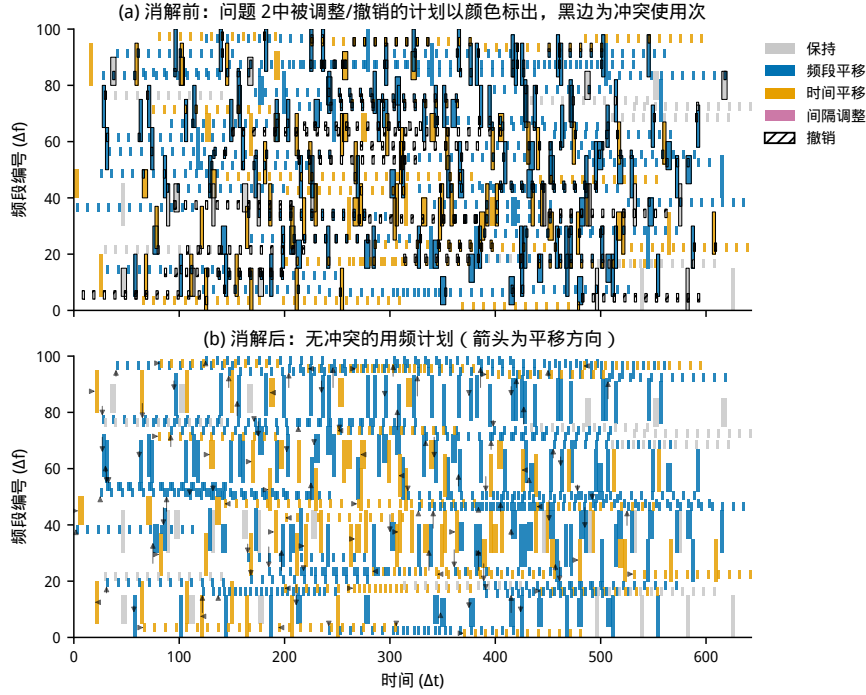


图 4 问题 2 的冲突消解方案。(a) 消解前：按所采取的动作着色（灰 = 保持、蓝 = 频段平移、橙 = 时间平移、斜线 = 撤销），黑边为发生冲突的使用次；(b) 消解后：138 个存留计划在时频平面上已无任何冲突，箭头指示每个被平移计划的移动方向；在 $643\Delta t$ 的横轴尺度下， $|\delta| \leq \Phi$ 、 $|\tau| \leq \mathcal{T}$ 的位移不足一个印刷点，本图只用于展示全局占用格局，逐计划的动作与幅度见附录表 19。图例中的“间隔调整”一项在问题 2 中不可用（见问题 4，图 8）。

5.6 最优性证据

表 7 逐级如实报告“值 / 证明下界 / 状态”。据此，本文能够而且只能够作如下最优性声明：

定理 5.5 (撤销层的全局最优性). 在假设 1–8 之下，问题 2 的撤销三级取值 (0, 0, 12) 是全局最优的：即不撤销任何 A、B 类计划是可行的，且在此前提下必须撤销的 C 类计划数最少为 12。

证明. 第一、二级的求解状态为 OPTIMAL 且值与证明下界同为 0，故存在不撤销任何 A、B 类计划的可行解，且 0 是最小值。第三级在固定 $L_1 = L_2 = 0$ 之后状态为 OPTIMAL，值 12 与证明下界 12 相等，即对偶间隙为零。由命题 5.4，所加的现任解割不改变该级最优值，故 12 是受限问题的全局最小值。□

表 7 问题 2 逐级求解证据。“CP-SAT 值”为报告值，“CP-SAT 下界”为求解器给出的已证明下界；状态 OPTIMAL 表示该级已闭合对偶间隙，FEASIBLE 表示到达时限仍有间隙。HiGHS 为对同一模型的独立建模；“Time limit reached”表示 HiGHS 在其时限内未能证明最优，此时其界不构成额外信息。

目标层级	CP-SAT 值	CP-SAT 下界	CP-SAT 状态	用时/s	HiGHS LP 下界	HiGHS MILP 值	HiGHS MILP 下界	HiGHS 状态
cancelA	0	0.00	OPTIMAL	0.5	0.00	0	0.00	Optimal
cancelB	0	0.00	OPTIMAL	1.2	0.00	0	0.00	Optimal
cancelC	12	12.00	OPTIMAL	467.2	0.00	-	0.00	Time limit reached
adjustA	16	8.00	FEASIBLE	300.4	-	-	0.00	Time limit reached
adjustB	37	20.00	FEASIBLE	300.4	-	-	0.00	Time limit reached
adjustC	71	40.00	FEASIBLE	300.7	-	-	0.00	Time limit reached
amplitude	750	419.00	FEASIBLE	240.5	-	-	0.00	Time limit reached

推论 5.6 (零撤销不可行). (i) 任何无冲突消解方案都至少要撤销一个计划；(ii) 在坚持不撤销任何 A 、 B 类计划的前提下，被撤销的 C 类计划数恰好最少为 12。

证明. (ii) 即定理 5.5 的第三级结论。(i)：若某方案一个计划都不撤销，则它的前三级取值为 $(0, 0, 0)$ ，特别地 $L_1 = L_2 = 0$ 与 $L_3 = 0$ ；但定理 5.5 已证明在 $L_1 = L_2 = 0$ 之下 L_3 的最小值为 $12 > 0$ ，矛盾。□

注意 (i) 是无条件的，而“至少撤销 12 个”不是：表 8 中允许撤销 A 、 B 类的读法可以用更少的撤销总数达到无冲突。

对于调整各级与幅度级，求解器在给定预算内未能闭合间隙（未通过：并非各级均证明最优）。本文因此严格区分两类陈述：表 5 的调整数量是**已知最好的可行方案**，本文不声称它们全局最优。

下界只在它成立的范围内成立。 逐级求解在第 k 级是把前 $k - 1$ 级固定为表中所报的值之后再最小化的。因此表 7 的下界 $\underline{v}_k$ 只对**第一个未闭合间隙的层级**（本文为“调整 A ”）构成对词典序真优解相应分量的界，即该分量落在 $[\underline{v}_4, v_4]$ 内；其后各级的 $\underline{v}_k$ 是“在 $L_4 = v_4, \dots, L_{k-1} = v_{k-1}$ 这些附加等式之下”的下界——若真优解的 L_4 严格小于 v_4 ，它根本不在那个受限可行域里，该下界对它没有约束力。本文在正文与摘要中一律按此措辞陈述，不把后续各级的区间说成真优解的区间。

条件最优性 (MDR-0012)。 把被撤销计划的集合（而不仅是它们的个数）固定为表 5 所报告的那 12 个 C 类计划，再逐级最小化调整三级与幅度级，得到表 28。在这一受限问题上，“调整 A ”一级的值与证明下界均为 16，状态 OPTIMAL。于是可以作一个严格且诚实的陈述：**在与本文方案相同的撤销集合之下，16 个 A 类计划被调整已是最优。**必须强调这是受限问题的最优值，不是词典序真优解的分量——若换一个撤销集合，该值可能更小。

受限再解同时是最后一次改进。 受限问题的可行集是完整问题可行集的子集，因此受限再解找到的任何点在完整问题中同样可行。若它在词典序上严格优于主搜索报告的现任解，继续报告后者就等于明知有更优解而不报。本文因此在受限再解之后加一步判定：更优即采用，并重算表 5 的分类统计、全部幅度统计与独立校验，同时登记这一事件。本问是否发生了采用：未采用（主搜索解不劣）；问题 4 见第 7 节（未采用（主搜索解不劣））。

采用发生时两张表的分工是：表 7 为无约束逐级搜索的轨迹（值、证明下界与状态），表 28 才是最终报告解在调整三级与幅度级上的取值；采用不改变撤销集合，两表的撤销三级一致。

一个与求解器无关的下界。 两个相互冲突的计划不可能都保持原样，故任一无冲突方案中保持不变的计划集合是冲突图 G 的独立集，于是

$$\#\{\text{保持}\} \leq \alpha(G), \quad \#\{\text{被调整或被撤销}\} \geq N - \alpha(G), \quad (9)$$

其中 $\alpha(G) = 76$ (CP-SAT, 状态 OPTIMAL)，故任何方案至少要触动 74 个计划。本文方案触动了 $124 + 12$ 个，满足式 (9)。该界只依赖冲突图，与消解模型、动作上限与求解器都无关，是本文唯一一个完全独立的下界；它比 74 更紧的版本（团覆盖、列生成）未在本文实现，见第 9.2 节。

需要强调：由命题 5.4，调整各级的不确定性不会影响定理 5.5，因为撤销三级的等式是在其自身证明最优之后才被固定的。

为什么必须撤销这 12 个计划？ 附录表 18 给出被撤销计划的局部环境。原始冲突图的度不是撤销的直接原因：其他计划平移之后，相关的邻域已经换了一个；表中确有邻居数只有 1 的计划。真正的原因在表的最后一列——对每个被撤销的计划，逐一试遍它的全部合法单参数动作，没有任何一个动作能使它与本方案的存留计划都不冲突（该列全部为 0）。换句话说，撤销不是因为资源耗尽（占用率仅 22.9%），也不是因为它本身冲突多，而是因为在 $\pm 10\Delta f$ 、 $\pm 5\Delta t$ 的邻域内它相对于已经安排好的其余 138 个计划无处可去；而把高优先级的邻居挪开又与优先级条款相悖——表中被撤销计划的原始邻居里，A、B 类占了绝大多数。

5.7 目标方案的对照分析

假设 8 是本文最主观的一个建模选择，必须用数据检验其影响。为此本文对同一模型求解四种目标方案：P（本文主方案，类别内嵌七级词典序）、T（先总撤销数、再总调整数，类别优先级降为次级偏好）、S（完全不含优先级，只看总量与幅度）、W（纯加权和， $w_A : w_B : w_C = 3 : 2 : 1$ ，撤销权重 $100w$ 、调整权重 $30w$ ）。

表 8 四种目标方案在问题 2 上的对照。方案 P 即正文主结果（各级时限 $[20, 20, 600, 300, 300, 300, 240]$ s，撤销三级已证明最优，定理 5.5）；T、S、W 每级时限 30 s，且都以当时已知的最好解作为现任解热启动（MDR-0013），故它们是在本文预算下得到的可行点，不作最优性声明。表中“预算/用时”两列使这一差异对读者可见。

目标方案	撤销 A/B/C	调整 A/B/C	调整合计	归一化幅度	预算/s	用时/s	各级最优	独立校验
P: 类别内嵌七级词典序（主方案）	0/0/12	16/37/71	124	75.0	1780	1612.8	否	通过
T: 总量优先	2/4/1	11/34/70	115	70.3	360	160.1	否	通过
S: 不含优先级	2/4/1	13/33/69	115	69.7	90	93.4	否	通过
W: 纯加权和	0/8/4	9/27/67	103	63.6	30	32.5	否	通过

表 8 给出了一个清晰而重要的结论：方案 T 与 S 能把撤销总数从 12 降到 7，但代价是转而撤销若干 A 类与 B 类计划（表 8 的“撤销 A/B/C”列）。题目明确要求“A 类用频装备优先级最高，B 类其次，C 类优先级最低，高优先级用频装备的用频计划应尽量

保持”，在这一要求下，少撤销几个 C 类计划绝不足以补偿多撤销一个 A 类计划。纯加权方案 W 的撤销总数为 12。因此方案 P 的选择不是任意的：在本文预算下得到的这四个可行方案中，只有 P 在问题 2 与问题 4 中都不撤销任何 A、B 类计划。这是一个关于这四个具体方案的陈述，不是关于四种读法各自最优解的陈述——后者需要把四个方案都解到最优，本文预算不允许；按 MDR-0013，四个方案之间已经交叉热启动并做过一轮修复，因此不会出现“某一行的解在另一行自己的目标下更好”这种自相矛盾。图 5 以图形给出同一对照。

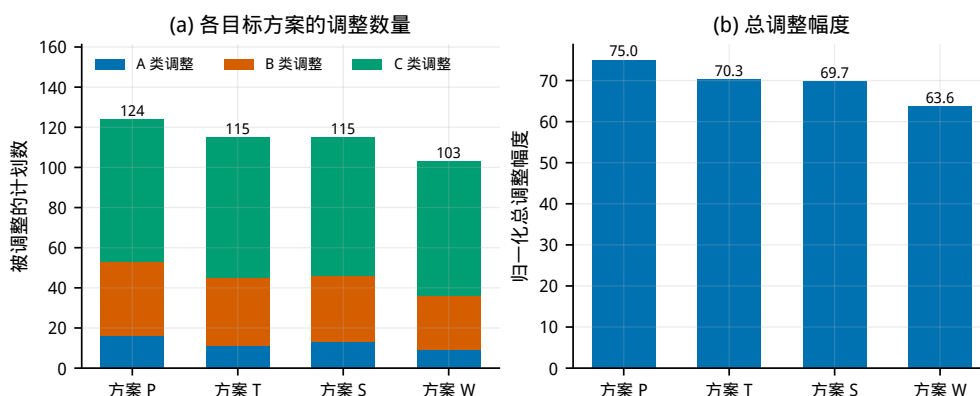


图 5 四种目标方案的对照。(a) 各方案下被调整的计划数（按类别堆叠）；(b) 归一化总调整幅度。方案 P（本文主方案）以较多的调整数量换取“完全不撤销 A、B 类计划”。

6 问题三：在无冲突方案上最多还能安排多少 C 类装备

本节回答：在第 5 节的无冲突方案之上、不增加时频资源，最多还能安排多少 C 类用频计划？结论是 109 个，且 CP-SAT 对偶界、HiGHS 线性松弛界与 MILP 界三者同时等于该值，即它是已证明的最大值。

6.1 题意的两种解释

题目原文为：“基于问题 2 求解得到的无冲突用频计划，如果不限制频段和时间的平移幅度，在不增加时频资源的前提下，最多还能安排多少 C 类用频装备？”。其中“平移”一词只对已经有位置的计划有意义，而新增装备本无“原位置”可平移，故存在两种读法：

解释 A（主解释） 问题 2 的方案原样保留；新增 C 类计划可放在资源区域内任意位置，不受 $10\Delta f/5\Delta t$ 的约束。此时“不限制平移幅度”是对新增计划放置自由度的说明。

解释 B 既有计划也可不受幅度限制地再次平移（频宽、时长、间隔、次数均不变）以腾出空间，在同一区域内最大化新增数量。

本文以 A 为主解释并据此填写 result3.xlsx，理由有二：其一，结果模板只有“序号、频段区间、时间区间”三列，恰好是新增计划的完整描述，无处记录既有计划的新位置，若按 B 作答，评阅者无法复核既有计划是否仍然无冲突；其二，A 的解答可仅凭 result2.xlsx 与 result3.xlsx 独立复核。但两种解释的答案相差约五倍，故本文在第 6.5 节为 B 给出严格的上下界，不回避这一歧义（MDR-0010）。

6.2 集合装填模型

按假设 9, 资源区域记为 $\mathcal{R} = \{0, \dots, B-1\} \times [0, T_{\text{end}})$ (与第 5 节的模型单元全集 $\mathcal{C}_2$ 区分: 这里不允许越出 T_{end}), $T_{\text{end}} = 643$ (问题 2 方案的最晚结束时刻)。记问题 2 存留计划占用的单元集合为 $\mathcal{O} = \bigcup_{i \in \mathcal{S}_2} U(P_i)$, 自由单元 $\mathcal{F} = \mathcal{R} \setminus \mathcal{O}$ 。一个新增 C 类计划由其频段起点 f 与首次使用起点 s 完全确定, 占用单元集合

$$U_{f,s} = [f, f+w_C) \times \bigcup_{k=0}^{n_C-1} [s+k\pi_C, s+d_C+k\pi_C), \quad (w_C, d_C, g_C, n_C) = (3, 2, 8, 12), \quad (10)$$

其时间跨度 $\sigma = d_C + (n_C - 1)\pi_C = 112$, $|U_{f,s}| = w_C d_C n_C = 72$ 。候选位置集合为

$$\mathcal{P} = \{(f, s) : 0 \leq f \leq B - w_C, 0 \leq s, s + \sigma \leq T_{\text{end}}, U_{f,s} \subseteq \mathcal{F}\}, \quad (11)$$

即“整块都落在自由单元上”的位置。引入 $x_{f,s} \in \{0, 1\}$, 问题 3 为

$$\max \sum_{(f,s) \in \mathcal{P}} x_{f,s} \quad \text{s.t.} \quad \sum_{(f,s) \in \mathcal{P}: (b,t) \in U_{f,s}} x_{f,s} \leq 1, \quad \forall (b, t) \in \mathcal{F}. \quad (12)$$

模型 (12) 是一个标准的集合装填问题 [5]。与命题 5.1 同理, 单元约束精确刻画了“新增计划彼此不冲突”, 而 $U_{f,s} \subseteq \mathcal{F}$ 已保证“新增与既有不冲突”。本实例中 $|\mathcal{P}| = 1344$, 去重后的单元约束 4378 条——变量数约为问题 2 的 1/3、约束行数约为 1/8 (表 6 与表 9), 且不含词典序的多级固定, 因而可以求到证明最优。

候选位置由一次向量化扫描生成: 对每个频段起点 f , 先求“该 3 频段带上全部空闲的时刻”的布尔向量, 再用长度为 $d_C n_C = 24$ 的时间偏移模板做滑窗与运算, 复杂度 $O(B \cdot T_{\text{end}} \cdot d_C n_C)$ (注意与每个计划占用的 $w_C d_C n_C = 72$ 个单元相区别)。求解前先用“按时间、再按频段”的首次适配贪心得到一个可行解作为热启动 (下界 103)。

6.3 上界链

最优性不能只依赖求解器的一句话, 本文给出一条从组合论证到线性松弛的上界链。

命题 6.1 (面积守恒与容量上界). 平移只改变 (f, s) 而不改变 (w, d, g, n) , 故任一计划占用的单元数 wdn 与其位置无关, 既有计划占用的单元总数 $S = \sum_{i \in \mathcal{S}_2} w_i d_i n_i$ 是常数。由于每个新增 C 类计划恰占 72 个单元, 新增数量 M 必满足

$$M \leq \left\lfloor \frac{B T_{\text{end}} - S}{w_C d_C n_C} \right\rfloor = \left\lfloor \frac{48484}{72} \right\rfloor = 673. \quad (13)$$

该上界对解释 A 与解释 B 同时成立, 因为它只用到“平移不改变占用面积”。

式 (13) 与新增计划的单元集合 (10)、候选集合 (11) 一起, 把“还能放多少”完全化为一个有限的组合问题。进一步按频段逐条统计自由单元可得更紧的逐频段容量界 658; 若区域内没有任何既有计划, 则上界为 893。这些是纯计数界, 通常很松; 真正紧的界来自组合搜索本身: CP-SAT 的对偶界与 HiGHS 对模型 (12) 的线性松弛。

表 9 问题 3 的解与上界链。三个独立的上界（CP-SAT 对偶界、HiGHS 线性松弛界、HiGHS MILP 界）与解值同为 109，故对偶间隙为零。末行为备选解释 B 的区间（第 6.5 节）。

项目	数值
时间范围 T_end（问题 2 方案最晚结束时刻）	643
既有计划数 / 占用单元 / 自由单元	138 / 15816 / 48484
候选放置位置数 (f,s)	1344
AtMostOne 约束数	4378
贪心下界	103
CP-SAT 最优解数量（状态）	109 (OPTIMAL)
CP-SAT 上界	109
HiGHS LP 松弛上界	109
HiGHS MILP 解 / 上界	109 / 109
自由单元容量界 floor(自由单元数/72)	673
逐频段容量界	658
空区域容量界（无既有计划）	893
最终结果 / 最紧上界 / gap	109 / 109 / 0
时频利用率（前 / 后）	24.6% / 36.8%
备选解释 B（既有计划可任意平移，MDR-0010）：下界 / 上界	572 / 673（重排移动 138 个既有计划）

6.4 结果：109 个新增计划，且已被证明最优

定理 6.2 (问题 3 的最优性). 在解释 A 与假设 9 之下，基于本文问题 2 的方案，最多还能安排 109 个 C 类用频装备，且这一数量是全局最优的。

证明. 可行性：CP-SAT 返回了一个含 109 个位置的可行解，该解经独立校验器复核——每个新增计划的参数、边界、与既有计划以及彼此之间的零冲突全部通过（通过）。最优性：CP-SAT 在状态 OPTIMAL 下给出对偶界 109；HiGHS 对同一模型的线性松弛最优值为 109.00，故整数最优值 $\leq \lfloor 109.00 \rfloor = 109$ ；HiGHS MILP 亦以状态 Optimal 给出界 109。三者与解值相等，间隙 0。□

安排这 109 个新增计划后，时频资源利用率由 24.6% 提高到 36.8%（图 6，清单见附录表 20，写入 result3.xlsx 共 109 行）。

一个反直觉但重要的观察。 最紧的上界 109 远小于容量界 673：自由单元足够放下 673 个新计划，实际却只能放下 109 个。差距来自碎片化——既有计划把自由区域切割成大量无法容纳 $3 \times 2 \times 12$ 这一形状的碎片。这引出下面的引理，它既解释了图 6 的条带结构，也说明了什么样的布局才能逼近容量界。

引理 6.3 (C 类计划的相位平铺). C 类计划的周期为 $\pi_C = d_C + g_C = 2 + 8$ ，单次时长 $d_C = 2$ ，故 $\pi_C/d_C = 5$ 为整数。在同一条 w_C 频段宽的带内，取首次起点为 $s, s + d_C, s + 2d_C, s + 3d_C, s + 4d_C$ 的五个计划互不冲突，且在其公共跨度内恰好铺满该带的全部单元。

证明. 第 r 个计划 ($r = 0, \dots, 4$) 占用的时刻集合为 $\{s + rd_C + k\pi_C + j : 0 \leq j < d_C, 0 \leq k < n_C\}$ ，即模 π_C 余数落在 $[rd_C, (r+1)d_C)$ 中的时刻。由于 $\pi_C = 5d_C$ ，五个

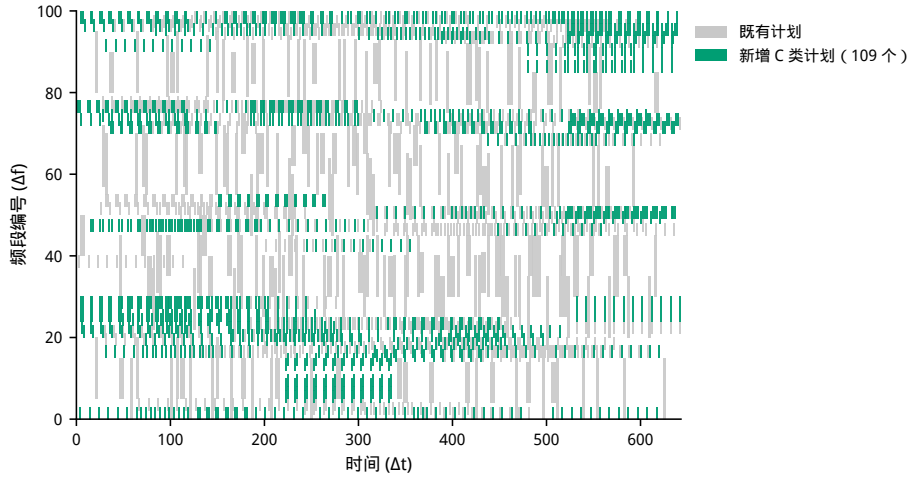


图 6 在问题 2 的无冲突方案（灰）之上最多可再安排 109 个 C 类用频计划（绿）。资源区域为 100 个频段 $\times [0, 643]$ ；新增计划与既有计划、以及新增计划彼此之间均无时频冲突。时频利用率由 24.6% 提升至 36.8%。新增计划呈现明显的水平条带结构，这是引理 6.3 所刻画的相位平铺。

余数区间恰构成 $\{0, \dots, \pi_C - 1\}$ 的一个划分，故五个计划两两不交且并集覆盖公共跨度内的全部时刻。□

引理 6.3 说明：局部利用率可以达到 100%，但前提是既有计划的边界与这一相位划分对齐。因此“还能放多少”主要取决于既有布局的对齐程度，而不取决于自由单元的总数。

这一依赖有多大？用产物回答（MDR-0014）。第 5.7 节的四个目标方案给出四个都合法的问题 2 无冲突方案。在每一个之上跑同一个装填模型，得到表 29：可再安排的数量在 109 到 146 之间（相差 34%），而对应的自由单元数只相差 1.8%——量级完全不同，这正是引理 6.3 所预言的：决定性因素是布局与相位划分的对齐，不是自由单元的总数。（反过来也要说清楚：若一个布局的占用集合是另一个的子集，则可装填数必然单调不减；表 29 中的四个布局互不包含，因此不存在这种单调关系。）

因此定理 6.2 的 109 是“在本文这一个问题 2 方案之上的最大值”，而非跨所有无冲突方案的最大值——这一措辞上的限定必须保留。联合优化（同时选择问题 2 的方案与新增计划的布局）本文未做，见第 9.2 节。

优先级条款的第二重代价。 表 29 还给出一个题目未问、但对用频管理有意义的结论：四个布局中最不利于后续新增的是方案 P（可再安排 109 个、撤销 12 个），最有利的是方案 S（146 个、撤销 7 个）；主方案 P 是唯一不撤销 A、B 类的方案，它是否恰为最不利者：通过。若为“通过”，则坚持优先级条款的代价是双重的：当期多撤销若干 C 类计划，留给后续新增的空间也更小。

6.5 备选解释 B：允许既有计划重新布局

若题意为解释 B，问题变为“既有计划与新增计划的联合布局优化”，规模远超解释 A。本文不给出无依据的点估计，而给出一个可核查的区间：

- 上界：由命题 6.1， $M \leq 673$ （该界只依赖面积守恒，对 A、B 同时成立）。

- **下界：**任何一个可行布局都给出下界。本文按“占用单元数降序的首次适配”把全部 138 个既有计划重新放置 (w, d, g, n 全部不变, 且区域包含性与零冲突经同一独立校验器复核, 通过), 再在剩余自由单元上用同一模型 (12) 求最大新增数, 得 572 (状态 OPTIMAL, 即该布局下的精确最大值)。解释 A 的最优值 109 本身也是 B 的一个下界。

$$572 \leq M_{\text{解释 B}} \leq 673. \quad (14)$$

采用解释 B 时, 时频资源的利用率将成倍提高 (图 7)。**两种解释的答案相差约五倍** (109 对 ≥ 572), 差别完全来自“是否允许既有计划再次平移”。本文无法断定命题人的本意, 故以 A 作答并把 B 的区间作为正式的备选分析给出。

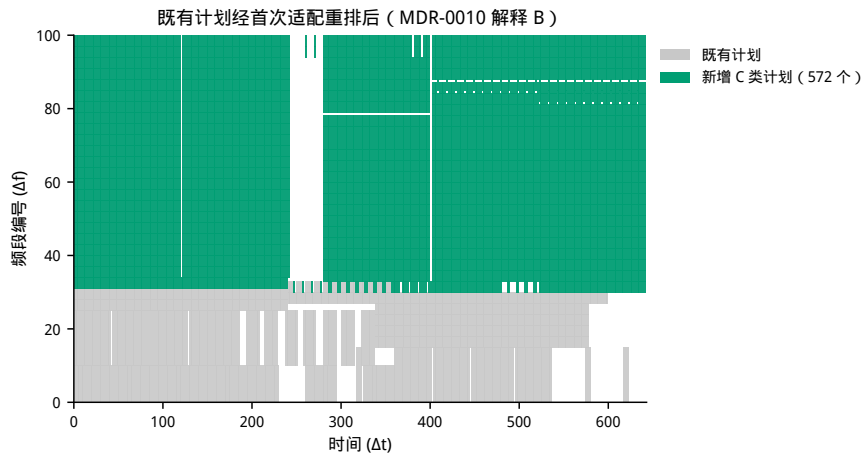


图 7 备选解释 B 下的布局：若允许既有计划不受平移幅度限制地重新放置（按占用单元数降序的首次适配重排，138 个计划全部移动，各自的频宽、时长、间隔与次数均不变），同一资源区域可再安排 572 个 C 类计划。与图 6 对照可见，重排把既有计划压到低频段与早时刻，从而腾出大片连续的自由区域。

7 问题四：允许 C 类装备调整用频间隔

本节回答：若允许部分 C 类装备把用频间隔调整不超过 $\Gamma = 10$ 个 Δt ，冲突消解能改善多少？结论是在不增加时频资源 ($e_i \leq 643$) 的前提下撤销数由 12 降到 8，代价是多调整 1 个计划，A、B 两类仍一个未撤。

7.1 动作集合的扩张

问题 4 允许部分 C 类装备调整用频间隔时长，与原间隔的差异不超过 $\Gamma = 10$ 。按假设 7，在式 (3) 的基础上为 C 类计划增加一族动作

$$\{\text{gap}_{\Delta g} : 0 < |\Delta g| \leq \Gamma, g_i + \Delta g \geq 1\}, \quad c(i) = C, \quad (15)$$

其余参数（频段区间、首次使用时间区间、使用次数）保持不变，仍满足“只能调整其中一个参数”。其幅度按 $|\Delta g|/\Gamma$ 归一化计入 L_7 (式 (7))。模型、目标与求解算法与问题 2 完全相同，只是 C 类每个计划的可选动作数显著增加 (表 6 与附录表 21 的“每计

划最多动作数”一行)，模型规模相应增大：变量由 4589 增至 6013、单元由 54768 增至 56092、约束由 35406 增至 41724。

注意式 (15) 的动作会改变计划的结束时刻 $e_i = s_i + d_i + (n_i - 1)(d_i + g'_i)$ ，因而可能多占用时间资源；本文为此加入硬约束 $e_i \leq T_{\text{cap}}$ ，见第 7.3 节。

命题 7.1 (问题 4 不劣于问题 2 的报告解). 设 T_{cap} 为问题 4 所加的资源守恒上限 (本文取原始计划的最晚结束时刻 643)。问题 4 的动作集合是问题 2 的超集，但附加了 $e_i \leq T_{\text{cap}}$ ，因此一般而言两者的可行域互不包含。不过，凡是最晚结束时刻不超过 T_{cap} 的问题 2 可行解都仍是问题 4 的可行解；本文报告的问题 2 方案满足这一条件 (其最晚结束时刻为 $643 \leq 643$)，故问题 4 的报告向量在词典序上不劣于问题 2 的报告向量 (本文在阶段内断言该性质：通过)。

注 7.2. 若去掉 $e_i \leq T_{\text{cap}}$ ，动作集合的超集关系即给出无条件的包含，此时问题 4 的最优向量不劣于问题 2 的最优向量；本文一并报告了这一不设限变体 (第 7.3 节)。

7.2 求解结果与对照

表 10 问题 4：时频冲突消解的统计结果 (题目表 1 格式)。

类别	保留数量	调整数量	撤销数量
A 类	3	17	0
B 类	3	37	0
C 类	11	71	8
合计	17	125	8

表 11 问题 4 与问题 2 的逐类别对照。

类别	保留 (问 2)	保留 (问 4)	调整 (问 2)	调整 (问 4)	撤销 (问 2)	撤销 (问 4)
A 类	4	3	16	17	0	0
B 类	3	3	37	37	0	0
C 类	7	11	71	71	12	8
合计	14	17	124	125	12	8
归一化幅度	-	-	75.0	73.8	-	-

表 10 给出题目要求的表 1 格式结果：保留 17 个、调整 125 个、撤销 8 个，目标向量 $(0, 0, 8, 17, 37, 71, 738)$ ；与问题 2 一样，A、B 两类计划仍然一个也没有撤销。写入 result4.xlsx 共 133 行，逐计划动作清单见附录表 24，被撤销计划的局部环境见附录表 22。

核心结论由表 11 给出：允许 C 类调整间隔，可把必须撤销的计划从 12 个减少到 8 个 (−4，降幅四分之一)，代价是多调整 1 个计划、归一化总幅度增加 −1.2。词典序不劣性检查通过 (通过)，与命题 7.1 一致。在 125 个被调整的计划中，25 个使用了新增的间隔调整动作 (平均 $|\Delta g| = 3.28$ 、最大 7)，其余仍为频段平移 (70 个) 或时间平移 (30 个)。

间隔调整为什么有效？ 频段平移与时间平移都是把整个使用序列刚性地搬动，而改变间隔是把序列在时间轴上拉伸或压缩：它同时改变了第 $1, 2, \dots, n-1$ 次使用的位置，且各次的位移量 $k\Delta g$ 互不相同。对于 C 类这种“次数多、周期短”的计划 ($n = 12$, $\pi_C = 2 + 8$)，一次 $|\Delta g| = 1$ 的调整就能让最后一次使用移动 $n_C - 1 = 12 - 1$ 个 Δt ，远超时间平移上限 $\mathcal{T} = 5$ 。这正是它能解开那些“平移无解”的死结的原因。

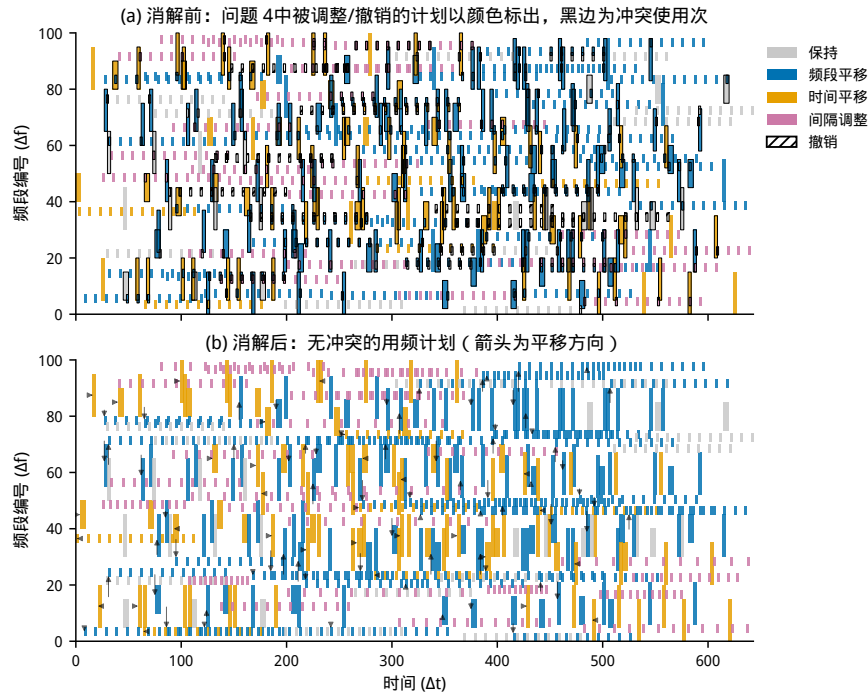


图 8 问题 4 的冲突消解方案，图例与图 4 相同，另增“间隔调整”一类（25 个 C 类计划改变了使用间隔）。

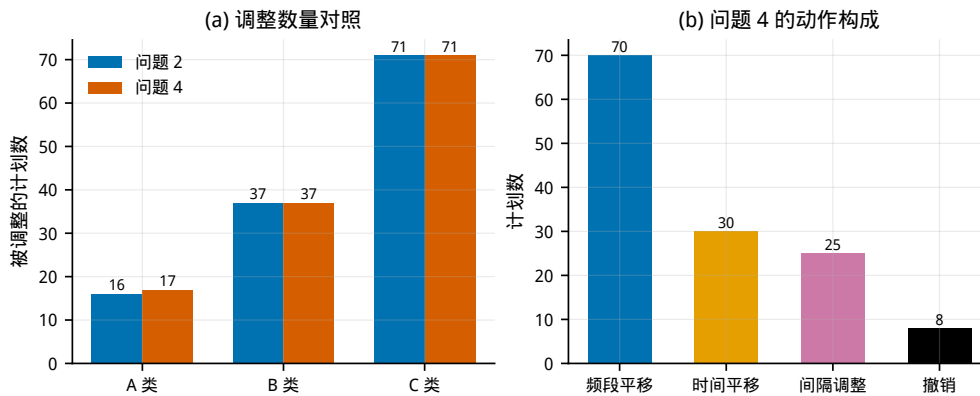


图 9 问题 4 与问题 2 的对照。(a) 各类被调整的计划数；(b) 问题 4 所采取动作的构成。允许 C 类调整间隔后，必须撤销的计划由 12 个降为 8 个。

表 12 问题 4 逐级求解证据（列的含义同表 7）。与问题 2 不同，本题的“撤销 C”一级在时限内未能闭合间隙，故 8 是已知最好的可行值而非已证明的最优值。

目标层级	CP-SAT 值	CP-SAT 下界	CP-SAT 状态	用时/s	HiGHS LP 下界	HiGHS MILP 值	HiGHS MILP 下界	HiGHS 状态
cancelA	0	0.00	OPTIMAL	0.7	0.00	0	0.00	Optimal
cancelB	0	0.00	OPTIMAL	1.5	0.00	0	0.00	Optimal
cancelC	8	5.00	FEASIBLE	2400.9	0.00	-	0.00	Time limit reached
adjustA	17	6.00	FEASIBLE	500.4	-	-	0.00	Time limit reached
adjustB	37	19.00	FEASIBLE	501.0	-	-	0.00	Time limit reached
adjustC	71	46.00	FEASIBLE	500.8	-	-	0.00	Time limit reached
amplitude	738	357.00	FEASIBLE	400.6	-	-	0.00	Time limit reached

7.3 最优性证据与时间范围的代价

表 12 显示：撤销 A、B 两级为 0 且状态 OPTIMAL（故“不撤销任何 A、B 类计划”在问题 4 中同样被证明可行且最优）；撤销 C 一级的值 8 与证明下界 5 之间仍有间隙，状态 FEASIBLE。因此本文对问题 4 的表述是：8 是已知最好的可行方案，且严格优于问题 2 的 12；其真实最优值落在 $[5, 8]$ 之间。这里体现了现任解保持算法的价值：热启动提示的向量为 $(0, 0, 12, 16, 37, 71, 750)$ ，其第三个分量即提示解的撤销 C 数量，而词典序搜索在保持前两级不变的前提下把它进一步改进到 8。

时间资源守恒 (MDR-0011)。 间隔变长会把最后一次使用推后，因此“改间隔”这一动作有可能多占用时间资源。题目只在问题 3 中写明“不增加时频资源”，问题 4 的题面未重复这一条；但既然同一份用频计划所能占用的时频资源在问题 3 中被明确限定，本文对问题 4 采用同一尺度：把 $e_i \leq T_{\text{cap}} = 643$ （即问题 2 方案的最晚结束时刻）作为硬约束写进模型，在动作层面直接过滤掉会越界的动作，并由独立校验器断言存留计划的最晚结束时刻不超过 T_{cap} 。本文报告的问题 4 方案因此满足 $643 \leq 643$ ，**没有多占用任何时间资源**，撤销数仍由 12 降到 8。

为量化这一约束的代价，本文另解一次不设限变体（同一动作集合，去掉 $e_i \leq T_{\text{cap}}$ ，以受限解热启动，故其向量不会比受限解更差）：它给出向量 $(0, 0, 8, 15, 37, 74, 751)$ 、撤销总数 8、最晚结束时刻 686。该变体是否严格优于受限解：通过；两者是否给出同一向量：未通过。两个版本的对照见表 11，读者可据此自行判断“是否值得用时间跨度换回用频计划”。

交叉验证的诚实说明。 问题 4 的分离权重单目标在其时限内只回到一个更差的向量，未能复现词典序解（未通过）。这不构成矛盾——词典序解严格更优，且由命题 5.4 其值永不劣于任何已知可行解——但它说明：在该规模上，单次加权求解弱于逐级求解。HiGHS 在其证明最优的各级与 CP-SAT 一致（通过）。

8 模型检验

组合优化的结果不能只靠“求解器说它是最优的”来支撑。本节按五条独立的证据线检验本文的模型与结果：（一）与求解器代码分离的可行性校验；（二）算法之间的交叉验证；（三）最优性的证明与界；（四）穷举对照与确定性；（五）灵敏度与可扩展性。每一条都给出可核查的产物，并明确区分“已证明”与“已知最好”。

8.1 与求解器分离的独立校验

本文实现了一套**独立校验器**，它刻意不复用检测与求解的任何代码：对给定的一组计划，它按定义 (1) 把每个计划显式展开为时频单元的集合，再用集合求交逐对判断冲突。这是命题 4.1 的“另一侧”——区间算术与集合展开是同一事实的两种算法实现，二者一致才说明实现正确，而不是同一段代码自证。

校验内容逐题不同：问题 1 复核冲突对集合（真值 297 对，缺失 0、多余 0）；问题 2、4 逐计划检查“恰有一个动作”、“调整后与原计划恰有一个参数不同”、 $|\delta| \leq \Phi$ 、 $|\tau| \leq \mathcal{T}$ 、 $|\Delta g| \leq \Gamma$ 、 $g' \geq 1$ 与边界条件，再对存留计划做零冲突扫描，并重新计算表 5、10 与目标向量与求解器报告比对；问题 3 检查新增计划的参数、区域包含性，以及“既有-既有”、“既有-新增”、“新增-新增”三类零冲突。四个校验全部通过（表 13：通过、通

表 13 四个问题的独立校验结论。校验器与检测/求解代码分离，按定义展开时频单元集合并用集合求交复核。

对象	独立校验方法	校验内容	结论
问题 1 冲突检测	cell-set intersection	真值 297 对，报告 297 对，缺失 0，多余 0	通过
问题 2 消解方案	independent per-plan legality checks + cell-set conflict scan	150 个决策，存留 138，剩余冲突 0	通过
问题 3 新增计划	parameter/bound checks + cell-set conflict scan over existing and new plans	新增 109，剩余冲突 0	通过
问题 4 消解方案	independent per-plan legality checks + cell-set conflict scan	150 个决策，存留 142，剩余冲突 0	通过

过、通过、通过)，备选解释 B 的重排布局亦通过同一校验（通过，另加“ w, d, g, n 不变”的参数不变性断言）。**结果工作簿在四个校验全部通过之前不会被写出——这是流水线中的硬门禁。**

8.2 算法之间的交叉验证

两种检测算法。 成对枚举（算法 1）与按频段分桶的扫描线给出完全相同的 297 对冲突；在 $n = 150, \dots, 600$ 的合成实例上二者也始终一致（通过，附录表 27）。加上第 8.1 节的单元集合校验器，问题 1 的答案由三种互不相同的算法共同确认。

两个求解器。 本文用 HiGHS[15] 对问题 2、4 的模型独立建模（0–1 线性规划形式），逐级给出线性松弛下界与限时 MILP 结果。在 HiGHS 自身证明最优的每一级上，其值与 CP-SAT 完全一致（通过、通过）。问题 3 更进一步：CP-SAT 对偶界、HiGHS 线性松弛界与 HiGHS MILP 界三者同时等于 109（表 9）。

两种标量化。 命题 5.2 给出词典序的等价单目标形式。本文以此做第二次独立搜索（不以词典序解热启动，而以仓库内的现任解热启动）：问题 2 返回了与逐级求解相同的目标向量（通过）。问题 4 则未能复现（未通过）——单次加权求解只回到一个更差的向量。本文如实报告这一点：它不是矛盾（词典序解严格更优，且由命题 5.4 永不劣于任何已知可行解），而是说明在该规模上逐级求解强于单次加权求解。

诚实的界定。 需要指出：两种标量化都以同一个现任解为起点，因此问题 2 的“一致”主要说明两条独立的搜索路径都没有找到更好的解，而不等于独立地证明了最优性。本文的最优性证据只有一个来源——表 7、12、9 中的“证明下界”一列。

8.3 最优性：已证明的与未证明的

本文对最优性的声明严格限定在求解器闭合了对偶间隙的层级上：

- **已证明最优：**问题 2 的撤销三级 (0, 0, 12)（定理 5.5，三级状态均为 OPTIMAL，值与证明下界相等）；问题 4 的撤销 A、B 两级（均为 0，状态 OPTIMAL）；问题 3 的 109（定理 6.2，三个独立上界同时等于解值，间隙 0）。
- **已知最好但未证明最优：**问题 2 的调整三级与幅度级（未通过）；问题 4 的撤销 C 级与其后各级（未通过）。它们的真实最优值落在表 7、12 给出的 $[\underline{v}_k, \bar{v}_k]$ 区间内。

由命题 5.4，现任解上界割不改变任何一级的最优值，因此上述“已证明”的层级确实是全局最优；同时该割保证报告向量在词典序上永不劣于任何已知可行解，故未证明的

层级至少不会比先前任何一次搜索更差。线性松弛下界在本模型上很弱（表 7 中撤销级的 LP 下界为 0），这是装填型约束的典型现象：分数解可以把每个计划“摊薄”到多个动作上而不违反任何单元约束。因此紧的界只能来自 CP-SAT 的组合推理。

8.4 穷举对照与确定性

最优性证据之外，还需检验求解器被正确使用——目标是否真的按词典序最小化了、约束是否真的编码了“无冲突”。为此本文构造 24 个随机微型实例（计划数 ≤ 6 、动作集合缩小），对每个实例枚举全部动作组合（最多 229376 种）直接求出词典序最优向量，再与本文求解器的结果比对：24/24 **全部一致**（通过，附录表 25）。同样的实例用相同随机种子重复求解，24/24 给出完全相同的解（通过）。

局限：确定性只在微型实例上检验过。150 个计划的完整模型单次求解需数十分钟，本文未做重复求解，故 150 规模上的确定性记为“未检验”。由于 CP-SAT 的多线程搜索在固定种子下仍可能受线程调度影响，大规模模型的位对位可复现性不应被假定；但本文的结论不依赖于此——撤销层的最优性由证明下界保证，与是否得到同一个最优解无关。

8.5 灵敏度分析

表 14 消解方案对最大平移幅度 $(\Phi, \mathcal{T})$ 的灵敏度。第四行 (10, 5) 为题目给定值，即正文的问题 2 主结果。每个情形的解都通过了独立校验（通过）。

最大频段平移	最大时间平移	撤销 A/B/C	调整 A/B/C	调整合计	归一化幅度	各级最优	用时/s
5	2	0/0/59	19/36/26	81	31.0	是	5.2
5	5	0/0/42	16/35/40	91	46.7	否	251.7
10	2	0/0/24	19/36/59	114	64.4	否	262.6
10	5	0/0/12	16/37/71	124	75.0	否	1612.8
15	5	0/0/0	12/34/72	118	98.4	否	237.2
20	10	0/0/0	5/36/83	124	136.9	否	241.6

题目把最大平移幅度定为 $(\Phi, \mathcal{T}) = (10, 5)$ ，但未说明这一取值的来源，故它是最值得做灵敏度分析的参数。本文在 6 组取值上重解同一模型（表 14、图 10），得到三条结论：

1. **撤销数量对频段平移上限极其敏感。**把 Φ 由 10 放宽到 15，撤销数由 12 直接降到 0——一个计划都不必撤销；反之在 $\mathcal{T} = 5$ 不变时把 Φ 收紧到 5，撤销数升至 42；若同时把 $\mathcal{T}$ 收紧到 2，则升至 59。这再次印证第 4 节的判断：**撤销源于局部拥塞与幅度上限，而非资源总量不足。**
2. **频域比时域更“贵”。**两个上限的增量不等（ $+5\Delta f$ 对 $+3\Delta t$ ），直接比较总降幅会误导，故按单位增量折算： Φ 由 5 增至 10（ $\mathcal{T}$ 固定为题给的 5）使撤销数由 42 降到 12，合每 $1\Delta f$ 少撤销 6.0 个； $\mathcal{T}$ 由 2 增至 5（ Φ 固定为题给的 10）使撤销数由 24 降到 12，合每 $1\Delta t$ 少撤销 4.0 个。**频域的单位效果约为时域的 1.50 倍**——这与问题 1 中“B-C 类冲突占六成以上、而 B 类频宽最大”的统计一致。该比值依赖于所取的两个基准点，换一组对照会得到不同的数值，故只作数量级判断，不作精确结论。

3. 一条有实际价值的管理建议。若能把最大频段平移幅度由 $10\Delta f$ 放宽到 $15\Delta f$ ，代价仅是多调整一个计划、归一化总幅度由 75.0 升至表 14 第五行所示的水平，却可以完全避免撤销。对用频管理部门而言，这意味着“适度放宽频段调整权限”比“增加带宽”更具性价比。

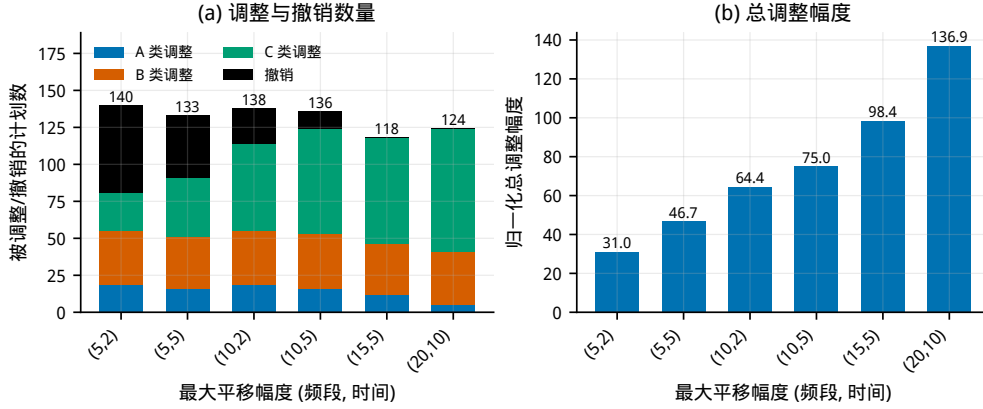


图 10 消解方案对最大平移幅度的灵敏度。(a) 被调整（按类别堆叠）与被撤销（黑）的计划数；(b) 归一化总调整幅度。频段平移上限由 $10\Delta f$ 放宽到 $15\Delta f$ 时撤销数降为 0，在 $\mathcal{T} = 5$ 不变时收紧到 $5\Delta f$ 则升至 42，说明冲突主要源于频域拥塞。

单调性检查及其有效范围（诚实说明）。 动作集合更大的情形其最优向量应当不劣于基准，更小的情形应当不优于基准；该检查通过（通过）。但必须指出：自现任解保持算法引入后，动作集合更大的情形 $((15, 5), (20, 10))$ 中基准解本身可行，会成为现任解并给出上界割，因而其“不劣于基准”是由构造保证的，不再是独立检验；只有动作集合更小的情形 $((5, 2), (5, 5), (10, 2))$ 中基准解一般不可行、不产生割，其“不优于基准”仍是真正的检验。

其他不确定性来源。 目标函数的形式化（假设 8）由四方案对照量化（表 8、附录表 23）；问题 3 的题意歧义由解释 A 与 B 的区间式 (14) 量化； T_{end} 的两种取法（问题 2 方案的最晚结束时刻与原始计划的最晚结束时刻）在本实例中相同（均为 643），故不产生差异。

8.6 可扩展性

附录表 27 与图 11 给出 $n = 150, \dots, 600$ 的合成实例基准。两种检测算法均为亚秒级（最大 0.17 s 与 0.02 s），说明**检测不是瓶颈**；消解模型的变量数与约束数随 n 近似线性增长。**局限**：在固定 120 s 时限下，全部合成实例的求解状态均为 FEASIBLE（未通过），即本文没有证据表明该方法在更大规模上仍能证明最优。这与问题的 NP-难性（命题 5.3）一致：可扩展的是建模与求可行解，不是证明最优性。

9 模型评价与推广

本节回答：本文的方法在什么范围内可信？优点是“无冲突”的充要编码与求解器分离的独立校验，使每个结果都可被逐位复核；局限是最优性只在撤销层与问题 3 被证明，调整层只有值与下界，若干题意读法本文只能并列给出而无法判定。

9.1 模型的优点

1. **统一的单元格语言。**四个问题——检测、消解、增容、放宽间隔——被统一到“时频单元集合是否相交”这一个判据上（命题 4.1、5.1）。这既使模型精确（单元约束是无冲突的充要编码，不是近似或松弛），又使校验独立（同一判据的另一种算法实现即构成校验器）。
2. **可证明的部分被明确证明。**问题 2 撤销层的全局最优性（定理 5.5）与问题 3 的最优性（定理 6.2）都给出了闭合的对偶间隙，后者更由三个独立上界同时确认。推论 5.6 直接回答了“能否一个都不撤销”这一自然追问。
3. **未证明的部分被诚实标注。**调整各级只报告“值 / 证明下界 / 状态”（表 7、12），不作最优性声明。现任解上界割（命题 5.4）在数学上保证报告值不劣于任何已知可行解，从而消除了“交叉验证击败主解”这类自相矛盾。
4. **主观选择被量化。**目标函数的形式化是本文最主观的一步，四方案对照（表 8）把“优先级条款值多少”变成了可以看见的数字：坚持不撤销 A、B 类计划的代价，是撤销总数从 7 上升到 12。
5. **题意歧义被正面处理。**问题 3 的两种读法给出相差约五倍的答案，本文不回避，而是以模板一致性为准选定主解释，并为另一解释给出可核查的区间式 (14)。
6. **结果可复现，且复现的边界被写明。**论文报告的全部数值产生于同一次云端运行，每个数字由阶段产物自动注入论文，每个阶段记录代码版本、参数与输入输出的 SHA-256（附录 C）；支撑材料另附不依赖云端账号的本地驱动脚本。唯一需要外部输入的是热启动现任解，本文报告了它的来源、本轮是否改进了它，以及不加载它时的冷启动结果（第 5.4 节）。

9.2 模型的局限

1. **调整层的最优性未闭合。**问题 2 的调整三级与幅度级、问题 4 的撤销 C 级均未证明最优（未通过、未通过）。其间隙较宽，主要原因是线性松弛在装填型约束上很弱（撤销级的 LP 下界为 0）。改进方向是加入更强的有效不等式（如基于冲突图团的覆盖不等式）或列生成。
2. **问题 3 的答案依赖于具体的问题 2 方案。**定理 6.2 的 109 是“在本文这一个无冲突方案之上”的最大值。本文在 4 个无冲突方案（表 8 的四种读法）上各解一次装填，可装填数介于 109 与 146 之间（相差 34%），而对应的自由单元数只相差 1.8%——由引理 6.3，可装填数量主要取决于既有布局与相位划分的对齐程度，而非自由单元的总数。若要回答“跨所有无冲突方案的最大值”，需把问题 2 与问题 3 合并为一个联合优化，规模将显著增大。
3. **问题 4 的“不增加时频资源”是本文自加的解释。**题目只在问题 3 写明该要求，问题 4 未重复；本文按同一尺度加入硬约束 $e_i \leq 643$ （第 7.3 节），并同时报告去掉该约束的变体（撤销 8、最晚结束时刻 686）。若命题人本意是不设限，应以变体为准；两种读法的结果都已给出，本文不替评阅者做这一判断。

4. **离散化与动作集合是题目给定的。** Δf 、 Δt 的粒度、“只改一个参数”的限制、平移幅度上限均为外生。连续化模型、多参数联合调整、功率或方向性维度都未建模。
5. **物理层机制未建模**（假设 10）。真实电磁兼容需要保护带、空间隔离、互调与杂散抑制等；本文的“冲突”是纯几何的。好在模型对此是可扩展的：只需把 $U(P)$ 沿频率（或时间）方向膨胀一个保护宽度，全部命题与算法原样成立，只是冲突对数会上升。
6. **大规模的可证明最优性未验证。**合成实例在固定时限下只达到可行解（未通过）。

9.3 模型的推广

（一）带保护带的冲突判据。 把频段方向的占用由 $[f, f + w)$ 膨胀为 $[f - \rho, f + w + \rho)$ (ρ 为保护频段数)，时间方向同理膨胀为收发转换时间，命题 4.1–5.1 与全部算法不变。这使模型可直接用于真实的频谱指配。

（二）在线与滚动消解。 实际用频计划是陆续提报的。把已批准的计划视为固定占用、新提报的计划视为待放置对象，问题即退化为问题 3 的装填模型；若允许有限次回溯调整，则是问题 2 模型在小邻域上的重解。由于检测是亚秒级的（表 27），在线判定完全可以实时完成。

（三）鲁棒性与不确定性。 若计划的起始时刻带有抖动 $\pm\epsilon$ ，可把每次使用的时间区间膨胀 ϵ 后重解，得到“对抖动免疫”的方案；方案的代价随 ϵ 的增长曲线正是第 8.5 节灵敏度分析的变体。

（四）同类问题。 本文的“每个对象恰选一个位置、位置之间不得共享资源单元、按优先级词典序最优”的结构，同样适用于会议室与航班时刻分配、光网络波长指配、数据中心作业的时间–机架放置等问题；单元格 AtMostOne 编码与现任解保持的词典序搜索可原样迁移。

9.4 给用频管理部门的建议

基于本文的定量结果，提出三条建议：**其一**，适度放宽频段调整权限比增加带宽更有效——把最大频段平移幅度由 $10\Delta f$ 提高到 $15\Delta f$ ，即可把必须撤销的计划数从 12 降为 0（第 8.5 节）；**其二**，允许 C 类装备微调用频间隔是低成本的松弛——在不增加时频资源 ($e_i \leq 643$) 的前提下，它把撤销数从 12 降到 8（表 11），代价是多调整 1 个计划；**其三**，若提报时能引导 C 类装备按引理 6.3 的相位错开（首次时刻相差 d_C 的整数倍），同一频带的利用率可接近 100%，从源头上减少冲突。

AI 工具使用声明

本研究在问题分析、方案比较、程序编写与调试、云端实验组织、数值核验、图表绘制以及论文文字组织等环节使用了生成式 AI 工具（Anthropic Claude Code，模型 Claude Fable 5.1）；所有数据处理、计算与排版在 Modal 云平台执行。使用过程与人工核验方式详见支撑材料中的《AI 工具使用详情.pdf》。

参考文献

- [1] KARP R M. Reducibility among combinatorial problems[M]//MILLER R E, THATCHER J W. Complexity of Computer Computations. New York: Plenum Press, 1972: 85-103.
- [2] GAREY M R, JOHNSON D S. Computers and intractability: A guide to the theory of NP-completeness[M]. San Francisco: W. H. Freeman, 1979.
- [3] HALE W K. Frequency assignment: Theory and applications[J]. Proceedings of the IEEE, 1980, 68(12): 1497-1514.
- [4] AARDAL K I, VAN HOESEL S P M, KOSTER A M C A, et al. Models and solution techniques for frequency assignment problems[J]. Annals of Operations Research, 2007, 153(1): 79-129.
- [5] PADBERG M W. On the facial structure of set packing polyhedra[J]. Mathematical Programming, 1973, 5(1): 199-215.
- [6] LODI A, MARTELLO S, MONACI M. Two-dimensional packing problems: A survey[J]. European Journal of Operational Research, 2002, 141(2): 241-252.
- [7] EHRGOTT M. Multicriteria optimization[M]. 2nd ed. Berlin: Springer, 2005.
- [8] DE BERG M, CHEONG O, VAN KREVELD M, et al. Computational geometry: Algorithms and applications[M]. 3rd ed. Berlin: Springer, 2008.
- [9] CORMEN T H, LEISERSON C E, RIVEST R L, et al. Introduction to algorithms[M]. 3rd ed. Cambridge, MA: MIT Press, 2009.
- [10] GOLUMBIC M C. Algorithmic graph theory and perfect graphs[M]. 2nd ed. Amsterdam: Elsevier, 2004.
- [11] WOLSEY L A. Integer programming[M]. 2nd ed. Hoboken, NJ: Wiley, 2020.
- [12] ACHTERBERG T, WUNDERLING R. Mixed integer programming: Analyzing 12 years of progress[M]//JÜNGER M, REINELT G. Facets of Combinatorial Optimization. Berlin: Springer, 2013: 449-481.
- [13] FOWLER R J, PATERSON M S, TANIMOTO S L. Optimal packing and covering in the plane are NP-complete[J/OL]. Information Processing Letters, 1981, 12(3): 133-137. DOI: 10.1016/0020-0190(81)90111-3.
- [14] PERRON L, DIDIER F. CP-SAT solver, OR-Tools v9.12[Z]. 2025.
- [15] HUANGFU Q, HALL J A J. Parallelizing the dual revised simplex method[J]. Mathematical Programming Computation, 2018, 10(1): 119-142.

A 补充表格

本附录收录正文中引用但因篇幅置于此处的完整表格。全部表格由 tables 阶段生成 (.tex 与同名 .csv 并存)，数值与结果工作簿一致。

A.1 问题 1：冲突强度、度分布与全部冲突对

表 15 冲突强度分布：横跨 k 对使用次的冲突对数量。多数冲突对只在一次使用上相撞。

冲突使用次对数 k	冲突对数量
1	236
2	45
3	3
4	2
6	2
7	4
8	2
10	2
12	1

表 16 冲突图中度最大的 10 个用频装备。

装备编号	冲突计划数 (度)
B009	8
B016	8
B024	8
B027	8
B030	8
C076	8
B008	7
B011	7
B013	7
B018	7

表 17 问题 1 检测出的全部 297 对时频冲突（按装备编号字典序，与 result1.xlsx 一致）。“冲突使用次对数”为该计划对中出现时间重叠的使用次对数，“频段重叠宽度”以 Δf 计。

序号	装备 1	装备 2	类别对	冲突使用次对数	频段重叠宽度
1	A001	B001	AB	1	10
2	A001	C002	AC	1	3
3	A001	C012	AC	1	3
4	A001	C022	AC	1	3
5	A002	B010	AB	1	10
6	A002	C009	AC	1	2
7	A002	C010	AC	1	2
8	A002	C019	AC	1	3
9	A002	C029	AC	1	3
10	A003	B009	AB	2	8
11	A003	C008	AC	2	3
12	A003	C018	AC	1	3
13	A003	C028	AC	1	3
14	A004	B009	AB	1	2
15	A004	B014	AB	1	10
16	A004	B018	AB	1	5
17	A004	C016	AC	1	3
18	A004	C017	AC	1	3
19	A004	C027	AC	1	1
20	A005	B007	AB	1	10
21	A005	B012	AB	1	10

序号	装备 1	装备 2	类别对	冲突使用次对数	频段重叠宽度
22	A005	C014	AC	1	3
23	A005	C024	AC	1	3
24	A005	C054	AC	1	3
25	A006	B004	AB	1	5
26	A006	B008	AB	1	10
27	A006	C006	AC	1	3
28	A007	B011	AB	1	10
29	A007	B022	AB	1	7
30	A007	C023	AC	1	3
31	A007	C053	AC	1	1
32	A008	C028	AC	1	1
33	A008	C038	AC	1	3
34	A008	C039	AC	1	3
35	A008	C049	AC	1	3
36	A008	C058	AC	1	1
37	A008	C059	AC	1	3
38	A009	B012	AB	1	5
39	A009	C033	AC	2	1
40	A009	C043	AC	1	3
41	A009	C053	AC	1	3
42	A009	C054	AC	1	2
43	A010	C027	AC	1	3
44	A010	C037	AC	1	3
45	A010	C057	AC	1	3
46	A011	B027	AB	1	5
47	A011	C034	AC	1	3
48	A011	C044	AC	1	3
49	A011	C074	AC	1	3
50	A012	B024	AB	1	5
51	A012	B028	AB	1	5
52	A012	C036	AC	1	3
53	A012	C046	AC	1	3
54	A012	C056	AC	1	3
55	A012	C076	AC	1	3
56	A013	B016	AB	1	7
57	A013	C022	AC	1	3
58	A013	C032	AC	1	3
59	A013	C041	AC	1	2
60	A013	C052	AC	1	3
61	A014	C040	AC	1	3
62	A014	C050	AC	1	3
63	A014	C070	AC	1	3
64	A014	C080	AC	1	1
65	A014	C090	AC	1	3
66	A015	B030	AB	2	5
67	A015	C067	AC	1	2
68	A015	C077	AC	1	3
69	A015	C078	AC	2	3
70	A016	C067	AC	1	3

序号	装备 1	装备 2	类别对	冲突使用次对数	频段重叠宽度
71	A016	C087	AC	1	3
72	A017	B033	AB	1	8
73	A017	C065	AC	1	3
74	A017	C075	AC	2	3
75	A017	C085	AC	1	3
76	A018	B028	AB	1	10
77	A018	B033	AB	1	3
78	A018	C065	AC	1	1
79	A018	C066	AC	1	3
80	A018	C076	AC	1	1
81	A018	C085	AC	1	3
82	A019	C073	AC	1	3
83	A019	C082	AC	1	3
84	A020	C039	AC	1	3
85	A020	C069	AC	1	3
86	A020	C079	AC	1	3
87	A020	C089	AC	1	3
88	B001	C001	BC	2	3
89	B001	C002	BC	1	3
90	B001	C012	BC	1	3
91	B002	B011	BB	1	10
92	B002	C003	BC	2	3
93	B002	C013	BC	2	3
94	B002	C014	BC	1	3
95	B003	C004	BC	1	3
96	B003	C005	BC	1	3
97	B003	C015	BC	1	3
98	B003	C024	BC	1	3
99	B004	C016	BC	1	3
100	B004	C017	BC	1	3
101	B004	C027	BC	1	3
102	B005	C009	BC	1	3
103	B005	C010	BC	1	2
104	B005	C019	BC	1	3
105	B005	C029	BC	1	3
106	B005	C030	BC	1	3
107	B006	C001	BC	1	3
108	B006	C011	BC	2	3
109	B006	C012	BC	1	1
110	B006	C021	BC	1	3
111	B006	C051	BC	1	3
112	B006	C052	BC	1	3
113	B007	C004	BC	1	3
114	B007	C013	BC	1	3
115	B007	C014	BC	1	3
116	B007	C024	BC	2	3
117	B008	C006	BC	1	3
118	B008	C015	BC	2	3
119	B008	C016	BC	1	3

序号	装备 1	装备 2	类别对	冲突使用次对数	频段重叠宽度
120	B008	C025	BC	1	1
121	B008	C026	BC	1	3
122	B008	C055	BC	1	3
123	B009	C008	BC	2	1
124	B009	C017	BC	2	1
125	B009	C018	BC	1	1
126	B009	C027	BC	2	3
127	B009	C028	BC	1	3
128	B009	C057	BC	1	3
129	B010	C009	BC	1	3
130	B010	C010	BC	1	2
131	B010	C030	BC	1	3
132	B011	C012	BC	1	2
133	B011	C013	BC	1	3
134	B011	C023	BC	1	3
135	B011	C043	BC	1	3
136	B011	C053	BC	2	3
137	B012	C024	BC	1	3
138	B012	C025	BC	1	2
139	B012	C054	BC	1	3
140	B013	B018	BB	3	10
141	B013	B023	BB	1	10
142	B013	C015	BC	1	3
143	B013	C025	BC	2	3
144	B013	C026	BC	1	3
145	B013	C036	BC	1	3
146	B013	C055	BC	1	3
147	B014	C017	BC	1	3
148	B014	C027	BC	1	3
149	B014	C037	BC	1	3
150	B014	C056	BC	1	3
151	B014	C057	BC	1	3
152	B015	C018	BC	1	3
153	B015	C028	BC	1	3
154	B015	C038	BC	2	3
155	B015	C039	BC	1	3
156	B015	C058	BC	1	3
157	B015	C059	BC	1	3
158	B016	C021	BC	1	3
159	B016	C022	BC	1	3
160	B016	C031	BC	1	3
161	B016	C041	BC	2	3
162	B016	C042	BC	1	1
163	B016	C051	BC	1	3
164	B016	C052	BC	1	3
165	B017	C014	BC	1	3
166	B017	C033	BC	1	1
167	B017	C043	BC	1	3
168	B017	C054	BC	1	3

序号	装备 1	装备 2	类别对	冲突使用次对数	频段重叠宽度
169	B018	B023	BB	2	5
170	B018	C026	BC	1	3
171	B018	C036	BC	1	3
172	B018	C055	BC	1	3
173	B018	C056	BC	1	3
174	B019	B024	BB	2	10
175	B019	C028	BC	1	2
176	B019	C058	BC	1	2
177	B020	C019	BC	2	3
178	B020	C020	BC	1	3
179	B020	C029	BC	1	3
180	B020	C030	BC	1	3
181	B020	C040	BC	1	3
182	B021	C031	BC	1	1
183	B021	C032	BC	1	3
184	B021	C041	BC	1	3
185	B021	C042	BC	1	3
186	B021	C051	BC	1	1
187	B021	C052	BC	1	3
188	B021	C072	BC	1	3
189	B022	C033	BC	1	3
190	B022	C044	BC	1	3
191	B022	C054	BC	1	2
192	B022	C073	BC	1	3
193	B022	C074	BC	1	2
194	B023	C034	BC	1	3
195	B023	C035	BC	2	3
196	B023	C045	BC	1	3
197	B023	C074	BC	1	1
198	B023	C075	BC	1	3
199	B024	C037	BC	1	3
200	B024	C046	BC	1	3
201	B024	C047	BC	1	3
202	B024	C076	BC	1	2
203	B024	C077	BC	1	3
204	B024	C086	BC	1	3
205	B025	B040	BB	2	5
206	B025	C049	BC	1	3
207	B026	C031	BC	1	1
208	B026	C042	BC	1	3
209	B026	C061	BC	1	3
210	B026	C062	BC	2	3
211	B026	C071	BC	2	3
212	B027	B032	BB	2	13
213	B027	B033	BB	2	2
214	B027	C033	BC	1	3
215	B027	C043	BC	1	3
216	B027	C044	BC	2	3
217	B027	C063	BC	1	3

序号	装备 1	装备 2	类别对	冲突使用次对数	频段重叠宽度
218	B027	C073	BC	1	3
219	B028	C036	BC	1	3
220	B028	C045	BC	2	3
221	B028	C066	BC	1	3
222	B028	C075	BC	1	3
223	B028	C076	BC	1	1
224	B029	B034	BB	2	15
225	B029	B038	BB	1	5
226	B029	C046	BC	1	3
227	B029	C047	BC	1	3
228	B029	C076	BC	1	2
229	B029	C077	BC	1	3
230	B029	C086	BC	2	3
231	B030	C038	BC	1	3
232	B030	C048	BC	2	3
233	B030	C049	BC	1	3
234	B030	C068	BC	1	3
235	B030	C069	BC	1	3
236	B030	C078	BC	2	3
237	B030	C079	BC	1	3
238	B031	C042	BC	1	3
239	B031	C061	BC	2	3
240	B031	C062	BC	2	3
241	B031	C071	BC	2	3
242	B031	C072	BC	1	3
243	B031	C081	BC	1	3
244	B032	C063	BC	1	3
245	B032	C073	BC	1	3
246	B033	C064	BC	2	3
247	B033	C065	BC	2	3
248	B033	C075	BC	1	3
249	B034	C067	BC	1	3
250	B034	C076	BC	1	2
251	B034	C086	BC	1	3
252	B034	C087	BC	1	3
253	B035	C048	BC	1	3
254	B035	C068	BC	1	3
255	B035	C069	BC	1	3
256	B035	C078	BC	2	3
257	B035	C079	BC	1	3
258	B035	C088	BC	1	3
259	B035	C089	BC	1	3
260	B036	C070	BC	1	3
261	B036	C090	BC	1	3
262	B037	C064	BC	1	3
263	B037	C074	BC	1	3
264	B037	C075	BC	1	2
265	B037	C083	BC	1	2
266	B037	C084	BC	2	3

序号	装备 1	装备 2	类别对	冲突使用次对数	频段重叠宽度
267	B038	C065	BC	1	1
268	B038	C076	BC	1	3
269	B038	C085	BC	1	3
270	B038	C086	BC	1	3
271	B039	C067	BC	1	2
272	B039	C068	BC	1	3
273	B039	C078	BC	1	3
274	B039	C088	BC	1	3
275	B040	C049	BC	1	3
276	B040	C069	BC	1	3
277	B040	C070	BC	1	3
278	B040	C090	BC	2	1
279	C005	C025	CC	2	2
280	C019	C029	CC	7	2
281	C021	C051	CC	2	2
282	C022	C032	CC	2	3
283	C023	C033	CC	4	1
284	C025	C055	CC	4	1
285	C027	C057	CC	10	2
286	C028	C058	CC	7	3
287	C031	C051	CC	8	3
288	C033	C043	CC	12	1
289	C037	C077	CC	3	3
290	C038	C048	CC	6	2
291	C039	C049	CC	7	2
292	C043	C053	CC	8	2
293	C050	C070	CC	1	2
294	C061	C071	CC	6	1
295	C067	C077	CC	3	1
296	C069	C079	CC	7	1
297	C076	C086	CC	10	2

A.2 问题 2：被撤销计划与逐计划动作

表 18 问题 2 中被撤销计划的局部环境。“冲突邻居数”为该计划在冲突图中的度，“其中 A/B 类”为这些邻居中高优先级装备的个数，“可用平移动作数”为式 (3) 中该计划的合法平移动作数。多数被撤销的计划被高优先级邻居包围；表中也有邻居数仅为 1 的计划，说明原始度数不是撤销的直接原因，真正起作用的是最后一列——没有任何一个合法单参数动作能使它与存留计划都不冲突。

装备编号	频段	首次时间	原冲突邻居数	其中 A/B 类	合法平移动作数	其中在本方案下无冲突
C010	[4,7)	[8,10)	3	3	24	0
C017	[36,39)	[95,97)	4	4	30	0
C018	[20,23)	[107,109)	3	3	30	0
C019	[12,15)	[86,88)	4	3	30	0
C024	[62,65)	[150,152)	4	4	30	0
C034	[62,65)	[294,296)	2	2	30	0
C035	[57,60)	[272,274)	1	1	30	0
C037	[31,34)	[270,272)	4	3	30	0
C054	[64,67)	[218,220)	5	5	30	0
C055	[52,55)	[213,215)	4	3	30	0
C067	[33,36)	[451,453)	5	4	30	0
C090	[3,6)	[482,484)	3	3	23	0

表 19 问题 2 的逐计划动作清单（只列出被调整或被撤销的计划，与 result2.xlsx 一致）。

装备编号	动作	原频段	新频段	原首次时间	新首次时间
A003	频段平移	[20,30)	[30,40)	[75,80)	[75,80)
A005	时间平移	[60,70)	[60,70)	[125,130)	[129,134)
A006	时间平移	[40,50)	[40,50)	[0,5)	[4,9)
A007	频段平移	[73,83)	[65,75)	[175,180)	[175,180)
A008	频段平移	[15,25)	[5,15)	[211,216)	[211,216)
A009	频段平移	[65,75)	[62,72)	[228,233)	[228,233)
A010	时间平移	[30,40)	[30,40)	[265,270)	[267,272)
A011	时间平移	[60,70)	[60,70)	[273,278)	[272,277)
A012	频段平移	[40,50)	[31,41)	[300,305)	[300,305)
A013	时间平移	[82,92)	[82,92)	[190,195)	[186,191)
A014	频段平移	[2,12)	[4,14)	[348,353)	[348,353)
A015	时间平移	[25,35)	[25,35)	[385,390)	[387,392)
A016	频段平移	[30,40)	[35,45)	[415,420)	[415,420)
A017	频段平移	[50,60)	[52,62)	[432,437)	[432,437)
A018	频段平移	[45,55)	[35,45)	[451,456)	[451,456)
A020	频段平移	[10,20)	[4,14)	[375,380)	[375,380)
B001	时间平移	[80,95)	[80,95)	[15,18)	[20,23)
B002	频段平移	[65,80)	[57,72)	[27,30)	[27,30)
B003	频段平移	[50,65)	[55,70)	[30,33)	[30,33)
B005	频段平移	[5,20)	[0,15)	[57,60)	[57,60)
B006	频段平移	[85,100)	[79,94)	[95,98)	[95,98)
B007	频段平移	[60,75)	[55,70)	[62,65)	[62,65)
B008	频段平移	[40,55)	[31,46)	[85,88)	[85,88)
B009	时间平移	[22,37)	[22,37)	[77,80)	[81,84)

装备编号	动作	原频段	新频段	原首次时间	新首次时间
B010	时间平移	[5,20)	[5,20)	[24,27)	[20,23)
B011	频段平移	[70,85)	[78,93)	[155,158)	[155,158)
B012	时间平移	[55,70)	[55,70)	[167,170)	[169,172)
B013	时间平移	[45,60)	[45,60)	[180,183)	[185,188)
B014	时间平移	[30,45)	[30,45)	[185,188)	[190,193)
B015	频段平移	[15,30)	[25,40)	[197,200)	[197,200)
B016	频段平移	[85,100)	[78,93)	[235,238)	[235,238)
B017	频段平移	[60,75)	[57,72)	[202,205)	[202,205)
B018	频段平移	[40,55)	[49,64)	[225,228)	[225,228)
B019	时间平移	[25,40)	[25,40)	[217,220)	[219,222)
B020	时间平移	[0,15)	[0,15)	[124,127)	[120,123)
B021	频段平移	[80,95)	[78,93)	[295,298)	[295,298)
B022	频段平移	[65,80)	[75,90)	[307,310)	[307,310)
B023	时间平移	[50,65)	[50,65)	[310,313)	[308,311)
B024	时间平移	[30,45)	[30,45)	[305,308)	[308,311)
B025	频段平移	[15,30)	[25,40)	[337,340)	[337,340)
B026	频段平移	[80,95)	[77,92)	[375,378)	[375,378)
B027	频段平移	[65,80)	[55,70)	[342,345)	[342,345)
B028	频段平移	[45,60)	[52,67)	[365,368)	[365,368)
B029	时间平移	[30,45)	[30,45)	[357,360)	[362,365)
B030	频段平移	[15,30)	[25,40)	[384,387)	[384,387)
B031	频段平移	[83,98)	[77,92)	[415,418)	[415,418)
B032	频段平移	[67,82)	[76,91)	[427,430)	[427,430)
B033	时间平移	[52,67)	[52,67)	[430,433)	[425,428)
B035	频段平移	[15,30)	[20,35)	[457,460)	[457,460)
B037	频段平移	[58,73)	[52,67)	[462,465)	[462,465)
B038	频段平移	[40,55)	[30,45)	[485,488)	[485,488)
B039	时间平移	[20,35)	[20,35)	[477,480)	[479,482)
B040	频段平移	[5,20)	[0,15)	[424,427)	[424,427)
C001	频段平移	[90,93)	[95,98)	[40,42)	[40,42)
C002	频段平移	[82,85)	[76,79)	[27,29)	[27,29)
C004	频段平移	[60,63)	[52,55)	[31,33)	[31,33)
C005	频段平移	[55,58)	[52,55)	[33,35)	[33,35)
C006	频段平移	[47,50)	[50,53)	[25,27)	[25,27)
C007	频段平移	[35,38)	[37,40)	[2,4)	[2,4)
C009	频段平移	[13,16)	[18,21)	[31,33)	[31,33)
C010	撤销	[4,7)	-	[8,10)	-
C011	时间平移	[96,99)	[96,99)	[80,82)	[82,84)
C012	频段平移	[83,86)	[75,78)	[65,67)	[65,67)
C013	时间平移	[70,73)	[70,73)	[75,77)	[80,82)
C014	频段平移	[65,68)	[72,75)	[92,94)	[92,94)
C015	频段平移	[50,53)	[49,52)	[110,112)	[110,112)
C016	频段平移	[42,45)	[50,53)	[88,90)	[88,90)
C017	撤销	[36,39)	-	[95,97)	-
C018	撤销	[20,23)	-	[107,109)	-
C019	撤销	[12,15)	-	[86,88)	-
C020	时间平移	[2,5)	[2,5)	[65,67)	[66,68)
C021	频段平移	[95,98)	[97,100)	[125,127)	[125,127)
C022	频段平移	[86,89)	[93,96)	[147,149)	[147,149)

装备编号	动作	原频段	新频段	原首次时间	新首次时间
C023	频段平移	[76,79)	[75,78)	[171,173)	[171,173)
C024	撤销	[62,65)	-	[150,152)	-
C025	频段平移	[54,57)	[49,52)	[132,134)	[132,134)
C026	时间平移	[46,49)	[46,49)	[145,147)	[143,145)
C027	频段平移	[33,36)	[27,30)	[165,167)	[165,167)
C028	频段平移	[24,27)	[19,22)	[168,170)	[168,170)
C029	频段平移	[11,14)	[15,18)	[137,139)	[137,139)
C030	频段平移	[6,9)	[15,18)	[122,124)	[122,124)
C031	时间平移	[94,97)	[94,97)	[265,267)	[268,270)
C032	频段平移	[86,89)	[96,99)	[246,248)	[246,248)
C033	频段平移	[74,77)	[72,75)	[252,254)	[252,254)
C034	撤销	[62,65)	-	[294,296)	-
C035	撤销	[57,60)	-	[272,274)	-
C036	时间平移	[46,49)	[46,49)	[266,268)	[269,271)
C037	撤销	[31,34)	-	[270,272)	-
C038	时间平移	[22,25)	[22,25)	[285,287)	[283,285)
C039	时间平移	[16,19)	[16,19)	[264,266)	[265,267)
C040	频段平移	[5,8)	[1,4)	[242,244)	[242,244)
C041	频段平移	[90,93)	[97,100)	[303,305)	[303,305)
C042	频段平移	[83,86)	[93,96)	[326,328)	[326,328)
C043	时间平移	[72,75)	[72,75)	[253,255)	[250,252)
C044	频段平移	[66,69)	[70,73)	[334,336)	[334,336)
C045	频段平移	[53,56)	[49,52)	[317,319)	[317,319)
C046	频段平移	[42,45)	[45,48)	[327,329)	[327,329)
C047	频段平移	[36,39)	[45,48)	[341,343)	[341,343)
C048	时间平移	[21,24)	[21,24)	[346,348)	[345,347)
C050	频段平移	[5,8)	[1,4)	[306,308)	[306,308)
C051	时间平移	[94,97)	[94,97)	[225,227)	[222,224)
C052	频段平移	[86,89)	[94,97)	[204,206)	[204,206)
C053	频段平移	[71,74)	[75,78)	[213,215)	[213,215)
C054	撤销	[64,67)	-	[218,220)	-
C055	撤销	[52,55)	-	[213,215)	-
C056	时间平移	[41,44)	[41,44)	[202,204)	[201,203)
C057	频段平移	[32,35)	[22,25)	[185,187)	[185,187)
C058	频段平移	[24,27)	[22,25)	[218,220)	[218,220)
C059	时间平移	[16,19)	[16,19)	[201,203)	[200,202)
C060	时间平移	[2,5)	[2,5)	[198,200)	[194,196)
C061	频段平移	[90,93)	[95,98)	[450,452)	[450,452)
C062	频段平移	[86,89)	[92,95)	[420,422)	[420,422)
C064	频段平移	[61,64)	[67,70)	[455,457)	[455,457)
C065	频段平移	[54,57)	[49,52)	[478,480)	[478,480)
C066	时间平移	[45,48)	[45,48)	[442,444)	[439,441)
C067	撤销	[33,36)	-	[451,453)	-
C068	频段平移	[27,30)	[17,20)	[461,463)	[461,463)
C069	时间平移	[15,18)	[15,18)	[441,443)	[443,445)
C070	频段平移	[6,9)	[15,18)	[415,417)	[415,417)
C071	时间平移	[92,95)	[92,95)	[390,392)	[392,394)
C072	频段平移	[86,89)	[96,99)	[386,388)	[386,388)
C073	频段平移	[77,80)	[72,75)	[398,400)	[398,400)

装备编号	动作	原频段	新频段	原首次时间	新首次时间
C074	频段平移	[64,67)	[70,73)	[396,398)	[396,398)
C075	频段平移	[57,60)	[49,52)	[391,393)	[391,393)
C076	频段平移	[43,46)	[45,48)	[384,386)	[384,386)
C077	频段平移	[31,34)	[22,25)	[361,363)	[361,363)
C078	频段平移	[26,29)	[17,20)	[389,391)	[389,391)
C079	频段平移	[17,20)	[14,17)	[391,393)	[391,393)
C080	时间平移	[0,3)	[0,3)	[368,370)	[370,372)
C081	时间平移	[95,98)	[95,98)	[485,487)	[484,486)
C082	频段平移	[82,85)	[91,94)	[507,509)	[507,509)
C085	频段平移	[51,54)	[46,49)	[492,494)	[492,494)
C086	频段平移	[42,45)	[48,51)	[405,407)	[405,407)
C087	频段平移	[37,40)	[45,48)	[525,527)	[525,527)
C088	时间平移	[21,24)	[21,24)	[528,530)	[531,533)
C090	撤销	[3,6)	-	[482,484)	-

A.3 问题 3：新增计划清单

表 20 问题 3 在解释 A 下新增的 109 个 C 类用频计划（与 result3.xlsx 一致；间隔时长与使用次数按假设 9 与附件 C 类相同）。

序号	频段区间	首次时间区间	间隔时长	使用次数
1	[75,78)	[1,3)	8	12
2	[0,3)	[3,5)	8	12
3	[75,78)	[3,5)	8	12
4	[24,27)	[4,6)	8	12
5	[27,30)	[4,6)	8	12
6	[72,75)	[4,6)	8	12
7	[97,100)	[4,6)	8	12
8	[21,24)	[5,7)	8	12
9	[24,27)	[6,8)	8	12
10	[27,30)	[6,8)	8	12
11	[97,100)	[6,8)	8	12
12	[20,23)	[7,9)	8	12
13	[95,98)	[8,10)	8	12
14	[46,49)	[14,16)	8	12
15	[46,49)	[17,19)	8	12
16	[73,76)	[27,29)	8	12
17	[15,18)	[30,32)	8	12
18	[90,93)	[31,33)	8	12
19	[70,73)	[36,38)	8	12
20	[70,73)	[38,40)	8	12
21	[27,30)	[48,50)	8	12
22	[23,26)	[49,51)	8	12
23	[0,3)	[68,70)	8	12
24	[18,21)	[73,75)	8	12
25	[15,18)	[74,76)	8	12
26	[46,49)	[79,81)	8	12

序号	频段区间	首次时间区间	间隔时长	使用次数
27	[46,49)	[81,83)	8	12
28	[94,97)	[84,86)	8	12
29	[27,30)	[90,92)	8	12
30	[21,24)	[91,93)	8	12
31	[24,27)	[91,93)	8	12
32	[97,100)	[127,129)	8	12
33	[0,3)	[130,132)	8	12
34	[27,30)	[133,135)	8	12
35	[24,27)	[143,145)	8	12
36	[75,78)	[149,151)	8	12
37	[52,55)	[151,153)	8	12
38	[52,55)	[153,155)	8	12
39	[97,100)	[159,161)	8	12
40	[94,97)	[160,162)	8	12
41	[21,24)	[163,165)	8	12
42	[19,22)	[166,168)	8	12
43	[72,75)	[185,187)	8	12
44	[75,78)	[185,187)	8	12
45	[75,78)	[187,189)	8	12
46	[72,75)	[188,190)	8	12
47	[18,21)	[192,194)	8	12
48	[46,49)	[195,197)	8	12
49	[97,100)	[201,203)	8	12
50	[19,22)	[210,212)	8	12
51	[22,25)	[220,222)	8	12
52	[4,7)	[222,224)	8	12
53	[7,10)	[222,224)	8	12
54	[12,15)	[222,224)	8	12
55	[5,8)	[224,226)	8	12
56	[8,11)	[224,226)	8	12
57	[13,16)	[224,226)	8	12
58	[41,44)	[244,246)	8	12
59	[15,18)	[257,259)	8	12
60	[0,3)	[258,260)	8	12
61	[97,100)	[288,290)	8	12
62	[93,96)	[300,302)	8	12
63	[73,76)	[316,318)	8	12
64	[14,17)	[319,321)	8	12
65	[49,52)	[319,321)	8	12
66	[93,96)	[324,326)	8	12
67	[97,100)	[331,333)	8	12
68	[19,22)	[337,339)	8	12
69	[22,25)	[337,339)	8	12
70	[22,25)	[339,341)	8	12
71	[17,20)	[341,343)	8	12
72	[0,3)	[362,364)	8	12
73	[70,73)	[370,372)	8	12
74	[73,76)	[370,372)	8	12
75	[70,73)	[372,374)	8	12

序号	频段区间	首次时间区间	间隔时长	使用次数
76	[15,18)	[377,379)	8	12
77	[18,21)	[385,387)	8	12
78	[92,95)	[388,390)	8	12
79	[20,23)	[403,405)	8	12
80	[49,52)	[403,405)	8	12
81	[97,100)	[408,410)	8	12
82	[67,70)	[437,439)	8	12
83	[45,48)	[447,449)	8	12
84	[97,100)	[452,454)	8	12
85	[85,88)	[479,481)	8	12
86	[89,92)	[479,481)	8	12
87	[67,70)	[482,484)	8	12
88	[73,76)	[495,497)	8	12
89	[0,3)	[506,508)	8	12
90	[97,100)	[506,508)	8	12
91	[15,18)	[509,511)	8	12
92	[48,51)	[510,512)	8	12
93	[49,52)	[522,524)	8	12
94	[85,88)	[522,524)	8	12
95	[88,91)	[522,524)	8	12
96	[91,94)	[522,524)	8	12
97	[94,97)	[522,524)	8	12
98	[70,73)	[523,525)	8	12
99	[49,52)	[524,526)	8	12
100	[92,95)	[524,526)	8	12
101	[70,73)	[525,527)	8	12
102	[49,52)	[526,528)	8	12
103	[94,97)	[526,528)	8	12
104	[72,75)	[527,529)	8	12
105	[94,97)	[528,530)	8	12
106	[97,100)	[528,530)	8	12
107	[72,75)	[529,531)	8	12
108	[24,27)	[531,533)	8	12
109	[27,30)	[531,533)	8	12

A.4 问题 4：模型规模、目标方案与逐计划动作

表 21 问题 4 的模型规模与求解设置。

项目	数值
决策变量数（计划-动作对）	6013
时频单元数（含动作扩展）	56092
变量-单元隶属数	594432
单元格 AtMostOne 约束数（去重前 / 去重后）	53679 / 41724
成对蕴含团约束数 / 强制撤销割数	83827 / 0
可能相互冲突的计划对数	2267
每计划最多动作数（A/B/C）	32/32/49
CP-SAT 词典序总用时/s	4308.7
热启动现任解目标向量（MDR-0009）	(0, 0, 12, 16, 37, 71, 750)
词典序求解起点	分离权重解
各级均证明最优	否
HiGHS 交叉验证总用时/s	423.5
HiGHS 证明最优的各级与 CP-SAT 一致	是
分离权重单目标解与词典序向量一致	否

表 22 问题 4 中被撤销计划的局部环境(列的含义同表 18)。

装备编号	频段	首次时间	原冲突邻居数	其中 A/B 类	合法调整动作数	其中在本方案下无冲突
C016	[42,45)	[88,90)	3	3	47	0
C022	[86,89)	[147,149)	4	3	47	0
C025	[54,57)	[132,134)	5	3	47	0
C027	[33,36)	[165,167)	6	5	47	0
C047	[36,39)	[341,343)	2	2	47	0
C053	[71,74)	[213,215)	4	3	47	0
C067	[33,36)	[451,453)	5	4	47	0
C077	[31,34)	[361,363)	5	3	47	0

表 23 四种目标方案在问题 4 上的对照（列的含义同表 8，另给出使用间隔调整动作的计划数）。

目标方案	撤销 A/B/C	调整 A/B/C	调整合计	归一化幅度	间隔调整数	预算/s	用时/s	独立校验
P	0/0/8	17/37/71	125	73.8	25	4340	4308.7	通过
T	2/4/0	12/33/70	115	69.7	10	480	214.8	通过
S	2/4/0	13/33/69	115	69.7	10	240	124.7	通过
W	1/4/7	8/31/64	103	59.5	12	40	48.7	通过

表 24 问题 4 的逐计划动作清单（只列出被调整或被撤销的计划，与 result4.xlsx 一致）。

装备编号	动作	原频段	新频段	原首次时间	新首次时间	原间隔	新间隔
A001	时间平移	[80,90)	[80,90)	[35,40)	[40,45)	60	60
A003	频段平移	[20,30)	[10,20)	[75,80)	[75,80)	60	60
A004	时间平移	[35,45)	[35,45)	[95,100)	[93,98)	60	60
A005	时间平移	[60,70)	[60,70)	[125,130)	[130,135)	60	60
A006	时间平移	[40,50)	[40,50)	[0,5)	[4,9)	60	60
A007	时间平移	[73,83)	[73,83)	[175,180)	[180,185)	60	60
A008	频段平移	[15,25)	[25,35)	[211,216)	[211,216)	60	60
A009	频段平移	[65,75)	[60,70)	[228,233)	[228,233)	60	60
A010	时间平移	[30,40)	[30,40)	[265,270)	[267,272)	60	60
A011	时间平移	[60,70)	[60,70)	[273,278)	[272,277)	60	60
A012	频段平移	[40,50)	[31,41)	[300,305)	[300,305)	60	60
A013	频段平移	[82,92)	[76,86)	[190,195)	[190,195)	60	60
A014	频段平移	[2,12)	[6,16)	[348,353)	[348,353)	60	60
A015	时间平移	[25,35)	[25,35)	[385,390)	[389,394)	60	60
A017	频段平移	[50,60)	[52,62)	[432,437)	[432,437)	60	60
A018	频段平移	[45,55)	[35,45)	[451,456)	[451,456)	60	60
A020	频段平移	[10,20)	[5,15)	[375,380)	[375,380)	60	60
B001	时间平移	[80,95)	[80,95)	[15,18)	[16,19)	40	40
B002	频段平移	[65,80)	[55,70)	[27,30)	[27,30)	40	40

装备编号	动作	原频段	新频段	原首次时间	新首次时间	原间隔	新间隔
B005	时间平移	[5,20)	[5,20)	[57,60)	[59,62)	40	40
B006	时间平移	[85,100)	[85,100)	[95,98)	[99,102)	40	40
B007	频段平移	[60,75)	[50,65)	[62,65)	[62,65)	40	40
B008	频段平移	[40,55)	[35,50)	[85,88)	[85,88)	40	40
B009	频段平移	[22,37)	[30,45)	[77,80)	[77,80)	40	40
B010	时间平移	[5,20)	[5,20)	[24,27)	[21,24)	40	40
B011	频段平移	[70,85)	[79,94)	[155,158)	[155,158)	40	40
B012	时间平移	[55,70)	[55,70)	[167,170)	[171,174)	40	40
B013	时间平移	[45,60)	[45,60)	[180,183)	[176,179)	40	40
B014	时间平移	[30,45)	[30,45)	[185,188)	[186,189)	40	40
B015	频段平移	[15,30)	[25,40)	[197,200)	[197,200)	40	40
B016	时间平移	[85,100)	[85,100)	[235,238)	[231,234)	40	40
B017	频段平移	[60,75)	[55,70)	[202,205)	[202,205)	40	40
B018	频段平移	[40,55)	[50,65)	[225,228)	[225,228)	40	40
B019	时间平移	[25,40)	[25,40)	[217,220)	[219,222)	40	40
B020	频段平移	[0,15)	[5,20)	[124,127)	[124,127)	40	40
B021	频段平移	[80,95)	[75,90)	[295,298)	[295,298)	40	40
B022	频段平移	[65,80)	[75,90)	[307,310)	[307,310)	40	40
B023	时间平移	[50,65)	[50,65)	[310,313)	[308,311)	40	40
B024	时间平移	[30,45)	[30,45)	[305,308)	[308,311)	40	40
B025	频段平移	[15,30)	[25,40)	[337,340)	[337,340)	40	40
B026	频段平移	[80,95)	[75,90)	[375,378)	[375,378)	40	40
B027	频段平移	[65,80)	[55,70)	[342,345)	[342,345)	40	40
B028	频段平移	[45,60)	[51,66)	[365,368)	[365,368)	40	40
B029	时间平移	[30,45)	[30,45)	[357,360)	[362,365)	40	40
B030	频段平移	[15,30)	[25,40)	[384,387)	[384,387)	40	40
B031	频段平移	[83,98)	[75,90)	[415,418)	[415,418)	40	40
B032	频段平移	[67,82)	[75,90)	[427,430)	[427,430)	40	40
B033	时间平移	[52,67)	[52,67)	[430,433)	[425,428)	40	40
B035	频段平移	[15,30)	[6,21)	[457,460)	[457,460)	40	40
B036	时间平移	[0,15)	[0,15)	[495,498)	[490,493)	40	40
B037	频段平移	[58,73)	[52,67)	[462,465)	[462,465)	40	40
B038	频段平移	[40,55)	[30,45)	[485,488)	[485,488)	40	40
B039	时间平移	[20,35)	[20,35)	[477,480)	[473,476)	40	40
B040	时间平移	[5,20)	[5,20)	[424,427)	[428,431)	40	40
C001	间隔调整	[90,93)	[90,93)	[40,42)	[40,42)	8	9
C002	频段平移	[82,85)	[77,80)	[27,29)	[27,29)	8	8
C004	频段平移	[60,63)	[70,73)	[31,33)	[31,33)	8	8
C005	间隔调整	[55,58)	[55,58)	[33,35)	[33,35)	8	9
C006	间隔调整	[47,50)	[47,50)	[25,27)	[25,27)	8	5
C007	时间平移	[35,38)	[35,38)	[2,4)	[1,3)	8	8
C009	频段平移	[13,16)	[23,26)	[31,33)	[31,33)	8	8
C010	频段平移	[4,7)	[2,5)	[8,10)	[8,10)	8	8
C011	间隔调整	[96,99)	[96,99)	[80,82)	[80,82)	8	12
C012	频段平移	[83,86)	[76,79)	[65,67)	[65,67)	8	8
C014	间隔调整	[65,68)	[65,68)	[92,94)	[92,94)	8	10
C015	间隔调整	[50,53)	[50,53)	[110,112)	[110,112)	8	13
C016	撤销	[42,45)	-	[88,90)	-	8	-
C017	频段平移	[36,39)	[27,30)	[95,97)	[95,97)	8	8

装备编号	动作	原频段	新频段	原首次时间	新首次时间	原间隔	新间隔
C018	间隔调整	[20,23)	[20,23)	[107,109)	[107,109)	8	3
C019	频段平移	[12,15)	[2,5)	[86,88)	[86,88)	8	8
C020	时间平移	[2,5)	[2,5)	[65,67)	[64,66)	8	8
C021	间隔调整	[95,98)	[95,98)	[125,127)	[125,127)	8	5
C022	撤销	[86,89)	-	[147,149)	-	8	-
C023	间隔调整	[76,79)	[76,79)	[171,173)	[171,173)	8	14
C024	频段平移	[62,65)	[70,73)	[150,152)	[150,152)	8	8
C025	撤销	[54,57)	-	[132,134)	-	8	-
C026	间隔调整	[46,49)	[46,49)	[145,147)	[145,147)	8	10
C027	撤销	[33,36)	-	[165,167)	-	8	-
C028	频段平移	[24,27)	[22,25)	[168,170)	[168,170)	8	8
C029	间隔调整	[11,14)	[11,14)	[137,139)	[137,139)	8	9
C030	频段平移	[6,9)	[2,5)	[122,124)	[122,124)	8	8
C031	间隔调整	[94,97)	[94,97)	[265,267)	[265,267)	8	5
C032	间隔调整	[86,89)	[86,89)	[246,248)	[246,248)	8	9
C033	频段平移	[74,77)	[70,73)	[252,254)	[252,254)	8	8
C034	频段平移	[62,65)	[70,73)	[294,296)	[294,296)	8	8
C035	频段平移	[57,60)	[47,50)	[272,274)	[272,274)	8	8
C036	时间平移	[46,49)	[46,49)	[266,268)	[269,271)	8	8
C037	频段平移	[31,34)	[22,25)	[270,272)	[270,272)	8	8
C038	时间平移	[22,25)	[22,25)	[285,287)	[283,285)	8	8
C040	频段平移	[5,8)	[2,5)	[242,244)	[242,244)	8	8
C042	频段平移	[83,86)	[90,93)	[326,328)	[326,328)	8	8
C043	时间平移	[72,75)	[72,75)	[253,255)	[256,258)	8	8
C044	间隔调整	[66,69)	[66,69)	[334,336)	[334,336)	8	9
C045	频段平移	[53,56)	[48,51)	[317,319)	[317,319)	8	8
C046	频段平移	[42,45)	[45,48)	[327,329)	[327,329)	8	8
C047	撤销	[36,39)	-	[341,343)	-	8	-
C049	频段平移	[17,20)	[19,22)	[313,315)	[313,315)	8	8
C050	间隔调整	[5,8)	[5,8)	[306,308)	[306,308)	8	11
C051	间隔调整	[94,97)	[94,97)	[225,227)	[225,227)	8	12
C052	间隔调整	[86,89)	[86,89)	[204,206)	[204,206)	8	9
C053	撤销	[71,74)	-	[213,215)	-	8	-
C054	频段平移	[64,67)	[70,73)	[218,220)	[218,220)	8	8
C055	间隔调整	[52,55)	[52,55)	[213,215)	[213,215)	8	13
C056	间隔调整	[41,44)	[41,44)	[202,204)	[202,204)	8	15
C057	频段平移	[32,35)	[22,25)	[185,187)	[185,187)	8	8
C058	频段平移	[24,27)	[19,22)	[218,220)	[218,220)	8	8
C059	间隔调整	[16,19)	[16,19)	[201,203)	[201,203)	8	13
C062	频段平移	[86,89)	[96,99)	[420,422)	[420,422)	8	8
C063	频段平移	[73,76)	[72,75)	[433,435)	[433,435)	8	8
C064	频段平移	[61,64)	[69,72)	[455,457)	[455,457)	8	8
C065	频段平移	[54,57)	[49,52)	[478,480)	[478,480)	8	8
C066	时间平移	[45,48)	[45,48)	[442,444)	[440,442)	8	8
C067	撤销	[33,36)	-	[451,453)	-	8	-
C068	间隔调整	[27,30)	[27,30)	[461,463)	[461,463)	8	14
C069	频段平移	[15,18)	[21,24)	[441,443)	[441,443)	8	8
C070	频段平移	[6,9)	[0,3)	[415,417)	[415,417)	8	8
C071	频段平移	[92,95)	[93,96)	[390,392)	[390,392)	8	8

装备编号	动作	原频段	新频段	原首次时间	新首次时间	原间隔	新间隔
C072	频段平移	[86,89)	[93,96)	[386,388)	[386,388)	8	8
C073	频段平移	[77,80)	[72,75)	[398,400)	[398,400)	8	8
C074	频段平移	[64,67)	[72,75)	[396,398)	[396,398)	8	8
C075	频段平移	[57,60)	[48,51)	[391,393)	[391,393)	8	8
C076	频段平移	[43,46)	[45,48)	[384,386)	[384,386)	8	8
C077	撤销	[31,34)	-	[361,363)	-	8	-
C078	频段平移	[26,29)	[22,25)	[389,391)	[389,391)	8	8
C079	间隔调整	[17,20)	[17,20)	[391,393)	[391,393)	8	3
C081	频段平移	[95,98)	[96,99)	[485,487)	[485,487)	8	8
C082	频段平移	[82,85)	[90,93)	[507,509)	[507,509)	8	8
C085	频段平移	[51,54)	[45,48)	[492,494)	[492,494)	8	8
C086	频段平移	[42,45)	[48,51)	[405,407)	[405,407)	8	8
C087	频段平移	[37,40)	[45,48)	[525,527)	[525,527)	8	8
C088	间隔调整	[21,24)	[21,24)	[528,530)	[528,530)	8	7
C089	间隔调整	[15,18)	[15,18)	[497,499)	[497,499)	8	5
C090	间隔调整	[3,6)	[3,6)	[482,484)	[482,484)	8	12

A.5 模型检验：微型实例穷举对照与规模基准

表 25 24 个随机微型实例上的穷举对照。对每个实例枚举全部动作组合，求出词典序最优向量，与本文的 CP-SAT 求解器结果逐一对比；最大枚举组合数为 229376。

实例	计划	频段	冲突对	组合数	穷举最优向量	撤销非零	CP-SAT 向量	一致	用时/s
1	4	6	2	2688	(0, 0, 0, 0, 0, 2, 3)	0	(0, 0, 0, 0, 0, 2, 3)	是	0.01
2	5	7	4	16464	(0, 0, 1, 0, 0, 1, 1)	1	(0, 0, 1, 0, 0, 1, 1)	是	0.01
3	6	8	2	163840	(0, 0, 0, 1, 0, 0, 1)	0	(0, 0, 0, 1, 0, 0, 1)	是	0.18
4	4	9	1	4096	(0, 0, 0, 0, 0, 1, 2)	0	(0, 0, 0, 0, 0, 1, 2)	是	0.01
5	5	6	8	13440	(0, 0, 2, 1, 0, 1, 3)	2	(0, 0, 2, 1, 0, 1, 3)	是	0.00
6	6	7	5	172032	(0, 0, 0, 0, 0, 3, 5)	0	(0, 0, 0, 0, 0, 3, 5)	是	0.05
7	4	8	1	2304	(0, 0, 0, 0, 0, 1, 2)	0	(0, 0, 0, 0, 0, 1, 2)	是	0.01
8	5	9	4	28672	(0, 0, 0, 1, 0, 2, 4)	0	(0, 0, 0, 1, 0, 2, 4)	是	0.03
9	6	6	9	175616	(0, 0, 1, 0, 2, 2, 8)	1	(0, 0, 1, 0, 2, 2, 8)	是	0.02
10	4	7	5	3072	(0, 0, 0, 1, 2, 1, 6)	0	(0, 0, 0, 1, 2, 1, 6)	是	0.00
11	5	8	3	18432	(0, 0, 0, 2, 0, 0, 4)	0	(0, 0, 0, 2, 0, 0, 4)	是	0.01
12	6	9	6	196608	(0, 0, 0, 3, 3, 0, 11)	0	(0, 0, 0, 3, 3, 0, 11)	是	0.03
13	4	6	2	2240	(0, 0, 0, 0, 1, 0, 2)	0	(0, 0, 0, 0, 1, 0, 2)	是	0.00
14	5	7	3	21952	(0, 0, 0, 3, 0, 0, 3)	0	(0, 0, 0, 3, 0, 0, 3)	是	0.01
15	6	8	6	172032	(0, 0, 1, 1, 2, 0, 4)	1	(0, 0, 1, 1, 2, 0, 4)	是	0.03
16	4	9	0	3072	(0, 0, 0, 0, 0, 0, 0)	0	(0, 0, 0, 0, 0, 0, 0)	是	0.01
17	5	6	6	18816	(0, 0, 0, 1, 3, 1, 8)	0	(0, 0, 0, 1, 3, 1, 8)	是	0.01
18	6	7	8	107520	(0, 1, 1, 1, 0, 1, 3)	2	(0, 1, 1, 1, 0, 1, 3)	是	0.01
19	4	8	3	2016	(0, 0, 0, 2, 0, 2, 7)	0	(0, 0, 0, 2, 0, 2, 7)	是	0.00
20	5	9	3	24576	(0, 0, 0, 0, 2, 1, 5)	0	(0, 0, 0, 0, 2, 1, 5)	是	0.02
21	6	6	7	80640	(0, 1, 0, 4, 1, 0, 10)	1	(0, 1, 0, 4, 1, 0, 10)	是	0.02
22	4	7	5	3584	(0, 0, 0, 1, 2, 0, 4)	0	(0, 0, 0, 1, 2, 0, 4)	是	0.00
23	5	8	7	24576	(0, 1, 0, 3, 0, 1, 6)	1	(0, 1, 0, 3, 0, 1, 6)	是	0.01
24	6	9	5	229376	(0, 0, 0, 0, 1, 2, 6)	0	(0, 0, 0, 0, 1, 2, 6)	是	0.05

表 27 合成实例上的规模基准 ($n = 150, \dots, 600$, 时间范围随 n 等比放大以保持密度)。CP-SAT 一列采用固定 120 s 时限, 故其状态只说明 “在该预算内找到了可行解”, 不代表可证明最优。

n	时间范围	冲突对	成对检测/s (中位数)	扫描线/s (中位数)	变量数	约束数	建模/s	CP-SAT 状态	求解/s	撤销合计	调整合计
150	651	329	0.01	0.004	4582	32154	0.3	FEASIBLE	121.2	20	116
300	1191	717	0.04	0.008	9276	64600	0.6	FEASIBLE	122.4	72	192
600	2252	1423	0.17	0.018	18485	132121	1.5	FEASIBLE	124.6	171	369

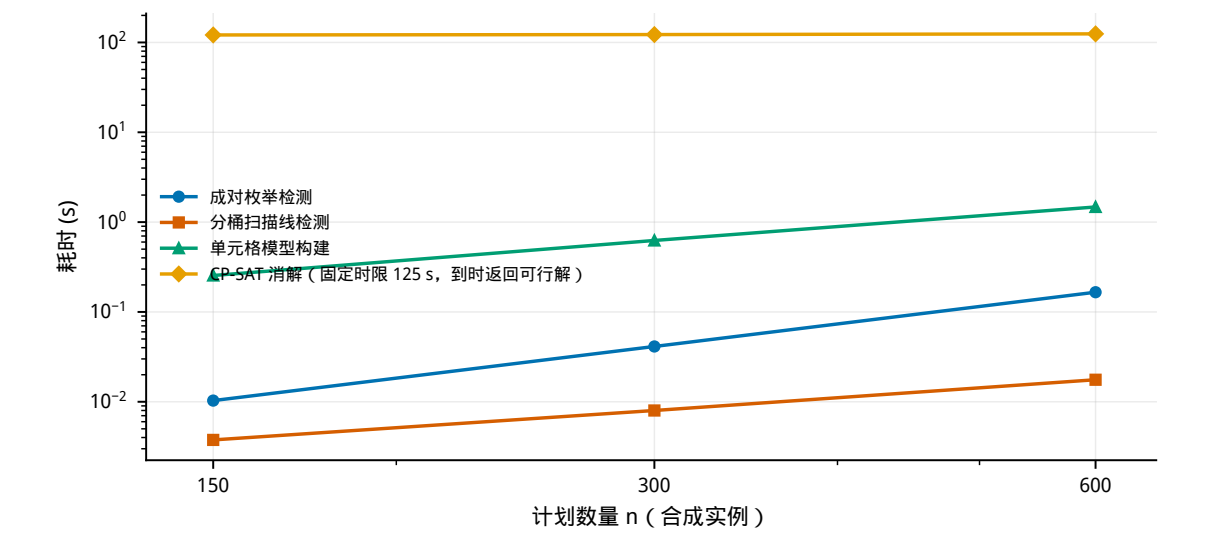


图 11 算法的规模基准。合成实例 $n = 150, \dots, 600$ 下两种检测算法与模型构建的用时, 以及 CP-SAT 在固定 120 s 时限下的求解时间 (纵轴为对数刻度)。检测算法始终为亚秒级, 求解时间则受时限截断。

A.6 最优性证据的两张补充表

表 28 问题 2 的条件最优性 (MDR-0012): 把被撤销计划的集合固定为表 5 所报告的那 12 个 C 类计划之后, 对调整三级与幅度级逐级最小化的结果。此处的 “已证明下界” 与状态 OPTIMAL 只对该受限问题成立, 不构成对词典序真优解相应分量的界。

目标层级	值	已证明下界	状态	用时/s
adjustA	16	16.00	OPTIMAL	3.1
adjustB	37	37.00	OPTIMAL	3.1
adjustC	71	71.00	OPTIMAL	3.3
amplitude	750	750.00	OPTIMAL	4.9

表 29 问题 3 的答案对 “采用哪一个问题 2 方案” 的依赖。四个方案均为无冲突方案且均通过独立校验; 在每个方案的存留计划布局之上求解同一个模型 (12)。各方案的时间范围可能不同, 故单列给出, 不同布局的自由单元数不可直接相减。

问题 2 方案	撤销数	存留计划数	时间范围/ Δt	自由单元数	可再安排 C 类数	上界	CP-SAT 状态	独立校验
P	12	138	643	48484	109	109	OPTIMAL	通过
S	7	143	643	48712	146	146	OPTIMAL	通过
T	7	143	643	48712	143	143	OPTIMAL	通过
W	12	138	643	49348	143	143	OPTIMAL	通过

B 支撑材料文件清单

下表列出支撑材料压缩包内的全部文件（相对路径）及其用途。压缩包内另含 MANIFEST.json，记录每个文件的 SHA-256。

文件	说明
AI_USAGE.md	AI 工具使用记录（源文件）
AI 工具使用详情.pdf	AI 工具使用详情
MANIFEST.json	支撑材料成员清单与 SHA-256
Makefile	本地快捷命令（只提交云端任务）
README.md	仓库说明与复现入口
REPRODUCE.md	复现说明
UNLICENSE	公有领域声明（Unlicense）
docs/ARCHITECTURE.md	仓库架构与唯一计算面的设计
docs/CONTRIBUTING.md	协作与提交规范
docs/DATA_NOTES.md	数据口径与附件解析约定
docs/ENGINEERING_STANDARD.md	工程标准与门禁定义
docs/MODELING_STANDARD.md	建模与写作标准（审稿清单）
docs/RESULTS.md	本轮结果总览与逐项证据
docs/REVIEW_RESPONSE.md	评审意见的逐条处理说明
docs/RUNBOOK.md	运行手册（完整命令序列与实测用时）
docs/adr/0001-cloud-only-execution.md	工程规范、建模标准、决策记录
docs/adr/0002-stage-run-manifest-model.md	工程规范、建模标准、决策记录
docs/adr/0003-independent-repos-vendored-runtime.md	工程规范、建模标准、决策记录
docs/mdr/0000-template.md	建模决策记录（题意解释、假设、目标与方法的选择理由）
docs/mdr/0001-conflict-semantics.md	建模决策记录（题意解释、假设、目标与方法的选择理由）
docs/mdr/0002-resolution-options.md	建模决策记录（题意解释、假设、目标与方法的选择理由）
docs/mdr/0003-lexicographic-objective.md	建模决策记录（题意解释、假设、目标与方法的选择理由）

文件	说明
docs/mdr/0004-solver-and-formulation.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0005-q3-packing-interpretation.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0006-q4-interval-adjustment.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0007-verification-design.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0008-warm-starts-and-time-limits.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0009-incumbent-preserving-lexicographic-search.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0010-q3-unlimited-shift-interpretation.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0011-q4-horizon-cap.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0012-conditional-optimality.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0013-scheme-comparison-fairness.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0014-q3-layout-dependence.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/0015-hardness-and-shift-semantics.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/mdr/README.md	建模决策记录(题意解释、假设、目标与方法的选择理由)
docs/problem_brief.md	题目要点与交付清单
forge/__init__.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/config.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/context.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/contracts.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/events.py	云端运行时(阶段执行、清单、契约、排版、打包)
forge/hashing.py	云端运行时(阶段执行、清单、契约、排版、打包)

文件	说明
<code>forge/manifest.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/modal_app.py</code>	云端应用定义（镜像、卷、四种规格的执行函数）
<code>forge/packaging.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/pdfqa.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/plotting.py</code>	绘图规范（矢量 PDF + 预览 PNG、色板、字体）
<code>forge/runner.py</code>	阶段注册与执行（依赖、清单、错误捕获）
<code>forge/tex.py</code>	云端运行时（阶段执行、清单、契约、排版、打包）
<code>forge/xlsx.py</code>	从组委会模板生成结果工作簿
<code>manuscript/ai_usage.tex</code>	论文 LaTeX 源码
<code>manuscript/main.tex</code>	论文 LaTeX 源码
<code>manuscript/references.bib</code>	论文 LaTeX 源码
<code>manuscript/sections/00_abstract.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/01_restatement.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/02_analysis.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/03_assumptions.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/04_models.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/08_validation.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/09_evaluation.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/10_ai_statement.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/ai_usage_body.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/appendix_repro.tex</code>	论文 LaTeX 源码
<code>manuscript/sections/appendix_tables.tex</code>	论文 LaTeX 源码
<code>pipelines/__init__.py</code>	本题建模与求解程序
<code>pipelines/common/__init__.py</code>	通用阶段：数据接入、门禁、论文、质量检查、打包发布
<code>pipelines/common/stages.py</code>	通用阶段：数据接入、门禁、论文、质量检查、打包发布
<code>pipelines/common/validation.py</code>	通用阶段：数据接入、门禁、论文、质量检查、打包发布
<code>pipelines/d/__init__.py</code>	本题建模与求解程序

文件	说明
pipelines/d/contracts.py	本题建模与求解程序
pipelines/d/detect.py	问题 1: 成对检测与按频段分桶的扫描线检测
pipelines/d/highs_util.py	本题建模与求解程序
pipelines/d/pack.py	问题 3: 候选放置、集合装填模型与重排构造
pipelines/d/plans.py	本题建模与求解程序
pipelines/d/report.py	论文图与表的生成
pipelines/d/resolve.py	问题 2/4: 动作集合、单元格模型、词典序与加权求解
pipelines/d/stages.py	本题各阶段的编排
pipelines/d/synth.py	微型/合成实例与穷举参照解
pipelines/d/validators.py	与求解器分离的独立校验器(按定义展开单元集合求交)
pyproject.toml	项目元数据与静态检查配置
runs/20260912-212914-2b9c081/bench/events.jsonl	结构化运行日志
runs/20260912-212914-2b9c081/bench/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-212914-2b9c081/bench/numbers.json	该阶段登记的论文数字宏
runs/20260912-212914-2b9c081/bench/scaling.json	阶段数据产物
runs/20260912-212914-2b9c081/bench/tiny.json	阶段数据产物
runs/20260912-212914-2b9c081/coldstart/alternative_decisions.json	阶段数据产物
runs/20260912-212914-2b9c081/coldstart/alternatives.json	阶段数据产物
runs/20260912-212914-2b9c081/coldstart/decisions.json	阶段数据产物
runs/20260912-212914-2b9c081/coldstart/events.jsonl	结构化运行日志
runs/20260912-212914-2b9c081/coldstart/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-212914-2b9c081/coldstart/numbers.json	该阶段登记的论文数字宏
runs/20260912-212914-2b9c081/coldstart/resolve_d_plans.json	阶段数据产物

文件	说明
runs/20260912-212914-2b9c081/coldstart/solver_report.json	求解器逐级证据（值、证明下界、状态、用时）
runs/20260912-212914-2b9c081/coldstart/validation_report.json	该阶段结果的独立校验报告
runs/20260912-212914-2b9c081/compare/comparison.json	阶段数据产物
runs/20260912-212914-2b9c081/compare/events.jsonl	结构化运行日志
runs/20260912-212914-2b9c081/compare/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-212914-2b9c081/compare/numbers.json	该阶段登记的论文数字宏
runs/20260912-212914-2b9c081/detect/conflicts.json	阶段数据产物
runs/20260912-212914-2b9c081/detect/events.jsonl	结构化运行日志
runs/20260912-212914-2b9c081/detect/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-212914-2b9c081/detect/numbers.json	该阶段登记的论文数字宏
runs/20260912-212914-2b9c081/detect/plans.json	阶段数据产物
runs/20260912-212914-2b9c081/detect/stats.json	阶段数据产物
runs/20260912-212914-2b9c081/detect/validation_report.json	该阶段结果的独立校验报告
runs/20260912-212914-2b9c081/figures/events.jsonl	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_bench_scaling.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_bench_scaling.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q1_gantt.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q1_gantt.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q1_structure.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q1_structure.png	论文图（矢量 PDF 与预览 PNG）

文件	说明
runs/20260912-212914-2b9c081/figures/figures/fig_q2_resolution.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q2_resolution.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q2_schemes.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q2_schemes.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q3_layout.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q3_layout.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q3_repack.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q3_repack.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q4_comparison.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q4_comparison.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q4_resolution.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_q4_resolution.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_sensitivity.pdf	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/figures/fig_sensitivity.png	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/manifest.json	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/figures/numbers.json	论文图（矢量 PDF 与预览 PNG）
runs/20260912-212914-2b9c081/ingest/data/附件 1__Sheet1.parquet	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-212914-2b9c081/ingest/events.jsonl	原始数据接入结果（散列、题目文本、规范化数据）
runs/20260912-212914-2b9c081/ingest/inventory.json	原始数据接入结果（散列、题目文本、规范化数据）

文件	说明
runs/20260912-212914-2b9c081/ingest/manifest.json	原始数据接入结果(散列、题目文本、规范化数据)
runs/20260912-212914-2b9c081/ingest/problem_text.txt	原始数据接入结果(散列、题目文本、规范化数据)
runs/20260912-212914-2b9c081/lint/events.jsonl	静态检查报告
runs/20260912-212914-2b9c081/lint/lint_report.json	静态检查报告
runs/20260912-212914-2b9c081/lint/manifest.json	静态检查报告
runs/20260912-212914-2b9c081/lint/mypy.txt	静态检查报告
runs/20260912-212914-2b9c081/lint/ruff_check.txt	静态检查报告
runs/20260912-212914-2b9c081/lint/ruff_format.txt	静态检查报告
runs/20260912-212914-2b9c081/pack/events.jsonl	结构化运行日志
runs/20260912-212914-2b9c081/pack/layouts.json	阶段数据产物
runs/20260912-212914-2b9c081/pack/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-212914-2b9c081/pack/new_plans.json	阶段数据产物
runs/20260912-212914-2b9c081/pack/numbers.json	该阶段登记的论文数字宏
runs/20260912-212914-2b9c081/pack/pack_report.json	阶段数据产物
runs/20260912-212914-2b9c081/pack/repack_plans.json	阶段数据产物
runs/20260912-212914-2b9c081/pack/validation_report.json	该阶段结果的独立校验报告
runs/20260912-212914-2b9c081/qa/events.jsonl	论文与结果文件质量门禁报告
runs/20260912-212914-2b9c081/qa/manifest.json	论文与结果文件质量门禁报告
runs/20260912-212914-2b9c081/qa/qa_report.json	论文与结果文件质量门禁报告
runs/20260912-212914-2b9c081/resolve/alternative_decisions.json	阶段数据产物
runs/20260912-212914-2b9c081/resolve/alternatives.json	阶段数据产物
runs/20260912-212914-2b9c081/resolve/decisions.json	阶段数据产物

文件	说明
runs/20260912-212914-2b9c081/resolve/events.js onl	结构化运行日志
runs/20260912-212914-2b9c081/resolve/manifest. json	阶段清单(输入输出散列、环境、参数)
runs/20260912-212914-2b9c081/resolve/numbers.j son	该阶段登记的论文数字宏
runs/20260912-212914-2b9c081/resolve/resolved_ plans.json	阶段数据产物
runs/20260912-212914-2b9c081/resolve/solver_re port.json	求解器逐级证据(值、证明下界、状态、用时)
runs/20260912-212914-2b9c081/resolve/validatio n_report.json	该阶段结果的独立校验报告
runs/20260912-212914-2b9c081/resolve_interval/ alternative_decisions.json	阶段数据产物
runs/20260912-212914-2b9c081/resolve_interval/ alternatives.json	阶段数据产物
runs/20260912-212914-2b9c081/resolve_interval/ decisions.json	阶段数据产物
runs/20260912-212914-2b9c081/resolve_interval/ events.jsonl	结构化运行日志
runs/20260912-212914-2b9c081/resolve_interval/ manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-212914-2b9c081/resolve_interval/ numbers.json	该阶段登记的论文数字宏
runs/20260912-212914-2b9c081/resolve_interval/ resolved_plans.json	阶段数据产物
runs/20260912-212914-2b9c081/resolve_interval/ solver_report.json	求解器逐级证据(值、证明下界、状态、用时)
runs/20260912-212914-2b9c081/resolve_interval/ validation_report.json	该阶段结果的独立校验报告
runs/20260912-212914-2b9c081/resolve_interval/ variant_uncapped.json	阶段数据产物
runs/20260912-212914-2b9c081/results/events.js onl	结果工作簿与求解输出
runs/20260912-212914-2b9c081/results/manifest. json	结果工作簿与求解输出
runs/20260912-212914-2b9c081/results/numbers.j son	结果工作簿与求解输出

文件	说明
runs/20260912-212914-2b9c081/results/result1.xlsx	结果工作簿与求解输出
runs/20260912-212914-2b9c081/results/result2.xlsx	结果工作簿与求解输出
runs/20260912-212914-2b9c081/results/result3.xlsx	结果工作簿与求解输出
runs/20260912-212914-2b9c081/results/result4.xlsx	结果工作簿与求解输出
runs/20260912-212914-2b9c081/results/results_report.json	结果工作簿与求解输出
runs/20260912-212914-2b9c081/sensitivity/cases.json	阶段数据产物
runs/20260912-212914-2b9c081/sensitivity/event_s.jsonl	结构化运行日志
runs/20260912-212914-2b9c081/sensitivity/manifest.json	阶段清单(输入输出散列、环境、参数)
runs/20260912-212914-2b9c081/sensitivity/numbers.json	该阶段登记的论文数字宏
runs/20260912-212914-2b9c081/tables/events.jsonl	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/manifest.json	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/numbers.json	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_bench_scaling.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_bench_scaling.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_bench_tiny.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_bench_tiny.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_data.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_data.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q1_category.csv	论文表(LaTeX 片段与 CSV)

文件	说明
runs/20260912-212914-2b9c081/tables/tables/tab_q1_category.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q1_conflicts.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q1_conflicts.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q1_multiplicity.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q1_multiplicity.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q1_summary.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q1_summary.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q1_top_degree.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q1_top_degree.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_adjustments.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_adjustments.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_cancelled.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_cancelled.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_conditional.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_conditional.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_model.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_model.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_schemes.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_schemes.tex	论文表(LaTeX 片段与 CSV)

文件	说明
runs/20260912-212914-2b9c081/tables/tables/tab_q2_solver.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_solver.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_table1.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q2_table1.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q3_layouts.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q3_layouts.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q3_plans.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q3_plans.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q3_summary.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q3_summary.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_adjustments.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_adjustments.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_cancelled.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_cancelled.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_compare.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_compare.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_conditional.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_conditional.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_model.csv	论文表(LaTeX 片段与 CSV)

文件	说明
runs/20260912-212914-2b9c081/tables/tables/tab_q4_model.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_schemes.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_schemes.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_solver.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_solver.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_table1.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_table1.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_variant.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_q4_variant.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_sensitivity.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_sensitivity.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_validation.csv	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/tables/tables/tab_validation.tex	论文表(LaTeX 片段与 CSV)
runs/20260912-212914-2b9c081/test/coverage.json	测试报告与覆盖率
runs/20260912-212914-2b9c081/test/events.jsonl	测试报告与覆盖率
runs/20260912-212914-2b9c081/test/junit.xml	测试报告与覆盖率
runs/20260912-212914-2b9c081/test/manifest.json	测试报告与覆盖率
runs/20260912-212914-2b9c081/test/pytest.txt	测试报告与覆盖率
runs/20260912-212914-2b9c081/test/test_report.json	测试报告与覆盖率
runs/20260912-212914-2b9c081/validate/data_profile.json	数据契约校验报告与数据画像
runs/20260912-212914-2b9c081/validate/events.jsonl	数据契约校验报告与数据画像

文件	说明
runs/20260912-212914-2b9c081/validate/manifest.json	数据契约校验报告与数据画像
runs/20260912-212914-2b9c081/validate/validation_report.json	数据契约校验报告与数据画像
tests/__init__.py	单元 / 性质 / 集成测试
tests/conftest.py	单元 / 性质 / 集成测试
tests/integration/__init__.py	单元 / 性质 / 集成测试
tests/integration/test_ingest_validate.py	单元 / 性质 / 集成测试
tests/unit/__init__.py	单元 / 性质 / 集成测试
tests/unit/test_config.py	单元 / 性质 / 集成测试
tests/unit/test_contracts.py	单元 / 性质 / 集成测试
tests/unit/test_d_hints.py	单元 / 性质 / 集成测试
tests/unit/test_d_pack.py	单元 / 性质 / 集成测试
tests/unit/test_d_plans_detect.py	单元 / 性质 / 集成测试
tests/unit/test_d_report.py	单元 / 性质 / 集成测试
tests/unit/test_d_resolve.py	单元 / 性质 / 集成测试
tests/unit/test_hashing.py	单元 / 性质 / 集成测试
tests/unit/test_manifest.py	单元 / 性质 / 集成测试
tests/unit/test_packaging_tex.py	单元 / 性质 / 集成测试
tests/unit/test_runner.py	单元 / 性质 / 集成测试
tests/unit/test_xlsx.py	单元 / 性质 / 集成测试
tools/__init__.py	本地控制面：提交云端任务、下载、校验散列
tools/cli.py	本地控制面：提交云端任务、下载、校验散列
tools/run_local.py	不依赖云端的本地驱动脚本

C 复现记录

论文中的全部数值、表格与图形均由同一次云端运行（run 20260912-212914-2b9c081, 代码版本 4d895244bfdd06c85e8fec274d57eb331d5b4728, 构建日期 2026-09-12）生成。各阶段状态与输出摘要如下。

阶段	状态	用时/s	输出摘要 (SHA-256 前 12 位)
bench	completed	374.0	d8dd8de945aa
coldstart	completed	2686.0	80937018244e
compare	completed	1.0	5948163a2fca
detect	completed	1.0	f45963852b4a

阶段	状态	用时/s	输出摘要 (SHA-256 前 12 位)
figures	completed	17.0	7863b6bd623b
fmt	completed	1.0	8fb31c07c424
ingest	completed	0.0	1c6f1c5a038c
lint	completed	20.0	a9c8365db560
pack	completed	109.0	796e88a956d0
paper	running	0.0	
qa	failed	1.0	
resolve	completed	2550.0	d4899aa501ac
resolve_interval	completed	7807.0	a7aa2df9db2c
results	completed	2.0	6f4a2dc4aedd
sensitivity	completed	1003.0	6f95b8983a55
tables	completed	2.0	4ad5e01d9277
test	completed	15.0	6bc4ffc701d1
validate	completed	0.0	4dad00dd8173

C.1 复现步骤

本仓库的设计原则是“唯一计算面”：一切数据解析、数值计算、优化求解、测试、绘图、 $\text{\LaTeX}$ 排版与打包都在云端 (Modal) 执行，本地只负责编辑文本、提交任务、下载产物并核对 SHA-256。**关于上表的时序说明。**该表由 paper 阶段在编译本 PDF 之前生成，因此它记录的是那一刻的运行状态：paper 一行显示为 running（其自身状态尚未写入），而 qa、package、release 三个阶段按依赖顺序排在 paper 之后，其行要么缺失、要么是前一次编译循环留下的状态与输出摘要。这三行不应被用作审计线索：交付物的权威记录是发布目录中的 release_manifest.json（逐文件 SHA-256）与支撑材料内的 REPRODUCE.md、MANIFEST.json，它们都在 paper 之后生成。复现步骤如下：

1. 安装 Modal 客户端并完成鉴权后，在仓库根目录执行 `python3 tools/cli.py provision` 验证云端工具链 ($\text{\TeX}$ Live、中文字体、CP-SAT、HiGHS)。
2. 数据接入与契约校验: `python3 tools/cli.py run ingest,validate,detect --new-run --size small`。
3. 两个重阶段分离运行 (各约 40 分钟): `run resolve --size large --spawn` 与 `run resolve_interval --size large --spawn`，随后 `wait <ID>` 或轮询 `status`。
4. 其余科学阶段: `run pack,bench,sensitivity --size large`，再 `run compare,results,figures,tables`。
5. 门禁与论文: `run lint,test` 与 `run paper,qa,package,release`。
6. 下载与核对: `download --stage <name>` 会逐文件核对 SHA-256; `release --version <tag>` 生成交付目录与发布清单。

7. **不使用云端账号的复现路径**：支撑材料根目录的 `tools/run_local.py` 调用同一批阶段函数，只把云端卷换成本地目录。安装 `pandas numpy openpyxl pyarrow networkx matplotlib ortools highspy` 后执行 `python3 tools/run_local.py --quick`，数分钟内得到四个结果工作簿与全部图表；去掉 `--quick` 则使用与本文相同量级的求解预算。排版阶段需另行安装 `TEX Live` 与中文字体。

各阶段的确切参数（求解器线程数、逐级时限）记录在仓库的运行手册中，也记录在每个阶段的 `manifest.json` 内。

C.2 可复现性的保证

- **固定输入**：题目 PDF 与附件在仓库中原样保存，`ingest` 阶段记录其 SHA-256，任何代码不得改写。
- **固定环境**：容器镜像的依赖版本固定；每个阶段的 `manifest.json` 记录 Python 版本、平台、CPU 数与全部包版本。
- **固定随机源**：所有随机性经统一的种子接口派生，求解器的随机种子固定。
- **固定热启动**：词典序求解的现任解提示保存在仓库的 `configs/hints/` 中（附带来源 `run`、代码版本与目标向量）。由命题 5.4，带提示重跑所得向量在词典序上不会劣于提示，因此本文结果可由仓库直接复现。提示只影响搜索起点，不改变可行域，也不改变任何一级的最优值。
- **数字的单一来源**：论文正文不允许手工键入结果数字；每个数值或来自阶段登记的宏，或来自 `tables` 阶段生成的表格文件。若两个阶段登记了同名而不同值的数字，论文构建会报告冲突并由质检门禁拦截。

C.3 已知的不确定性

150 个计划的完整模型采用多线程求解，单次求解耗时数十分钟，本文未做重复求解以验证位对位的确定性（记为“未检验”）；微型实例上的确定性检验为 24/24 通过。需要强调：本文已证明最优的结论（定理 5.5、6.2）由求解器的证明下界保证，与是否复现出同一个最优解无关；未证明最优的层级则本就以区间形式报告。

D 完整程序

以下按文件收录建模、求解、检验、绘图、排版与打包的全部源程序，文件名与附录 B 一致。运行方式见附录 C。

Makefile

```
1 # Thin wrappers over tools/cli.py. Nothing here computes locally.
2 PY := python3
3 CLI := $(PY) tools/cli.py
4 STAGE := ingest
5 SIZE := small
6 RUN :=
7 RUNFLAG := $(if $(RUN),--run-id $(RUN),)
8 FORCEFLAG := $(if $(FORCE),--force,)
```

```

9
10 .PHONY: fmt help provision run pipeline exec status logs runs download release lint test paper qa package smoke
11
12 help:
13     @echo "make provision          # verify cloud toolchain"
14     @echo "make run STAGE=q1 SIZE=medium # run one stage (RUN=<id> to continue a run, FORCE=1 to rerun)"
15     @echo "make pipeline          # ingest,validate,lint,test in one container"
16     @echo "make status | logs STAGE=q1 | runs | download STAGE=results"
17     @echo "make paper | qa | package | release VERSION=v1.0.0"
18
19 provision:
20     $(CLI) provision
21
22 run:
23     $(CLI) run $(STAGE) $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
24
25 pipeline:
26     $(CLI) run ingest,validate,lint,test $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
27
28 exec:
29     $(CLI) exec --code '$(CODE)' $(RUNFLAG)
30
31 status:
32     $(CLI) status $(RUNFLAG)
33
34 logs:
35     $(CLI) logs $(STAGE) $(RUNFLAG)
36
37 runs:
38     $(CLI) runs
39
40 download:
41     $(CLI) download $(RUNFLAG) $(if $(STAGE),--stage $(STAGE),)
42
43 lint:
44     $(CLI) run lint $(RUNFLAG) $(FORCEFLAG)
45
46 test:
47     $(CLI) run test $(RUNFLAG) $(FORCEFLAG)
48
49 paper:
50     $(CLI) run paper $(RUNFLAG) --size $(SIZE) $(FORCEFLAG)
51
52 qa:
53     $(CLI) run qa $(RUNFLAG) $(FORCEFLAG)
54
55 package:
56     $(CLI) run package $(RUNFLAG) $(FORCEFLAG)
57
58 release:
59     $(CLI) run release $(RUNFLAG) $(FORCEFLAG) && $(CLI) release --version $(VERSION) $(RUNFLAG) --force
60
61 smoke:
62     $(CLI) run ingest,validate,lint,test,paper,qa,package,release --new-run --param require_results=false --param
        ↪ allow_missing_results=true
63
64 fmt:
65     $(CLI) fmt $(RUNFLAG)

```

configs/default.toml

```

1 # Forge 运行时配置（默认档）。Stage 代码通过 ctx.cfg("a.b.c") 读取；
2 # `--profile <name>` 会叠加 configs/<name>.toml; `--param config_overrides='{...}'` 可临时覆盖。
3
4 [run]
5 seed = 20260912
6
7 [gates]

```

```

8 lint_required = ["ruff_check", "ruff_format", "mypy"]
9 mypy_paths = ["forge"]
10
11 [paper]
12 main = "main.tex"
13 # 需在同一 run 中已完成的 stage; 其 tables/*.tex 与 figures/*.pdf/png 会被并入排版
14 sources = ["figures", "tables"]
15 max_body_pages = 30
16 max_overfull_pt = 20.0
17 code_globs = [
18     "forge/**/*.py", "pipelines/**/*.py", "tools/**/*.py", "tests/**/*.py",
19     "configs/*.toml", "configs/*.json", "Makefile", "pyproject.toml",
20 ]
21
22 [qa]
23 required_stages = ["lint", "test"]
24 results_stage = "results"
25 require_results = true
26 identity_denylist = [] # 硬门禁（正则）：出现即失败
27 identity_watchlist = ["大学", "学院", "赛区", "队号", "指导教师", "[\\w.-]+@[\\w.-]+\\.\\w+"] # 软提示：写入报告供复核
28
29 [package]
30 stages = ["ingest", "validate", "detect", "resolve", "coldstart", "pack", "resolve_interval", "compare", "bench", "
    ↳ sensitivity", "results", "figures", "tables", "lint", "test", "qa"]
31 code_globs = [
32     "forge/**/*.py", "pipelines/**/*.py", "tools/**/*.py", "tests/**/*.py", "configs/**",
33     "manuscript/**/*.tex", "manuscript/**/*.bib", "docs/**/*.md", "Makefile", "pyproject.toml",
34     "README.md", "UNLICENSE", "AI_USAGE.md",
35 ]
36 max_bytes = 20971520
37
38 [release]
39 paper_name = "论文.pdf"

```

configs/project.json

```

1 {
2     "letter": "D",
3     "package": "d",
4     "slug": "spectrum-conflict",
5     "title": "时频冲突检测与消解",
6     "app": "cumcm2026-d-spectrum-conflict",
7     "volume": "cumcm2026-d-artifacts",
8     "problem_pdf": "D题.pdf",
9     "attachments_dir": "附件",
10    "templates_dir": "附件/附件2",
11    "result_files": [
12        "result1.xlsx",
13        "result2.xlsx",
14        "result3.xlsx",
15        "result4.xlsx"
16    ],
17    "github_repo": "cumcm2026-d-spectrum-conflict"
18 }

```

forge/__init__.py

```

1 """Forge — cloud-only research engineering runtime (CUMCM 2026).
2
3 Every computation (data parsing, modelling, solving, tests, figures, LaTeX, PDF QA,
4 packaging) runs inside Modal. The local machine only edits text, drives git,
5 submits jobs, downloads artifacts and verifies transport hashes.
6 """
7
8 __version__ = "1.0.0"

```

forge/config.py

```
1 """TOML configuration with profile overlays and dotted access."""
2
3 from __future__ import annotations
4
5 import json
6 import tomllib
7 from pathlib import Path
8 from typing import Any
9
10 from .hashing import sha256_json
11
12 _MISSING = object()
13
14
15 def deep_merge(base: dict[str, Any], override: dict[str, Any]) -> dict[str, Any]:
16     out = dict(base)
17     for key, value in override.items():
18         if isinstance(value, dict) and isinstance(out.get(key), dict):
19             out[key] = deep_merge(out[key], value)
20         else:
21             out[key] = value
22     return out
23
24
25 class Config:
26     """Read-only dotted-access view over a merged configuration mapping."""
27
28     def __init__(self, data: dict[str, Any]) -> None:
29         self._data = json.loads(json.dumps(data))
30
31     def get(self, key: str, default: Any = None) -> Any:
32         cursor: Any = self._data
33         for part in key.split("."):
34             if not isinstance(cursor, dict) or part not in cursor:
35                 return default
36             cursor = cursor[part]
37         return cursor
38
39     def __getitem__(self, key: str) -> Any:
40         value = self.get(key, _MISSING)
41         if value is _MISSING:
42             raise KeyError(key)
43         return value
44
45     def section(self, key: str) -> dict[str, Any]:
46         value = self.get(key, {})
47         return dict(value) if isinstance(value, dict) else {}
48
49     def as_dict(self) -> dict[str, Any]:
50         return json.loads(json.dumps(self._data))
51
52     @property
53     def digest(self) -> str:
54         return sha256_json(self._data)
55
56
57 def load_project(repo: Path) -> dict[str, Any]:
58     return json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))
59
60
61 def load_config(repo: Path, profile: str | None = None, overrides: dict[str, Any] | None = None) -> Config:
62     data = tomllib.loads((repo / "configs" / "default.toml").read_text(encoding="utf-8"))
63     if profile:
64         data = deep_merge(data, tomllib.loads((repo / "configs" / f"{profile}.toml").read_text(encoding="utf-8")))
65     if overrides:
66         data = deep_merge(data, overrides)
67     return Config(data)
```

forge/context.py

```
1 """Stage execution context: paths, config, params, logging, seeds and paper numbers."""
2
3 from __future__ import annotations
4
5 import json
6 import random
7 import re
8 import zlib
9 from pathlib import Path
10 from typing import Any
11
12 from .config import Config, load_config, load_project
13 from .events import EventLogger
14
15 _NUMBER_KEY = re.compile(r"^[A-Za-z]+$")
16
17
18 class StageContext:
19     """Everything a stage needs. Stages must write only under ``stage_dir``."""
20
21     def __init__(self, run_id: str, stage: str, params: dict[str, Any], repo: Path, vol: Path) -> None:
22         self.run_id = run_id
23         self.stage = stage
24         self.params = dict(params)
25         self.repo = repo
26         self.vol = vol
27         self.project = load_project(repo)
28         overrides = self.params.get("config_overrides") or None
29         self.config: Config = load_config(repo, self.params.get("profile"), overrides)
30         self.run_dir = vol / "runs" / run_id
31         self.stage_dir = self.run_dir / stage
32         self.stage_dir.mkdir(parents=True, exist_ok=True)
33         self.log = EventLogger(self.stage_dir / "events.jsonl", run_id, stage)
34         self._numbers: dict[str, str] = {}
35
36     # ---- raw inputs (immutable, at repo root, exactly as delivered by the organisers) ----
37     @property
38     def problem_pdf(self) -> Path:
39         return self.repo / self.project["problem_pdf"]
40
41     @property
42     def attachments(self) -> Path:
43         return self.repo / self.project["attachments_dir"]
44
45     @property
46     def templates(self) -> Path:
47         return self.repo / self.project["templates_dir"]
48
49     # ---- outputs ----
50     def out(self, *parts: str) -> Path:
51         path = self.stage_dir.joinpath(*parts)
52         path.parent.mkdir(parents=True, exist_ok=True)
53         return path
54
55     def dep(self, stage: str) -> Path:
56         """Directory of a completed dependency stage in the same run (raises otherwise)."""
57         directory = self.run_dir / stage
58         manifest = directory / "manifest.json"
59         if not manifest.exists():
60             raise RuntimeError(f"dependency stage '{stage}' has not run in run {self.run_id}")
61         status = json.loads(manifest.read_text(encoding="utf-8")).get("status")
62         if status != "completed":
63             raise RuntimeError(f"dependency stage '{stage}' has status {status!r} in run {self.run_id}")
64         return directory
65
66     def has_stage(self, stage: str) -> bool:
67         manifest = self.run_dir / stage / "manifest.json"
68         return manifest.exists() and json.loads(manifest.read_text(encoding="utf-8")).get("status") == "completed"
```

```

69
70 def dep_data(self, stage: str, name: str) -> Any:
71     return json.loads((self.dep(stage) / name).read_text(encoding="utf-8"))
72
73 # ---- parameters / configuration ----
74 def param(self, key: str, default: Any = None) -> Any:
75     return self.params.get(key, default)
76
77 def cfg(self, key: str, default: Any = None) -> Any:
78     return self.config.get(key, default)
79
80 # ---- determinism ----
81 def seed(self, salt: str = "") -> int:
82     base = int(self.cfg("run.seed", 0))
83     return (base + zlib.crc32(salt.encode("utf-8"))) % (2**31 - 1)
84
85 def seed_everything(self, salt: str = "") -> int:
86     value = self.seed(salt)
87     random.seed(value)
88     try:
89         import numpy as np
90
91         np.random.seed(value) # noqa: NPY002 - legacy global RNG used by third-party libraries
92     except ImportError: # pragma: no cover
93         pass
94     self.log.info("seed", salt=salt, value=value)
95     return value
96
97 # ---- numbers that flow into the paper as \val<Key> macros ----
98 def number(self, key: str, value: Any, fmt: str | None = None) -> str:
99     if not _NUMBER_KEY.match(key):
100         raise ValueError(f"paper number keys must be letters only (TeX macro names); got {key!r}")
101     if fmt is not None:
102         text = format(value, fmt)
103     elif isinstance(value, bool):
104         text = str(value)
105     elif isinstance(value, int):
106         text = str(value)
107     elif isinstance(value, float):
108         text = f"{{value:.4f}}"
109     else:
110         text = str(value)
111     self._numbers[key] = text
112     return text
113
114 def flush_numbers(self) -> None:
115     if not self._numbers:
116         return
117     path = self.stage_dir / "numbers.json"
118     merged: dict[str, str] = {}
119     if path.exists():
120         merged.update(json.loads(path.read_text(encoding="utf-8")))
121     merged.update(self._numbers)
122     path.write_text(json.dumps(merged, ensure_ascii=False, indent=2, sort_keys=True), encoding="utf-8")
123
124 # ---- json helpers ----
125 def write_json(self, name: str, obj: Any) -> Path:
126     path = self.out(name)
127     path.write_text(json.dumps(obj, ensure_ascii=False, indent=2, default=str), encoding="utf-8")
128     return path
129
130 def read_json(self, path: Path) -> Any:
131     return json.loads(path.read_text(encoding="utf-8"))

```

forge/contracts.py

```
1 """Lightweight DataFrame contracts (schema + range + monotonicity + custom checks)."""
2
3 from __future__ import annotations
4
5 import re
6 from collections.abc import Callable
7 from dataclasses import dataclass
8 from typing import Any
9
10 import numpy as np
11 import pandas as pd
12
13
14 @dataclass(frozen=True)
15 class Column:
16     name: str
17     kind: str = "any" # int | float | number | str | datetime | any
18     nullable: bool = False
19     min: float | None = None
20     max: float | None = None
21     regex: str | None = None
22     unique: bool = False
23     monotonic: str | None = None # increasing | non_decreasing | decreasing | non_increasing
24     allowed: tuple[Any, ...] | None = None
25     step: float | None = None # constant difference between consecutive values
26
27
28 @dataclass(frozen=True)
29 class FrameContract:
30     name: str
31     columns: tuple[Column, ...]
32     min_rows: int = 1
33     max_rows: int | None = None
34     extra_columns: str = "forbid" # forbid | allow
35     checks: tuple[Callable[[pd.DataFrame], list[str]], ...] = ()
36
37
38 def normalise_columns(df: pd.DataFrame) -> pd.DataFrame:
39     out = df.copy()
40     out.columns = pd.Index([str(c).strip() for c in out.columns])
41     return out
42
43
44 def validate_frame(df: pd.DataFrame, contract: FrameContract) -> dict[str, Any]:
45     df = normalise_columns(df)
46     errors: list[str] = []
47     warnings: list[str] = []
48     expected = [c.name for c in contract.columns]
49     missing = [c for c in expected if c not in df.columns]
50     extra = [c for c in df.columns if c not in expected]
51     if missing:
52         errors.append(f"missing columns: {missing}")
53     if extra and contract.extra_columns == "forbid":
54         errors.append(f"unexpected columns: {extra}")
55     n = len(df)
56     if n < contract.min_rows:
57         errors.append(f"rows {n} < min_rows {contract.min_rows}")
58     if contract.max_rows is not None and n > contract.max_rows:
59         errors.append(f"rows {n} > max_rows {contract.max_rows}")
60
61     for col in contract.columns:
62         if col.name not in df.columns:
63             continue
64         series = df[col.name]
65         nulls = int(series.isna().sum())
66         if nulls and not col.nullable:
67             errors.append(f"{col.name}: {nulls} null values")
68         values = series.dropna()
```

```

69     if col.kind in {"int", "float", "number"}:
70         numeric = pd.to_numeric(values, errors="coerce")
71         bad = int(numeric.isna().sum())
72         if bad:
73             errors.append(f"{col.name}: {bad} non-numeric values")
74         numeric = numeric.dropna().astype(float)
75         if col.kind == "int" and len(numeric) and not np.all(np.mod(numeric.to_numpy(), 1) == 0):
76             errors.append(f"{col.name}: non-integer values present")
77         if col.min is not None and len(numeric) and float(numeric.min()) < col.min:
78             errors.append(f"{col.name}: min {numeric.min()} < {col.min}")
79         if col.max is not None and len(numeric) and float(numeric.max()) > col.max:
80             errors.append(f"{col.name}: max {numeric.max()} > {col.max}")
81         if col.monotonic and len(numeric) > 1:
82             diff = np.diff(numeric.to_numpy())
83             ok = {
84                 "increasing": bool((diff > 0).all()),
85                 "non_decreasing": bool((diff >= 0).all()),
86                 "decreasing": bool((diff < 0).all()),
87                 "non_increasing": bool((diff <= 0).all()),
88             }[col.monotonic]
89             if not ok:
90                 errors.append(f"{col.name}: not {col.monotonic}")
91         if col.step is not None and len(numeric) > 1:
92             diff = np.diff(numeric.to_numpy())
93             if not np.allclose(diff, col.step):
94                 errors.append(f"{col.name}: step is not constant {col.step} (min {diff.min()}, max {diff.max()})")
95     elif col.kind == "str":
96         if col.regex:
97             pattern = re.compile(col.regex)
98             bad_values = [str(v) for v in values.astype(str) if not pattern.fullmatch(str(v))]
99             if bad_values:
100                 errors.append(f"{col.name}: {len(bad_values)} values violate {col.regex}/, e.g. {bad_values[:3]}")
101     elif col.kind == "datetime":
102         parsed = pd.to_datetime(values, errors="coerce")
103         bad = int(parsed.isna().sum())
104         if bad:
105             errors.append(f"{col.name}: {bad} unparsable datetimes")
106     if col.unique and values.duplicated().any():
107         errors.append(f"{col.name}: {int(values.duplicated().sum())} duplicated values")
108     if col.allowed is not None:
109         unexpected = sorted(set(values.unique()) - set(col.allowed), key=str)
110         if unexpected:
111             errors.append(f"{col.name}: values outside allowed set: {unexpected[:5]}")
112
113     for check in contract.checks:
114         try:
115             errors.extend(check(df))
116         except Exception as exc:
117             errors.append(f"check {getattr(check, '__name__', check)!r} raised {exc!r}")
118
119     return {
120         "contract": contract.name,
121         "ok": not errors,
122         "rows": n,
123         "columns": list(df.columns),
124         "errors": errors,
125         "warnings": warnings,
126     }

```

forge/events.py

```

1  """Structured JSONL event logging with a human-readable mirror on stdout."""
2
3  from __future__ import annotations
4
5  import json
6  import time
7  from datetime import UTC, datetime

```

```

8 from pathlib import Path
9 from typing import Any
10
11
12 def _short(value: Any, limit: int = 200) -> str:
13     text = value if isinstance(value, str) else json.dumps(value, ensure_ascii=False, default=str)
14     return text if len(text) <= limit else text[: limit - 1] + "..."
15
16
17 class EventLogger:
18     """Append-only event stream: one JSON object per line, mirrored to stdout."""
19
20     def __init__(self, path: Path, run_id: str, stage: str) -> None:
21         path.parent.mkdir(parents=True, exist_ok=True)
22         self._fh = path.open("a", encoding="utf-8")
23         self.path = path
24         self.run_id = run_id
25         self.stage = stage
26         self._t0 = time.monotonic()
27
28     def emit(self, event: str, level: str = "info", **fields: Any) -> None:
29         record = {
30             "ts": datetime.now(UTC).isoformat(timespec="milliseconds"),
31             "elapsed_s": round(time.monotonic() - self._t0, 3),
32             "run_id": self.run_id,
33             "stage": self.stage,
34             "level": level,
35             "event": event,
36             **fields,
37         }
38         self._fh.write(json.dumps(record, ensure_ascii=False, default=str) + "\n")
39         self._fh.flush()
40         extra = " ".join(f"{k}={_short(v)}" for k, v in fields.items())
41         print(f"[{record['elapsed_s']:9.3f}s] {level.upper():5s} {self.stage}:{event} {extra}", flush=True)
42
43     def info(self, event: str, **fields: Any) -> None:
44         self.emit(event, "info", **fields)
45
46     def warn(self, event: str, **fields: Any) -> None:
47         self.emit(event, "warn", **fields)
48
49     def error(self, event: str, **fields: Any) -> None:
50         self.emit(event, "error", **fields)
51
52     def metric(self, name: str, value: Any, **fields: Any) -> None:
53         self.emit("metric", "info", name=name, value=value, **fields)
54
55     def close(self) -> None:
56         if not self._fh.closed:
57             self._fh.close()

```

forge/hashing.py

```

1 """Content hashing helpers (SHA-256 everywhere; JSON hashed in canonical form)."""
2
3 from __future__ import annotations
4
5 import hashlib
6 import json
7 from collections.abc import Iterable
8 from pathlib import Path
9 from typing import Any
10
11 CHUNK = 1 << 20
12
13
14 def sha256_file(path: Path) -> str:
15     h = hashlib.sha256()

```

```

16     with path.open("rb") as fh:
17         for chunk in iter(lambda: fh.read(CHUNK), b""):
18             h.update(chunk)
19     return h.hexdigest()
20
21
22 def sha256_bytes(data: bytes) -> str:
23     return hashlib.sha256(data).hexdigest()
24
25
26 def sha256_json(obj: Any) -> str:
27     payload = json.dumps(obj, sort_keys=True, ensure_ascii=False, separators=(",", ":"), default=str)
28     return sha256_bytes(payload.encode("utf-8"))
29
30
31 def hash_tree(root: Path, ignore: Iterable[str] = ()) -> dict[str, dict[str, Any]]:
32     """Return ``{relative_posix_path: {"sha256": ..., "bytes": ...}}`` for every file under ``root``.
33
34     Any path component contained in ``ignore`` excludes the file (directory names or file names).
35     The result is sorted by path so that its canonical JSON digest is stable.
36     """
37     ignored = set(ignore)
38     out: dict[str, dict[str, Any]] = {}
39     if not root.exists():
40         return out
41     for path in sorted(root.rglob("*")):
42         if not path.is_file():
43             continue
44         rel = path.relative_to(root)
45         if ignored & set(rel.parts):
46             continue
47         out[rel.as_posix()] = {"sha256": sha256_file(path), "bytes": path.stat().st_size}
48     return out
49
50
51 def tree_digest(tree: dict[str, dict[str, Any]]) -> str:
52     return sha256_json({k: v["sha256"] for k, v in sorted(tree.items())})

```

forge/manifest.py

```

1  """Run/stage manifests: the unit of lineage, reproducibility and audit."""
2
3  from __future__ import annotations
4
5  import importlib.metadata
6  import json
7  import os
8  import platform
9  import sys
10 import traceback
11 from dataclasses import asdict, dataclass, field, fields
12 from datetime import UTC, datetime
13 from pathlib import Path
14 from typing import Any
15
16 from .hashing import hash_tree, tree_digest
17
18 TRACKED_PACKAGES = (
19     "numpy",
20     "scipy",
21     "pandas",
22     "pyarrow",
23     "openpyxl",
24     "matplotlib",
25     "scikit-learn",
26     "statsmodels",
27     "ortools",
28     "highspy",

```

```

29     "numba",
30     "cvxpy",
31     "networkx",
32     "pymupdf",
33     "pytest",
34     "hypothesis",
35     "ruff",
36     "mypy",
37 )
38 RECORD_FILES = ("manifest.json", "events.jsonl") # run records, excluded from the outputs digest
39
40
41 def utc_now() -> str:
42     return datetime.now(UTC).isoformat(timespec="seconds")
43
44
45 def runtime_info() -> dict[str, Any]:
46     packages: dict[str, str] = {}
47     for name in TRACKED_PACKAGES:
48         try:
49             packages[name] = importlib.metadata.version(name)
50         except importlib.metadata.PackageNotFoundError:
51             continue
52     return {
53         "python": sys.version.split()[0],
54         "platform": platform.platform(),
55         "cpu_count": os.cpu_count(),
56         "modal_task_id": os.getenv("MODAL_TASK_ID"),
57         "modal_is_remote": os.getenv("MODAL_IS_REMOTE") == "1" or os.getenv("MODAL_TASK_ID") is not None,
58         "packages": packages,
59         "env": {k: os.environ.get(k) for k in ("OMP_NUM_THREADS", "OPENBLAS_NUM_THREADS", "PYTHONHASHSEED")},
60     }
61
62
63 @dataclass
64 class Manifest:
65     schema: str = "forge.manifest/1"
66     run_id: str = ""
67     stage: str = ""
68     status: str = "running" # running | completed | failed
69     started_at: str = field(default_factory=utc_now)
70     finished_at: str | None = None
71     duration_s: float | None = None
72     code_ref: dict[str, Any] = field(default_factory=dict)
73     params: dict[str, Any] = field(default_factory=dict)
74     config_digest: str = ""
75     deps: dict[str, str] = field(default_factory=dict)
76     inputs: dict[str, dict[str, Any]] = field(default_factory=dict)
77     outputs: dict[str, dict[str, Any]] = field(default_factory=dict)
78     outputs_digest: str = ""
79     metrics: dict[str, Any] = field(default_factory=dict)
80     runtime: dict[str, Any] = field(default_factory=dict)
81     error: dict[str, Any] | None = None
82
83 @classmethod
84 def start(
85     cls,
86     run_id: str,
87     stage: str,
88     params: dict[str, Any],
89     code_ref: dict[str, Any],
90     config_digest: str,
91     deps: dict[str, str],
92     inputs: dict[str, dict[str, Any]],
93 ) -> Manifest:
94     manifest = cls(
95         run_id=run_id,
96         stage=stage,
97         params=params,
98         code_ref=code_ref,
99         config_digest=config_digest,

```

```

100         deps=deps,
101         inputs=inputs,
102     )
103     manifest.runtime = runtime_info()
104     return manifest
105
106     def complete(self, stage_dir: Path, metrics: dict[str, Any]) -> None:
107         self.outputs = hash_tree(stage_dir, ignore=RECORD_FILES)
108         self.outputs_digest = tree_digest(self.outputs)
109         self.metrics = json.loads(json.dumps(metrics, default=str))
110         self.status = "completed"
111         self._finish()
112
113     def fail(self, exc: BaseException) -> None:
114         self.error = {"type": type(exc).__name__, "message": str(exc), "traceback": traceback.format_exc()}
115         self.status = "failed"
116         self._finish()
117
118     def _finish(self) -> None:
119         self.finished_at = utc_now()
120         started = datetime.fromisoformat(self.started_at)
121         self.duration_s = round((datetime.fromisoformat(self.finished_at) - started).total_seconds(), 3)
122
123     def write(self, path: Path) -> None:
124         path.parent.mkdir(parents=True, exist_ok=True)
125         path.write_text(json.dumps(asdict(self), ensure_ascii=False, indent=2, default=str), encoding="utf-8")
126
127     @classmethod
128     def read(cls, path: Path) -> Manifest:
129         raw = json.loads(path.read_text(encoding="utf-8"))
130         known = {f.name for f in fields(cls)}
131         return cls(**{k: v for k, v in raw.items() if k in known})
132
133     def verify_outputs(self, stage_dir: Path) -> list[str]:
134         """Return a list of mismatches between the manifest and the files on disk (empty == verified)."""
135         problems: list[str] = []
136         current = hash_tree(stage_dir, ignore=RECORD_FILES)
137         for rel, meta in self.outputs.items():
138             if rel not in current:
139                 problems.append(f"missing: {rel}")
140             elif current[rel]["sha256"] != meta["sha256"]:
141                 problems.append(f"sha256 mismatch: {rel}")
142         for rel in current:
143             if rel not in self.outputs:
144                 problems.append(f"unexpected file: {rel}")
145         return problems

```

forge/modal_app.py

```

1  """Modal application — the only compute surface of this repository.
2
3  Local side (tools/cli.py) only submits:
4      modal run forge/modal_app.py --stage ingest --run-id <id> --params '{...}' --size small
5  Inside the container the repository is mounted read-only at /repo and the persistent
6  volume at /vol; stages write exclusively to /vol/runs/<run_id>/<stage>/.
7  """
8
9  from __future__ import annotations
10
11  import contextlib
12  import io
13  import json
14  import os
15  import sys
16  import traceback
17  from pathlib import Path
18
19  import modal

```

```

20
21 _HERE = Path(__file__).resolve()
22 REPO = _HERE.parents[1]
23
24
25 def _load_project() -> dict:
26     for candidate in (REPO / "configs" / "project.json", Path("/repo/configs/project.json")):
27         if candidate.exists():
28             return json.loads(candidate.read_text(encoding="utf-8"))
29         raise FileNotFoundError("configs/project.json not found next to forge/modal_app.py")
30
31
32 PROJECT = _load_project()
33 app = modal.App(PROJECT["app"])
34 volume = modal.Volume.from_name(PROJECT["volume"], create_if_missing=True)
35
36 APT_PACKAGES = [
37     "texlive-xetex",
38     "texlive-lang-chinese",
39     "texlive-latex-base",
40     "texlive-latex-recommended",
41     "texlive-latex-extra",
42     "texlive-science",
43     "texlive-pictures",
44     "texlive-bibtex-extra",
45     "texlive-fonts-recommended",
46     "fonts-noto-cjk",
47     "fonts-noto-core",
48     "fonts-droid-fallback",
49     "fonts-wqy-microhei",
50     "fontconfig",
51     "poppler-utils",
52     "zip",
53     "unzip",
54 ]
55 PIP_PACKAGES = [
56     "numpy==2.2.6",
57     "scipy==1.15.3",
58     "pandas==2.2.3",
59     "pyarrow==20.0.0",
60     "openpyxl==3.1.5",
61     "matplotlib==3.10.3",
62     "scikit-learn==1.6.1",
63     "statsmodels==0.14.4",
64     "ortools==9.12.4544",
65     "highspy==1.10.0",
66     "numba==0.61.2",
67     "cvxpy==1.6.5",
68     "networkx==3.4.2",
69     "pydantic==2.11.4",
70     "pymupdf==1.26.7",
71     "pytest==8.3.5",
72     "pytest-cov==6.1.1",
73     "hypothesis==6.131.9",
74     "ruff==0.11.10",
75     "mypy==1.15.0",
76     "pandas-stubs==2.2.3.250308",
77     "types-openpyxl==3.1.5.20250602",
78 ]
79 SKIP_PARTS = {
80     ".git",
81     "artifacts",
82     "releases",
83     ".forge",
84     "__pycache__",
85     ".mypy_cache",
86     ".ruff_cache",
87     ".pytest_cache",
88     ".hypothesis",
89     ".DS_Store",
90 }

```

```

91
92
93 def _ignore(path: Path) -> bool:
94     return bool(SKIP_PARTS & set(path.parts)) or path.suffix in {".pyc", ".pyo"}
95
96
97 image = (
98     modal.Image.debian_slim(python_version="3.11")
99     .apt_install(*APT_PACKAGES)
100     .pip_install(*PIP_PACKAGES)
101     .env(
102         {
103             "OMP_NUM_THREADS": "1",
104             "OPENBLAS_NUM_THREADS": "1",
105             "MKL_NUM_THREADS": "1",
106             "MPLBACKEND": "Agg",
107             "MPLCONFIGDIR": "/tmp/mpl",
108             "PYTHONHASHSEED": "0",
109             "PYTHONUNBUFFERED": "1",
110             "PYTHONDONTWRITEBYTECODE": "1",
111             "PYTHONPATH": "/repo",
112             "FORGE_REPO": "/repo",
113             "FORGE_VOL": "/vol",
114             "MODAL_IS_REMOTE": "1",
115             "TEXMFVAR": "/tmp/texmf-var",
116             "TEXMFCONFIG": "/tmp/texmf-config",
117             "HOME": "/root",
118         }
119     )
120     .add_local_dir(REPO, "/repo", ignore=_ignore)
121 )
122
123 VOLUME_MOUNT = {"vol": volume}
124
125
126 def _bootstrap() -> None:
127     if "/repo" not in sys.path:
128         sys.path.insert(0, "/repo")
129     os.chdir("/repo")
130     Path("/tmp/mpl").mkdir(exist_ok=True)
131
132
133 def _run(stage: str, run_id: str, params_json: str, force: bool) -> dict:
134     _bootstrap()
135     volume.reload()
136     from forge.runner import execute
137
138     params = json.loads(params_json or "{}")
139     return execute(
140         stage_name=stage,
141         run_id=run_id,
142         params=params,
143         force=force,
144         repo=Path("/repo"),
145         vol=Path("/vol"),
146         commit=volume.commit,
147     )
148
149
150 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=2, memory=4096, timeout=2 * 3600)
151 def run_small(stage: str, run_id: str, params: str, force: bool) -> dict:
152     return _run(stage, run_id, params, force)
153
154
155 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=8, memory=16384, timeout=6 * 3600)
156 def run_medium(stage: str, run_id: str, params: str, force: bool) -> dict:
157     return _run(stage, run_id, params, force)
158
159
160 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=32, memory=65536, timeout=12 * 3600)
161 def run_large(stage: str, run_id: str, params: str, force: bool) -> dict:

```

```

162     return _run(stage, run_id, params, force)
163
164
165 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=4, memory=8192, timeout=3600)
166 def exec_python(code: str, run_id: str) -> str:
167     """Run an ad-hoc Python snippet in the cloud (exploration, status, debugging). Never local."""
168     _bootstrap()
169     volume.reload()
170     buffer = io.StringIO()
171     scope = {
172         "__name__": "__forge_exec__",
173         "RUN_DIR": Path("/vol/runs") / run_id,
174         "VOL": Path("/vol"),
175         "REPO": Path("/repo"),
176         "RUNS": Path("/vol/runs"),
177     }
178     with contextlib.redirect_stdout(buffer), contextlib.redirect_stderr(buffer):
179         try:
180             exec(compile(code, "<forge-exec>", "exec"), scope) # noqa: S102 - explicit cloud REPL
181         except BaseException:
182             traceback.print_exc(file=buffer)
183     volume.commit()
184     return buffer.getvalue()
185
186
187 @app.function(image=image, volumes=VOLUME_MOUNT, cpu=2, memory=4096, timeout=600)
188 def provision() -> dict:
189     """Prove the toolchain: interpreter, packages, TeX engine, fonts, solver availability."""
190     import importlib.metadata
191     import subprocess
192
193     _bootstrap()
194     info: dict = {"modal_task_id": os.getenv("MODAL_TASK_ID"), "cpu_count": os.cpu_count()}
195     info["python"] = sys.version.split()[0]
196     info["packages"] = {p.split("==")[0]: importlib.metadata.version(p.split("==")[0]) for p in PIP_PACKAGES}
197     info["xelatex"] = subprocess.run(["xelatex", "--version"], capture_output=True, text=True).stdout.splitlines()[0]
198     info["bibtex"] = subprocess.run(["bibtex", "--version"], capture_output=True, text=True).stdout.splitlines()[0]
199     info["fc_cjk"] = subprocess.run(["fc-match", "Noto Serif CJK SC"], capture_output=True, text=True).stdout.strip()
200     info["kpse"] = {
201         name: bool(subprocess.run(["kpsewhich", name], capture_output=True, text=True).stdout.strip())
202         for name in (
203             "ctexart.cls",
204             "gbt7714.sty",
205             "siunitx.sty",
206             "algorithm2e.sty",
207             "booktabs.sty",
208             "tikz.sty",
209             "listings.sty",
210             "cleveref.sty",
211             "tcolorbox.sty",
212             "pgfplots.sty",
213         )
214     }
215     from ortools.sat.python import cp_model
216
217     model = cp_model.CpModel()
218     x = model.NewIntVar(0, 10, "x")
219     model.Add(x >= 3)
220     model.Minimize(x)
221     solver = cp_model.CpSolver()
222     info["cpsat"] = {"status": solver.StatusName(solver.Solve(model)), "x": solver.Value(x)}
223     import highspy
224
225     info["highs"] = highspy.Highs().version()
226     from forge.plotting import cjk_font_family
227
228     info["figure_cjk_font"] = cjk_font_family()
229     return info
230
231
232 @app.local_entrypoint()

```

```

233 def main(
234     stage: str = "ingest",
235     run_id: str = "",
236     params: str = "{}",
237     force: bool = False,
238     size: str = "small",
239     spawn: bool = False,
240 ) -> None:
241     """Submit one stage (or ``all`` with params.stages) to the cloud and print its result."""
242     if stage == "provision":
243         print("PROVISION " + json.dumps(provision.remote(), ensure_ascii=False, default=str))
244         return
245     if stage == "exec":
246         payload = json.loads(params or "{}")
247         print(exec_python.remote(payload.get("code", ""), run_id or "adhoc"))
248         return
249     fn = {"small": run_small, "medium": run_medium, "large": run_large}[size]
250     if spawn:
251         call = fn.spawn(stage, run_id, params, force)
252         print(f"FUNCTION_CALL_ID {call.object_id}")
253         return
254     result = fn.remote(stage, run_id, params, force)
255     print("STAGE_RESULT " + json.dumps(result, ensure_ascii=False, default=str))

```

forge/packaging.py

```

1  """Deterministic packaging of support materials (stable member order and timestamps)."""
2
3  from __future__ import annotations
4
5  import zipfile
6  from collections.abc import Iterable
7  from pathlib import Path
8  from typing import Any
9
10 from .hashing import sha256_file
11
12 FIXED_TIME = (2026, 1, 1, 0, 0, 0)
13 SKIP_PARTS = {
14     ".git",
15     "__pycache__",
16     ".DS_Store",
17     ".mypy_cache",
18     ".ruff_cache",
19     ".pytest_cache",
20     ".hypothesis",
21     "artifacts",
22     "releases",
23     ".forge",
24     "build",
25 }
26
27
28 def code_members(repo: Path, globs: Iterable[str]) -> list[tuple[str, Path]]:
29     found: dict[str, Path] = {}
30     for pattern in globs:
31         for path in sorted(repo.glob(pattern)):
32             if not path.is_file():
33                 continue
34             rel = path.relative_to(repo)
35             if SKIP_PARTS & set(rel.parts) or path.suffix == ".pyc":
36                 continue
37             found[rel.as_posix()] = path
38     return sorted(found.items())
39
40
41 def run_members(run_dir: Path, stages: Iterable[str], *, prefix: str = "runs") -> list[tuple[str, Path]]:
42     members: list[tuple[str, Path]] = []

```

```

43     for stage in stages:
44         directory = run_dir / stage
45         if not directory.is_dir():
46             continue
47         for path in sorted(directory.rglob("*")):
48             rel = path.relative_to(directory)
49             if path.is_file() and not (SKIP_PARTS & set(rel.parts)):
50                 members.append((f"{prefix}/{run_dir.name}/{stage}/{rel.as_posix()}", path))
51     return members
52
53
54 def build_zip(members: Iterable[tuple[str, Path]], out: Path, *, comment: str = "") -> dict[str, Any]:
55     ordered = sorted(dict(members).items())
56     out.parent.mkdir(parents=True, exist_ok=True)
57     total = 0
58     with zipfile.ZipFile(out, "w", compression=zipfile.ZIP_DEFLATED, compresslevel=9) as zf:
59         if comment:
60             zf.comment = comment.encode("utf-8")
61         for name, source in ordered:
62             info = zipfile.ZipInfo(name, date_time=FIXED_TIME)
63             info.compress_type = zipfile.ZIP_DEFLATED
64             info.external_attr = 0o644 << 16
65             data = source.read_bytes()
66             total += len(data)
67             zf.writestr(info, data)
68     with zipfile.ZipFile(out) as zf:
69         bad = zf.testzip()
70         count = len(zf.namelist())
71     if bad:
72         raise RuntimeError(f"zip member failed CRC check: {bad}")
73     return {
74         "path": str(out),
75         "members": count,
76         "uncompressed_bytes": total,
77         "bytes": out.stat().st_size,
78         "sha256": sha256_file(out),
79     }

```

forge/pdfqa.py

```

1  """PDF inspection for quality gates (PyMuPDF)."""
2
3  from __future__ import annotations
4
5  import re
6  from collections.abc import Iterable
7  from pathlib import Path
8  from typing import Any
9
10 A4_PT = (595.276, 841.89)
11
12
13 def inspect_pdf(path: Path, *, with_text: bool = True) -> dict[str, Any]:
14     import fitz # PyMuPDF
15
16     doc = fitz.open(path)
17     pages: list[dict[str, Any]] = []
18     texts: list[str] = []
19     fonts: dict[int, dict[str, Any]] = {}
20     unembedded: list[str] = []
21     for index, page in enumerate(doc, start=1):
22         rect = page.rect
23         pages.append(
24             {
25                 "page": index,
26                 "width_pt": round(rect.width, 2),
27                 "height_pt": round(rect.height, 2),
28                 "a4": abs(rect.width - A4_PT[0]) < 3 and abs(rect.height - A4_PT[1]) < 3,

```

```

29         }
30     )
31     if with_text:
32         texts.append(page.get_text("text"))
33     for font in page.get_fonts(full=True):
34         xref, ftype, basefont = font[0], font[2], font[3]
35         if xref in fonts:
36             continue
37         embedded = ftype == "Type3"
38         if not embedded and xref:
39             try:
40                 embedded = len(doc.extract_font(xref)[3]) > 0
41             except Exception:
42                 embedded = False
43         fonts[xref] = {"name": basefont, "type": ftype, "embedded": bool(embedded)}
44         if not embedded:
45             unembedded.append(basefont)
46     return {
47         "pages": len(doc),
48         "page_boxes": pages,
49         "all_a4": all(p["a4"] for p in pages),
50         "fonts": list(fonts.values()),
51         "unembedded_fonts": sorted(set(unembedded)),
52         "texts": texts,
53         "metadata": doc.metadata,
54     }
55
56
57 def scan_strings(texts: Iterable[str], patterns: Iterable[str], *, context: int = 40) -> list[dict[str, Any]]:
58     compiled = [(p, re.compile(p)) for p in patterns]
59     hits: list[dict[str, Any]] = []
60     for page_no, text in enumerate(texts, start=1):
61         for raw, pattern in compiled:
62             for match in pattern.finditer(text):
63                 start, end = max(0, match.start() - context), min(len(text), match.end() + context)
64                 hits.append({"page": page_no, "pattern": raw, "context": text[start:end].replace("\n", " ")})
65     return hits

```

forge/plotting.py

```

1  """House style for paper figures: vector PDF output, CJK-capable fonts, accessible palette."""
2
3  from __future__ import annotations
4
5  import logging
6  import warnings
7  from pathlib import Path
8  from typing import Any
9
10 OKABE_ITO = ["#0072B2", "#D55E00", "#009E73", "#E69F00", "#CC79A7", "#56B4E9", "#F0E442", "#000000"]
11 # (family name as matplotlib registers it, font file). TTC files expose only their first face,
12 # which is why the Noto CJK collection is addressed by its JP face name.
13 _CJK_CANDIDATES = (
14     ("WenQuanYi Micro Hei", "/usr/share/fonts/truetype/wqy/wqy-microhei.ttc"),
15     ("Noto Sans CJK JP", "/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc"),
16     ("Droid Sans Fallback", "/usr/share/fonts/truetype/droid/DroidSansFallbackFull.ttf"),
17 )
18 _PROBE_TEXT = "温度 T 水分 C 药材 频段 关键词 abc 0.25"
19 _STATE: dict[str, Any] = {}
20
21
22 class _Catcher(logging.Handler):
23     def __init__(self) -> None:
24         super().__init__()
25         self.records: list[str] = []
26
27     def emit(self, record: logging.LogRecord) -> None:
28         self.records.append(record.getMessage())

```

```

29
30
31 def _probe(family: str) -> bool:
32     """True when ``family`` (with DejaVu Sans as Latin fallback) renders the probe text into PDF cleanly."""
33     from matplotlib import pyplot as plt
34
35     catcher = _Catcher()
36     loggers = [
37         logging.getLogger(n) for n in ("matplotlib.font_manager", "matplotlib.backends.backend_pdf", "matplotlib")
38     ]
39     for lg in loggers:
40         lg.addHandler(catcher)
41     try:
42         with warnings.catch_warnings(record=True) as caught:
43             warnings.simplefilter("always")
44             fig, ax = plt.subplots(figsize=(2.5, 1))
45             ax.text(0.05, 0.5, _PROBE_TEXT, fontfamily=[family, "DejaVu Sans"])
46             ax.set_axis_off()
47             probe = Path("/tmp") / f"forge_font_probe_{family.replace(' ', '_')}.pdf"
48             fig.savefig(probe, format="pdf")
49             plt.close(fig)
50             messages = [str(w.message) for w in caught] + catcher.records
51             bad = [m for m in messages if "missing from font" in m or "not found" in m.lower()]
52             return not bad
53     except Exception:
54         return False
55     finally:
56         for lg in loggers:
57             lg.removeHandler(catcher)
58
59
60 def cjk_font_family() -> str:
61     """Pick a CJK font that renders into PDF without missing glyphs (cached per process)."""
62     if "family" in _STATE:
63         return str(_STATE["family"])
64     import matplotlib
65
66     matplotlib.use("Agg")
67     from matplotlib import font_manager
68
69     chosen = "DejaVu Sans"
70     for family, path in _CJK_CANDIDATES:
71         if not Path(path).exists():
72             continue
73         try:
74             font_manager.fontManager.addfont(path)
75         except Exception:
76             continue
77         if _probe(family):
78             chosen = family
79             break
80     _STATE["family"] = chosen
81     return chosen
82
83
84 def setup(font_size: float = 9.0) -> str:
85     import matplotlib
86
87     matplotlib.use("Agg")
88     from cycler import cycler
89     from matplotlib import pyplot as plt
90
91     family = cjk_font_family()
92     plt.rcParams.update(
93         {
94             "font.family": [family, "DejaVu Sans"],
95             "font.size": font_size,
96             "axes.titlesize": font_size + 1,
97             "axes.labelsize": font_size,
98             "legend.fontsize": font_size - 1,
99             "xtick.labelsize": font_size - 1,

```

```

100         "ytick.labelsize": font_size - 1,
101         "axes.prop_cycle": cycler(color=OKABE_ITO),
102         "axes.spines.top": False,
103         "axes.spines.right": False,
104         "axes.grid": True,
105         "grid.alpha": 0.25,
106         "grid.linewidth": 0.6,
107         "lines.linewidth": 1.4,
108         "figure.dpi": 150,
109         "savefig.dpi": 300,
110         "pdf.fonttype": 42,
111         "ps.fonttype": 42,
112         "axes.unicode_minus": False,
113         "figure.constrained_layout.use": True,
114     }
115 )
116 return family
117
118
119 def new_figure(width_in: float = 6.3, height_in: float = 3.4, **kwargs: Any) -> tuple[Any, Any]:
120     from matplotlib import pyplot as plt
121
122     setup()
123     return plt.subplots(figsize=(width_in, height_in), **kwargs)
124
125
126 def save(fig: Any, pdf_path: Path, *, png_preview: bool = True, dpi: int = 200) -> dict[str, Any]:
127     from matplotlib import pyplot as plt
128
129     pdf_path.parent.mkdir(parents=True, exist_ok=True)
130     fig.savefig(pdf_path, format="pdf")
131     out = {"pdf": str(pdf_path), "bytes": pdf_path.stat().st_size}
132     if png_preview:
133         png = pdf_path.with_suffix(".png")
134         fig.savefig(png, format="png", dpi=dpi)
135         out["png"] = str(png)
136     plt.close(fig)
137     return out

```

forge/runner.py

```

1  """Stage registry and executor. A stage is a function ``fn(ctx) -> metrics``.
2
3  Guarantees: one manifest per (run, stage); dependencies must be completed in the same run
4  (or copied from ``from_run``); a completed stage is skipped unless ``force``; failures are
5  recorded with a traceback and re-raised; the volume is committed after every stage.
6  """
7
8  from __future__ import annotations
9
10 import importlib
11 import json
12 import shutil
13 import time
14 from collections.abc import Callable, Iterable
15 from dataclasses import dataclass
16 from pathlib import Path
17 from typing import Any
18
19 from .context import StageContext
20 from .hashing import hash_tree, tree_digest
21 from .manifest import Manifest
22
23 StageFn = Callable[[StageContext], dict[str, Any] | None]
24 CODE_IGNORE = (
25     ".git",
26     "artifacts",
27     "releases",

```

```

28     "__pycache__",
29     ".mypy_cache",
30     ".ruff_cache",
31     ".pytest_cache",
32     ".hypothesis",
33     ".DS_Store",
34     ".forge",
35 )
36
37
38 @dataclass(frozen=True)
39 class StageSpec:
40     name: str
41     fn: StageFn
42     deps: tuple[str, ...]
43     description: str
44
45
46 STAGES: dict[str, StageSpec] = {}
47
48
49 def stage(name: str, deps: Iterable[str] = (), description: str = "") -> Callable[[StageFn], StageFn]:
50     def decorator(fn: StageFn) -> StageFn:
51         if name in STAGES:
52             raise ValueError(f"duplicate stage registration: {name}")
53         doc = (fn.__doc__ or "").strip().splitlines()
54         STAGES[name] = StageSpec(name, fn, tuple(deps), description or (doc[0] if doc else ""))
55         return fn
56
57     return decorator
58
59
60 def discover(repo: Path) -> None:
61     project = json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))
62     importlib.import_module("pipelines.common.stages")
63     importlib.import_module(f"pipelines.{project['package']}.stages")
64
65
66 def list_stages(repo: Path) -> list[dict[str, Any]]:
67     discover(repo)
68     return [{"name": s.name, "deps": list(s.deps), "description": s.description} for s in STAGES.values()]
69
70
71 def _raw_input_prefixes(repo: Path) -> tuple[str, ...]:
72     project = json.loads((repo / "configs" / "project.json").read_text(encoding="utf-8"))
73     return (project["problem_pdf"], project["attachments_dir"].rstrip("/") + "/")
74
75
76 def execute(
77     stage_name: str,
78     run_id: str,
79     params: dict[str, Any],
80     force: bool,
81     repo: Path,
82     vol: Path,
83     commit: Callable[[], None] | None = None,
84 ) -> dict[str, Any]:
85     discover(repo)
86     if stage_name == "all":
87         results = [execute(s, run_id, params, force, repo, vol, commit) for s in params.get("stages", [])]
88         return {
89             "stage": "all",
90             "run_id": run_id,
91             "results": results,
92             "status": "completed" if all(r.get("status") in {"completed", "skipped"} for r in results) else "failed",
93         }
94     if stage_name not in STAGES:
95         raise KeyError(f"unknown stage {stage_name!r}; known stages: {sorted(STAGES)}")
96     spec = STAGES[stage_name]
97     run_dir = vol / "runs" / run_id
98     stage_dir = run_dir / stage_name

```

```

99     manifest_path = stage_dir / "manifest.json"
100
101     if manifest_path.exists() and not force:
102         previous = Manifest.read(manifest_path)
103         if previous.status == "completed":
104             return {
105                 "stage": stage_name,
106                 "run_id": run_id,
107                 "status": "skipped",
108                 "reason": "already completed in this run (use --force to rerun)",
109                 "outputs_digest": previous.outputs_digest,
110             }
111
112     deps_digest: dict[str, str] = {}
113     for dep in spec.deps:
114         dep_manifest = run_dir / dep / "manifest.json"
115         if not dep_manifest.exists() and params.get("from_run"):
116             source = vol / "runs" / str(params["from_run"]) / dep
117             if (source / "manifest.json").exists():
118                 shutil.copytree(source, run_dir / dep, dirs_exist_ok=True)
119             if not dep_manifest.exists():
120                 raise RuntimeError(
121                     f"stage '{stage_name}' requires '{dep}', which has not run in run {run_id} "
122                     f"(run it first, or pass from_run=<run_id> to reuse its outputs)"
123                 )
124             dep_state = Manifest.read(dep_manifest)
125             if dep_state.status != "completed":
126                 raise RuntimeError(f"dependency '{dep}' has status {dep_state.status!r} in run {run_id}")
127             deps_digest[dep] = dep_state.outputs_digest
128
129     if stage_dir.exists():
130         shutil.rmtree(stage_dir)
131     ctx = StageContext(run_id, stage_name, params, repo, vol)
132     code_tree = hash_tree(repo, ignore=CODE_IGNORE)
133     prefixes = _raw_input_prefixes(repo)
134     inputs = {k: v for k, v in code_tree.items() if k.startswith(prefixes)}
135     code_ref = {**dict(params.get("code_ref") or {}), "code_digest": tree_digest(code_tree), "files": len(code_tree)}
136     manifest = Manifest.start(run_id, stage_name, params, code_ref, ctx.config.digest, deps_digest, inputs)
137     manifest.write(manifest_path)
138     ctx.log.info("stage.start", deps=list(spec.deps), description=spec.description, size=params.get("size"))
139     t0 = time.monotonic()
140     try:
141         metrics = spec.fn(ctx) or {}
142         ctx.flush_numbers()
143         manifest.complete(stage_dir, metrics)
144         ctx.log.info("stage.completed", duration_s=round(time.monotonic() - t0, 3), metrics=metrics)
145     except BaseException as exc:
146         manifest.fail(exc)
147         ctx.log.error("stage.failed", error=repr(exc))
148         manifest.write(manifest_path)
149         ctx.log.close()
150         if commit:
151             commit()
152         raise
153     manifest.write(manifest_path)
154     ctx.log.close()
155     if commit:
156         commit()
157     return {
158         "stage": stage_name,
159         "run_id": run_id,
160         "status": "completed",
161         "duration_s": manifest.duration_s,
162         "metrics": manifest.metrics,
163         "outputs_digest": manifest.outputs_digest,
164         "modal_task_id": manifest.runtime.get("modal_task_id"),
165     }

```

forge/tex.py

```
1 """XeLaTeX compilation and log analysis (only ever executed inside Modal)."""
2
3 from __future__ import annotations
4
5 import os
6 import re
7 import subprocess
8 from pathlib import Path
9 from typing import Any
10
11 _UNDEF_REF = re.compile(r"LaTeX Warning: Reference `([~]+)' on page \d+ undefined")
12 _UNDEF_CIT = re.compile(r"LaTeX Warning: Citation `([~]+)' on page \d+ undefined")
13 _OVERFULL = re.compile(r"Overfull \hbox \((([~]+)pt too wide\)")
14 _MISSING = re.compile(r"Missing character: There is no (.?) in font")
15 _ERROR = re.compile(r"! (.*)$", re.M)
16 _PAGES = re.compile(r"Output written on .*? \((\d+) pages?)")
17
18
19 def tex_env() -> dict[str, str]:
20     env = dict(os.environ)
21     env.setdefault("TEXMFVAR", "/tmp/texmf-var")
22     env.setdefault("TEXMFCONFIG", "/tmp/texmf-config")
23     env.setdefault("HOME", "/tmp")
24     env["max_print_line"] = "2000"
25     env["error_line"] = "254"
26     env["half_error_line"] = "238"
27     return env
28
29
30 def summarize_log(log: str) -> dict[str, Any]:
31     overfull = [float(x) for x in _OVERFULL.findall(log)]
32     missing = _MISSING.findall(log)
33     pages = _PAGES.search(log)
34     return {
35         "undefined_references": sorted(set(_UNDEF_REF.findall(log))),
36         "undefined_citations": sorted(set(_UNDEF_CIT.findall(log))),
37         "overfull_count": len(overfull),
38         "overfull_max_pt": max(overfull) if overfull else 0.0,
39         "missing_characters": len(missing),
40         "missing_character_samples": sorted(set(missing))[:10],
41         "errors": _ERROR.findall(log)[:10],
42         "pages_from_log": int(pages.group(1)) if pages else None,
43     }
44
45
46 def aux_page(aux: Path, label: str) -> int | None:
47     """Page number recorded for ``\label{label}`` in the .aux file (hyperref-compatible)."""
48     if not aux.exists():
49         return None
50     pattern = re.compile(r"\newlabel\{" + re.escape(label) + r"\}\{([~]*)\{(\d+)\}\}")
51     match = pattern.search(aux.read_text(encoding="utf-8", errors="replace"))
52     return int(match.group(1)) if match else None
53
54
55 def compile_tex(
56     workdir: Path,
57     main: str,
58     jobname: str,
59     *,
60     passes: int = 3,
61     bibtex: bool = True,
62     timeout: int = 1200,
63 ) -> dict[str, Any]:
64     env = tex_env()
65     runs: list[dict[str, Any]] = []
66     log_path = workdir / f"{jobname}.log"
67
68     def xelatex(tag: str) -> None:
```

```

69     proc = subprocess.run(
70         ["xelatex", "-interaction=nonstopmode", "-halt-on-error", "-file-line-error", f"-jobname={jobname}", main],
71         cwd=workdir,
72         capture_output=True,
73         text=True,
74         env=env,
75         timeout=timeout,
76         check=False,
77     )
78     runs.append({"pass": tag, "rc": proc.returncode})
79     if proc.returncode != 0:
80         log = log_path.read_text(encoding="utf-8", errors="replace") if log_path.exists() else proc.stdout
81         errors = _ERROR.findall(log)
82         raise RuntimeError(
83             f"xelatex failed on pass {tag} for {main}: "
84             + " | ".join(errors[:5])
85             + "\n--- log tail ---\n"
86             + log[-4000:]
87         )
88
89     xelatex("1")
90     aux = workdir / f"{jobname}.aux"
91     if bibtex and aux.exists():
92         aux_text = aux.read_text(encoding="utf-8", errors="replace")
93         if "\\citation" in aux_text and "\\bibdata" in aux_text:
94             proc = subprocess.run(
95                 ["bibtex", jobname], cwd=workdir, capture_output=True, text=True, env=env, timeout=timeout, check=False
96             )
97             runs.append({"pass": "bibtex", "rc": proc.returncode, "tail": proc.stdout[-1500:]})
98     for i in range(2, passes + 1):
99         xelatex(str(i))
100     log_text = log_path.read_text(encoding="utf-8", errors="replace")
101     return {
102         "pdf": workdir / f"{jobname}.pdf",
103         "log": log_path,
104         "aux": aux,
105         "runs": runs,
106         "summary": summarize_log(log_text),
107     }
108
109
110 def tex_escape(text: str) -> str:
111     replacements = {
112         "\\": r"\textbackslash{}",
113         "&": r"\&",
114         "%": r"%",
115         "$": r"\$",
116         "#": r"\#",
117         "_": r"\_",
118         "{": r"\{",
119         "}": r"\}",
120         "~": r"\textasciitilde{}",
121         "^": r"\textasciicircum{}",
122     }
123     return "".join(replacements.get(ch, ch) for ch in text)
124
125
126 def tex_path(path: str) -> str:
127     """Typeset a file path so that it can break across lines (hyperref/xurl); falls back to \\texttt."""
128     if any(ch in path for ch in "{}%#\\") or not path.isascii():
129         return f"\\texttt{{{tex_escape(path)}}}" # CJK names need the CJK mono font, not the url font
130     return f"\\nolinkurl{{{path}}}"

```

forge/xlsx.py

```
1 """Workbook I/O: reading attachments, filling result templates, checking result contracts."""
2
3 from __future__ import annotations
4
5 import math
6 from collections.abc import Iterable, Sequence
7 from dataclasses import dataclass
8 from pathlib import Path
9 from typing import Any
10
11 import openpyxl
12 import pandas as pd
13
14 BIG_SHEET_CELLS = 300_000
15
16
17 def read_sheets(path: Path, header: int | None = 0) -> dict[str, pd.DataFrame]:
18     with pd.ExcelFile(path) as xf:
19         return {str(name): xf.parse(name, header=header) for name in xf.sheet_names}
20
21
22 def sheet_headers(path: Path) -> dict[str, list[Any]]:
23     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
24     out: dict[str, list[Any]] = {}
25     for ws in wb.worksheets:
26         row = list(next(ws.iter_rows(min_row=1, max_row=1, values_only=True), ()))
27         while row and row[-1] is None:
28             row.pop()
29         out[str(ws.title)] = row
30     wb.close()
31     return out
32
33
34 def _clean(value: Any, decimals: int | None) -> Any:
35     if hasattr(value, "item") and not isinstance(value, str | bytes):
36         try:
37             value = value.item()
38         except (ValueError, TypeError):
39             pass
40     if isinstance(value, float):
41         if math.isnan(value) or math.isinf(value):
42             return None
43         return round(value, decimals) if decimals is not None else value
44     return value
45
46
47 def write_result(
48     template: Path,
49     out: Path,
50     sheets: dict[str, tuple[Sequence[Any], Iterable[Sequence[Any]]]],
51     *,
52     decimals: int | None = 4,
53     float_format: str = "0.0000",
54 ) -> dict[str, Any]:
55     """Fill a competition result template.
56
57     ``sheets`` maps sheet name -> (header_row, rows). The template's A1 text is always kept;
58     header cells after A1 are taken from ``header_row[1:]``. Sheets not mentioned keep their
59     template content. Floats are rounded to ``decimals`` and formatted with ``float_format``.
60     Very large sheets fall back to openpyxl's write-only mode (no per-cell number format).
61     """
62     template_wb = openpyxl.load_workbook(template)
63     names = list(template_wb.sheetnames)
64     unknown = [n for n in sheets if n not in names]
65     if unknown:
66         raise KeyError(f"sheets {unknown} not in template {template.name} (has {names})")
67     a1 = {ws.title: ws.cell(row=1, column=1).value for ws in template_wb.worksheets}
68     materialised = {name: (list(header), [list(r) for r in rows]) for name, (header, rows) in sheets.items()}
```

```

69     cells = sum(max(len(h), 1) * (len(rows) + 1) for h, rows in materialised.values())
70     stats: dict[str, Any] = {}
71     out.parent.mkdir(parents=True, exist_ok=True)
72
73     if cells > BIG_SHEET_CELLS:
74         wb = openpyxl.Workbook(write_only=True)
75         for name in names:
76             ws = wb.create_sheet(name)
77             if name in materialised:
78                 header, rows = materialised[name]
79                 full_header = [a1[name], *header[1:]]
80                 ws.append(full_header)
81                 for row in rows:
82                     ws.append([_clean(v, decimals) for v in row])
83                 stats[name] = {"rows": len(rows), "cols": len(full_header)}
84             else:
85                 for tpl_row in template_wb[name].iter_rows(values_only=True):
86                     ws.append(list(tpl_row))
87         wb.save(out)
88     else:
89         wb = template_wb
90         for name, (header, rows) in materialised.items():
91             index = wb.sheetnames.index(name)
92             wb.remove(wb[name])
93             ws = wb.create_sheet(name, index) # fresh sheet: no stale template cells or dimensions
94             full_header = [a1[name], *header[1:]]
95             for j, value in enumerate(full_header, start=1):
96                 ws.cell(row=1, column=j, value=_clean(value, None))
97             for i, row in enumerate(rows, start=2):
98                 for j, value in enumerate(row, start=1):
99                     cleaned = _clean(value, decimals)
100                     cell = ws.cell(row=i, column=j, value=cleaned)
101                     if isinstance(cleaned, float) and j > 1:
102                         cell.number_format = float_format
103             stats[name] = {"rows": len(rows), "cols": len(full_header)}
104         wb.save(out)
105     return {"path": str(out), "sheets": stats, "write_only": cells > BIG_SHEET_CELLS, "cells": cells}
106
107
108 @dataclass(frozen=True)
109 class SheetContract:
110     name: str
111     min_rows: int = 1 # data rows (excluding the header)
112     max_rows: int | None = None
113     header_len: int | None = None # exact number of header cells, if known
114     numeric_from_col: int | None = None # 0-based; every data cell from this column must be numeric
115     allow_blank: bool = False # blank numeric cells permitted (e.g. moving-boundary tables)
116     decimals: int | None = 4 # numeric cells must carry at most this many decimals
117     text_columns: tuple[int, ...] = () # 0-based columns that must be non-empty text
118
119
120 @dataclass(frozen=True)
121 class WorkbookContract:
122     file: str
123     template: str
124     sheets: tuple[SheetContract, ...]
125
126
127 def check_workbook(path: Path, contract: WorkbookContract, template: Path) -> dict[str, Any]:
128     errors: list[str] = []
129     if not path.exists():
130         return {"file": contract.file, "ok": False, "errors": ["file missing"], "sheets": {}}
131     try:
132         tpl_headers = sheet_headers(template)
133     except Exception as exc:
134         tpl_headers = {}
135         errors.append(f"template unreadable: {exc!r}")
136     wb = openpyxl.load_workbook(path, read_only=True, data_only=True)
137     report: dict[str, Any] = {}
138     names = wb.sheetnames
139     if tpl_headers and names != list(tpl_headers):

```

```

140     errors.append(f"sheet names {names} differ from template {list(tpl_headers)}")
141 for sc in contract.sheets:
142     if sc.name not in names:
143         errors.append(f"sheet {sc.name!r} missing")
144         continue
145     ws = wb[sc.name]
146     rows = ws.iter_rows(values_only=True)
147     first = next(rows, None)
148     header: list[Any] = list(first) if first is not None else []
149     while header and header[-1] is None:
150         header.pop()
151     tpl = tpl_headers.get(sc.name)
152     if tpl and header and header[0] != tpl[0]:
153         errors.append(f"{sc.name}: A1 {header[0]!r} differs from template {tpl[0]!r}")
154     if sc.header_len is not None and len(header) != sc.header_len:
155         errors.append(f"{sc.name}: header has {len(header)} cells, expected {sc.header_len}")
156     n = 0
157     bad_numeric = blank = bad_decimals = bad_text = 0
158     for row in rows:
159         values = list(row)
160         if all(v is None or (isinstance(v, str) and not v.strip()) for v in values):
161             continue
162         n += 1
163         if sc.numeric_from_col is not None:
164             for v in values[sc.numeric_from_col : len(header) or None]:
165                 if v is None or (isinstance(v, str) and not v.strip()):
166                     blank += 1
167                 elif not isinstance(v, int | float) or isinstance(v, bool):
168                     bad_numeric += 1
169                 elif sc.decimals is not None and isinstance(v, float) and abs(round(v, sc.decimals) - v) > 1e-12:
170                     bad_decimals += 1
171         for j in sc.text_columns:
172             v = values[j] if j < len(values) else None
173             if v is None or not str(v).strip():
174                 bad_text += 1
175     if n < sc.min_rows:
176         errors.append(f"{sc.name}: {n} data rows < {sc.min_rows}")
177     if sc.max_rows is not None and n > sc.max_rows:
178         errors.append(f"{sc.name}: {n} data rows > {sc.max_rows}")
179     if bad_numeric:
180         errors.append(f"{sc.name}: {bad_numeric} non-numeric cells in numeric region")
181     if blank and not sc.allow_blank:
182         errors.append(f"{sc.name}: {blank} blank cells in numeric region")
183     if bad_decimals:
184         errors.append(f"{sc.name}: {bad_decimals} cells with more than {sc.decimals} decimals")
185     if bad_text:
186         errors.append(f"{sc.name}: {bad_text} empty cells in required text columns")
187     report[sc.name] = {
188         "rows": n,
189         "header": [str(h) for h in header[:6]] + (["..."] if len(header) > 6 else []),
190         "blank": blank,
191     }
192 wb.close()
193 return {"file": contract.file, "ok": not errors, "errors": errors, "sheets": report}

```

pipelines/__init__.py

pipelines/common/__init__.py

pipelines/common/stages.py

```
1 """Problem-agnostic stages: ingest, lint, test, paper, qa, package, release."""
2
3 from __future__ import annotations
4
5 import importlib
6 import json
7 import os
8 import re
9 import shlex
10 import shutil
11 import subprocess
12 import xml.etree.ElementTree as ET
13 from datetime import UTC, datetime
14 from pathlib import Path
15 from typing import Any
16
17 from forge import packaging, pdfqa, xlsx
18 from forge.context import StageContext
19 from forge.hashing import hash_tree, sha256_file
20 from forge.runner import stage
21 from forge.tex import aux_page, compile_tex, tex_escape, tex_path
22
23 PLANNED_STAGE_FILES = {
24     "lint": ["ruff_check.txt", "ruff_format.txt", "mypy.txt", "lint_report.json", "manifest.json", "events.jsonl"],
25     "test": ["junit.xml", "coverage.json", "pytest.txt", "test_report.json", "manifest.json", "events.jsonl"],
26     "qa": ["qa_report.json", "manifest.json", "events.jsonl"],
27 }
28 _BOOT_STAGES = ("ingest", "validate", "fmt")
29 MEMBER_DESCRIPTIONS = (
30     (r"~forge/plotting\.py$", "绘图规范 (矢量 PDF + 预览 PNG、色板、字体)"),
31     (r"~forge/xlsx\.py$", "从组委会模板生成结果工作簿"),
32     (r"~forge/runner\.py$", "阶段注册与执行 (依赖、清单、错误捕获)"),
33     (r"~forge/modal_app\.py$", "云端应用定义 (镜像、卷、四种规格的执行函数)"),
34     (r"~forge/", "云端运行时 (阶段执行、清单、契约、排版、打包)"),
35     (r"~pipelines/common/", "通用阶段: 数据接入、门禁、论文、质量检查、打包发布"),
36     (r"~pipelines/d/validators\.py$", "与求解器分离的独立校验器 (按定义展开单元集合求交)"),
37     (r"~pipelines/d/detect\.py$", "问题 1: 成对检测与按频段分桶的扫描线检测"),
38     (r"~pipelines/d/resolve\.py$", "问题 2/4: 动作集合、单元格模型、词典序与加权求解"),
39     (r"~pipelines/d/pack\.py$", "问题 3: 候选放置、集合装填模型与重排构造"),
40     (r"~pipelines/d/report\.py$", "论文图与表的生成"),
41     (r"~pipelines/d/synth\.py$", "微型/合成实例与穷举参照解"),
42     (r"~pipelines/d/stages\.py$", "本题各阶段的编排"),
43     (r"~pipelines/", "本题建模与求解程序"),
44     (r"~tools/run_local\.py$", "不依赖云端的本地驱动脚本"),
45     (r"~tools/", "本地控制面: 提交云端任务、下载、校验散列"),
46     (r"~tests/", "单元 / 性质 / 集成测试"),
47     (r"~configs/hints/", "词典序求解的热启动现任解 (含来源 run 与目标向量)"),
48     (r"~configs/", "运行配置与项目元数据"),
49     (r"~manuscript/", "论文 LaTeX 源码"),
50     (r"~docs/mdr/", "建模决策记录 (题意解释、假设、目标与方法的选择理由)"),
51     (r"~docs/ARCHITECTURE\.md$", "仓库架构与唯一计算面的设计"),
52     (r"~docs/CONTRIBUTING\.md$", "协作与提交规范"),
53     (r"~docs/DATA_NOTES\.md$", "数据口径与附件解析约定"),
54     (r"~docs/ENGINEERING_STANDARD\.md$", "工程标准与门禁定义"),
55     (r"~docs/MODELING_STANDARD\.md$", "建模与写作标准 (审稿清单)"),
56     (r"~docs/RESULTS\.md$", "本轮结果总览与逐项证据"),
57     (r"~docs/RUNBOOK\.md$", "运行手册 (完整命令序列与实测用时)"),
58     (r"~docs/REVIEW_RESPONSE\.md$", "评审意见的逐条处理说明"),
59     (r"~docs/problem_brief\.md$", "题目要点与交付清单"),
60     (r"~docs/", "工程规范、建模标准、决策记录"),
61     (r"/ingest/", "原始数据接入结果 (散列、题目文本、规范化数据)"),
62     (r"/validate/", "数据契约校验报告与数据画像"),
63     (r"/results/", "结果工作簿与求解输出"),
64     (r"/figures/", "论文图 (矢量 PDF 与预览 PNG)"),
65     (r"/tables/", "论文表 (LaTeX 片段与 CSV)"),
66     (r"/lint/", "静态检查报告"),
67     (r"/test/", "测试报告与覆盖率"),
68     (r"/qa/", "论文与结果文件质量门禁报告"),
```

```

69 (r"manifest\.json$", "阶段清单 (输入输出散列、环境、参数)"),
70 (r"events\.jsonl$", "结构化运行日志"),
71 (r"numbers\.json$", "该阶段登记的论文数字宏"),
72 (r"validation_report\.json$", "该阶段结果的独立校验报告"),
73 (r"solver_report\.json$", "求解器逐级证据 (值、证明下界、状态、用时)"),
74 (r"^AI工具使用详情\.pdf$", "AI 工具使用详情"),
75 (r"^AI_USAGE\.md$", "AI 工具使用记录 (源文件)"),
76 (r"^Makefile$", "本地快捷命令 (只提交云端任务)"),
77 (r"^pyproject\.toml$", "项目元数据与静态检查配置"),
78 (r"^README\.md$", "仓库说明与复现入口"),
79 (r"^CHANGELOG\.md$", "变更记录"),
80 (r"^UNLICENSE$", "公有领域声明 (Unlicense)"),
81 (r"^REPRODUCE\.md$", "复现说明"),
82 (r"^MANIFEST\.json$", "支撑材料成员清单与 SHA-256"),
83 (r"\.parquet$", "规范化后的输入数据"),
84 (r"\.json$", "阶段数据产物"),
85 (r"\.csv$", "表格数据 (与 LaTeX 表同源)"),
86 )
87
88
89 def _describe(name: str) -> str:
90     for pattern, text in MEMBER_DESCRIPTIONS:
91         if re.search(pattern, name):
92             return text
93     return ""
94
95
96 def _to_parquet(df: Any, target: Path) -> None:
97     frame = df.copy()
98     frame.columns = [str(c).strip() for c in frame.columns]
99     try:
100         frame.to_parquet(target, index=False)
101     except Exception:
102         for col in frame.columns:
103             if frame[col].dtype == object:
104                 frame[col] = frame[col].map(
105                     lambda v: None if v is None or (isinstance(v, float) and v != v) else str(v)
106                 )
107         frame.to_parquet(target, index=False)
108
109
110 @stage("ingest", description="Hash raw inputs, extract the problem text, convert every attachment sheet to parquet")
111 def ingest(ctx: StageContext) -> dict[str, Any]:
112     import fitz
113
114     document = fitz.open(ctx.problem_pdf)
115     text = "\n\n".join(page.get_text("text", sort=True) for page in document)
116     ctx.out("problem_text.txt").write_text(text, encoding="utf-8")
117     inventory: dict[str, Any] = {
118         "problem_pdf": {
119             "path": ctx.project["problem_pdf"],
120             "sha256": sha256_file(ctx.problem_pdf),
121             "pages": len(document),
122         },
123         "files": [],
124     }
125     for path in sorted(ctx.attachments.rglob("*.xlsx")):
126         if path.name.startswith("-$"):
127             continue
128         rel = path.relative_to(ctx.attachments).as_posix()
129         is_template = path.parent == ctx.templates or ctx.templates in path.parents
130         entry: dict[str, Any] = {
131             "path": rel,
132             "sha256": sha256_file(path),
133             "bytes": path.stat().st_size,
134             "template": is_template,
135             "sheets": [],
136         }
137         for name, df inxlsx.read_sheets(path).items():
138             sheet: dict[str, Any] = {"name": name, "rows": len(df), "columns": [str(c) for c in df.columns]}
139             if not is_template:

```

```

140         safe = re.sub(r"~\w.-+", "_", f"{path.stem}_{name}")
141         target = ctx.out("data", f"{safe}.parquet")
142         _to_parquet(df, target)
143         sheet["parquet"] = target.relative_to(ctx.stage_dir).as_posix()
144         sheet["dtypes"] = {str(c): str(t) for c, t in df.dtypes.items()}
145         sheet["head"] = df.head(5).astype(str).values.tolist()
146         entry["sheets"].append(sheet)
147         inventory["files"].append(entry)
148         ctx.log.info("ingest.file", path=rel, template=is_template, sheets=[s["name"] for s in entry["sheets"]])
149         ctx.write_json("inventory.json", inventory)
150         return {"files": len(inventory["files"]), "problem_chars": len(text)}
151
152
153 @stage("lint", description="Static gates: ruff check, ruff format --check, mypy")
154 def lint(ctx: StageContext) -> dict[str, Any]:
155     returncodes: dict[str, int] = {}
156
157     def run(name: str, cmd: list[str]) -> None:
158         proc = subprocess.run(cmd, cwd=str(ctx.repo), capture_output=True, text=True, check=False)
159         ctx.out(f"{name}.txt").write_text(proc.stdout + ("\n" + proc.stderr if proc.stderr else ""), encoding="utf-8")
160         returncodes[name] = proc.returncode
161         ctx.log.info("lint.tool", tool=name, rc=proc.returncode, tail=(proc.stdout + proc.stderr)[-400:])
162
163     run("ruff_check", ["ruff", "check", "--no-cache", "--output-format", "concise", "."])
164     run("ruff_format", ["ruff", "format", "--check", "--no-cache", "."])
165     run("mypy", ["mypy", "--cache-dir", "/tmp/mypy", *ctx.cfg("gates.mypy_paths", ["forge"])]))
166     required = list(ctx.cfg("gates.lint_required", ["ruff_check", "ruff_format", "mypy"]))
167     passed = all(returncodes.get(name, 1) == 0 for name in required)
168     ctx.write_json("lint_report.json", {"passed": passed, "required": required, "returncodes": returncodes})
169     if not passed:
170         raise RuntimeError(f"lint gate failed: {returncodes}")
171     return {"passed": passed, **returncodes}
172
173
174 @stage("test", description="pytest (unit, property, integration) with JUnit and coverage reports")
175 def test(ctx: StageContext) -> dict[str, Any]:
176     env = {
177         **os.environ,
178         "COVERAGE_FILE": "/tmp/.coverage",
179         "HYPOTHESIS_STORAGE_DIRECTORY": "/tmp/hypothesis",
180         "FORGE_RUN_DIR": str(ctx.run_dir),
181     }
182     cmd = [
183         "python",
184         "-m",
185         "pytest",
186         "tests",
187         "-q",
188         "-p",
189         "no:cacheprovider",
190         "--rootdir",
191         str(ctx.repo),
192         f"--junitxml={ctx.out('junit.xml')}",
193         "--cov=forge",
194         "--cov=pipelines",
195         f"--cov-report=json:{ctx.out('coverage.json')}",
196         "--cov-report=term",
197     ]
198     extra = ctx.param("pytest_args")
199     if extra:
200         cmd += shlex.split(str(extra))
201     proc = subprocess.run(cmd, cwd=str(ctx.repo), capture_output=True, text=True, env=env, check=False)
202     ctx.out("pytest.txt").write_text(proc.stdout + "\n" + proc.stderr, encoding="utf-8")
203     print(proc.stdout[-5000:], flush=True)
204     summary: dict[str, Any] = {"returncode": proc.returncode}
205     junit = ctx.stage_dir / "junit.xml"
206     if junit.exists():
207         root = ET.parse(junit).getroot()
208         suite = root if root.tag == "testsuite" else root.find("testsuite")
209         if suite is not None:
210             summary.update({k: int(suite.get(k, 0)) for k in ("tests", "failures", "errors", "skipped")})

```

```

211 coverage = ctx.stage_dir / "coverage.json"
212 if coverage.exists():
213     summary["coverage_percent"] = round(json.loads(coverage.read_text())["totals"]["percent_covered"], 2)
214 ctx.write_json("test_report.json", summary)
215 if proc.returncode != 0:
216     raise RuntimeError(f"tests failed: {summary}\n{proc.stdout[-2500:]}")
217 return summary
218
219
220 def _support_members(ctx: StageContext, *, planned: bool) -> list[tuple[str, Path]]:
221     members = packaging.code_members(ctx.repo, ctx.cfg("package.code_globs", []))
222     for name in ctx.cfg("package.stages", []):
223         directory = ctx.run_dir / name
224         if (directory / "manifest.json").exists():
225             members += packaging.run_members(ctx.run_dir, [name])
226         elif planned and name in PLANNED_STAGE_FILES:
227             members += [(f"runs/{ctx.run_id}/{name}/{f}", directory / f) for f in PLANNED_STAGE_FILES[name]]
228     members.append(("AI工具使用详情.pdf", ctx.run_dir / "paper" / "AI工具使用详情.pdf"))
229     members.append(("REPRODUCE.md", ctx.run_dir / "package" / "REPRODUCE.md"))
230     members.append(("MANIFEST.json", ctx.run_dir / "package" / "MANIFEST.json"))
231     return sorted(dict(members).items())
232
233
234 def _load_numbers(run_dir: Path) -> tuple[dict[str, str], list[str]]:
235     numbers: dict[str, str] = {}
236     conflicts: list[str] = []
237     for path in sorted(run_dir.glob("*/numbers.json")):
238         for key, value in json.loads(path.read_text(encoding="utf-8")).items():
239             if key in numbers and numbers[key] != value:
240                 conflicts.append(f"{key}: {numbers[key]!r} vs {value!r} ({path.parent.name})")
241             numbers[key] = value
242     return numbers, conflicts
243
244
245 def _stage_rows(ctx: StageContext) -> list[dict[str, Any]]:
246     rows = []
247     for manifest in sorted(ctx.run_dir.glob("*/manifest.json")):
248         data = json.loads(manifest.read_text(encoding="utf-8"))
249         rows.append(
250             {
251                 "stage": data["stage"],
252                 "status": data["status"],
253                 "duration_s": data.get("duration_s"),
254                 "outputs_digest": data.get("outputs_digest", ""),
255                 "modal_task_id": data.get("runtime", {}).get("modal_task_id"),
256             }
257         )
258     return rows
259
260
261 @stage(
262     "paper",
263     description="Assemble generated inputs (numbers, tables, figures, code, support list) and compile with XeLaTeX",
264 )
265 def paper(ctx: StageContext) -> dict[str, Any]:
266     sources = list(ctx.cfg("paper.sources", []))
267     for source in sources:
268         ctx.dep(source)
269     build = Path("/tmp/build")
270     shutil.rmtree(build, ignore_errors=True)
271     shutil.copytree(
272         ctx.repo / "manuscript", build, ignore=shutil.ignore_patterns("generated", "*.aux", "*.log", "*.pdf")
273     )
274     generated = build / "generated"
275     (generated / "tables").mkdir(parents=True, exist_ok=True)
276     (build / "figures").mkdir(exist_ok=True)
277
278     numbers, conflicts = _load_numbers(ctx.run_dir)
279     for conflict in conflicts:
280         ctx.log.warn("paper.number_conflict", detail=conflict)
281     lines = ["% generated by forge — do not edit"] + [

```

```

282         f"\newcommand{{\val{key}}}{{\tex_escape(value)}}}" for key, value in sorted(numbers.items())
283     ]
284     (generated / "numbers.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")
285
286     n_tables = n_figures = 0
287     for source in sources:
288         directory = ctx.run_dir / source
289         for table in sorted((directory / "tables").glob("*.tex")) if (directory / "tables").is_dir() else []:
290             shutil.copy2(table, generated / "tables" / table.name)
291             n_tables += 1
292         for figure in sorted((directory / "figures").glob("*")) if (directory / "figures").is_dir() else []:
293             if figure.suffix.lower() in {".pdf", ".png"}:
294                 shutil.copy2(figure, build / "figures" / figure.name)
295                 n_figures += 1
296
297     code_ref = ctx.params.get("code_ref") or {}
298     meta = [
299         f"\newcommand{{\RunId}}{{\tex_escape(ctx.run_id)}}",
300         f"\newcommand{{\CodeSha}}{{\tex_escape(str(code_ref.get('git_sha', '')))}}",
301         f"\newcommand{{\BuildDate}}{{\datetime.now(UTC).strftime('%Y-%m-%d')}}",
302     ]
303     (generated / "meta.tex").write_text("\n".join(meta) + "\n", encoding="utf-8")
304
305     files = packaging.code_members(ctx.repo, ctx.cfg("paper.code_globs", []))
306     listing = ["% generated by forge — complete source listing"]
307     for rel, src in files:
308         target = build / "code" / rel
309         target.parent.mkdir(parents=True, exist_ok=True)
310         shutil.copy2(src, target)
311         language = {"py": "Python", ".tex": "[LaTeX]TeX"}.get(src.suffix, "{")
312         listing.append(f"\needspace{{6\baselineskip}}\subsection*{{\tex_path(rel)}}")
313         listing.append(f"\lstinputlisting[language={language}][{code}/{rel}]")
314     (generated / "code_listing.tex").write_text("\n".join(listing) + "\n", encoding="utf-8")
315
316     members = _support_members(ctx, planned=True)
317     rows = [
318         "% generated by forge — support material file list",
319         "\begin{longtable}{{@{}p{0.64\linewidth}p{0.32\linewidth}@{}}}",
320         "\toprule 文件 & 说明 \\\midrule \\\endhead \\\bottomrule \\\endfoot",
321     ]
322     rows += [f"{{\tex_path(name)}} & {{\tex_escape(_describe(name))}} \\\\" for name, _ in members]
323     rows.append("\end{longtable}")
324     (generated / "support_files.tex").write_text("\n".join(rows) + "\n", encoding="utf-8")
325     ctx.write_json("support_members.json", [name for name, _ in members])
326
327     stage_rows = _stage_rows(ctx)
328     repro = [
329         "% generated by forge — reproducibility record",
330         "\begin{longtable}{{@{}llrl@{}}}",
331         "\toprule 阶段 & 状态 & 用时/s & 输出摘要 (SHA-256 前 12 位) \\\midrule \\\endhead \\\bottomrule \\\endfoot",
332     ]
333     repro += [
334         f"\texttt{{\tex_escape(r['stage'])}} & {{r['status']}} & {{r['duration_s']}} or 0:.1f & "
335         f"\texttt{{r['outputs_digest'][:12]}} \\\\"
336         for r in stage_rows
337     ]
338     repro.append("\end{longtable}")
339     (generated / "repro_table.tex").write_text("\n".join(repro) + "\n", encoding="utf-8")
340
341     result = compile_tex(build, str(ctx.cfg("paper.main", "main.tex")), "main", passes=3, bibtex=True)
342     shutil.copy2(result["pdf"], ctx.out("main.pdf"))
343     shutil.copy2(result["log"], ctx.out("main.log"))
344     if result["aux"].exists():
345         shutil.copy2(result["aux"], ctx.out("main.aux"))
346     body_end = aux_page(result["aux"], "page:body-end")
347     abstract_end = aux_page(result["aux"], "page:abstract-end")
348     ai_summary = None
349     if (build / "ai_usage.tex").exists():
350         ai = compile_tex(build, "ai_usage.tex", "ai_usage", passes=2, bibtex=False)
351         shutil.copy2(ai["pdf"], ctx.out("AI工具使用详情.pdf"))
352         ai_summary = ai["summary"]

```

```

353 info = pdfqa.inspect_pdf(ctx.out("main.pdf"), with_text=False)
354 body_pages = (body_end - abstract_end) if body_end and abstract_end else None
355 report = {
356     "pages": info["pages"],
357     "all_a4": info["all_a4"],
358     "abstract_end_page": abstract_end,
359     "body_end_page": body_end,
360     "body_pages": body_pages,
361     "log": result["summary"],
362     "unembedded_fonts": info["unembedded_fonts"],
363     "fonts": info["fonts"],
364     "numbers": len(numbers),
365     "number_conflicts": conflicts,
366     "tables": n_tables,
367     "figures": n_figures,
368     "code_files": len(files),
369     "support_members": len(members),
370     "ai_usage_log": ai_summary,
371     "stages": stage_rows,
372 }
373 ctx.write_json("paper_report.json", report)
374 summary = result["summary"]
375 return {
376     "pages": info["pages"],
377     "body_pages": body_pages,
378     "undefined_refs": len(summary["undefined_references"]) + len(summary["undefined_citations"]),
379     "overfull_max_pt": summary["overfull_max_pt"],
380     "missing_chars": summary["missing_characters"],
381     "figures": n_figures,
382     "tables": n_tables,
383 }
384
385
386 @stage("qa", deps=("paper",), description="Quality gates over the compiled paper and the result workbooks")
387 def qa(ctx: StageContext) -> dict[str, Any]:
388     for required in ctx.cfg("qa.required_stages", ["lint", "test"]):
389         ctx.dep(required)
390     paper_dir = ctx.dep("paper")
391     report = json.loads((paper_dir / "paper_report.json").read_text(encoding="utf-8"))
392     checks: list[dict[str, Any]] = []
393
394     def check(name: str, ok: bool, detail: Any = "") -> None:
395         checks.append({"name": name, "ok": bool(ok), "detail": detail})
396         ctx.log.info("qa.check", name=name, ok=bool(ok), detail=detail)
397
398     max_body = int(ctx.cfg("paper.max_body_pages", 30))
399     check("abstract_single_page", report["abstract_end_page"] == 1, report["abstract_end_page"])
400     check(
401         "body_pages_within_limit",
402         report["body_pages"] is not None and report["body_pages"] <= max_body,
403         f"{report['body_pages']} <= {max_body}",
404     )
405     log = report["log"]
406     check(
407         "no_undefined_references",
408         not log["undefined_references"] and not log["undefined_citations"],
409         {"refs": log["undefined_references"][:10], "cites": log["undefined_citations"][:10]},
410     )
411     check("no_tex_errors", not log["errors"], log["errors"][:5])
412     check("fonts_embedded", not report["unembedded_fonts"], report["unembedded_fonts"])
413     check("a4_pages", report["all_a4"])
414     check(
415         "overfull_within_limit",
416         log["overfull_max_pt"] <= float(ctx.cfg("paper.max_overfull_pt", 20.0)),
417         f"max {log['overfull_max_pt']} pt, count {log['overfull_count']}",
418     )
419     check("no_missing_characters", log["missing_characters"] == 0, log["missing_character_samples"])
420     check("no_number_conflicts", not report["number_conflicts"], report["number_conflicts"][:5])
421     check("ai_usage_pdf_present", (paper_dir / "AI工具使用详情.pdf").exists())
422
423     info = pdfqa.inspect_pdf(paper_dir / "main.pdf", with_text=True)

```

```

424 deny_hits = pdfqa.scan_strings(info["texts"], ctx.cfg("qa.identity_denylist", []))
425 watch_hits = pdfqa.scan_strings(info["texts"], ctx.cfg("qa.identity_watchlist", []))
426 check("identity_denylist_clean", not deny_hits, deny_hits[:5])
427 check("paper_size_under_20mb", (paper_dir / "main.pdf").stat().st_size <= 20 * 1024 * 1024)
428
429 results_reports: list[dict[str, Any]] = []
430 results_stage = str(ctx.cfg("qa.results_stage", "results"))
431 require_results = bool(ctx.param("require_results", ctx.cfg("qa.require_results", True)))
432 if ctx.has_stage(results_stage):
433     contracts = importlib.import_module(f"pipelines.{ctx.project['package']}.contracts")
434     results_dir = ctx.run_dir / results_stage
435     for contract in contracts.RESULT_CONTRACTS:
436         rep = xlsx.check_workbook(results_dir / contract.file, contract, ctx.templates / contract.template)
437         results_reports.append(rep)
438         check(f"result:{contract.file}", rep["ok"], rep["errors"][:5])
439 else:
440     check("results_stage_present", not require_results, f"stage '{results_stage}' not completed in this run")
441
442 passed = all(c["ok"] for c in checks)
443 ctx.write_json(
444     "qa_report.json",
445     {
446         "passed": passed,
447         "checks": checks,
448         "identity_watchlist_hits": watch_hits,
449         "results": results_reports,
450         "paper": {k: v for k, v in report.items() if k != "fonts"},
451     },
452 )
453 if not passed:
454     raise RuntimeError("QA gate failed: " + ", ".join(c["name"] for c in checks if not c["ok"]))
455 return {"passed": passed, "checks": len(checks), "watchlist_hits": len(watch_hits)}
456
457
458 def _reproduce_md(ctx: StageContext) -> str:
459     rows = _stage_rows(ctx)
460     code_ref = ctx.params.get("code_ref") or {}
461     manifest = json.loads((ctx.run_dir / "ingest" / "manifest.json").read_text(encoding="utf-8"))
462     packages = manifest.get("runtime", {}).get("packages", {})
463     lines = [
464         f"# 复现说明 (run `{ctx.run_id}`)",
465         "",
466         "本目录内所有结果均由 Modal 云端按下列阶段生成; 本地只提交任务、下载和核对 SHA-256。",
467         "",
468         f"- 代码版本: git `{code_ref.get('git_sha', '')}` (分支 `{code_ref.get('branch', '')}`)",
469         f"- 运行环境: Python {manifest.get('runtime', {}).get('python', '')}, "
470         f"{manifest.get('runtime', {}).get('platform', '')}",
471         f"- 依赖版本: " + ", ".join(f"{k}=={v}" for k, v in sorted(packages.items())),
472         "",
473         "## 阶段",
474         "",
475         "| 阶段 | 状态 | 用时 (s) | 输出摘要 | Modal 任务 |",
476         "|---|---|---:|---|---|",
477     ]
478     lines += [
479         f"| {r['stage']} | {r['status']} | {r['duration_s'] or 0:.1f} | `{r['outputs_digest'][:16]}` | "
480         f"`{r['modal_task_id'] or ''}` |"
481         for r in rows
482     ]
483     lines += [
484         "",
485         "## 复现命令",
486         "",
487         "### A. 云端 (本文使用的方式, 需要 Modal 账号)",
488         "",
489         "```bash",
490         "python3 tools/cli.py provision",
491         "python3 tools/cli.py run ingest,validate --new-run",
492         *[f"python3 tools/cli.py run {r['stage']} --size medium" for r in rows if r["stage"] not in _BOOT_STAGES],
493         "python3 tools/cli.py release --version <tag>",
494         "```",
495     ]

```

```

495     """
496     "各阶段的确切参数（线程数、逐级时限）见每个阶段目录下 `manifest.json` 的 `params` 字段，",
497     "以及 `docs/RUNBOOK.md` 第 7 节。",
498     """
499     "### B. 本地（不需要任何云端账号）",
500     """
501     "```bash",
502     "pip install pandas numpy openpyxl pyarrow networkx matplotlib ortools highspy",
503     "python3 tools/run_local.py --quick          # 数分钟，得到 result1-4.xlsx、图与表",
504     "python3 tools/run_local.py                  # 使用默认（较大）预算",
505     "```",
506     """
507     "本地驱动脚本调用的是同一批阶段函数，只是把云端卷换成本地目录；",
508     "附件需按原样放在 `附件/` 下。论文排版阶段（paper/qa/package/release）需要 TeX Live 与中文字体，",
509     "不在本地默认序列中。",
510     """
511     "每个阶段目录下的 `manifest.json` 记录输入、输出散列、参数与环境；`events.jsonl` 为结构化日志。",
512     """
513 ]
514 return "\n".join(lines)
515
516
517 @stage("package", deps=("qa",), description="Build the deterministic support-materials zip and its member manifest")
518 def package(ctx: StageContext) -> dict[str, Any]:
519     planned = set(json.loads((ctx.dep("paper") / "support_members.json").read_text(encoding="utf-8")))
520     ctx.out("REPRODUCE.md").write_text(_reproduce_md(ctx), encoding="utf-8")
521     members = [(name, path) for name, path in _support_members(ctx, planned=False) if name != "MANIFEST.json"]
522     missing = [name for name, path in members if not path.exists()]
523     if missing:
524         raise RuntimeError(f"support members missing on disk: {missing[:10]}")
525     actual = {name for name, _ in members} | {"MANIFEST.json"}
526     drift = sorted(actual ^ planned)
527     if drift and not ctx.param("allow_member_drift"):
528         raise RuntimeError(f"support material members differ from the list printed in the paper: {drift[:20]}")
529     manifest = {
530         "run_id": ctx.run_id,
531         "created_at": datetime.now(UTC).isoformat(timespec="seconds"),
532         "members": [
533             {"path": name, "sha256": sha256_file(path), "bytes": path.stat().st_size} for name, path in members
534         ],
535     }
536     manifest_path = ctx.out("MANIFEST.json")
537     manifest_path.write_text(json.dumps(manifest, ensure_ascii=False, indent=2), encoding="utf-8")
538     members.append(("MANIFEST.json", manifest_path))
539     zip_info = packaging.build_zip(members, ctx.out("支撑材料.zip"), comment=f"forge run {ctx.run_id}")
540     limit = int(ctx.cfg("package.max_bytes", 20 * 1024 * 1024))
541     ctx.write_json("package_report.json", {**zip_info, "limit_bytes": limit, "drift": drift})
542     if zip_info["bytes"] > limit:
543         raise RuntimeError(f"support zip {zip_info['bytes']} bytes exceeds limit {limit}")
544     return {"members": zip_info["members"], "bytes": zip_info["bytes"], "drift": len(drift)}
545
546
547 @stage(
548     "release",
549     deps=("package",),
550     description="Assemble deliverables (paper, result workbooks, support zip) with SHA-256 manifest",
551 )
552 def release(ctx: StageContext) -> dict[str, Any]:
553     qa_report = json.loads((ctx.dep("qa") / "qa_report.json").read_text(encoding="utf-8"))
554     if not qa_report["passed"] and not ctx.param("allow_failed_qa"):
555         raise RuntimeError("release refused: QA gate did not pass")
556     paper_dir = ctx.dep("paper")
557     files: dict[str, Path] = {str(ctx.cfg("release.paper_name", "论文.pdf")): paper_dir / "main.pdf"}
558     results_stage = str(ctx.cfg("qa.results_stage", "results"))
559     if ctx.has_stage(results_stage):
560         for name in ctx.project.get("result_files", []):
561             files[name] = ctx.run_dir / results_stage / name
562     elif not ctx.param("allow_missing_results"):
563         raise RuntimeError("release refused: results stage missing")
564     files["支撑材料.zip"] = ctx.dep("package") / "支撑材料.zip"
565     files["AI工具使用详情.pdf"] = paper_dir / "AI工具使用详情.pdf"

```

```

566 manifest: dict[str, Any] = {
567     "run_id": ctx.run_id,
568     "code_ref": ctx.params.get("code_ref"),
569     "files": {},
570     "qa_passed": qa_report["passed"],
571     "created_at": datetime.now(UTC).isoformat(timespec="seconds"),
572 }
573 for name, source in files.items():
574     if not source.exists():
575         raise RuntimeError(f"release file missing: {name} ({source})")
576     target = ctx.out(name)
577     shutil.copy2(source, target)
578     manifest["files"][name] = {"sha256": sha256_file(target), "bytes": target.stat().st_size}
579 notes = [
580     f"# 发布说明 — run {ctx.run_id}",
581     "",
582     f"- 代码: `{(ctx.params.get('code_ref') or {}).get('git_sha', '')}`",
583     f"- QA: {'通过' if qa_report['passed'] else '未通过 (强制发布)'}",
584     "",
585     "| 文件 | 字节 | SHA-256 |",
586     "|---|---:|---|",
587 ]
588 notes += [f"| {n} | {m['bytes']} | `{m['sha256']}`" for n, m in manifest["files"].items()]
589 ctx.out("RELEASE_NOTES.md").write_text("\n".join(notes) + "\n", encoding="utf-8")
590 ctx.write_json("release_manifest.json", manifest)
591 return {
592     "files": len(files),
593     "paper_bytes": manifest["files"][str(ctx.cfg("release.paper_name", "论文.pdf"))]["bytes"],
594 }
595
596
597 FMT_IGNORE = (".git", "artifacts", "releases", "__pycache__", ".forge", ".DS_Store", ".mypy_cache", ".ruff_cache")
598
599
600 @stage("fmt", description="ruff --fix and ruff format in the cloud; changed files are exported for a local copy-back")
601 def fmt(ctx: StageContext) -> dict[str, Any]:
602     work = Path("/tmp/fmt")
603     shutil.rmtree(work, ignore_errors=True)
604     shutil.copytree(ctx.repo, work, ignore=shutil.ignore_patterns(*FMT_IGNORE, "*.pyc"))
605     before = hash_tree(work, ignore=FMT_IGNORE)
606     fix = subprocess.run(
607         ["ruff", "check", "--fix", "--no-cache", "--output-format", "concise", "."],
608         cwd=work,
609         capture_output=True,
610         text=True,
611         check=False,
612     )
613     form = subprocess.run(["ruff", "format", "--no-cache", "."], cwd=work, capture_output=True, text=True, check=False)
614     after = hash_tree(work, ignore=FMT_IGNORE)
615     changed = sorted(rel for rel in after if rel not in before or after[rel]["sha256"] != before[rel]["sha256"])
616     for rel in changed:
617         shutil.copy2(work / rel, ctx.out("files", rel))
618     ctx.out("ruff_fix.txt").write_text(fix.stdout + fix.stderr, encoding="utf-8")
619     ctx.out("ruff_format.txt").write_text(form.stdout + form.stderr, encoding="utf-8")
620     ctx.write_json("fmt_report.json", {"changed": changed, "remaining": fix.stdout.strip().splitlines()[-1:]})
621     ctx.log.info("fmt.done", changed=len(changed), remaining=fix.stdout.strip().splitlines()[-1:])
622     return {"changed": len(changed)}

```

pipelines/common/validation.py

```

1 """Shared implementation of the ``validate`` stage (input data contracts + profiling)."""
2
3 from __future__ import annotations
4
5 from typing import Any
6
7 import pandas as pd
8

```

```

9 from forge.context import StageContext
10 from forge.contracts import FrameContract, normalise_columns, validate_frame
11
12
13 def run_validation(ctx: StageContext, contracts: dict[str, FrameContract]) -> dict[str, Any]:
14     """Validate every ingested parquet named in ``contracts`` (key = parquet stem)."""
15     data_dir = ctx.dep("ingest") / "data"
16     reports: list[dict[str, Any]] = []
17     profile: dict[str, Any] = {}
18     for stem, contract in contracts.items():
19         path = data_dir / f"{stem}.parquet"
20         if not path.exists():
21             reports.append({"contract": contract.name, "ok": False, "errors": [f"parquet missing: {stem}"], "rows": 0})
22             continue
23         df = normalise_columns(pd.read_parquet(path))
24         report = validate_frame(df, contract)
25         report["parquet"] = path.name
26         reports.append(report)
27         desc = df.describe(include="all").astype(str).to_dict()
28         profile[stem] = {
29             "rows": len(df),
30             "columns": list(df.columns),
31             "describe": desc,
32             "nulls": {c: int(df[c].isna().sum()) for c in df.columns},
33         }
34         ctx.log.info(
35             "validate.frame", contract=contract.name, ok=report["ok"], rows=report["rows"], errors=report["errors"][:5]
36         )
37     ok = all(r["ok"] for r in reports)
38     ctx.write_json("validation_report.json", {"ok": ok, "frames": reports})
39     ctx.write_json("data_profile.json", profile)
40     if not ok:
41         failed = [r["contract"] for r in reports if not r["ok"]]
42         raise RuntimeError(f"input contracts violated: {failed}")
43     return {"ok": ok, "frames": len(reports)}

```

pipelines/d/__init__.py

pipelines/d/contracts.py

```

1 """Data contracts for Problem D (time-frequency conflict detection and resolution)."""
2
3 from __future__ import annotations
4
5 import re
6
7 import pandas as pd
8
9 from forge.contracts import Column, FrameContract
10 from forge.xlsx import SheetContract, WorkbookContract
11
12 INTERVAL = re.compile(r"^\s*(\d+),(\d+)\s*$")
13
14
15 def _intervals_well_formed(df: pd.DataFrame) -> list[str]:
16     errors: list[str] = []
17     for col, upper in (("频段区间", 100), ("时间区间", None)):
18         for value in df[col].astype(str):
19             match = INTERVAL.match(value.strip())
20             if not match:
21                 errors.append(f"{col}: malformed interval {value!r}")
22                 continue
23             lo, hi = int(match.group(1)), int(match.group(2))
24             if not lo < hi:

```

```

25         errors.append(f"{col}: empty interval {value!r}")
26     if upper is not None and hi > upper:
27         errors.append(f"{col}: {value!r} exceeds {upper} bands")
28     return errors[:20]
29
30
31 INPUT_CONTRACTS: dict[str, FrameContract] = {
32     "附件1_Sheet1": FrameContract(
33         name="D.附件1.用频计划",
34         columns=(
35             Column("用频装备编号", "str", regex=r"[ABC]\d{3}", unique=True),
36             Column("频段区间", "str", regex=r"[\d+, \d+\)"),
37             Column("时间区间", "str", regex=r"[\d+, \d+\)"),
38             Column("间隔时长", "int", min=1),
39             Column("使用次数", "int", min=1),
40         ),
41         min_rows=150,
42         max_rows=150,
43         checks=(intervals_well_formed,),
44     ),
45 }
46
47 RESULT_CONTRACTS: list[WorkbookContract] = [
48     WorkbookContract(
49         "result1.xlsx", "result1.xlsx", (SheetContract("Sheet1", min_rows=1, header_len=3, text_columns=(1, 2)),)
50     ),
51     WorkbookContract(
52         "result2.xlsx", "result2.xlsx", (SheetContract("Sheet1", min_rows=1, header_len=4, text_columns=(0,)),)
53     ),
54     WorkbookContract("result3.xlsx", "result3.xlsx", (SheetContract("Sheet1", min_rows=0, header_len=3),)),
55     WorkbookContract(
56         "result4.xlsx", "result4.xlsx", (SheetContract("Sheet1", min_rows=1, header_len=5, text_columns=(0,)),)
57     ),
58 ]

```

pipelines/d/detect.py

```

1  """Conflict detection (Problem 1): pairwise two-pointer method and band-bucketed sweep line.
2
3  Two plans conflict when their band intervals overlap and at least one pair of uses overlaps in time
4  (half-open intervals, MDR-0001). ``detect_pairwise`` is the  $O(n^2 (m_i + m_j))$  reference and
5  ``detect_bandsweep`` the  $O(N \log N + K)$  scalable method; both must return the same pair set.
6  """
7
8  from __future__ import annotations
9
10 from collections import Counter, defaultdict
11 from typing import Any
12
13 from pipelines.d.plans import CATEGORIES, Plan
14
15
16 def intervals_overlap(a0: int, a1: int, b0: int, b1: int) -> bool:
17     """Half-open intervals ``[a0,a1)`` and ``[b0,b1)`` intersect."""
18     return a0 < b1 and b0 < a1
19
20
21 def freq_overlap(a: Plan, b: Plan) -> bool:
22     return intervals_overlap(a.f, a.f_end, b.f, b.f_end)
23
24
25 def uses_overlap(a: Plan, b: Plan) -> bool:
26     """True when some use of ``a`` overlaps some use of ``b`` in time (two-pointer merge over sorted uses)."""
27     ua, ub = a.uses(), b.uses()
28     i = j = 0
29     while i < len(ua) and j < len(ub):
30         s1, e1 = ua[i]
31         s2, e2 = ub[j]

```

```

32         if s1 < e2 and s2 < e1:
33             return True
34         if e1 <= e2:
35             i += 1
36         else:
37             j += 1
38     return False
39
40
41 def overlapping_uses(a: Plan, b: Plan) -> list[tuple[int, int]]:
42     """All (k, l) use pairs of ``a`` and ``b`` that overlap in time (full O(m_a m_b) enumeration)."""
43     return [
44         (k, l)
45         for k, (s1, e1) in enumerate(a.uses())
46         for l, (s2, e2) in enumerate(b.uses())
47         if intervals_overlap(s1, e1, s2, e2)
48     ]
49
50
51 def plans_conflict(a: Plan, b: Plan) -> bool:
52     return freq_overlap(a, b) and uses_overlap(a, b)
53
54
55 def detect_pairwise(plans: list[Plan]) -> list[tuple[int, int]]:
56     """Reference detector: every pair, band test first, then two-pointer time test. Returns sorted index pairs."""
57     out: list[tuple[int, int]] = []
58     n = len(plans)
59     for i in range(n):
60         a = plans[i]
61         for j in range(i + 1, n):
62             b = plans[j]
63             if freq_overlap(a, b) and uses_overlap(a, b):
64                 out.append((i, j))
65     return out
66
67
68 def detect_bandsweep(plans: list[Plan]) -> list[tuple[int, int]]:
69     """Sweep line per band: bucket every use of every plan into the bands it occupies, sort by start,
70     and report pairs whose uses overlap. Complexity O(N log N + K) with N = total (use, band) items."""
71     buckets: dict[int, list[tuple[int, int, int]]] = defaultdict(list)
72     for idx, p in enumerate(plans):
73         for start, end in p.uses():
74             for band in range(p.f, p.f_end):
75                 buckets[band].append((start, end, idx))
76     found: set[tuple[int, int]] = set()
77     for items in buckets.values():
78         items.sort()
79         active: list[tuple[int, int]] = [] # (end, idx), pruned lazily
80         for start, end, idx in items:
81             active = [(e, k) for e, k in active if e > start]
82             for _, k in active:
83                 found.add((k, idx) if k < idx else (idx, k))
84             active.append((end, idx))
85     return sorted(found)
86
87
88 def pair_key(a: str, b: str) -> str:
89     return "".join(sorted((a, b)))
90
91
92 def conflict_records(plans: list[Plan], pairs: list[tuple[int, int]]) -> list[dict[str, Any]]:
93     """Per conflict pair: ids, categories and the number of overlapping use pairs."""
94     records = []
95     for i, j in pairs:
96         a, b = plans[i], plans[j]
97         uses = overlapping_uses(a, b)
98         records.append(
99             {
100                 "id1": a.pid,
101                 "id2": b.pid,
102                 "pair": pair_key(a.cat, b.cat),

```

```

103         "use_pairs": len(uses),
104         "first_overlap_time": min(max(a.uses()[k][0], b.uses()[l][0]) for k, l in uses),
105         "band_overlap": min(a.f_end, b.f_end) - max(a.f, b.f),
106     }
107 )
108 return records
109
110
111 def graph_stats(plans: list[Plan], pairs: list[tuple[int, int]]) -> dict[str, Any]:
112     """Conflict-graph statistics: category-pair counts, degrees, components, involvement."""
113     import networkx as nx
114
115     graph = nx.Graph()
116     graph.add_nodes_from(range(len(plans)))
117     graph.add_edges_from(pairs)
118     degrees = dict(graph.degree())
119     by_pair: Counter[str] = Counter(pair_key(plans[i].cat, plans[j].cat) for i, j in pairs)
120     involved = {c: sum(1 for i, p in enumerate(plans) if p.cat == c and degrees[i] > 0) for c in CATEGORIES}
121     counts = {c: sum(1 for p in plans if p.cat == c) for c in CATEGORIES}
122     components = sorted((len(c) for c in nx.connected_components(graph) if len(c) > 1), reverse=True)
123     degree_hist: Counter[int] = Counter(degrees.values())
124     return {
125         "pairs": len(pairs),
126         "by_pair": {k: int(by_pair.get(k, 0)) for k in ("AA", "AB", "AC", "BB", "BC", "CC")},
127         "plans_involved": int(sum(involved.values())),
128         "plans_involved_by_cat": involved,
129         "plans_by_cat": counts,
130         "max_degree": int(max(degrees.values(), default=0)),
131         "mean_degree": float(sum(degrees.values()) / max(len(plans), 1)),
132         "degree_hist": {str(k): int(v) for k, v in sorted(degree_hist.items())},
133         "components": len(components),
134         "largest_component": int(components[0]) if components else 0,
135         "component_sizes": components,
136         "density": float(len(pairs) / max(len(plans) * (len(plans) - 1) / 2, 1)),
137         "top_degree": sorted(
138             ({f"pid": plans[i].pid, "degree": degrees[i]} for i in range(len(plans))),
139             key=lambda r: (-int(r["degree"]), str(r["pid"]))),
140         )[:10],
141     }

```

pipelines/d/highs_util.py

```

1  """HiGHS process-level settings shared by the resolution and packing models."""
2
3  from __future__ import annotations
4
5  _THREADS: list[int] = []
6
7
8  def highs_threads(requested: int) -> int:
9      """Thread count to pass to HiGHS in this process.
10
11      HiGHS initialises its global task executor on the first ``run()`` and a later solve that asks for a
12      different ``threads`` value fails silently (model status "Not Set", objective 0). The first value used
13      in the process is therefore kept for every subsequent solve."""
14      if not _THREADS:
15          _THREADS.append(max(1, int(requested)))
16      return _THREADS[0]

```

pipelines/d/pack.py

```
1 """Problem 3: pack as many additional C-class plans as possible into the free time-frequency cells left by
2 the Problem-2 solution (MDR-0005). Variables x[f,s] (band start, first-use start); AtMostOne per cell.
3 """
4
5 from __future__ import annotations
6
7 import time
8 from dataclasses import replace
9 from typing import Any
10
11 import numpy as np
12
13 from pipelines.d.highs_util import highs_threads
14 from pipelines.d.plans import BANDS, Plan
15
16 C_TEMPLATE = {"w": 3, "d": 2, "g": 8, "n": 12}
17
18
19 def template_span(template: dict[str, int]) -> int:
20     return template["d"] + (template["n"] - 1) * (template["d"] + template["g"])
21
22
23 def template_offsets(template: dict[str, int]) -> np.ndarray:
24     """Time offsets (relative to the first-use start) occupied by one plan of the template."""
25     period = template["d"] + template["g"]
26     return np.array([k * period + j for k in range(template["n"]) for j in range(template["d"])], dtype=np.int64)
27
28
29 def occupancy_grid(plans: list[Plan], horizon: int, bands: int = BANDS) -> np.ndarray:
30     """Boolean (bands × horizon) grid of occupied cells."""
31     grid = np.zeros((bands, horizon), dtype=bool)
32     for p in plans:
33         for start, end in p.uses():
34             grid[p.f : p.f_end, start:end] = True
35     return grid
36
37
38 def candidate_placements(grid: np.ndarray, template: dict[str, int]) -> np.ndarray:
39     """All (f, s) whose template cells are free and end within the horizon; shape (m, 2)."""
40     bands, horizon = grid.shape
41     span = template_span(template)
42     offsets = template_offsets(template)
43     free = ~grid
44     width = template["w"]
45     n_s = horizon - span + 1
46     if n_s <= 0:
47         return np.zeros((0, 2), dtype=np.int64)
48     rows = []
49     for f in range(bands - width + 1):
50         strip = np.all(free[f : f + width], axis=0) # time slots free on all bands of the strip
51         windows = strip[np.arange(n_s)[:], None] + offsets[None, :] # (n_s, cells per plan)
52         ok = np.all(windows, axis=1)
53         for s in np.flatnonzero(ok):
54             rows.append((f, int(s)))
55     return np.array(rows, dtype=np.int64).reshape(-1, 2)
56
57
58 def cell_rows(candidates: np.ndarray, template: dict[str, int], horizon: int) -> list[list[int]]:
59     """Deduplicated cell rows (lists of candidate indices) with at least two candidates."""
60     offsets = template_offsets(template)
61     cells: dict[int, list[int]] = {}
62     for v, (f, s) in enumerate(candidates.tolist()):
63         for band in range(f, f + template["w"]):
64             base = band * horizon + s
65             for off in offsets.tolist():
66                 cells.setdefault(base + off, []).append(v)
67     rows = {tuple(r) for r in cells.values() if len(r) >= 2}
68     return [list(r) for r in sorted(rows)]
```

```

69
70
71 def greedy_pack(candidates: np.ndarray, template: dict[str, int], grid: np.ndarray) -> list[int]:
72     """First-fit greedy (by start time, then band): a feasible lower bound and a CP-SAT hint."""
73     occ = grid.copy()
74     offsets = template_offsets(template)
75     order = np.lexsort((candidates[:, 0], candidates[:, 1]))
76     chosen = []
77     for v in order.tolist():
78         f, s = candidates[v]
79         block = occ[f : f + template["w"]][:, s + offsets]
80         if not block.any():
81             occ[f : f + template["w"], s + offsets] = True
82             chosen.append(v)
83     return chosen
84
85
86 def solve_pack_cpsat(
87     rows: list[list[int]],
88     n_candidates: int,
89     *,
90     hint: list[int] | None,
91     seed: int,
92     workers: int = 8,
93     time_limit: float = 900.0,
94 ) -> dict[str, Any]:
95     from ortools.sat.python import cp_model
96
97     t0 = time.monotonic()
98     cp = cp_model.CpModel()
99     x = [cp.NewBoolVar(f"x{v}") for v in range(n_candidates)]
100     for row in rows:
101         cp.AddAtMostOne(x[v] for v in row)
102     cp.Maximize(sum(x))
103     if hint:
104         hinted = set(hint)
105         for v in range(n_candidates):
106             cp.AddHint(x[v], v in hinted)
107     solver = cp_model.CpSolver()
108     solver.parameters.random_seed = int(seed)
109     solver.parameters.num_workers = int(workers)
110     solver.parameters.max_time_in_seconds = float(time_limit)
111     status = solver.Solve(cp)
112     chosen = (
113         [v for v in range(n_candidates) if solver.Value(x[v])]
114         if status in (cp_model.OPTIMAL, cp_model.FEASIBLE)
115         else []
116     )
117     return {
118         "solver": "cp-sat",
119         "status": solver.StatusName(status),
120         "value": len(chosen),
121         "bound": float(solver.BestObjectiveBound()) if chosen else None,
122         "chosen": chosen,
123         "seconds": round(time.monotonic() - t0, 3),
124         "n_vars": n_candidates,
125         "n_rows": len(rows),
126     }
127
128
129 def highs_pack(
130     rows: list[list[int]], n_candidates: int, *, integer: bool, time_limit: float, seed: int, threads: int = 8
131 ) -> dict[str, Any]:
132     """LP relaxation upper bound (integer=False) or a MILP cross-check with its dual bound (integer=True)."""
133     import highsipy
134
135     t0 = time.monotonic()
136     h = highsipy.Highs()
137     h.setOptionValue("output_flag", False)
138     h.setOptionValue("time_limit", float(time_limit))
139     h.setOptionValue("random_seed", int(seed))

```

```

140 h.setOptionValue("threads", highs_threads(threads))
141 h.setOptionValue("mip_rel_gap", 0.0)
142 h.setOptionValue("mip_abs_gap", 0.0)
143 n = n_candidates
144 h.addVars(n, np.zeros(n), np.ones(n))
145 if integer:
146     h.changeColsIntegrality(n, np.arange(n, dtype=np.int32), np.array([highspy.HighsVarType.kInteger] * n))
147 starts, index = [], []
148 nnz = 0
149 for row in rows:
150     starts.append(nnz)
151     index.extend(row)
152     nnz += len(row)
153 h.addRows(
154     len(rows),
155     np.full(len(rows), -highspy.kHighsInf),
156     np.ones(len(rows)),
157     nnz,
158     np.array(starts, dtype=np.int32),
159     np.array(index, dtype=np.int32),
160     np.ones(nnz),
161 )
162 h.changeColsCost(n, np.arange(n, dtype=np.int32), -np.ones(n)) # maximise sum x
163 h.run()
164 status = h.modelStatusToString(h.getModelStatus())
165 info = h.getInfo()
166 out: dict[str, Any] = {"status": status, "seconds": round(time.monotonic() - t0, 3), "integer": integer}
167 if integer:
168     out["value"] = -float(info.objective_function_value)
169     out["bound"] = -float(info.mip_dual_bound)
170     sol = np.array(h.getSolution().col_value)
171     out["chosen"] = [int(v) for v in np.flatnonzero(sol > 0.5)]
172 else:
173     out["value"] = -float(info.objective_function_value)
174 return out
175
176
177 def plan_offsets(plan: Plan) -> np.ndarray:
178     """Time offsets (relative to the first-use start) occupied by ``plan``."""
179     return np.array([k * plan.period + j for k in range(plan.n) for j in range(plan.d)], dtype=np.int64)
180
181
182 def free_positions(grid: np.ndarray, width: int, offsets: np.ndarray, *, first_only: bool = False) -> np.ndarray:
183     """All (f, s) at which a plan of this shape fits in the free cells of ``grid``; shape (m, 2).
184
185     With ``first_only`` the scan stops at the lexicographically smallest (f, s), which is what first-fit
186     needs and keeps the repack linear in the number of plans rather than in the number of positions."""
187     bands, horizon = grid.shape
188     span = int(offsets[-1]) + 1
189     n_s = horizon - span + 1
190     if n_s <= 0 or width > bands:
191         return np.zeros((0, 2), dtype=np.int64)
192     free = ~grid
193     rows = []
194     windows_index = np.arange(n_s)[:, None] + offsets[None, :]
195     for f in range(bands - width + 1):
196         strip = np.all(free[f : f + width], axis=0)
197         ok = np.flatnonzero(np.all(strip[windows_index], axis=1))
198         if first_only:
199             if len(ok):
200                 return np.array([(f, int(ok[0]))], dtype=np.int64)
201             continue
202         rows.extend((f, int(s)) for s in ok)
203     return np.array(rows, dtype=np.int64).reshape(-1, 2)
204
205
206 ORDERS: dict[str, Any] = {
207     "area": lambda p: -(p.w * p.d * p.n), -p.w, p.pid),
208     "width": lambda p: (-p.w, -(p.d * p.n), p.pid),
209     "span": lambda p: -(p.d + (p.n - 1) * (p.d + p.g)), -p.w, p.pid),
210     "duty": lambda p: -(p.w * p.d) / (p.d + p.g), -p.w, p.pid),

```

```

211 }
212
213
214 def repack_existing(
215     plans: list[Plan], horizon: int, bands: int = BANDS, order: str = "area", time_first: bool = False
216 ) -> tuple[list[Plan], np.ndarray]:
217     """First-fit decreasing re-placement of every plan (MDR-0010, interpretation B).
218
219     Each plan keeps ``w``, ``d``, ``g`` and ``n`` — only its band start and first-use start move, by an
220     unrestricted amount — so the number of cells it occupies is unchanged. Plans are placed largest-first
221     at the first free ``(f, s)`` in the scan order (``time_first`` scans ``(s, f)`` instead of ``(f, s)``),
222     which compacts them and leaves a large contiguous free region. Raises when a plan cannot be placed."""
223     grid = np.zeros((bands, horizon), dtype=bool)
224     ordered = sorted(plans, key=ORDERS[order])
225     placed: list[Plan] = []
226     for plan in ordered:
227         offsets = plan_offsets(plan)
228         spots = free_positions(grid, plan.w, offsets, first_only=not time_first)
229         if not len(spots):
230             raise RuntimeError(f"first-fit decreasing could not place {plan.pid} within {bands}x{horizon}")
231         if time_first:
232             f, s = min(((int(a), int(b)) for a, b in spots), key=lambda fs: (fs[1], fs[0]))
233         else:
234             f, s = (int(v) for v in spots[0])
235         grid[f : f + plan.w, s + offsets] = True
236         placed.append(replace(plan, f=f, s=s))
237     return sorted(placed, key=lambda p: p.pid), grid
238
239
240 def plans_from_choice(candidates: np.ndarray, chosen: list[int], template: dict[str, int]) -> list[Plan]:
241     """New C plans (ids N001, N002, ...) in (start, band) order."""
242     picked = sorted((int(candidates[v, 1]), int(candidates[v, 0])) for v in chosen)
243     return [
244         Plan(f"N{k:03d}", "C", f, template["w"], s, template["d"], template["g"], template["n"])
245         for k, (s, f) in enumerate(picked, start=1)
246     ]

```

pipelines/d/plans.py

```

1 """Plan data model for Problem D: parsing attachment rows, use intervals, cells and adjustments.
2
3 A plan occupies the band interval ``[f, f+w)`` during ``n`` uses; use ``k`` (0-based) occupies the
4 time interval ``[s + k(d+g), s + d + k(d+g))``. All intervals are half-open integers (MDR-0001).
5 """
6
7 from __future__ import annotations
8
9 import re
10 from dataclasses import asdict, dataclass, replace
11 from pathlib import Path
12 from typing import Any
13
14 BANDS = 100
15 CATEGORIES = ("A", "B", "C")
16 INTERVAL = re.compile(r"^\s*[\s*(\d+)\s*,\s*(\d+)\s*(\d+)\s*$")
17
18
19 def parse_interval(text: Any) -> tuple[int, int]:
20     """Parse ``[lo,hi)`` into ``(lo, hi)``; rejects malformed or empty intervals."""
21     match = INTERVAL.match(str(text))
22     if not match:
23         raise ValueError(f"malformed half-open interval: {text!r}")
24     lo, hi = int(match.group(1)), int(match.group(2))
25     if lo >= hi:
26         raise ValueError(f"empty interval: {text!r}")
27     return lo, hi
28
29

```

```

30 def fmt_interval(lo: int, hi: int) -> str:
31     """Format a half-open interval the way the attachments do: ``[lo,hi)``."""
32     return f"[{lo},{hi})"
33
34
35 @dataclass(frozen=True)
36 class Plan:
37     """One frequency-use plan (immutable)."""
38
39     pid: str
40     cat: str
41     f: int # first band occupied
42     w: int # number of bands occupied
43     s: int # start of the first use
44     d: int # duration of every use
45     g: int # gap between consecutive uses
46     n: int # number of uses
47
48     @property
49     def period(self) -> int:
50         return self.d + self.g
51
52     @property
53     def f_end(self) -> int:
54         return self.f + self.w
55
56     @property
57     def end(self) -> int:
58         """Exclusive end time of the last use."""
59         return self.s + self.d + (self.n - 1) * self.period
60
61     def uses(self) -> list[tuple[int, int]]:
62         """Half-open time intervals of the ``n`` uses, in chronological order."""
63         return [(self.s + k * self.period, self.s + k * self.period + self.d) for k in range(self.n)]
64
65     @property
66     def n_cells(self) -> int:
67         return self.w * self.d * self.n
68
69     def freq_text(self) -> str:
70         return fmt_interval(self.f, self.f_end)
71
72     def time_text(self) -> str:
73         return fmt_interval(self.s, self.s + self.d)
74
75     def shifted(self, df: int = 0, dt: int = 0, gap: int | None = None) -> Plan:
76         """Copy with the band start moved by ``df``, the time start by ``dt`` and/or a new gap."""
77         return replace(self, f=self.f + df, s=self.s + dt, g=self.g if gap is None else gap)
78
79     def record(self) -> dict[str, Any]:
80         return asdict(self)
81
82
83 def plan_from_record(rec: dict[str, Any]) -> Plan:
84     return Plan(
85         str(rec["pid"]),
86         str(rec["cat"]),
87         int(rec["f"]),
88         int(rec["w"]),
89         int(rec["s"]),
90         int(rec["d"]),
91         int(rec["g"]),
92         int(rec["n"]),
93     )
94
95
96 def plan_from_row(pid: Any, freq: Any, time: Any, gap: Any, count: Any) -> Plan:
97     """Build a plan from one attachment row (equipment id, band interval, time interval, gap, count)."""
98     pid = str(pid).strip()
99     f0, f1 = parse_interval(freq)
100     t0, t1 = parse_interval(time)

```

```

101     return Plan(pid, pid[0], f0, f1 - f0, t0, t1 - t0, int(gap), int(count))
102
103
104 def load_plans(parquet: Path) -> list[Plan]:
105     """Read the ingested attachment sheet (parquet) into plans, in file order."""
106     import pandas as pd
107
108     df = pd.read_parquet(parquet)
109     df.columns = [str(c).strip() for c in df.columns]
110     return [
111         plan_from_row(r["用频装备编号"], r["频段区间"], r["时间区间"], r["间隔时长"], r["使用次数"])
112         for _, r in df.iterrows()
113     ]
114
115
116 def horizon(plans: list[Plan]) -> int:
117     """Latest exclusive end time over the plans (0 for an empty list)."""
118     return max((p.end for p in plans), key=int, default=0)
119
120
121 def by_category(plans: list[Plan]) -> dict[str, list[Plan]]:
122     out: dict[str, list[Plan]] = {c: [] for c in CATEGORIES}
123     for p in plans:
124         out.setdefault(p.cat, []).append(p)
125     return out

```

pipelines/d/report.py

```

1  """Paper figures and tables for Problem D (stages ``figures`` and ``tables``).
2
3  Figures follow the house style in ``forge.plotting`` (vector PDF + PNG preview, Okabe-Ito palette with a
4  fixed category → hue mapping, one axis per panel, legends whenever two or more series are shown).
5  Tables are booktabs fragments (``tables/*.tex``) with a CSV twin; the paper wraps them in ``table``.
6  """
7
8  from __future__ import annotations
9
10 import csv
11 from collections.abc import Sequence
12 from typing import Any
13
14 from forge import plotting
15 from forge.context import StageContext
16 from forge.runner import stage
17 from forge.tex import tex_escape
18 from pipelines.d.detect import overlapping_uses, plans_conflict
19 from pipelines.d.plans import CATEGORIES, Plan, horizon, plan_from_record
20 from pipelines.d.resolve import Limits, enumerate_options
21
22 CAT_COLOR = {"A": "#0072B2", "B": "#D55E00", "C": "#009E73"} # fixed identity → hue (Okabe-Ito slots 1-3)
23 ACTION_COLOR = {"freq": "#0072B2", "time": "#E69F00", "gap": "#CC79A7", "cancel": "#000000", "keep": "#BBBBBB"}
24 ACTION_LABEL = {"freq": "频段平移", "time": "时间平移", "gap": "间隔调整", "cancel": "撤销", "keep": "保持"}
25 GREY = "#C8C8C8"
26 REPORT_DEPS = ("detect", "resolve", "pack", "resolve_interval", "bench", "sensitivity")
27
28
29 def _plans(records: list[dict[str, Any]]) -> list[Plan]:
30     return [plan_from_record(r) for r in records]
31
32
33 def _rects(
34     ax: Any,
35     plans: Sequence[Plan],
36     face: dict[str, str],
37     *,
38     edge: dict[str, str] | None = None,
39     hatch_ids: set[str] = frozenset(),
40     alpha: float = 0.9,

```

```

41 ) -> None:
42     from matplotlib.collections import PatchCollection
43     from matplotlib.patches import Rectangle
44
45     edge = edge or {}
46     plain, plain_face, plain_edge, plain_lw = [], [], [], []
47     hatched, hatched_edge = [], []
48     for p in plans:
49         for start, end in p.uses():
50             rect = Rectangle((start, p.f), end - start, p.w)
51             if p.pid in hatch_ids:
52                 hatched.append(rect)
53                 hatched_edge.append(edge.get(p.pid, "#000000"))
54             else:
55                 plain.append(rect)
56                 plain_face.append(face.get(p.pid, GREY))
57                 plain_edge.append(edge.get(p.pid, "none"))
58                 plain_lw.append(0.6 if p.pid in edge else 0.0)
59     if plain:
60         ax.add_collection(
61             PatchCollection(plain, facecolors=plain_face, edgecolors=plain_edge, linewidths=plain_lw, alpha=alpha)
62         )
63     if hatched:
64         ax.add_collection(
65             PatchCollection(hatched, facecolors="none", edgecolors=hatched_edge, linewidths=0.6, hatch="////")
66         )
67
68
69 def _plane(ax: Any, horizon_t: int, *, xlabel: bool = True) -> None:
70     ax.set_xlim(0, horizon_t)
71     ax.set_ylim(0, 100)
72     ax.set_ylabel("频段编号 ( $\Delta f$ )")
73     if xlabel:
74         ax.set_xlabel("时间 ( $\Delta t$ )")
75     ax.grid(False)
76
77
78 def _legend(ax: Any, entries: list[tuple[str, str, str | None]], **kwargs: Any) -> None:
79     from matplotlib.patches import Patch
80
81     handles = [
82         Patch(
83             # colour "none" means the mark is drawn by its edge only (e.g. the black outline used for
84             # conflicting uses): give the swatch a white face and a black edge so that it is visible.
85             facecolor="white" if color == "none" else (color if hatch is None else "none"),
86             edgecolor="#000000" if (hatch or color == "none") else "none",
87             hatch=hatch or None,
88             label=label,
89         )
90         for label, color, hatch in entries
91     ]
92     ax.legend(handles=handles, frameon=False, **kwargs)
93
94
95 def _conflict_use_ids(plans: list[Plan], conflicts: list[dict[str, Any]]) -> set[tuple[str, int]]:
96     by_id = {p.pid: p for p in plans}
97     out: set[tuple[str, int]] = set()
98     for rec in conflicts:
99         a, b = by_id[rec["id1"]], by_id[rec["id2"]]
100         for k, l in overlapping_uses(a, b):
101             out.add((a.pid, k))
102             out.add((b.pid, l))
103     return out
104
105
106 def fig_q1_gantt(ctx: StageContext, plans: list[Plan], conflicts: list[dict[str, Any]]) -> None:
107     from matplotlib.collections import PatchCollection
108     from matplotlib.patches import Rectangle
109
110     horizon_t = horizon(plans)
111     fig, ax = plotting.new_figure(6.5, 3.4)

```

```

112     _rects(ax, plans, {p.pid: CAT_COLOR[p.cat] for p in plans}, alpha=0.75)
113     hot = _conflict_use_ids(plans, conflicts)
114     patches = [
115         Rectangle((start, p.f), end - start, p.w)
116         for p in plans
117         for k, (start, end) in enumerate(p.uses())
118         if (p.pid, k) in hot
119     ]
120     ax.add_collection(PatchCollection(patches, facecolors="none", edgecolors="#000000", linewidths=0.7))
121     _plane(ax, horizon_t)
122     _legend(
123         ax,
124         [
125             ("A 类", CAT_COLOR["A"], None),
126             ("B 类", CAT_COLOR["B"], None),
127             ("C 类", CAT_COLOR["C"], None),
128             ("冲突的使用次 (黑边)", "none", ""),
129         ],
130         loc="upper left",
131         bbox_to_anchor=(1.0, 1.0),
132         ncol=1,
133     )
134     plotting.save(fig, ctx.out("figures", "fig_q1_gantt.pdf"))
135
136
137 def fig_q1_structure(ctx: StageContext, stats: dict[str, Any]) -> None:
138     import numpy as np
139
140     fig, axes = plotting.new_figure(6.5, 2.7, ncols=2, gridspec_kw={"width_ratios": [1.0, 1.4]})
141     ax = axes[0]
142     matrix = np.zeros((3, 3))
143     for key, value in stats["by_pair"].items():
144         i, j = CATEGORIES.index(key[0]), CATEGORIES.index(key[1])
145         matrix[i, j] = value
146         matrix[j, i] = value
147     ax.imshow(matrix, cmap="Blues", vmin=0)
148     for i in range(3):
149         for j in range(3):
150             ax.text(
151                 j,
152                 i,
153                 f"{int(matrix[i, j])}",
154                 ha="center",
155                 va="center",
156                 color="#000000" if matrix[i, j] < matrix.max() * 0.6 else "#FFFFFF",
157             )
158     ax.set_xticks(range(3), [f"{c} 类" for c in CATEGORIES])
159     ax.set_yticks(range(3), [f"{c} 类" for c in CATEGORIES])
160     ax.set_title("(a) 冲突对数 (按类别对)")
161     ax.grid(False)
162     ax = axes[1]
163     hist = {int(k): v for k, v in stats["degree_hist"].items()}
164     degrees = sorted(hist)
165     ax.bar(degrees, [hist[d] for d in degrees], color="#0072B2", width=0.8)
166     for d in degrees:
167         ax.text(d, hist[d], str(hist[d]), ha="center", va="bottom", fontsize=7)
168     ax.set_xlabel("冲突图中的度 (与之冲突的计划数)")
169     ax.set_ylabel("计划数量")
170     ax.set_title("(b) 度分布")
171     ax.set_xticks(degrees)
172     plotting.save(fig, ctx.out("figures", "fig_q1_structure.pdf"))
173
174
175 def fig_resolution(
176     ctx: StageContext,
177     name: str,
178     plans: list[Plan],
179     decisions: list[dict[str, Any]],
180     conflicts: list[dict[str, Any]],
181     title: str,
182 ) -> None:

```

```

183 from matplotlib.patches import FancyArrowPatch
184
185 horizon_t = max(horizon(plans), max((plan_from_record(d["plan"])).end for d in decisions if d["plan"]), default=0)
186 fig, axes = plotting.new_figure(6.5, 5.2, nrows=2, sharex=True)
187 kinds = {d["pid"]: d["kind"] for d in decisions}
188 ax = axes[0]
189 face = {p.pid: (ACTION_COLOR[kinds[p.pid]] if kinds[p.pid] in {"freq", "time", "gap"} else GREY) for p in plans}
190 _rects(ax, [p for p in plans if kinds[p.pid] != "cancel"], face, alpha=0.85)
191 _rects(
192     ax,
193     [p for p in plans if kinds[p.pid] == "cancel"],
194     {},
195     hatch_ids={p.pid for p in plans if kinds[p.pid] == "cancel"},
196 )
197 hot = _conflict_use_ids(plans, conflicts)
198 from matplotlib.collections import PatchCollection
199 from matplotlib.patches import Rectangle
200
201 ax.add_collection(
202     PatchCollection(
203         [Rectangle((s, p.f), e - s, p.w) for p in plans for k, (s, e) in enumerate(p.uses()) if (p.pid, k) in hot],
204         facecolors="none",
205         edgecolors="#000000",
206         linewidths=0.5,
207     )
208 )
209 _plane(ax, horizon_t, xlabel=False)
210 ax.set_title(f"(a) 消解前: {title}中被调整/撤销的计划以颜色标出, 黑边为冲突使用次")
211 _legend(
212     ax,
213     [
214         ("保持", GREY, None),
215         ("频段平移", ACTION_COLOR["freq"], None),
216         ("时间平移", ACTION_COLOR["time"], None),
217         ("间隔调整", ACTION_COLOR["gap"], None),
218         ("撤销", "none", "/////"),
219     ],
220     loc="upper left",
221     bbox_to_anchor=(1.0, 1.0),
222 )
223 ax = axes[1]
224 after = [plan_from_record(d["plan"]) for d in decisions if d["plan"]]
225 _rects(
226     ax, after, {p.pid: (ACTION_COLOR[kinds[p.pid]] if kinds[p.pid] != "keep" else GREY) for p in after}, alpha=0.85
227 )
228 by_id = {p.pid: p for p in plans}
229 for p in after:
230     if kinds[p.pid] in {"freq", "time"}:
231         before = by_id[p.pid]
232         ax.add_patch(
233             FancyArrowPatch(
234                 (before.s, before.f + before.w / 2),
235                 (p.s, p.f + p.w / 2),
236                 arrowstyle="-|>",
237                 mutation_scale=6,
238                 color="#000000",
239                 linewidth=0.5,
240                 alpha=0.6,
241             )
242         )
243 _plane(ax, horizon_t)
244 ax.set_title("(b) 消解后: 无冲突的用频计划 (箭头为平移方向)")
245 plotting.save(fig, ctx.out("figures", f"{name}.pdf"))
246
247
248 def fig_schemes(ctx: StageContext, name: str, alternatives: dict[str, Any]) -> None:
249     import numpy as np
250
251     names = [n for n in ("P", "T", "S", "W") if alternatives.get(n, {}).get("canonical_vector")]
252     fig, axes = plotting.new_figure(6.5, 2.6, ncols=2)
253     ax = axes[0]

```

```

254     x = np.arange(len(names))
255     bottom = np.zeros(len(names))
256     for idx, c in enumerate(CATEGORIES):
257         vals = np.array([alternatives[n]["canonical_vector"][3 + idx] for n in names], dtype=float)
258         ax.bar(x, vals, bottom=bottom, color=CAT_COLOR[c], label=f"{c} 类调整", width=0.6)
259         bottom += vals
260     for i in range(len(names)):
261         ax.text(x[i], bottom[i], str(int(bottom[i])), ha="center", va="bottom", fontsize=7)
262     ax.set_xticks(x, [f"方案 {n}" for n in names])
263     ax.set_ylabel("被调整的计划数")
264     ax.set_title("(a) 各目标方案的调整数量")
265     ax.set_ylim(0, float(bottom.max()) * 1.3)
266     ax.legend(frameon=False, fontsize=7, ncol=3, loc="upper left")
267     ax = axes[1]
268     amps = [alternatives[n]["canonical_vector"][6] / 10.0 for n in names]
269     ax.bar(x, amps, color="#0072B2", width=0.6)
270     for i, a in enumerate(amps):
271         ax.text(x[i], a, f"{a:.1f}", ha="center", va="bottom", fontsize=7)
272     ax.set_xticks(x, [f"方案 {n}" for n in names])
273     ax.set_ylabel("归一化总调整幅度")
274     ax.set_title("(b) 总调整幅度")
275     plotting.save(fig, ctx.out("figures", f"{name}.pdf"))
276
277
278 def fig_q3_layout(
279     ctx: StageContext,
280     existing: list[Plan],
281     new_plans: list[Plan],
282     horizon_t: int,
283     *,
284     name: str = "fig_q3_layout",
285     title: str | None = None,
286 ) -> None:
287     fig, ax = plotting.new_figure(6.5, 3.4)
288     _rects(ax, existing, {p.pid: GREY for p in existing}, alpha=0.9)
289     _rects(ax, new_plans, {p.pid: CAT_COLOR["C"] for p in new_plans}, alpha=0.95)
290     _plane(ax, horizon_t)
291     if title:
292         ax.set_title(title)
293     _legend(
294         ax,
295         [{"既有计划", GREY, None}, (f"新增 C 类计划 ({len(new_plans)} 个)", CAT_COLOR["C"], None)],
296         loc="upper left",
297         bbox_to_anchor=(1.0, 1.0),
298     )
299     plotting.save(fig, ctx.out("figures", f"{name}.pdf"))
300
301
302 def fig_q4_comparison(
303     ctx: StageContext, q2_table: dict[str, Any], q4_table: dict[str, Any], q4_shifts: dict[str, Any]
304 ) -> None:
305     import numpy as np
306
307     fig, axes = plotting.new_figure(6.5, 2.6, ncols=2)
308     ax = axes[0]
309     x = np.arange(3)
310     width = 0.36
311     q2 = [q2_table[c]["adjust"] for c in CATEGORIES]
312     q4 = [q4_table[c]["adjust"] for c in CATEGORIES]
313     ax.bar(x - width / 2, q2, width, color="#0072B2", label="问题 2")
314     ax.bar(x + width / 2, q4, width, color="#D55E00", label="问题 4")
315     for i in range(3):
316         ax.text(x[i] - width / 2, q2[i], str(q2[i]), ha="center", va="bottom", fontsize=7)
317         ax.text(x[i] + width / 2, q4[i], str(q4[i]), ha="center", va="bottom", fontsize=7)
318     ax.set_xticks(x, [f"{c} 类" for c in CATEGORIES])
319     ax.set_ylabel("被调整的计划数")
320     ax.set_title("(a) 调整数量对照")
321     ax.legend(frameon=False)
322     ax = axes[1]
323     kinds = ["freq", "time", "gap", "cancel"]
324     counts = [q4_shifts[k]["count"] if k != "cancel" else q4_shifts["cancel"] for k in kinds]

```

```

325     ax.bar(range(4), counts, color=[ACTION_COLOR[k] for k in kinds], width=0.6)
326     for i, v in enumerate(counts):
327         ax.text(i, v, str(v), ha="center", va="bottom", fontsize=7)
328     ax.set_xticks(range(4), [ACTION_LABEL[k] for k in kinds])
329     ax.set_ylabel("计划数")
330     ax.set_title("(b) 问题 4 的动作构成")
331     plotting.save(fig, ctx.out("figures", "fig_q4_comparison.pdf"))
332
333
334 def fig_bench_scaling(ctx: StageContext, scaling: list[dict[str, Any]]) -> None:
335     from matplotlib.ticker import NullFormatter
336
337     fig, ax = plotting.new_figure(6.3, 3.0)
338     ns = [r["n"] for r in scaling]
339     limit = max([r["cpsat_seconds"] for r in scaling], default=0.0)
340     series = [
341         ("成对枚举检测", [r["pairwise_seconds"] for r in scaling], "#0072B2", "o"),
342         ("分桶扫描线检测", [r["sweep_seconds"] for r in scaling], "#D55E00", "s"),
343         ("单元格模型构建", [r["build_seconds"] for r in scaling], "#009E73", "^"),
344         (
345             f"CP-SAT 消解 (固定时限 {limit:.0f} s, 到时返回可行解)",
346             [r["cpsat_seconds"] for r in scaling],
347             "#E69F00",
348             "D",
349         ),
350     ]
351     for label, ys, color, marker in series:
352         ax.plot(ns, ys, marker=marker, markersize=4, color=color, label=label)
353     ax.set_xscale("log")
354     ax.set_yscale("log")
355     ax.set_xlabel("计划数量 n (合成实例)")
356     ax.set_ylabel("耗时 (s)")
357     ax.set_xticks(ns, [str(n) for n in ns])
358     ax.xaxis.set_minor_formatter(NullFormatter())
359     ax.legend(frameon=False, fontsize=7, loc="center left")
360     ax.set_xlim(ns[0] * 0.9, ns[-1] * 1.1)
361     plotting.save(fig, ctx.out("figures", "fig_bench_scaling.pdf"))
362
363
364 def fig_sensitivity(ctx: StageContext, cases: list[dict[str, Any]]) -> None:
365     import numpy as np
366
367     rows = [r for r in cases if r["adjust_total"] is not None]
368     labels = [f"({r['fmax']},{r['tmax']})" for r in rows]
369     x = np.arange(len(rows))
370     fig, axes = plotting.new_figure(6.5, 2.7, ncols=2)
371     ax = axes[0]
372     bottom = np.zeros(len(rows))
373     for idx, c in enumerate(CATEGORIES):
374         vals = np.array([r["vector"][3 + idx] for r in rows], dtype=float)
375         ax.bar(x, vals, bottom=bottom, color=CAT_COLOR[c], label=f"{c} 类调整", width=0.65)
376         bottom += vals
377     cancels = np.array([r["cancel_total"] for r in rows], dtype=float)
378     ax.bar(x, cancels, bottom=bottom, color="#000000", label="撤销", width=0.65)
379     for i in range(len(rows)):
380         ax.text(x[i], bottom[i] + cancels[i], str(int(bottom[i] + cancels[i])), ha="center", va="bottom", fontsize=7)
381     ax.set_xticks(x, labels, rotation=45, ha="right")
382     ax.set_xlabel("最大平移幅度 (频段, 时间)")
383     ax.set_ylabel("被调整/撤销的计划数")
384     ax.set_title("(a) 调整与撤销数量")
385     ax.set_ylim(0, float((bottom + cancels).max()) * 1.35)
386     ax.legend(frameon=False, fontsize=7, ncol=2, loc="upper left")
387     ax = axes[1]
388     ax.bar(x, [r["amplitude"] for r in rows], color="#0072B2", width=0.65)
389     for i, r in enumerate(rows):
390         ax.text(x[i], r["amplitude"], f"{r['amplitude']:.1f}", ha="center", va="bottom", fontsize=7)
391     ax.set_xticks(x, labels, rotation=45, ha="right")
392     ax.set_xlabel("最大平移幅度 (频段, 时间)")
393     ax.set_ylabel("归一化总调整幅度")
394     ax.set_title("(b) 总调整幅度")
395     plotting.save(fig, ctx.out("figures", "fig_sensitivity.pdf"))

```

```

396
397
398 @stage("figures", deps=REPORT_DEPS, description="Paper figures (vector PDF + PNG preview)")
399 def figures(ctx: StageContext) -> dict[str, Any]:
400     plans = _plans(ctx.dep_data("detect", "plans.json"))
401     conflicts = ctx.dep_data("detect", "conflicts.json")
402     stats = ctx.dep_data("detect", "stats.json")
403     fig_q1_gantt(ctx, plans, conflicts)
404     fig_q1_structure(ctx, stats)
405     q2_dec = ctx.dep_data("resolve", "decisions.json")
406     fig_resolution(ctx, "fig_q2_resolution", plans, q2_dec, conflicts, "问题 2")
407     fig_schemes(ctx, "fig_q2_schemes", ctx.dep_data("resolve", "alternatives.json"))
408     existing = _plans(ctx.dep_data("resolve", "resolved_plans.json"))
409     new_plans = _plans(ctx.dep_data("pack", "new_plans.json"))
410     pack_report = ctx.dep_data("pack", "pack_report.json")
411     fig_q3_layout(ctx, existing, new_plans, int(pack_report["horizon"]))
412     if "repack" in pack_report.get("alternatives", {}):
413         repack = ctx.dep_data("pack", "repack_plans.json")
414         fig_q3_layout(
415             ctx,
416             _plans(repack["existing"]),
417             _plans(repack["new"]),
418             int(pack_report["horizon"]),
419             name="fig_q3_repack",
420             title="既有计划经首次适配重排后 (MDR-0010 解释 B) ",
421         )
422     q4_dec = ctx.dep_data("resolve_interval", "decisions.json")
423     fig_resolution(ctx, "fig_q4_resolution", plans, q4_dec, conflicts, "问题 4")
424     q2_rep = ctx.dep_data("resolve", "solver_report.json")
425     q4_rep = ctx.dep_data("resolve_interval", "solver_report.json")
426     fig_q4_comparison(ctx, q2_rep["table"], q4_rep["table"], q4_rep["shifts"])
427     fig_bench_scaling(ctx, ctx.dep_data("bench", "scaling.json"))
428     fig_sensitivity(ctx, ctx.dep_data("sensitivity", "cases.json"))
429     files = sorted(p.name for p in (ctx.stage_dir / "figures").glob("*.pdf"))
430     ctx.number("FigureCount", len(files))
431     return {"figures": files}
432
433
434 # ----- tables
435 def _write_table(
436     ctx: StageContext,
437     name: str,
438     header: list[str],
439     rows: list[list[Any]],
440     *,
441     align: str | None = None,
442     long: bool = False,
443 ) -> None:
444     def cell(v: Any) -> str:
445         if isinstance(v, float):
446             return f"{v:.4f}".rstrip("0").rstrip(".") if abs(v - round(v)) > 1e-9 else f"{v:.0f}"
447         return tex_escape(str(v))
448
449     align = align or ("l" + "r" * (len(header) - 1))
450     lines = []
451     if long:
452         lines.append(f"\\begin{{longtable}}{{@{{}}}{align}@{{}}}}")
453         lines.append(
454             "\\toprule "
455             + " & ".join(tex_escape(h) for h in header)
456             + " \\\\ \\midrule \\endhead \\bottomrule \\endfoot"
457         )
458         lines += [" & ".join(cell(v) for v in row) + " \\\\\" for row in rows]
459         lines.append("\\end{longtable}")
460     else:
461         lines.append(f"\\begin{{tabular}}{{@{{}}}{align}@{{}}}}")
462         lines.append("\\toprule")
463         lines.append(" & ".join(tex_escape(h) for h in header) + " \\\\")
464         lines.append("\\midrule")
465         lines += [" & ".join(cell(v) for v in row) + " \\\\\" for row in rows]
466         lines.append("\\bottomrule")

```

```

467         lines.append("\\end{tabular}")
468     ctx.out("tables", f"{name}.tex").write_text("\n".join(lines) + "\n", encoding="utf-8")
469     with ctx.out("tables", f"{name}.csv").open("w", encoding="utf-8", newline="") as fh:
470         writer = csv.writer(fh)
471         writer.writerow(header)
472         writer.writerows(rows)
473
474
475 def _table1(table: dict[str, dict[str, int]]) -> list[list[Any]]:
476     rows = [[f"{c} 类", table[c]["keep"], table[c]["adjust"], table[c]["cancel"]] for c in CATEGORIES]
477     rows.append(
478         [
479             "合计",
480             sum(table[c]["keep"] for c in CATEGORIES),
481             sum(table[c]["adjust"] for c in CATEGORIES),
482             sum(table[c]["cancel"] for c in CATEGORIES),
483         ]
484     )
485     return rows
486
487
488 def _vector_text(vec: list[int] | None) -> str:
489     return "(" + ", ".join(str(v) for v in vec) + ")" if vec else "-"
490
491
492 def _adjustment_rows(plans: list[Plan], decisions: list[dict[str, Any]], with_gap: bool) -> list[list[Any]]:
493     by_id = {p.pid: p for p in plans}
494     rows = []
495     for dec in sorted(decisions, key=lambda d: d["pid"]):
496         if dec["kind"] == "keep":
497             continue
498         before = by_id[dec["pid"]]
499         after = plan_from_record(dec["plan"]) if dec["plan"] else None
500         row = [
501             dec["pid"],
502             ACTION_LABEL[dec["kind"]],
503             before.freq_text(),
504             after.freq_text() if after else "-",
505             before.time_text(),
506             after.time_text() if after else "-",
507         ]
508         if with_gap:
509             row += [before.g, after.g if after else "-"]
510         rows.append(row)
511     return rows
512
513
514 def _cancelled_rows(
515     plans: list[Plan],
516     decisions: list[dict[str, Any]],
517     conflicts: list[dict[str, Any]],
518     limits: Limits,
519 ) -> list[list[Any]]:
520     """Why a plan was cancelled.
521
522     The original conflict degree is *not* the binding reason — after the other plans move, the relevant
523     neighbourhood is a different one. The last column therefore counts the plan's legal single-parameter
524     actions that would leave it conflict-free **against the reported solution**; it is that count being
525     zero, not the degree, that forces the cancellation."""
526     by_id = {p.pid: p for p in plans}
527     neighbours: dict[str, list[str]] = {p.pid: [] for p in plans}
528     for rec in conflicts:
529         neighbours[rec["id1"]].append(rec["id2"])
530         neighbours[rec["id2"]].append(rec["id1"])
531     survivors = [plan_from_record(d["plan"]) for d in decisions if d.get("plan")]
532     rows = []
533     for dec in sorted(decisions, key=lambda d: d["pid"]):
534         if dec["kind"] != "cancel":
535             continue
536         plan = by_id[dec["pid"]]
537         nbrs = neighbours[plan.pid]

```

```

538     actions = [o for o in enumerate_options(plan, limits) if o.kind not in {"keep", "cancel"}]
539     free = 0
540     for opt in actions:
541         after = opt.apply(plan)
542         if after is not None and not any(plans_conflict(after, q) for q in survivors):
543             free += 1
544     rows.append(
545         [
546             plan.pid,
547             plan.freq_text(),
548             plan.time_text(),
549             len(nbrs),
550             sum(1 for n in nbrs if by_id[n].cat in {"A", "B"}),
551             len(actions),
552             free,
553         ]
554     )
555     return rows
556
557
558 def _conditional_table(ctx: StageContext, tag: str, rep: dict[str, Any]) -> None:
559     """Per-level evidence of the conditional problem (MDR-0012): the *set* of cancelled plans is fixed
560     to the reported solution and only the adjustment and amplitude levels are minimised. A bound proved
561     here bounds that restriction, never the unrestricted lexicographic optimum. No-op when the stage ran
562     without ``conditional_time_limits``."""
563     cond = rep.get("conditional") or {}
564     levels = cond.get("levels") or []
565     if not levels:
566         return
567     rows = [
568         [
569             entry["level"],
570             entry.get("value", "-"),
571             f"{entry['bound']:.2f}" if "bound" in entry else "-",
572             entry.get("status", "-"),
573             f"{entry.get('seconds', 0):.1f}",
574         ]
575         for entry in levels
576     ]
577     _write_table(
578         ctx,
579         f"tab_{tag}_conditional",
580         ["目标层级", "值", "已证明下界", "状态", "用时/s"],
581         rows,
582         align="lrrlr",
583     )
584
585
586 def _solver_tables(ctx: StageContext, tag: str, rep: dict[str, Any]) -> None:
587     """Per-level solver evidence (CP-SAT value/bound/status, HiGHS LP and MILP bounds) and model statistics."""
588     solver_rows = []
589     highs_levels = {e["level"]: e for e in rep["highs"]["levels"]}
590     for entry in rep["primary"]["levels"]:
591         h = highs_levels.get(entry["level"], {})
592         solver_rows.append(
593             [
594                 entry["level"],
595                 entry.get("value", "-"),
596                 f"{entry['bound']:.2f}" if "bound" in entry else "-",
597                 entry.get("status", "-"),
598                 f"{entry.get('seconds', 0):.1f}",
599                 f"{h['lp_value']:.2f}" if "lp_value" in h else "-",
600                 f"{h['mip_value']:.0f}" if "mip_value" in h and h["mip_value"] < 1e20 else "-",
601                 f"{h['mip_bound']:.2f}" if "mip_bound" in h and abs(h["mip_bound"]) < 1e20 else "-",
602                 h.get("mip_status", "-"),
603             ]
604         )
605     _write_table(
606         ctx,
607         f"tab_{tag}_solver",
608         [

```

```

609         "目标层级",
610         "CP-SAT 值",
611         "CP-SAT 下界",
612         "CP-SAT 状态",
613         "用时/s",
614         "HiGHS LP 下界",
615         "HiGHS MILP 值",
616         "HiGHS MILP 下界",
617         "HiGHS 状态",
618     ],
619     solver_rows,
620     align="lrrlrrrrl",
621 )
622 m = rep["model"]
623 _write_table(
624     ctx,
625     f"tab_{tag}_model",
626     ["项目", "数值"],
627     [
628         ["决策变量数 (计划-动作对)", m["vars"]],
629         ["时频单元数 (含动作扩展)", m["cells"]],
630         ["变量-单元隶属数", m["memberships"]],
631         ["单元格 AtMostOne 约束数 (去重前 / 去重后)", f"{m['rows_raw']} / {m['rows']}"],
632         [
633             "成对蕴含团约束数 / 强制撤销割数",
634             f"{m.get('strengthening_groups', 0)} / {m.get('forced_cancel_pairs', 0)}",
635         ],
636         ["可能相互冲突的计划对数", m.get("interacting_pairs", "-")],
637         [
638             "每计划最多动作数 (A/B/C)",
639             f"{m['options_per_plan']['A']} / {m['options_per_plan']['B']} / {m['options_per_plan']['C']}",
640         ],
641         ["CP-SAT 词典序总用时/s", f"{rep['primary']['seconds']:.1f}"],
642         ["热启动现解目标向量 (MDR-0009)", _vector_text(rep.get("hint", {}).get("vector"))],
643         [
644             "词典序求解起点",
645             {"file": "仓库内现任解", "weighted": "分离权重解"}.get(rep.get("hint", {}).get("start_from"), "冷启动"),
646         ],
647         ["各级均证明最优", "是" if rep.get("all_optimal") else "否"],
648         ["HiGHS 交叉验证总用时/s", f"{rep['highs']['seconds']:.1f}"],
649         ["HiGHS 证明最优的各级与 CP-SAT 一致", "是" if rep["highs_agree"] else "否"],
650         ["分离权重单目标解与词典序向量一致", "是" if rep["weighted_agree"] else "否"],
651     ],
652     align="lr",
653 )
654 _conditional_table(ctx, tag, rep)
655
656
657 @stage("tables", deps=REPORT_DEPS, description="Paper tables (booktabs .tex + CSV)")
658 def tables(ctx: StageContext) -> dict[str, Any]:
659     plans = _plans(ctx.dep_data("detect", "plans.json"))
660     stats = ctx.dep_data("detect", "stats.json")
661     conflicts = ctx.dep_data("detect", "conflicts.json")
662
663     # Given data, per category. The table asserts homogeneity instead of assuming it, so that the
664     # paper may quote one (w, d, g, n) per class only when every plan of that class really shares it.
665     data_rows = []
666     for cat in CATEGORIES:
667         group = [p for p in plans if p.cat == cat]
668         if not group:
669             continue
670         shapes = {(p.w, p.d, p.g, p.n) for p in group}
671         if len(shapes) != 1:
672             raise RuntimeError(f"category {cat} is not homogeneous: {sorted(shapes)}")
673         head = group[0]
674         data_rows.append(
675             [
676                 f"{cat} 类",
677                 len(group),
678                 head.w,
679                 head.d,

```

```

680         head.g,
681         head.n,
682         head.period,
683         head.d + (head.n - 1) * head.period,
684         head.n_cells,
685         len(group) * head.n_cells,
686     ]
687 )
688 ctx.number(f"DataPlans{cat}", len(group))
689 ctx.number(f"DataWidth{cat}", head.w)
690 ctx.number(f"DataDuration{cat}", head.d)
691 ctx.number(f"DataGap{cat}", head.g)
692 ctx.number(f"DataUses{cat}", head.n)
693 ctx.number(f"DataCells{cat}", head.n_cells)
694 _write_table(
695     ctx,
696     "tab_data",
697     [
698         "类别",
699         "装备数/个",
700         "频宽 w/ $\Delta f$ ",
701         "时长 d/ $\Delta t$ ",
702         "间隔 g/ $\Delta t$ ",
703         "次数 n/次",
704         "周期 (d+g)/ $\Delta t$ ",
705         "时间跨度/ $\Delta t$ ",
706         "单计划单元数/单元",
707         "该类单元合计/单元",
708     ],
709     data_rows,
710     align="lrrrrrrrr",
711 )
712
713 _write_table(
714     ctx,
715     "tab_q1_summary",
716     ["统计量", "数值"],
717     [
718         [
719             "用频计划数 (A/B/C) ",
720             f"{stats['plans_by_cat']['A']}/{stats['plans_by_cat']['B']}/{stats['plans_by_cat']['C']}",
721         ],
722         ["冲突对数", stats["pairs"]],
723         [
724             "相交的使用次对数 / 参与冲突的使用次数 / 使用次总数",
725             f"{stats['use_pairs']} / {stats['uses_in_conflict']} / {stats['uses_total']}",
726         ],
727         [
728             "卷入冲突的计划数 (A/B/C) ",
729             f"{stats['plans_involved']} ({stats['plans_involved_by_cat']['A']}/"
730             f"{stats['plans_involved_by_cat']['B']}/{stats['plans_involved_by_cat']['C']}) ",
731         ],
732         ["与任何计划都不冲突的孤立计划数", stats["isolated_plans"]],
733         ["冲突图连通分量数 (不少于 2 个节点)", stats["components"]],
734         ["最大连通分量规模", stats["largest_component"]],
735         ["最大度 / 平均度", f"{stats['max_degree']} / {stats['mean_degree']:.2f}"],
736         ["冲突图密度", f"{stats['density']:.4f}"],
737         [
738             "时频单元总数 / 被占用 / 重叠",
739             f"{stats['cells_total']} / {stats['cells_occupied']} / {stats['cells_overlap']}",
740         ],
741         ["时间范围 (最晚结束时刻,  $\Delta t$ ) ", stats["horizon"]],
742         [
743             "冲突图的最大独立集规模 (状态) ",
744             f"{stats['max_independent_set']['value']} ({stats['max_independent_set']['status']}) ",
745         ],
746         [
747             "由此得到的被触计划数下界 (计划总数减最大独立集规模) ",
748             len(plans) - stats["max_independent_set"]["value"],
749         ],
750     ],

```

```

751     align="lr",
752 )
753 matrix = [[f"{a} 类" + [stats["by_pair"][i].join(sorted((a, b)))] for b in CATEGORIES] for a in CATEGORIES]
754 _write_table(ctx, "tab_q1_category", ["类别", "A 类", "B 类", "C 类"], matrix)
755 _write_table(
756     ctx,
757     "tab_q1_top_degree",
758     ["装备编号", "冲突计划数 (度)"],
759     [[r["pid"], r["degree"]] for r in stats["top_degree"]],
760 )
761 mult = stats["use_pair_multiplicity"]
762 _write_table(ctx, "tab_q1_multiplicity", ["冲突使用次对数 k", "冲突对数量"], [[k, v] for k, v in mult.items()])
763 _write_table(
764     ctx,
765     "tab_q1_conflicts",
766     ["序号", "装备 1", "装备 2", "类别对", "冲突使用次对数", "频段重叠宽度"],
767     [
768         [k, r["id1"], r["id2"], r["pair"], r["use_pairs"], r["band_overlap"]]
769         for k, r in enumerate(sorted(conflicts, key=lambda r: (r["id1"], r["id2"])), start=1)
770     ],
771     align="rlllrr",
772     long=True,
773 )
774
775 q2 = ctx.dep_data("resolve", "solver_report.json")
776 q2_alt = ctx.dep_data("resolve", "alternatives.json")
777 q2_dec = ctx.dep_data("resolve", "decisions.json")
778 _write_table(ctx, "tab_q2_table1", ["类别", "保留数量", "调整数量", "撤销数量"], _table1(q2["table"]))
779 _write_table(
780     ctx,
781     "tab_q2_adjustments",
782     ["装备编号", "动作", "原频段", "新频段", "原首次时间", "新首次时间"],
783     _adjustment_rows(plans, q2_dec, False),
784     align="llllll",
785     long=True,
786 )
787
788 scheme_rows = []
789 for name, label in (
790     ("P", "P: 类别内嵌七级词典序 (主方案)"),
791     ("T", "T: 总量优先"),
792     ("S", "S: 不含优先级"),
793     ("W", "W: 纯加权和"),
794 ):
795     alt = q2_alt.get(name)
796     if not alt or not alt["canonical_vector"]:
797         continue
798     v = alt["canonical_vector"]
799     scheme_rows.append(
800         [
801             label,
802             f"{v[0]}/{v[1]}/{v[2]}",
803             f"{v[3]}/{v[4]}/{v[5]}",
804             sum(v[3:6]),
805             f"{v[6] / 10:.1f}",
806             f"{alt.get('budget_seconds', 0):.0f}",
807             f"{alt['seconds']:.1f}",
808             "是" if alt["all_optimal"] else "否",
809             "通过" if alt["validator_ok"] else "未通过",
810         ]
811     )
812
813 _write_table(
814     ctx,
815     "tab_q2_schemes",
816     ["目标方案", "撤销 A/B/C", "调整 A/B/C", "调整合计", "归一化幅度", "预算/s", "用时/s", "各级最优", "独立校验"],
817     scheme_rows,
818     align="lrrrrrrcc",
819 )
820
821 _solver_tables(ctx, "q2", q2)
822 _write_table(
823     ctx,
824     "tab_q2_cancelled",

```

```

822     ["装备编号", "频段", "首次时间", "原冲突邻居数", "其中 A/B 类", "合法平移动作数", "其中在本方案下无冲突"],
823     _cancelled_rows(
824         plans, q2_dec, conflicts, Limits(fmax=int(q2["limits"]["fmax"]), tmax=int(q2["limits"]["tmax"])),
825     ),
826     align="lllrrrr",
827 )
828
829 pack = ctx.dep_data("pack", "pack_report.json")
830 new_plans = _plans(ctx.dep_data("pack", "new_plans.json"))
831 bounds = pack["bounds"]
832 pack_rows = [
833     ["时间范围 T_end (问题 2 方案最晚结束时刻)", pack["horizon"]],
834     ["既有计划数 / 占用单元 / 自由单元", f"{pack['existing']} / {pack['cells_occupied']} / {pack['free_cells']}"],
835     ["候选放置位置数 (f,s)", pack["candidates"]],
836     ["AtMostOne 约束数", pack["rows"]],
837     ["贪心下界", pack["greedy"]],
838     ["CP-SAT 最优解数量 (状态)", f"{pack['cpsat']['value']} ({pack['cpsat']['status']})"],
839     ["CP-SAT 上界", bounds.get("cpsat", "-")],
840     ["HiGHS LP 松弛上界", bounds.get("lp", "-")],
841     [
842         "HiGHS MILP 解 / 上界",
843         f"{int(pack['highs_mip']['value'])} / {bounds.get('highs_mip', '-')}" if pack.get("highs_mip") else "-",
844     ],
845     ["自由单元容量界 floor(自由单元数/72)", bounds["free_cells"]],
846     ["逐频段容量界", bounds["per_band"]],
847     ["空区域容量界 (无既有计划)", pack["empty_region_bound"]],
848     ["最终结果 / 最紧上界 / gap", f"{pack['value']} / {pack['upper_bound']} / {pack['gap']}"],
849     ["时频利用率 (前 / 后)", f"{100 * pack['utilisation_before']:.1f}% / {100 * pack['utilisation_after']:.1f}%"],
850 ]
851 if pack.get("alternatives", {}).get("original_horizon"):
852     alt = pack["alternatives"]["original_horizon"]
853     pack_rows.append(
854         [f"备选: T_end 取原始计划最晚结束时刻 {alt['horizon']}", f"{alt['value']} (上界 {alt['upper_bound']})"]
855     )
856 if pack.get("alternatives", {}).get("repack"):
857     rep = pack["alternatives"]["repack"]
858     pack_rows.append(
859         [
860             "备选解释 B (既有计划可任意平移, MDR-0010): 下界 / 上界",
861             f"{rep['value']} / {rep['bounds']['free_cells']} (重排移动 {rep['existing_moved']} 个既有计划)",
862         ]
863     )
864 _write_table(ctx, "tab_q3_summary", ["项目", "数值"], pack_rows, align="lr")
865 _write_table(
866     ctx,
867     "tab_q3_plans",
868     ["序号", "频段区间", "首次时间区间", "间隔时长", "使用次数"],
869     [[k, p.freq_text(), p.time_text(), p.g, p.n] for k, p in enumerate(new_plans, start=1)],
870     align="rllrr",
871     long=True,
872 )
873 layouts = pack.get("layouts") or []
874 if layouts:
875     _write_table(
876         ctx,
877         "tab_q3_layouts",
878         [
879             "问题 2 方案",
880             "撤销数",
881             "存留计划数",
882             "时间范围/ $\Delta t$ ",
883             "自由单元数",
884             "可再安排 C 类数",
885             "上界",
886             "CP-SAT 状态",
887             "独立校验",
888         ],
889         [
890             [
891                 r["scheme"],
892                 r["cancelled"],

```

```

893         r["survivors"],
894         r["horizon"],
895         r["free_cells"],
896         r["new_plans"],
897         r["bound"],
898         r["status"],
899         "通过" if r["validator_ok"] else "未通过",
900     ]
901     for r in layouts
902 ],
903 align="lrrrrrrlc",
904 )
905
906 q4 = ctx.dep_data("resolve_interval", "solver_report.json")
907 q4_dec = ctx.dep_data("resolve_interval", "decisions.json")
908 q4_alt = ctx.dep_data("resolve_interval", "alternatives.json")
909 _write_table(ctx, "tab_q4_table1", ["类别", "保留数量", "调整数量", "撤销数量", _table1(q4["table"])])
910 cmp_rows = [
911     [
912         f"{c} 类",
913         q2["table"][c]["keep"],
914         q4["table"][c]["keep"],
915         q2["table"][c]["adjust"],
916         q4["table"][c]["adjust"],
917         q2["table"][c]["cancel"],
918         q4["table"][c]["cancel"],
919     ]
920     for c in CATEGORIES
921 ]
922 cmp_rows.append(
923     [
924         "合计",
925         sum(q2["table"][c]["keep"] for c in CATEGORIES),
926         sum(q4["table"][c]["keep"] for c in CATEGORIES),
927         sum(q2["table"][c]["adjust"] for c in CATEGORIES),
928         sum(q4["table"][c]["adjust"] for c in CATEGORIES),
929         sum(q2["table"][c]["cancel"] for c in CATEGORIES),
930         sum(q4["table"][c]["cancel"] for c in CATEGORIES),
931     ]
932 )
933 cmp_rows.append(
934     ["归一化幅度", "-", "-", f"{q2['amplitude_normalised']:.1f}", f"{q4['amplitude_normalised']:.1f}", "-", "-"]
935 )
936 _write_table(
937     ctx,
938     "tab_q4_compare",
939     ["类别", "保留(问2)", "保留(问4)", "调整(问2)", "调整(问4)", "撤销(问2)", "撤销(问4)"],
940     cmp_rows,
941 )
942 try:
943     variant = ctx.dep_data("resolve_interval", "variant_uncapped.json")
944 except Exception:
945     variant = None
946 if variant:
947     _write_table(
948         ctx,
949         "tab_q4_variant",
950         ["模型", "动作数", "目标向量", "撤销合计", "调整合计", "最晚结束时刻/ $\Delta t$ ", "用时/s", "独立校验"],
951         [
952             [
953                 "主模型: 结束时刻不超过原时间范围 (不增加时频资源)",
954                 q4["model"]["vars"],
955                 _vector_text(q4["vector"]),
956                 sum(q4["vector"][0:3]),
957                 sum(q4["vector"][3:6]),
958                 q4["horizon_after"],
959                 f"{q4['primary']['seconds']:.0f}",
960                 "通过",
961             ],
962             [
963                 "对照: 不设时间范围上限",

```

```

964         variant["vars"],
965         _vector_text(variant["vector"]),
966         sum(variant["vector"][0:3]),
967         sum(variant["vector"][3:6]),
968         variant["horizon_after"],
969         f"{variant['seconds']:.0f}",
970         "通过" if variant["validator_ok"] else "未通过",
971     ],
972 ],
973     align="lrlrrrrc",
974 )
975 _write_table(
976     ctx,
977     "tab_q4_adjustments",
978     ["装备编号", "动作", "原频段", "新频段", "原首次时间", "新首次时间", "原间隔", "新间隔"],
979     _adjustment_rows(plans, q4_dec, True),
980     align="llllllrr",
981     long=True,
982 )
983 scheme_rows4 = []
984 for name in ("P", "T", "S", "W"):
985     alt = q4_alt.get(name)
986     if alt and alt["canonical_vector"]:
987         v = alt["canonical_vector"]
988         scheme_rows4.append(
989             [
990                 name,
991                 f"{v[0]}/{v[1]}/{v[2]}",
992                 f"{v[3]}/{v[4]}/{v[5]}",
993                 sum(v[3:6]),
994                 f"{v[6] / 10:.1f}",
995                 alt["shifts"]["gap"]["count"],
996                 f"{alt.get('budget_seconds', 0):.0f}",
997                 f"{alt['seconds']:.1f}",
998                 "通过" if alt["validator_ok"] else "未通过",
999             ]
1000         )
1001 _write_table(
1002     ctx,
1003     "tab_q4_schemes",
1004     [
1005         "目标方案",
1006         "撤销 A/B/C",
1007         "调整 A/B/C",
1008         "调整合计",
1009         "归一化幅度",
1010         "间隔调整数",
1011         "预算/s",
1012         "用时/s",
1013         "独立校验",
1014     ],
1015     scheme_rows4,
1016     align="lrrrrrrrc",
1017 )
1018 _solver_tables(ctx, "q4", q4)
1019 _write_table(
1020     ctx,
1021     "tab_q4_cancelled",
1022     ["装备编号", "频段", "首次时间", "原冲突邻居数", "其中 A/B 类", "合法调整动作数", "其中在本方案下无冲突"],
1023     _cancelled_rows(
1024         plans,
1025         q4_dec,
1026         conflicts,
1027         Limits(
1028             fmax=int(q4["limits"]["fmax"]),
1029             tmax=int(q4["limits"]["tmax"]),
1030             gmax=int(q4["limits"]["gmax"]),
1031             gap_categories=tuple(q4["limits"]["gap_categories"]),
1032         ),
1033     ),
1034     align="lllrrrr",

```

```

1035 )
1036
1037 val_rows = []
1038 for stage_name, label in (
1039     ("detect", "问题 1 冲突检测"),
1040     ("resolve", "问题 2 消解方案"),
1041     ("pack", "问题 3 新增计划"),
1042     ("resolve_interval", "问题 4 消解方案"),
1043 ):
1044     rep = ctx.dep_data(stage_name, "validation_report.json")
1045     if stage_name == "detect":
1046         detail = (
1047             f"真值 {rep.get('truth_pairs')} 对, 报告 {rep.get('reported_pairs')} 对, "
1048             f"缺失 {len(rep.get('missing', []))}, 多余 {len(rep.get('extra', []))}"
1049         )
1050     elif stage_name == "pack":
1051         detail = f"新增 {rep.get('n_new')}, 剩余冲突 {rep.get('remaining_conflicts')}"
1052     else:
1053         detail = (
1054             f"{rep.get('n_decisions')} 个决策, 存留 {rep.get('n_survivors')}, "
1055             f"剩余冲突 {rep.get('remaining_conflicts')}"
1056         )
1057     val_rows.append([label, rep.get("method", ""), detail, "通过" if rep.get("ok") else "未通过"])
1058 _write_table(ctx, "tab_validation", ["对象", "独立校验方法", "校验内容", "结论"], val_rows, align="l11c")
1059
1060 tiny = ctx.dep_data("bench", "tiny.json")
1061 _write_table(
1062     ctx,
1063     "tab_bench_tiny",
1064     [
1065         "实例",
1066         "计划",
1067         "频段",
1068         "冲突对",
1069         "组合数",
1070         "穷举最优向量",
1071         "撤销非零",
1072         "CP-SAT 向量",
1073         "一致",
1074         "用时/s",
1075     ],
1076     [
1077         [
1078             r["instance"],
1079             r["plans"],
1080             r.get("bands", "-"),
1081             r["conflicts"],
1082             r["combinations"],
1083             _vector_text(r["exhaustive_vector"]),
1084             r.get("cancellations", 0),
1085             _vector_text(r["cpsat_vector"]),
1086             "是" if r["agree"] else "否",
1087             f"{r['exhaustive_seconds']:.2f}",
1088         ]
1089         for r in tiny
1090     ],
1091     align="rrrrrlrlcr",
1092     long=True,
1093 )
1094 scaling = ctx.dep_data("bench", "scaling.json")
1095 _write_table(
1096     ctx,
1097     "tab_bench_scaling",
1098     [
1099         "n",
1100         "时间范围",
1101         "冲突对",
1102         "成对检测/s (中位数)",
1103         "扫描线/s (中位数)",
1104         "变量数",
1105         "约束数",

```

```

1106         "建模/s",
1107         "CP-SAT 状态",
1108         "求解/s",
1109         "撤销合计",
1110         "调整合计",
1111     ],
1112     [
1113         [
1114             r["n"],
1115             r["horizon"],
1116             r["conflicts"],
1117             f"{r['pairwise_seconds']:.2f}",
1118             f"{r['sweep_seconds']:.3f}",
1119             r["vars"],
1120             r["rows"],
1121             f"{r['build_seconds']:.1f}",
1122             r["cpsat_status"],
1123             f"{r['cpsat_seconds']:.1f}",
1124             sum(r["vector"][0:3]) if r["vector"] else "-",
1125             sum(r["vector"][3:6]) if r["vector"] else "-",
1126         ]
1127         for r in scaling
1128     ],
1129     align="rrrrrrrlrrr",
1130 )
1131 cases = ctx.dep_data("sensitivity", "cases.json")
1132 _write_table(
1133     ctx,
1134     "tab_sensitivity",
1135     ["最大频段平移", "最大时间平移", "撤销 A/B/C", "调整 A/B/C", "调整合计", "归一化幅度", "各级最优", "用时/s"],
1136     [
1137         [
1138             r["fmax"],
1139             r["tmax"],
1140             f"{r['vector'][0]}/{r['vector'][1]}/{r['vector'][2]}",
1141             f"{r['vector'][3]}/{r['vector'][4]}/{r['vector'][5]}",
1142             r["adjust_total"],
1143             f"{r['amplitude']:.1f}",
1144             "是" if r["all_optimal"] else "否",
1145             f"{r['seconds']:.1f}",
1146         ]
1147         for r in cases
1148         if r["adjust_total"] is not None
1149     ],
1150     align="rrrrrrcr",
1151 )
1152 files = sorted(p.name for p in (ctx.stage_dir / "tables").glob("*.tex"))
1153 ctx.number("TableCount", len(files))
1154 return {"tables": files}

```

pipelines/d/resolve.py

```

1  """Conflict resolution (Problems 2 and 4): actions, time-frequency cell model, lexicographic CP-SAT,
2  separated-weight cross-checks and HiGHS MILP / LP bounds (MDR-0002, MDR-0003, MDR-0004).
3
4  Model. Plan ``i`` chooses exactly one action ``o`` (Boolean ``y[i,o]``). Two (plan, action) pairs
5  conflict iff they share a (band, time) cell, so ``AtMostOne`` over the variables covering each cell is an
6  exact encoding of "no conflict remains". Objectives are lists of levels; each level is a linear form in
7  the ``y`` variables and is minimised after fixing the optimal values of the previous levels.
8  """
9
10 from __future__ import annotations
11
12 import time
13 from dataclasses import dataclass, field
14 from typing import Any
15
16 from pipelines.d.highs_util import highs_threads

```

```

17 from pipelines.d.plans import BANDS, CATEGORIES, Plan
18
19 # amplitude normalisation (MDR-0003): |df|/10 + |dt|/5 + |dg|/10, stored x10 so that it is an integer
20 AMPLITUDE_SCALE = 10
21 FREQ_REF, TIME_REF, GAP_REF = 10, 5, 10
22
23
24 @dataclass(frozen=True)
25 class Option:
26     """One action for a plan: keep, band shift, time shift, gap change or cancel."""
27
28     kind: str
29     delta: int = 0
30
31     def apply(self, plan: Plan) -> Plan | None:
32         if self.kind == "cancel":
33             return None
34         if self.kind == "keep":
35             return plan
36         if self.kind == "freq":
37             return plan.shifted(df=self.delta)
38         if self.kind == "time":
39             return plan.shifted(dt=self.delta)
40         if self.kind == "gap":
41             return plan.shifted(gap=plan.g + self.delta)
42         raise ValueError(f"unknown action {self.kind!r}")
43
44     @property
45     def amplitude(self) -> int:
46         """Scaled amplitude (x10 of the normalised amplitude)."""
47         if self.kind == "freq":
48             return abs(self.delta) * AMPLITUDE_SCALE // FREQ_REF
49         if self.kind == "time":
50             return abs(self.delta) * AMPLITUDE_SCALE // TIME_REF
51         if self.kind == "gap":
52             return abs(self.delta) * AMPLITUDE_SCALE // GAP_REF
53         return 0
54
55     @property
56     def is_adjust(self) -> bool:
57         return self.kind in {"freq", "time", "gap"}
58
59     def label(self) -> str:
60         return self.kind if self.kind in {"keep", "cancel"} else f"{self.kind}{self.delta:+d}"
61
62
63 @dataclass(frozen=True)
64 class Limits:
65     fmax: int = 10
66     tmax: int = 5
67     gmax: int = 0
68     gap_categories: tuple[str, ...] = ()
69     allow_cancel: bool = True
70     horizon_cap: int = 0 # 0 = unrestricted; otherwise every surviving plan must end at or before the cap
71
72
73 def enumerate_options(plan: Plan, limits: Limits) -> list[Option]:
74     """All legal actions of a plan under the limits (band interval inside [0,100), start >= 0, gap >= 1).
75
76     When ``limits.horizon_cap`` is positive an action is legal only if the plan it produces ends at or
77     before the cap, i.e. the action consumes no time resource beyond the region already in use (MDR-0011).
78     """
79
80     def allowed(opt: Option) -> bool:
81         after = opt.apply(plan)
82         return not (limits.horizon_cap and after is not None and after.end > limits.horizon_cap)
83
84     opts = [Option("keep")]
85     for delta in range(-limits.fmax, limits.fmax + 1):
86         if delta and plan.f + delta >= 0 and plan.f + delta + plan.w <= BANDS:
87             opts.append(Option("freq", delta))

```

```

88     for delta in range(-limits.tmax, limits.tmax + 1):
89         if delta and plan.s + delta >= 0:
90             opts.append(Option("time", delta))
91     if plan.cat in limits.gap_categories:
92         for delta in range(-limits.gmax, limits.gmax + 1):
93             if delta and plan.g + delta >= 1:
94                 opts.append(Option("gap", delta))
95     opts = [o for o in opts if o.kind == "keep" or allowed(o)]
96     if limits.allow_cancel:
97         opts.append(Option("cancel"))
98     return opts
99
100
101 @dataclass
102 class CellModel:
103     """Variables ``y[v]`` (one per plan/action) and the cell rows over which at most one may be chosen."""
104
105     plans: list[Plan]
106     options: list[list[Option]]
107     var_plan: list[int] = field(default_factory=list)
108     var_option: list[int] = field(default_factory=list)
109     plan_vars: list[list[int]] = field(default_factory=list)
110     rows: list[list[int]] = field(default_factory=list)
111     horizon: int = 0
112     n_cells: int = 0
113     n_rows_raw: int = 0
114     memberships: int = 0
115
116     @property
117     def n_vars(self) -> int:
118         return len(self.var_plan)
119
120     def option_of(self, v: int) -> Option:
121         return self.options[self.var_plan[v]][self.var_option[v]]
122
123     def plan_of(self, v: int) -> Plan:
124         return self.plans[self.var_plan[v]]
125
126
127 def build_cell_model(plans: list[Plan], options: list[list[Option]]) -> CellModel:
128     """Expand every (plan, action) into the cells it occupies; keep rows with >= 2 variables (deduplicated)."""
129     model = CellModel(plans, options)
130     applied: list[Plan | None] = []
131     for i, plan in enumerate(plans):
132         model.plan_vars.append([])
133         for o, opt in enumerate(options[i]):
134             model.plan_vars[i].append(len(model.var_plan))
135             model.var_plan.append(i)
136             model.var_option.append(o)
137             applied.append(opt.apply(plan))
138     horizon = max((p.end for p in applied if p is not None), default=0)
139     model.horizon = horizon
140     cells: dict[int, list[int]] = {}
141     memberships = 0
142     for v, p in enumerate(applied):
143         if p is None:
144             continue
145         for start, end in p.uses():
146             for band in range(p.f, p.f_end):
147                 base = band * horizon
148                 for t in range(start, end):
149                     cells.setdefault(base + t, []).append(v)
150                     memberships += 1
151     model.n_cells = len(cells)
152     model.memberships = memberships
153     raw = [row for row in cells.values() if len(row) >= 2]
154     model.n_rows_raw = len(raw)
155     model.rows = [list(r) for r in sorted({tuple(row) for row in raw})]
156     return model
157
158

```

```

159 @dataclass
160 class Strengthening:
161     """Redundant but LP/propagation-strengthening constraints derived from the cell rows (MDR-0004).
162
163     ``groups``: for a variable v and a neighbouring plan j, the options of j that share a cell with v —
164     at most one of {v} together with the group may be chosen (a clique, since j picks exactly one option).
165     ``forced``: plan pairs whose non-cancel options all conflict pairwise → at least one must be cancelled.
166     """
167
168     groups: list[tuple[int, list[int]]] = field(default_factory=list)
169     forced: list[tuple[int, int]] = field(default_factory=list)
170     interacting_pairs: int = 0
171
172
173 def strengthen(model: CellModel) -> Strengthening:
174     conf: list[set[int]] = [set() for _ in range(model.n_vars)]
175     for row in model.rows:
176         for a in row:
177             conf[a].update(row)
178     out = Strengthening()
179     plan_pairs: set[tuple[int, int]] = set()
180     for v in range(model.n_vars):
181         conf[v].discard(v)
182         by_plan: dict[int, list[int]] = {}
183         for u in conf[v]:
184             by_plan.setdefault(model.var_plan[u], []).append(u)
185         for j, us in sorted(by_plan.items()):
186             out.groups.append((v, sorted(us)))
187             i = model.var_plan[v]
188             plan_pairs.add((min(i, j), max(i, j)))
189     non_cancel = [[v for v in vs if model.option_of(v).kind != "cancel"] for vs in model.plan_vars]
190     cancel_var = {
191         i: next((v for v in vs if model.option_of(v).kind == "cancel"), None) for i, vs in enumerate(model.plan_vars)
192     }
193     for i, j in sorted(plan_pairs):
194         if cancel_var[i] is None or cancel_var[j] is None:
195             continue
196         if all(set(non_cancel[j]) <= conf[v] for v in non_cancel[i]):
197             out.forced.append((i, j))
198     out.interacting_pairs = len(plan_pairs)
199     return out
200
201
202 def _cancel_of(model: CellModel, i: int) -> int:
203     return next(v for v in model.plan_vars[i] if model.option_of(v).kind == "cancel")
204
205
206 Level = tuple[str, dict[int, int]]
207
208
209 def _cancel_vars(model: CellModel, cats: tuple[str, ...]) -> dict[int, int]:
210     return {v: 1 for v in range(model.n_vars) if model.option_of(v).kind == "cancel" and model.plan_of(v).cat in cats}
211
212
213 def _adjust_vars(model: CellModel, cats: tuple[str, ...]) -> dict[int, int]:
214     return {v: 1 for v in range(model.n_vars) if model.option_of(v).is_adjust and model.plan_of(v).cat in cats}
215
216
217 def _amplitude_vars(model: CellModel) -> dict[int, int]:
218     return {v: model.option_of(v).amplitude for v in range(model.n_vars) if model.option_of(v).amplitude}
219
220
221 def _merge(*parts: dict[int, int]) -> dict[int, int]:
222     out: dict[int, int] = {}
223     for part in parts:
224         for v, c in part.items():
225             out[v] = out.get(v, 0) + c
226     return out
227
228
229 def canonical_levels(model: CellModel) -> list[Level]:

```

```

230     """The seven reporting levels: cancel A/B/C, adjust A/B/C, scaled amplitude (scheme P, MDR-0003)."""
231     levels: list[Level] = [(f"cancel{c}", _cancel_vars(model, (c,))) for c in CATEGORIES]
232     levels += [(f"adjust{c}", _adjust_vars(model, (c,))) for c in CATEGORIES]
233     levels.append((f"amplitude", _amplitude_vars(model)))
234     return levels
235
236
237 def scheme_levels(name: str, model: CellModel) -> list[Level]:
238     """Objective schemes P (primary), T (totals first), S (no priority), W (single weighted sum)."""
239     if name == "P":
240         return canonical_levels(model)
241     if name == "T":
242         levels: list[Level] = [(f"cancel", _cancel_vars(model, CATEGORIES)), (f"adjust", _adjust_vars(model, CATEGORIES))
243         ↪ ]
244         levels += [(f"changed{c}", _merge(_cancel_vars(model, (c,)), _adjust_vars(model, (c,)))) for c in CATEGORIES]
245         levels.append((f"amplitude", _amplitude_vars(model)))
246         return levels
247     if name == "S":
248         return [
249             (f"cancel", _cancel_vars(model, CATEGORIES)),
250             (f"adjust", _adjust_vars(model, CATEGORIES)),
251             (f"amplitude", _amplitude_vars(model)),
252         ]
253     if name == "W":
254         weight = {"A": 3, "B": 2, "C": 1}
255         terms: dict[int, int] = {}
256         for v in range(model.n_vars):
257             opt, plan = model.option_of(v), model.plan_of(v)
258             value = 0
259             if opt.kind == "cancel":
260                 value = 100 * weight[plan.cat]
261             elif opt.is_adjust:
262                 value = 30 * weight[plan.cat] + opt.amplitude
263             if value:
264                 terms[v] = value
265         return [(f"weighted", terms)]
266     raise KeyError(name)
267
268 def level_upper_bounds(model: CellModel, levels: list[Level]) -> list[int]:
269     """Upper bound of each level: each plan contributes at most the largest coefficient among its actions."""
270     bounds = []
271     for _, terms in levels:
272         total = 0
273         for vars_of_plan in model.plan_vars:
274             total += max((terms.get(v, 0) for v in vars_of_plan), default=0)
275         bounds.append(int(total))
276     return bounds
277
278
279 def separated_weights(bounds: list[int]) -> list[int]:
280     """Weights  $W_k = \prod_{j>k} (U_j + 1)$ : a single weighted sum then reproduces the lexicographic optimum."""
281     weights = [1] * len(bounds)
282     for k in range(len(bounds) - 2, -1, -1):
283         weights[k] = weights[k + 1] * (bounds[k + 1] + 1)
284     return weights
285
286
287 def evaluate(model: CellModel, chosen: list[int], levels: list[Level]) -> list[int]:
288     """Value of every level for a solution given as the chosen variable per plan."""
289     chosen_set = set(chosen)
290     return [int(sum(c for v, c in terms.items() if v in chosen_set)) for _, terms in levels]
291
292
293 def decisions_from_choice(model: CellModel, chosen: list[int]) -> list[dict[str, Any]]:
294     out = []
295     for i, plan in enumerate(model.plans):
296         opt = model.option_of(chosen[i])
297         after = opt.apply(plan)
298         out.append(
299             {

```

```

300         "pid": plan.pid,
301         "cat": plan.cat,
302         "kind": opt.kind,
303         "delta": opt.delta,
304         "label": opt.label(),
305         "plan": after.record() if after is not None else None,
306     }
307 )
308 return out
309
310
311 def table_from_decisions(decisions: list[dict[str, Any]]) -> dict[str, dict[str, int]]:
312     table = {c: {"keep": 0, "adjust": 0, "cancel": 0} for c in CATEGORIES}
313     for dec in decisions:
314         key = {"keep": "keep", "cancel": "cancel"}.get(dec["kind"], "adjust")
315         table[dec["cat"]][key] += 1
316     return table
317
318
319 def hint_from_decisions(model: CellModel, decisions: list[dict[str, Any]]) -> list[int]:
320     """Map recorded decisions (`pid`, `kind`, `delta`) onto this model's variables, one per plan.
321
322     Plans without a recorded decision, or whose recorded action does not exist under the current limits,
323     fall back to `keep`; the result is a warm-start *hint* (CP-SAT repairs infeasible hints), never a
324     constraint, so incumbents from a run with different limits can still be reused."""
325     by_pid = {str(d["pid"]): (str(d["kind"]), int(d.get("delta", 0))) for d in decisions}
326     hint: list[int] = []
327     for i, plan in enumerate(model.plans):
328         kind, delta = by_pid.get(plan.pid, ("keep", 0))
329         keep = model.plan_vars[i][0]
330         hint.append(
331             next(
332                 (v for v in model.plan_vars[i] if (model.option_of(v).kind, model.option_of(v).delta) == (kind, delta))
333                 ↪ ,
334                 keep,
335             )
336         )
337     return hint
338
339
340 def hint_is_feasible(model: CellModel, chosen: list[int]) -> bool:
341     """True when the chosen variables (one per plan) violate no cell row, i.e. no conflict remains."""
342     chosen_set = set(chosen)
343     return all(sum(1 for v in row if v in chosen_set) <= 1 for row in model.rows)
344
345
346 def _cp_base(model: CellModel, strength: Strengthening | None) -> tuple[Any, list[Any]]:
347     from ortools.sat.python import cp_model
348
349     cp = cp_model.CpModel()
350     y = [cp.NewBoolVar(f"y{v}") for v in range(model.n_vars)]
351     for vars_of_plan in model.plan_vars:
352         cp.AddExactlyOne(y[v] for v in vars_of_plan)
353     for row in model.rows:
354         cp.AddAtMostOne(y[v] for v in row)
355     if strength is not None:
356         for v, us in strength.groups:
357             cp.AddAtMostOne([y[v], *(y[u] for u in us)])
358         for i, j in strength.forced:
359             cp.AddBoolOr([y[_cancel_of(model, i)], y[_cancel_of(model, j)]])
360     return cp, y
361
362
363 def _cp_params(solver: Any, seed: int, workers: int, time_limit: float) -> None:
364     solver.parameters.random_seed = int(seed)
365     solver.parameters.num_workers = int(workers)
366     solver.parameters.max_time_in_seconds = float(time_limit)
367     if workers >= 4:
368         solver.parameters.extra_subsolvers.append("core")
369

```

```

370 def level_value(terms: dict[int, int], chosen: list[int]) -> int:
371     """Value of one level for a solution given as the chosen variable per plan."""
372     picked = set(chosen)
373     return int(sum(c for v, c in terms.items() if v in picked))
374
375
376 def solve_lexicographic_cpsat(
377     model: CellModel,
378     levels: list[Level],
379     *,
380     seed: int,
381     workers: int = 8,
382     time_limit: float | list[float] = 600.0,
383     hint: list[int] | None = None,
384     strength: Strengthening | None = None,
385     fix_vars: dict[int, int] | None = None,
386 ) -> dict[str, Any]:
387     """Minimise the levels in order with CP-SAT, fixing each level's value before the next.
388
389     Incumbent-preserving search (MDR-0009). A feasible ``hint`` becomes the incumbent ``x``; before level
390     ``k`` the cut ``L_k <= L_k(x)`` is added. Because ``x`` is feasible and satisfies every equality fixed
391     so far, that cut removes no optimal solution, so every level's optimum is unchanged — but the level can
392     never return a value worse than the incumbent. When the solver only matches the incumbent, whichever of
393     the two solutions is lexicographically smaller on the *remaining* levels is carried forward. A level that
394     reaches its time limit continues from its best-found value (recorded with its bound), so the returned
395     vector is always feasible; ``all_optimal`` tells whether every level was proven.
396
397     ``fix_vars`` pins individual variables to 0/1 before the search: it defines a *restricted* problem
398     (MDR-0012), so a bound proved under it bounds only that restriction, never the unrestricted optimum."""
399     from ortools.sat.python import cp_model
400
401     t0 = time.monotonic()
402     if isinstance(time_limit, int | float):
403         limits = [float(time_limit)] * len(levels)
404     else:
405         limits = [float(x) for x in time_limit]
406     limits += [limits[-1]] * (len(levels) - len(limits))
407     cp, y = _cp_base(model, strength)
408     for v, val in (fix_vars or {}).items():
409         cp.Add(y[v] == int(val))
410     incumbent: list[int] | None = list(hint) if hint is not None and hint_is_feasible(model, hint) else None
411     if hint is not None:
412         hinted = set(hint)
413         for v in range(model.n_vars):
414             cp.AddHint(y[v], v in hinted)
415     per_level: list[dict[str, Any]] = []
416     values: list[int] = []
417     chosen: list[int] | None = list(incumbent) if incumbent is not None else None
418     all_optimal = True
419     for k, ((name, terms), limit) in enumerate(zip(levels, limits)):
420         expr = cp_model.LinearExpr.WeightedSum([y[v] for v in terms], [c for c in terms.values()])
421         cut = level_value(terms, incumbent) if incumbent is not None else None
422         if cut is not None:
423             cp.Add(expr <= cut) # valid: the incumbent is feasible and meets every equality fixed so far
424         cp.Minimize(expr)
425         solver = cp_model.CpSolver()
426         _cp_params(solver, seed, workers, limit)
427         t1 = time.monotonic()
428         status = solver.Solve(cp)
429         name_status = solver.StatusName(status)
430         entry: dict[str, Any] = {"level": name, "status": name_status, "seconds": round(time.monotonic() - t1, 3)}
431         if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
432             value = round(solver.ObjectiveValue())
433             found = [next(v for v in vars_of_plan if solver.Value(y[v])) for vars_of_plan in model.plan_vars]
434             entry["bound"] = float(solver.BestObjectiveBound())
435         elif incumbent is not None and cut is not None:
436             # no solution recovered before the limit: fall back on the incumbent, which attains `cut`
437             value, found, entry["status"], entry["bound"] = cut, list(incumbent), "INCUMBENT", 0.0
438             name_status = "INCUMBENT"
439         else:
440             per_level.append(entry)

```

```

441         all_optimal = False
442         break
443     if incumbent is not None and cut == value:
444         # equally good here: keep whichever is lexicographically better on the levels still to come
445         tail = levels[k + 1 :]
446         if evaluate(model, incumbent, tail) < evaluate(model, found, tail):
447             found = list(incumbent)
448         incumbent = chosen = found
449         values.append(value)
450         entry["value"] = value
451         entry["cut"] = cut
452         per_level.append(entry)
453         all_optimal = all_optimal and status == cp_model.OPTIMAL
454         cp.Add(expr == value)
455         cp.ClearHints()
456         picked = set(incumbent)
457         for v in range(model.n_vars):
458             cp.AddHint(y[v], v in picked)
459     return {
460         "solver": "cp-sat",
461         "levels": per_level,
462         "vector": values,
463         "chosen": chosen,
464         "all_optimal": all_optimal,
465         "seconds": round(time.monotonic() - t0, 3),
466         "n_vars": model.n_vars,
467         "n_rows": len(model.rows),
468     }
469
470
471 def solve_weighted_cpsat(
472     model: CellModel,
473     levels: list[Level],
474     *,
475     seed: int,
476     workers: int = 8,
477     time_limit: float = 600.0,
478     hint: list[int] | None = None,
479     strength: Strengthening | None = None,
480 ) -> dict[str, Any]:
481     """Single CP-SAT solve of the separated-weight scalarisation (should reproduce the lexicographic vector)."""
482     from ortools.sat.python import cp_model
483
484     t0 = time.monotonic()
485     weights = separated_weights(level_upper_bounds(model, levels))
486     terms: dict[int, int] = {}
487     for w, (_, lv) in zip(weights, levels):
488         for v, c in lv.items():
489             terms[v] = terms.get(v, 0) + w * c
490     cp, y = _cp_base(model, strength)
491     if hint is not None:
492         hinted = set(hint)
493         for v in range(model.n_vars):
494             cp.AddHint(y[v], v in hinted)
495     cp.Minimize(cp_model.LinearExpr.WeightedSum([y[v] for v in terms], list(terms.values())))
496     solver = cp_model.CpSolver()
497     _cp_params(solver, seed, workers, time_limit)
498     status = solver.Solve(cp)
499     chosen = None
500     if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
501         chosen = [next(v for v in vars_of_plan if solver.Value(y[v])) for vars_of_plan in model.plan_vars]
502     return {
503         "solver": "cp-sat-weighted",
504         "status": solver.StatusName(status),
505         "weights": weights,
506         "objective": float(solver.ObjectiveValue()) if chosen else None,
507         "bound": float(solver.BestObjectiveBound()) if chosen else None,
508         "vector": evaluate(model, chosen, levels) if chosen else None,
509         "chosen": chosen,
510         "seconds": round(time.monotonic() - t0, 3),
511     }

```

```

512
513
514 def _highs_base(
515     model: CellModel,
516     *,
517     integer: bool,
518     time_limit: float,
519     seed: int,
520     threads: int,
521     strength: Strengthening | None = None,
522 ) -> Any:
523     import highsipy
524     import numpy as np
525
526     h = highsipy.Highs()
527     h.setOptionValue("output_flag", False)
528     h.setOptionValue("log_to_console", False)
529     h.setOptionValue("time_limit", float(time_limit))
530     h.setOptionValue("random_seed", int(seed))
531     h.setOptionValue("threads", highs_threads(threads))
532     h.setOptionValue("mip_rel_gap", 0.0)
533     h.setOptionValue("mip_abs_gap", 0.0)
534     n = model.n_vars
535     h.addVars(n, np.zeros(n), np.ones(n))
536     if integer:
537         h.changeColsIntegrality(n, np.arange(n, dtype=np.int32), np.array([highsipy.HighsVarType.kInteger] * n))
538     starts, index, lower, upper = [], [], [], []
539     nnz = 0
540     for vars_of_plan in model.plan_vars:
541         starts.append(nnz)
542         index.extend(vars_of_plan)
543         nnz += len(vars_of_plan)
544         lower.append(1.0)
545         upper.append(1.0)
546     for row in model.rows:
547         starts.append(nnz)
548         index.extend(row)
549         nnz += len(row)
550         lower.append(-highsipy.kHighsInf)
551         upper.append(1.0)
552     if strength is not None:
553         for v, us in strength.groups:
554             starts.append(nnz)
555             index.append(v)
556             index.extend(us)
557             nnz += len(us) + 1
558             lower.append(-highsipy.kHighsInf)
559             upper.append(1.0)
560         for i, j in strength.forced:
561             starts.append(nnz)
562             index.extend((_cancel_of(model, i), _cancel_of(model, j)))
563             nnz += 2
564             lower.append(1.0)
565             upper.append(highsipy.kHighsInf)
566     h.addRows(
567         len(starts),
568         np.array(lower),
569         np.array(upper),
570         nnz,
571         np.array(starts, dtype=np.int32),
572         np.array(index, dtype=np.int32),
573         np.ones(nnz),
574     )
575     return h
576
577
578 def solve_lexicographic_highs(
579     model: CellModel,
580     levels: list[Level],
581     *,
582     seed: int,

```

```

583 threads: int = 8,
584 time_limit: float = 600.0,
585 reference: list[int] | None = None,
586 hint: list[int] | None = None,
587 strength: Strengthening | None = None,
588 lp_time_limit: float | None = None,
589 ) -> dict[str, Any]:
590     """Independent cross-check in HiGHS: the same 0-1 model, level by level.
591
592     For every level the LP relaxation (previous levels fixed to the reference values, i.e. the CP-SAT vector
593     when given) yields a lower bound; the MILP is then run with a time limit. When the MILP proves optimality
594     its value must equal the reference; otherwise its dual bound and incumbent are recorded. The next level is
595     fixed to the reference value (or the MILP optimum when no reference is given)."""
596     import highspy
597     import numpy as np
598
599     t0 = time.monotonic()
600     n = model.n_vars
601     per_level: list[dict[str, Any]] = []
602     values: list[int] = []
603     chosen: list[int] | None = None
604     all_optimal = True
605     agree = True
606     idx_all = np.arange(n, dtype=np.int32)
607     fixed: list[tuple[np.ndarray, np.ndarray, float]] = [] # rows fixing the previous levels
608     lp_limit = float(lp_time_limit) if lp_time_limit is not None else float(time_limit)
609     for k, (name, terms) in enumerate(levels):
610         cost = np.zeros(n)
611         for v, c in terms.items():
612             cost[v] = c
613         entry: dict[str, Any] = {"level": name}
614         # A fresh solver per level and per relaxation: HiGHS' run clock accumulates over successive run()
615         # calls on one object, so re-using it would let the first level's time limit starve the others.
616         lp = _highs_base(model, integer=False, time_limit=lp_limit, seed=seed, threads=threads, strength=strength)
617         mip = _highs_base(model, integer=True, time_limit=time_limit, seed=seed, threads=threads, strength=strength)
618         for h in (lp, mip):
619             for f_idx, f_val, f_value in fixed:
620                 h.addRow(f_value, f_value, len(f_idx), f_idx, f_val)
621             if hint is not None:
622                 hinted = set(hint)
623                 start = highspy.HighsSolution()
624                 start.col_value = [1.0 if v in hinted else 0.0 for v in range(n)]
625                 start.value_valid = True
626                 mip.setSolution(start)
627             for tag, h in (("lp", lp), ("mip", mip)):
628                 h.changeColsCost(n, idx_all, cost)
629                 t1 = time.monotonic()
630                 h.run()
631                 status = h.getModelStatus()
632                 entry[f"{tag}_status"] = h.modelStatusToString(status)
633                 entry[f"{tag}_seconds"] = round(time.monotonic() - t1, 3)
634                 info = h.getInfo()
635                 if status == highspy.HighsModelStatus.kOptimal:
636                     entry[f"{tag}_value"] = float(info.objective_function_value)
637                 elif tag == "mip":
638                     entry["mip_value"] = float(info.objective_function_value)
639                 if tag == "mip":
640                     entry["mip_bound"] = float(info.mip_dual_bound)
641             proven = entry.get("mip_status") == "Optimal"
642             all_optimal = all_optimal and proven
643             if proven:
644                 value = round(entry["mip_value"])
645                 sol = np.array(mip.getSolution().col_value)
646                 chosen = [int(max(vars_of_plan, key=lambda v: sol[v])) for vars_of_plan in model.plan_vars]
647                 if reference is not None and k < len(reference) and value != reference[k]:
648                     agree = False
649                     entry["disagrees_with_reference"] = reference[k]
650             elif reference is not None and k < len(reference):
651                 value = int(reference[k])
652             else:
653                 per_level.append(entry)

```

```

654         break
655     if reference is not None and k < len(reference):
656         value = int(reference[k])
657         entry["fixed_value"] = value
658         values.append(value)
659         per_level.append(entry)
660         idx = np.array(list(terms), dtype=np.int32)
661         val = np.array([float(c) for c in terms.values()])
662         fixed.append((idx, val, float(value)))
663     return {
664         "solver": "highs",
665         "levels": per_level,
666         "vector": values,
667         "chosen": chosen,
668         "all_optimal": all_optimal,
669         "agrees_with_reference": agree,
670         "seconds": round(time.monotonic() - t0, 3),
671         "version": str(highspy.Highs().version()),
672     }

```

pipelines/d/stages.py

```

1  """Problem D stages: validate, detect (Q1), resolve (Q2), pack (Q3), resolve_interval (Q4), bench,
2  sensitivity, results; figures and tables live in ``pipelines.d.report`` (imported here so they register).
3
4  Conventions (docs/ENGINEERING_STANDARD.md): every stage is ``@stage(name, deps=...)`` returning
5  JSON-serialisable metrics; outputs go only to ``ctx.out(...)``; paper numbers via ``ctx.number``;
6  randomness only via ``ctx.seed_everything``; every optimisation result is re-checked by
7  ``pipelines.d.validators`` before it is written.
8  """
9
10 from __future__ import annotations
11
12 import json
13 import math
14 import time
15 from pathlib import Path
16 from typing import Any
17
18 from forge.context import StageContext
19 from forge.runner import stage
20 from forge.xlsx import write_result
21 from pipelines.common.validation import run_validation
22 from pipelines.d import validators
23 from pipelines.d.contracts import INPUT_CONTRACTS
24 from pipelines.d.detect import conflict_records, detect_bandsweep, detect_pairwise, graph_stats, overlapping_uses
25 from pipelines.d.pack import (
26     C_TEMPLATE,
27     candidate_placements,
28     cell_rows,
29     greedy_pack,
30     highs_pack,
31     occupancy_grid,
32     plans_from_choice,
33     repack_existing,
34     solve_pack_cpsat,
35     template_span,
36 )
37 from pipelines.d.plans import CATEGORIES, Plan, horizon, load_plans, plan_from_record
38 from pipelines.d.resolve import (
39     Limits,
40     build_cell_model,
41     canonical_levels,
42     decisions_from_choice,
43     enumerate_options,
44     evaluate,
45     hint_from_decisions,
46     hint_is_feasible,

```

```

47     scheme_levels,
48     solve_lexicographic_cpsat,
49     solve_lexicographic_highs,
50     solve_weighted_cpsat,
51     strengthen,
52     table_from_decisions,
53 )
54 from pipelines.d.synth import exhaustive_lexicographic, synthetic_instance, tiny_instance
55
56 PARQUET = "附件1__Sheet1.parquet"
57 Q2_LIMITS = Limits(fmax=10, tmax=5)
58 Q4_LIMITS = Limits(fmax=10, tmax=5, gmax=10, gap_categories=("C",))
59 PASS = "通过"
60 FAIL = "未通过"
61 # incumbents of an earlier verified run, used only as warm starts (configs/hints/*.json carry their provenance)
62 Q2_HINT = "configs/hints/q2_decisions.json"
63
64
65 def _limit_list(limit: Any, n: int) -> list[float]:
66     """Per-level time limits as a list of length ``n`` (a scalar is repeated, a short list is padded)."""
67     values = [float(limit)] * n if isinstance(limit, int | float) else [float(x) for x in limit]
68     return (values + [values[-1]] * n)[:n]
69
70
71 def _vector_text(vec: list[int] | None) -> str:
72     return "(" + ", ".join(str(v) for v in vec) + ")" if vec else "-"
73
74
75 def _max_independent_set(
76     n: int, edges: list[tuple[int, int]], *, seed: int, time_limit: float = 60.0
77 ) -> dict[str, Any]:
78     """Independence number  $\alpha(G)$  of the conflict graph, by CP-SAT.
79
80     Two conflicting plans can never both be kept unchanged, so the kept set is independent in  $G$ ; hence
81      $|\text{keep}| \leq \alpha(G)$  and at least  $n - \alpha(G)$  plans must be adjusted or cancelled in *any*
82     conflict-free resolution. This bound uses only the conflict graph, so it is independent of the
83     resolution model and of the action limits (Proposition on the touched-plan bound)."""
84     from ortools.sat.python import cp_model
85
86     cp = cp_model.CpModel()
87     x = [cp.NewBoolVar(f"x{i}") for i in range(n)]
88     for i, j in edges:
89         cp.AddAtMostOne([x[i], x[j]])
90     cp.Maximize(sum(x))
91     solver = cp_model.CpSolver()
92     solver.parameters.random_seed = int(seed)
93     solver.parameters.num_workers = 8
94     solver.parameters.max_time_in_seconds = float(time_limit)
95     status = solver.Solve(cp)
96     return {
97         "value": round(solver.ObjectiveValue()) if status in (cp_model.OPTIMAL, cp_model.FEASIBLE) else 0,
98         "bound": float(solver.BestObjectiveBound()),
99         "status": solver.StatusName(status),
100         "members": [i for i in range(n) if status in (cp_model.OPTIMAL, cp_model.FEASIBLE) and solver.Value(x[i])],
101     }
102
103
104 def _load_plans(ctx: StageContext) -> list[Plan]:
105     return load_plans(ctx.dep("ingest") / "data" / PARQUET)
106
107
108 def _records(plans: list[Plan]) -> list[dict[str, Any]]:
109     return [p.record() for p in plans]
110
111
112 def _plans(records: list[dict[str, Any]]) -> list[Plan]:
113     return [plan_from_record(r) for r in records]
114
115
116 @stage("validate", deps=("ingest",), description="Validate the 150 frequency-use plans against their contract")
117 def validate(ctx: StageContext) -> dict[str, Any]:

```

```

118     return run_validation(ctx, INPUT_CONTRACTS)
119
120
121 # ----- Q1
122 @stage("detect", deps=("ingest", "validate"), description="Q1: exact conflict detection with two algorithms")
123 def detect(ctx: StageContext) -> dict[str, Any]:
124     plans = _load_plans(ctx)
125     t0 = time.perf_counter()
126     pairs = detect_pairwise(plans)
127     t_pairwise = time.perf_counter() - t0
128     t0 = time.perf_counter()
129     sweep = detect_bandsweep(plans)
130     t_sweep = time.perf_counter() - t0
131     if pairs != sweep:
132         raise RuntimeError(f"detectors disagree: pairwise {len(pairs)} vs sweep {len(sweep)}")
133     records = conflict_records(plans, pairs)
134     stats = graph_stats(plans, pairs)
135     use_pairs = sum(r["use_pairs"] for r in records)
136     horizon_t = horizon(plans)
137     grid = occupancy_grid(plans, horizon_t)
138     import numpy as np
139
140     counts = np.zeros(grid.shape, dtype=int)
141     for p in plans:
142         for start, end in p.uses():
143             counts[p.f : p.f_end, start:end] += 1
144     validation = validators.validate_detection(plans, [(r["id1"], r["id2"]) for r in records])
145     if not validation["ok"]:
146         raise RuntimeError(f"independent detection check failed: {validation}")
147     multiplicity = {
148         str(k): int(sum(1 for r in records if r["use_pairs"] == k)) for k in sorted({r["use_pairs"] for r in records})
149     }
150     by_id = {p.pid: p for p in plans}
151     conflict_uses: set[tuple[str, int]] = set()
152     for rec in records:
153         a, b = by_id[rec["id1"]], by_id[rec["id2"]]
154         for k, m in overlapping_uses(a, b):
155             conflict_uses.add((a.pid, k))
156             conflict_uses.add((b.pid, m))
157     alpha = _max_independent_set(len(plans), [(i, j) for i, j in pairs], seed=ctx.seed("detect-mis"))
158     top_degree_max = max((r["degree"] for r in stats["top_degree"]), default=0)
159     top_degree_ids = [str(r["pid"]) for r in stats["top_degree"] if r["degree"] == top_degree_max]
160     stats.update(
161         {
162             "use_pairs": int(use_pairs),
163             "use_pair_multiplicity": multiplicity,
164             "uses_total": int(sum(p.n for p in plans)),
165             "uses_in_conflict": len(conflict_uses),
166             "isolated_plans": int(len(plans) - stats["plans_involved"]),
167             "top_degree_ids": top_degree_ids,
168             "max_independent_set": alpha,
169             "horizon": int(horizon_t),
170             "cells_total": int(counts.size),
171             "cells_occupied": int((counts >= 1).sum()),
172             "cells_overlap": int((counts >= 2).sum()),
173             "cells_max_multiplicity": int(counts.max()),
174             "cells_by_cat": {c: int(sum(p.n_cells for p in plans if p.cat == c)) for c in CATEGORIES},
175             "seconds_pairwise": t_pairwise,
176             "seconds_sweep": t_sweep,
177             "detectors_agree": True,
178             "pair_multiplicity_max": max((r["use_pairs"] for r in records), default=0),
179         }
180     )
181     ctx.write_json("plans.json", _records(plans))
182     ctx.write_json("conflicts.json", records)
183     ctx.write_json("stats.json", stats)
184     ctx.write_json("validation_report.json", validation)
185     ctx.number("QonePlans", len(plans))
186     ctx.number("QoneConflictPairs", stats["pairs"])
187     for key, value in stats["by_pair"].items():
188         ctx.number(f"QoneConflictPairs{key}", value)

```

```

189     ctx.number("QonePlansInConflict", stats["plans_involved"])
190     for c in CATEGORIES:
191         ctx.number(f"QonePlansInConflict{c}", stats["plans_involved_by_cat"][c])
192     ctx.number("QoneComponents", stats["components"])
193     ctx.number("QoneLargestComponent", stats["largest_component"])
194     ctx.number("QoneMaxDegree", stats["max_degree"])
195     ctx.number("QoneMeanDegree", stats["mean_degree"], ".2f")
196     ctx.number("QoneDensity", stats["density"], ".4f")
197     ctx.number("QoneUsePairs", use_pairs)
198     ctx.number("QoneUsesTotal", stats["uses_total"])
199     ctx.number("QoneConflictUses", stats["uses_in_conflict"])
200     ctx.number("QoneIsolatedPlans", stats["isolated_plans"])
201     ctx.number("QoneHorizon", horizon_t)
202     ctx.number("QoneCellsTotal", stats["cells_total"])
203     ctx.number("QoneCellsOccupied", stats["cells_occupied"])
204     ctx.number("QoneCellsOverlap", stats["cells_overlap"])
205     ctx.number("QoneOccupancyPercent", 100.0 * stats["cells_occupied"] / stats["cells_total"], ".1f")
206     ctx.number("QonePairwiseMs", 1000 * t_pairwise, ".0f")
207     ctx.number("QoneSweepMs", 1000 * t_sweep, ".0f")
208     ctx.number("QoneValidator", PASS)
209     ctx.number("QoneTopDegreeId", stats["top_degree"][0]["pid"] if stats["top_degree"] else "-")
210     ctx.number("QoneTopDegreeIds", "\n".join(top_degree_ids))
211     ctx.number("QoneTopDegreeCount", len(top_degree_ids))
212     ctx.number("QoneMaxIndependentSet", alpha["value"])
213     ctx.number("QoneMaxIndependentSetStatus", alpha["status"])
214     ctx.number("QoneMinTouched", len(plans) - alpha["value"])
215     ctx.log.info("detect.done", pairs=stats["pairs"], by_pair=stats["by_pair"], seconds=t_pairwise)
216     return {"pairs": stats["pairs"], "by_pair": stats["by_pair"], "use_pairs": use_pairs, "validator_ok": True}
217
218
219 # ----- Q2 / Q4
220 def _shift_stats(decisions: list[dict[str, Any]]) -> dict[str, Any]:
221     freq = [abs(d["delta"]) for d in decisions if d["kind"] == "freq"]
222     tim = [abs(d["delta"]) for d in decisions if d["kind"] == "time"]
223     gap = [abs(d["delta"]) for d in decisions if d["kind"] == "gap"]
224
225     def stat(xs: list[int]) -> dict[str, Any]:
226         return {"count": len(xs), "mean": (sum(xs) / len(xs)) if xs else 0.0, "max": max(xs, default=0), "sum": sum(xs)}
227
228     return {
229         "freq": stat(freq),
230         "time": stat(tim),
231         "gap": stat(gap),
232         "cancel": sum(1 for d in decisions if d["kind"] == "cancel"),
233     }
234
235
236 def _read_decisions(path: Path) -> list[dict[str, Any]]:
237     """Decisions from a stage output (a list) or an in-repo incumbent file ({'provenance', 'decisions'})."""
238     data = json.loads(path.read_text(encoding="utf-8"))
239     return list(data.get("decisions", [])) if isinstance(data, dict) else list(data)
240
241
242 def _load_hint(ctx: StageContext, model: Any, source: str | None) -> tuple[list[int] | None, dict[str, Any]]:
243     """Warm start (MDR-0004 addendum): ``source`` is a path relative to the run directory or to the repository.
244
245     Returns the hint (one variable per plan) and a small provenance record; a missing file means no hint."""
246     if not source:
247         return None, {"source": None}
248     for path in (ctx.run_dir / source, ctx.repo / source):
249         if path.exists():
250             decisions = _read_decisions(path)
251             hint = hint_from_decisions(model, decisions)
252             info = {
253                 "source": source,
254                 "path": str(path),
255                 "feasible": hint_is_feasible(model, hint),
256                 "vector": evaluate(model, hint, canonical_levels(model)),
257             }
258             ctx.log.info("hint.loaded", **info)

```

```

259         return hint, info
260     ctx.log.warn("hint.missing", source=source)
261     return None, {"source": source, "path": None}
262
263
264 def _run_resolution(
265     ctx: StageContext, plans: list[Plan], limits: Limits, prefix: str, *, default_hint: str | None = None
266 ) -> dict[str, Any]:
267     """Shared Q2/Q4 pipeline: model, lexicographic CP-SAT, cross-checks, alternatives, validation, outputs."""
268     workers = int(ctx.param("workers", 8))
269     level_limits = ctx.param("level_time_limits", None) or float(ctx.param("time_limit", 360))
270     highs_limit = float(ctx.param("highs_time_limit", 120))
271     weighted_limit = float(ctx.param("weighted_time_limit", 180))
272     alt_limit = float(ctx.param("alt_time_limit", 60))
273     seed = ctx.seed_everything(prefix)
274     options = [enumerate_options(p, limits) for p in plans]
275     t0 = time.perf_counter()
276     model = build_cell_model(plans, options)
277     strength = strengthen(model) if bool(ctx.param("strengthen", True)) else None
278     build_seconds = time.perf_counter() - t0
279     levels = canonical_levels(model)
280     ctx.log.info(
281         "model.built",
282         vars=model.n_vars,
283         rows=len(model.rows),
284         rows_raw=model.n_rows_raw,
285         groups=len(strength.groups) if strength else 0,
286         forced=len(strength.forced) if strength else 0,
287         seconds=build_seconds,
288     )
289
290     hint, hint_info = _load_hint(ctx, model, ctx.param("hint_from", default_hint))
291     # The separated-weight scalarisation runs first (MDR-0009): warm-started from the *file* incumbent it is an
292     # independent second search, and its solution seeds the lexicographic pass, so the reported vector can
293     # never be worse than either route. (Previously it was hinted by the lexicographic solution — circular.)
294     weighted = solve_weighted_cpsat(
295         model, levels, seed=seed, workers=workers, time_limit=weighted_limit, hint=hint, strength=strength
296     )
297     start = hint
298     if weighted["chosen"] is not None and hint_is_feasible(model, weighted["chosen"]):
299         if hint is None or list(weighted["vector"]) < evaluate(model, hint, levels):
300             start = weighted["chosen"]
301     hint_info["start_from"] = "file" if start is hint else "weighted"
302     hint_info["start_vector"] = evaluate(model, start, levels) if start is not None else None
303     primary = solve_lexicographic_cpsat(
304         model, levels, seed=seed, workers=workers, time_limit=level_limits, strength=strength, hint=start
305     )
306     ctx.log.info("cpsat.primary", vector=primary["vector"], levels=primary["levels"], seconds=primary["seconds"])
307     if primary["chosen"] is None or len(primary["vector"]) != len(levels):
308         raise RuntimeError(f"CP-SAT found no complete solution: {primary['levels']}")
309     if bool(ctx.param("require_optimal", False)) and not primary["all_optimal"]:
310         raise RuntimeError(f"CP-SAT did not prove optimality on every level: {primary['levels']}")
311     deterministic: bool | None = None
312     if bool(ctx.param("repeat", False)):
313         repeat = solve_lexicographic_cpsat(
314             model, levels, seed=seed, workers=workers, time_limit=level_limits, strength=strength
315         )
316         deterministic = repeat["chosen"] == primary["chosen"] and repeat["vector"] == primary["vector"]
317     weighted_agree = weighted["vector"] == primary["vector"]
318     ctx.log.info(
319         "cpsat.weighted",
320         vector=weighted["vector"],
321         status=weighted["status"],
322         agree=weighted_agree,
323         start_from=hint_info["start_from"],
324     )
325     highs = solve_lexicographic_highs(
326         model,
327         levels,
328         seed=seed,
329         threads=workers,

```

```

330         time_limit=highs_limit,
331         reference=primary["vector"],
332         hint=primary["chosen"],
333         strength=strength,
334         lp_time_limit=float(ctx.param("lp_time_limit", highs_limit)),
335     )
336     highs_agree = highs["agrees_with_reference"]
337     ctx.log.info("highs.done", levels=highs["levels"], all_optimal=highs["all_optimal"], seconds=highs["seconds"])
338
339     decisions = decisions_from_choice(model, primary["chosen"])
340     table = table_from_decisions(decisions)
341     validation = validators.validate_resolution(
342         plans,
343         decisions,
344         fmax=limits.fmax,
345         tmax=limits.tmax,
346         gmax=limits.gmax,
347         gap_categories=limits.gap_categories,
348         horizon_cap=limits.horizon_cap,
349         reported_table=table,
350         reported_vector=primary["vector"],
351     )
352     if not validation["ok"]:
353         raise RuntimeError(f"independent validation failed: {validation['errors']}")
354     survivors = [plan_from_record(d["plan"]) for d in decisions if d["plan"] is not None]
355
356     # --- objective schemes (MDR-0003). Every scheme gets the same per-level budget, and every scheme is
357     # warm-started from the lexicographically best solution known for *its own* levels among all solutions
358     # found so far (MDR-0013). With the incumbent cut of MDR-0009 this makes the reported vector of each
359     # scheme at least as good as every other scheme's solution evaluated under it, so no scheme can be
360     # left reporting a point that another row of the same table already dominates.
361     scheme_names = ("P", "T", "S", "W")
362     alternatives: dict[str, Any] = {}
363     alt_decisions_out: dict[str, Any] = {}
364     pool: list[list[int]] = [list(primary["chosen"])]
365
366     def _best_for(name: str) -> list[int]:
367         lv = scheme_levels(name, model)
368         return min(pool, key=lambda c: evaluate(model, c, lv))
369
370     def _record(name: str, res: dict[str, Any], chosen: list[int] | None, budget: float) -> None:
371         alt_decisions = decisions_from_choice(model, chosen) if chosen else []
372         alt_valid = (
373             validators.validate_resolution(
374                 plans,
375                 alt_decisions,
376                 fmax=limits.fmax,
377                 tmax=limits.tmax,
378                 gmax=limits.gmax,
379                 gap_categories=limits.gap_categories,
380                 horizon_cap=limits.horizon_cap,
381             )
382             if chosen
383             else {"ok": False}
384         )
385         alternatives[name] = {
386             "scheme_vector": res["vector"],
387             "canonical_vector": evaluate(model, chosen, levels) if chosen else None,
388             "table": table_from_decisions(alt_decisions) if chosen else None,
389             "all_optimal": res["all_optimal"],
390             "levels": res["levels"],
391             "n_levels": len(scheme_levels(name, model)),
392             "seconds": res["seconds"],
393             "budget_seconds": budget,
394             "validator_ok": bool(alt_valid["ok"]),
395             "shifts": _shift_stats(alt_decisions),
396         }
397         alt_decisions_out[name] = alt_decisions
398         ctx.log.info(
399             "alternative", scheme=name, vector=alternatives[name]["canonical_vector"], optimal=res["all_optimal"]
400         )

```

```

401
402 for name in scheme_names:
403     if name == "p":
404         _record("P", primary, primary["chosen"], sum(_limit_list(level_limits, len(levels))))
405         continue
406     lv = scheme_levels(name, model)
407     res = solve_lexicographic_cpsat(
408         model, lv, seed=seed, workers=workers, time_limit=alt_limit, hint=_best_for(name), strength=strength
409     )
410     if res["chosen"]:
411         pool.append(list(res["chosen"]))
412     _record(name, res, res["chosen"], alt_limit * len(lv))
413 # repair round: any scheme whose reported point is beaten by a solution found later is re-solved from it
414 for name in scheme_names[1:]:
415     lv = scheme_levels(name, model)
416     best = _best_for(name)
417     if evaluate(model, best, lv) < list(alternatives[name]["scheme_vector"]):
418         res = solve_lexicographic_cpsat(
419             model, lv, seed=seed, workers=workers, time_limit=alt_limit, hint=best, strength=strength
420         )
421         if res["chosen"]:
422             pool.append(list(res["chosen"]))
423         _record(name, res, res["chosen"], alternatives[name]["budget_seconds"] + alt_limit * len(lv))
424         ctx.log.info("alternative.repaired", scheme=name, vector=alternatives[name]["canonical_vector"])
425
426 # --- conditional optimality (MDR-0012). Fixing the *set* of cancelled plans to the one reported above
427 # turns the remaining levels into a restricted problem that CP-SAT can close; a bound proved here bounds
428 # the restriction only, never the unrestricted lexicographic optimum.
429 conditional: dict[str, Any] = {}
430 cond_limits = ctx.param("conditional_time_limits", None)
431 if cond_limits:
432     fix_vars: dict[int, int] = {}
433     for i, chosen_v in enumerate(primary["chosen"]):
434         cancel_v = next(v for v in model.plan_vars[i] if model.option_of(v).kind == "cancel")
435         fix_vars[cancel_v] = int(chosen_v == cancel_v)
436     cond = solve_lexicographic_cpsat(
437         model,
438         levels[3:],
439         seed=seed,
440         workers=workers,
441         time_limit=[float(x) for x in cond_limits],
442         hint=primary["chosen"],
443         strength=strength,
444         fix_vars=fix_vars,
445     )
446     cond_decisions = decisions_from_choice(model, cond["chosen"]) if cond["chosen"] else []
447     cond_valid = (
448         validators.validate_resolution(
449             plans,
450             cond_decisions,
451             fmax=limits.fmax,
452             tmax=limits.tmax,
453             gmax=limits.gmax,
454             gap_categories=limits.gap_categories,
455             horizon_cap=limits.horizon_cap,
456         )
457         if cond_decisions
458         else {"ok": False}
459     )
460     conditional = {
461         "fixed_cancelled": sorted(d["pid"] for d in decisions if d["kind"] == "cancel"),
462         "levels": cond["levels"],
463         "vector": cond["vector"],
464         "canonical_vector": evaluate(model, cond["chosen"], levels) if cond["chosen"] else None,
465         "all_optimal": cond["all_optimal"],
466         "seconds": cond["seconds"],
467         "validator_ok": bool(cond_valid["ok"]),
468     }
469     ctx.log.info("conditional", vector=cond["vector"], all_optimal=cond["all_optimal"])
470 # The restricted problem's feasible set is a subset of the unrestricted one, so a point found here
471 # is feasible there as well. When it is lexicographically better than the incumbent the unrestricted

```

```

472 # search reported, reporting the incumbent would mean knowingly publishing a dominated solution:
473 # adopt the restricted point instead and record that this happened.
474 cond_canonical = conditional["canonical_vector"]
475 adopted = bool(
476     cond["chosen"] and cond_valid["ok"] and cond_canonical and list(cond_canonical) < list(primary["vector"])
477 )
478 conditional["adopted"] = adopted
479 conditional["superseded_vector"] = list(primary["vector"]) if adopted else None
480 if adopted:
481     ctx.log.info("conditional.adopted", was=primary["vector"], now=cond_canonical)
482     primary = {**primary, "chosen": list(cond["chosen"]), "vector": list(cond_canonical)}
483     decisions = cond_decisions
484     table = table_from_decisions(decisions)
485     validation = validators.validate_resolution(
486         plans,
487         decisions,
488         fmax=limits.fmax,
489         tmax=limits.tmax,
490         gmax=limits.gmax,
491         gap_categories=limits.gap_categories,
492         horizon_cap=limits.horizon_cap,
493     )
494     if not validation["ok"]:
495         raise RuntimeError(f"adopted conditional solution failed validation: {validation['errors']}")
496 ctx.number(
497     f"{prefix}CondAdopted",
498     "未做条件求解" if not conditional else (PASS if conditional.get("adopted") else "未采用 (主搜索解不劣)"),
499 )
500 if conditional.get("superseded_vector"):
501     ctx.number(f"{prefix}CondSuperseded", _vector_text(conditional["superseded_vector"]))
502 shifts = _shift_stats(decisions)
503 report = {
504     "limits": limits.__dict__,
505     "model": {
506         "vars": model.n_vars,
507         "rows": len(model.rows),
508         "rows_raw": model.n_rows_raw,
509         "cells": model.n_cells,
510         "memberships": model.memberships,
511         "horizon": model.horizon,
512         "options_per_plan": {
513             c: max(len(options[i]) for i, p in enumerate(plans) if p.cat == c) for c in CATEGORIES
514         },
515         "build_seconds": build_seconds,
516         "strengthening_groups": len(strength.groups) if strength else 0,
517         "forced_cancel_pairs": len(strength.forced) if strength else 0,
518         "interacting_pairs": strength.interacting_pairs if strength else None,
519     },
520     "primary": {k: v for k, v in primary.items() if k != "chosen"},
521     "hint": hint_info,
522     "all_optimal": primary["all_optimal"],
523     "repeat_deterministic": deterministic,
524     "weighted": {k: v for k, v in weighted.items() if k != "chosen"},
525     "weighted_agree": weighted_agree,
526     "highs": {k: v for k, v in highs.items() if k != "chosen"},
527     "highs_agree": highs_agree,
528     "vector": primary["vector"],
529     "table": table,
530     "shifts": shifts,
531     "amplitude_normalised": primary["vector"][-1] / 10.0,
532     "horizon_after": horizon(survivors),
533     "conditional": conditional,
534 }
535 ctx.write_json("decisions.json", decisions)
536 ctx.write_json("resolved_plans.json", _records(survivors))
537 ctx.write_json("solver_report.json", report)
538 ctx.write_json("alternatives.json", alternatives)
539 ctx.write_json("alternative_decisions.json", alt_decisions_out)
540 ctx.write_json("validation_report.json", validation)
541
542 for c in CATEGORIES:

```

```

543     ctx.number(f"{prefix}Keep{c}", table[c]["keep"])
544     ctx.number(f"{prefix}Adjust{c}", table[c]["adjust"])
545     ctx.number(f"{prefix}Cancel{c}", table[c]["cancel"])
546     ctx.number(f"{prefix}KeepTotal", sum(table[c]["keep"] for c in CATEGORIES))
547     ctx.number(f"{prefix}AdjustTotal", sum(table[c]["adjust"] for c in CATEGORIES))
548     ctx.number(f"{prefix}CancelTotal", sum(table[c]["cancel"] for c in CATEGORIES))
549     ctx.number(f"{prefix}RemainingPlans", len(survivors))
550     ctx.number(f"{prefix}HorizonCap", limits.horizon_cap if limits.horizon_cap else "无")
551     ctx.number(f"{prefix}FreqShifted", shifts["freq"]["count"])
552     ctx.number(f"{prefix}TimeShifted", shifts["time"]["count"])
553     ctx.number(f"{prefix}GapChanged", shifts["gap"]["count"])
554     ctx.number(f"{prefix}MeanAbsFreqShift", shifts["freq"]["mean"], ".2f")
555     ctx.number(f"{prefix}MaxAbsFreqShift", shifts["freq"]["max"])
556     ctx.number(f"{prefix}SumAbsFreqShift", shifts["freq"]["sum"])
557     ctx.number(f"{prefix}MeanAbsTimeShift", shifts["time"]["mean"], ".2f")
558     ctx.number(f"{prefix}MaxAbsTimeShift", shifts["time"]["max"])
559     ctx.number(f"{prefix}SumAbsTimeShift", shifts["time"]["sum"])
560     ctx.number(f"{prefix}MeanAbsGapChange", shifts["gap"]["mean"], ".2f")
561     ctx.number(f"{prefix}MaxAbsGapChange", shifts["gap"]["max"])
562     ctx.number(f"{prefix}Amplitude", primary["vector"][-1] / 10.0, ".1f")
563     ctx.number(f"{prefix}AmplitudeScaled", primary["vector"][-1])
564     ctx.number(f"{prefix}Vars", model.n_vars)
565     ctx.number(f"{prefix}Rows", len(model.rows))
566     ctx.number(f"{prefix}RowsRaw", model.n_rows_raw)
567     ctx.number(f"{prefix}Memberships", model.memberships)
568     ctx.number(f"{prefix}Cells", model.n_cells)
569     ctx.number(f"{prefix}CpsatSeconds", primary["seconds"], ".1f")
570     ctx.number(f"{prefix}HighsSeconds", highs["seconds"], ".1f")
571     ctx.number(f"{prefix}WeightedSeconds", weighted["seconds"], ".1f")
572     ctx.number(f"{prefix}HighsAgree", PASS if highs_agree else FAIL)
573     ctx.number(f"{prefix}HighsAllOptimal", PASS if highs["all_optimal"] else FAIL)
574     ctx.number(f"{prefix}WeightedAgree", PASS if weighted_agree else FAIL)
575     ctx.number(f"{prefix}AllOptimal", PASS if primary["all_optimal"] else FAIL)
576     ctx.number(f"{prefix}Deterministic", "未检验" if deterministic is None else (PASS if deterministic else FAIL))
577     ctx.number(f"{prefix}ForcedCancelPairs", len(strength.forced) if strength else 0)
578     ctx.number(f"{prefix}InteractingPairs", strength.interacting_pairs if strength else 0)
579     ctx.number(f"{prefix}Groups", len(strength.groups) if strength else 0)
580     for entry in primary["levels"]:
581         key = entry["level"].capitalize()
582         ctx.number(f"{prefix}Value{key}", entry.get("value", "-"))
583         ctx.number(f"{prefix}Bound{key}", math.ceil(entry.get("bound", 0) - 1e-6) if "bound" in entry else "-")
584         ctx.number(f"{prefix}Status{key}", entry.get("status", "-"))
585         ctx.number(f"{prefix}Validator", PASS if validation["ok"] else FAIL)
586         ctx.number(f"{prefix}HorizonAfter", report["horizon_after"])
587         ctx.number(f"{prefix}HintUsed", "是" if hint is not None else "否")
588         ctx.number(f"{prefix}HintFeasible", PASS if hint_info.get("feasible") else FAIL)
589         ctx.number(f"{prefix}HintVector", _vector_text(hint_info.get("vector")))
590     hint_vec = hint_info.get("vector")
591     ctx.number(
592         f"{prefix}HintImproved",
593         "未使用提示" if hint_vec is None else ("严格改进" if list(primary["vector"]) < list(hint_vec) else "未改进"),
594     )
595     ctx.number(f"{prefix}StartVector", _vector_text(hint_info.get("start_vector")))
596     ctx.number(f"{prefix}Vector", _vector_text(primary["vector"]))
597     ctx.number(f"{prefix}Levels", len(levels))
598     for entry in highs["levels"]:
599         if "lp_value" in entry:
600             ctx.number(f"{prefix}LpBound{entry['level'].capitalize()}", entry["lp_value"], ".2f")
601     for name, alt in alternatives.items():
602         vec = alt["canonical_vector"]
603         ctx.number(f"{prefix}Scheme{name}Seconds", alt["seconds"], ".1f")
604         ctx.number(f"{prefix}Scheme{name}Budget", alt["budget_seconds"], ".0f")
605         if vec is not None:
606             ctx.number(f"{prefix}Scheme{name}AdjustTotal", sum(vec[3:6]))
607             ctx.number(f"{prefix}Scheme{name}CancelTotal", sum(vec[0:3]))
608             ctx.number(f"{prefix}Scheme{name}Amplitude", vec[6] / 10.0, ".1f")
609             ctx.number(f"{prefix}Scheme{name}AdjustA", vec[3])
610     if conditional:
611         for entry in conditional["levels"]:
612             key = entry["level"].capitalize()
613             ctx.number(f"{prefix}CondValue{key}", entry.get("value", "-"))

```

```

614         ctx.number(f"{prefix}CondBound{key}", math.ceil(entry.get("bound", 0) - 1e-6) if "bound" in entry else "-")
615         ctx.number(f"{prefix}CondStatus{key}", entry.get("status", "-"))
616         ctx.number(f"{prefix}CondAllOptimal", PASS if conditional["all_optimal"] else FAIL)
617         ctx.number(f"{prefix}CondSeconds", conditional["seconds"], ".0f")
618         ctx.number(f"{prefix}CondCancelled", len(conditional["fixed_cancelled"]))
619         ctx.number(f"{prefix}CondValidator", PASS if conditional["validator_ok"] else FAIL)
620     return {
621         "vector": primary["vector"],
622         "table": table,
623         "all_optimal": primary["all_optimal"],
624         "deterministic": deterministic,
625         "weighted_agree": weighted_agree,
626         "highs_agree": highs_agree,
627         "validator_ok": validation["ok"],
628         "cpsat_seconds": primary["seconds"],
629         "highs_seconds": highs["seconds"],
630         "horizon_after": report["horizon_after"],
631         "conditional_all_optimal": conditional.get("all_optimal"),
632     }
633
634
635 @stage("resolve", deps=("detect",), description="Q2: lexicographic conflict resolution (CP-SAT, HiGHS cross-check)")
636 def resolve(ctx: StageContext) -> dict[str, Any]:
637     plans = _plans(ctx.dep_data("detect", "plans.json"))
638     limits = Limits(fmax=int(ctx.param("fmax", Q2_LIMITS.fmax)), tmax=int(ctx.param("tmax", Q2_LIMITS.tmax)))
639     return _run_resolution(ctx, plans, limits, "Qtwo", default_hint=Q2_HINT)
640
641
642 @stage(
643     "resolve_interval",
644     deps=("detect",),
645     description="Q4: resolution with C-class gap adjustments (|dg| <= 10) as an additional single-parameter action",
646 )
647 def resolve_interval(ctx: StageContext) -> dict[str, Any]:
648     plans = _plans(ctx.dep_data("detect", "plans.json"))
649     # "no extra time-frequency resource" (MDR-0011): the resource region is the one the original plans
650     # already span, [0,100) x [0,T_end); an action that pushes a plan's last use past T_end is illegal.
651     cap = int(ctx.param("horizon_cap", horizon(plans)))
652     limits = Limits(
653         fmax=int(ctx.param("fmax", Q4_LIMITS.fmax)),
654         tmax=int(ctx.param("tmax", Q4_LIMITS.tmax)),
655         gmax=int(ctx.param("gmax", Q4_LIMITS.gmax)),
656         gap_categories=("C",),
657         horizon_cap=cap,
658     )
659     out = _run_resolution(ctx, plans, limits, "Qfour", default_hint=Q2_HINT)
660     # Unrestricted variant (MDR-0011): the same action set without the cap, warm-started from the capped
661     # solution. Because the capped solution is feasible there too, the incumbent cut of MDR-0009 makes the
662     # variant's reported vector no worse than the capped one; whether it is strictly better answers
663     # "does the cap cost anything?" without leaving the run.
664     free_limits = ctx.param("variant_time_limits", None)
665     if free_limits:
666         seed = ctx.seed("Qfour-uncapped")
667         open_limits = Limits(fmax=limits.fmax, tmax=limits.tmax, gmax=limits.gmax, gap_categories=("C",))
668         options = [enumerate_options(p, open_limits) for p in plans]
669         model = build_cell_model(plans, options)
670         levels = canonical_levels(model)
671         decisions = json.loads(ctx.out("decisions.json").read_text(encoding="utf-8"))
672         hint = hint_from_decisions(model, decisions)
673         res = solve_lexicographic_cpsat(
674             model,
675             levels,
676             seed=seed,
677             workers=int(ctx.param("workers", 8)),
678             time_limit=[float(x) for x in free_limits],
679             hint=hint,
680             strength=strengthen(model),
681         )
682         var_decisions = decisions_from_choice(model, res["chosen"]) if res["chosen"] else []
683         var_valid = (
684             validators.validate_resolution(

```

```

685         plans, var_decisions, fmax=limits.fmax, tmax=limits.tmax, gmax=limits.gmax, gap_categories=("C",)
686     )
687     if var_decisions
688     else {"ok": False}
689 )
690 survivors = [plan_from_record(d["plan"]) for d in var_decisions if d["plan"] is not None]
691 variant = {
692     "limits": open_limits.__dict__,
693     "vector": res["vector"],
694     "levels": res["levels"],
695     "table": table_from_decisions(var_decisions) if var_decisions else None,
696     "all_optimal": res["all_optimal"],
697     "seconds": res["seconds"],
698     "vars": model.n_vars,
699     "rows": len(model.rows),
700     "horizon_after": horizon(survivors),
701     "validator_ok": bool(var_valid["ok"]),
702     "strictly_better": list(res["vector"]) < list(out["vector"]),
703     "same_as_capped": list(res["vector"]) == list(out["vector"]),
704 }
705 if not var_valid["ok"]:
706     raise RuntimeError(f"unrestricted Q4 variant failed validation: {var_valid['errors']}")
707 ctx.write_json("variant_uncapped.json", variant)
708 ctx.number("QfourFreeVector", _vector_text(res["vector"]))
709 ctx.number("QfourFreeCancelTotal", sum(res["vector"][0:3]))
710 ctx.number("QfourFreeAdjustTotal", sum(res["vector"][3:6]))
711 ctx.number("QfourFreeHorizon", variant["horizon_after"])
712 ctx.number("QfourFreeVars", model.n_vars)
713 ctx.number("QfourFreeSeconds", res["seconds"], ".0f")
714 ctx.number("QfourFreeStrictlyBetter", PASS if variant["strictly_better"] else FAIL)
715 ctx.number("QfourFreeSameAsCapped", PASS if variant["same_as_capped"] else FAIL)
716 ctx.number("QfourFreeValidator", PASS)
717 ctx.log.info("q4.variant", vector=res["vector"], horizon=variant["horizon_after"])
718 out["variant_vector"] = res["vector"]
719 return out
720
721
722 @stage(
723     "coldstart",
724     deps=("detect", "resolve"),
725     description="Q2 solved from scratch (no stored incumbent), so that the reported result does not depend on it",
726 )
727 def coldstart(ctx: StageContext) -> dict[str, Any]:
728     """Answers the reviewer's question 'is the repository's answer the search's output, or its input?'.
729
730     The primary ``resolve`` stage warm-starts from ``configs/hints/q2_decisions.json`` (an incumbent of an
731     earlier run, provenance inside the file). This stage runs the identical model and objective with the
732     hint switched off, and reports the vector it reaches on its own together with the difference. Nothing
733     here feeds the delivered workbooks; it is evidence about the method, not about the answer."""
734     plans = _plans(ctx.dep_data("detect", "plans.json"))
735     limits = Limits(fmax=int(ctx.param("fmax", Q2_LIMITS.fmax)), tmax=int(ctx.param("tmax", Q2_LIMITS.tmax)))
736     ctx.params.setdefault("hint_from", "")
737     out = _run_resolution(ctx, plans, limits, "Cold", default_hint=None)
738     warm = ctx.dep_data("resolve", "solver_report.json")
739     cold_vec, warm_vec = list(out["vector"]), list(warm["vector"])
740     ctx.number("ColdMatchesWarm", PASS if cold_vec == warm_vec else FAIL)
741     ctx.number("ColdWarmVector", _vector_text(warm_vec))
742     ctx.number(
743         "ColdRelation",
744         "相同" if cold_vec == warm_vec else ("冷启动更优" if cold_vec < warm_vec else "冷启动更差"),
745     )
746     ctx.number("ColdCancelDelta", sum(cold_vec[0:3]) - sum(warm_vec[0:3]))
747     ctx.number("ColdAdjustDelta", sum(cold_vec[3:6]) - sum(warm_vec[3:6]))
748     ctx.log.info("coldstart", cold=cold_vec, warm=warm_vec)
749     return {"cold_vector": cold_vec, "warm_vector": warm_vec, "matches": cold_vec == warm_vec}
750
751
752 @stage(
753     "compare",
754     deps=("resolve", "resolve_interval"),
755     description="Q4 vs Q2: lexicographic comparison of the two resolutions",

```

```

756 )
757 def compare(ctx: StageContext) -> dict[str, Any]:
758     q2 = ctx.dep_data("resolve", "solver_report.json")
759     q4 = ctx.dep_data("resolve_interval", "solver_report.json")
760     q2_vec, q4_vec = list(q2["vector"]), list(q4["vector"])
761     not_worse = q4_vec <= q2_vec # Q4's action set is a superset of Q2's (MDR-0006)
762     comparison = {
763         "q2_vector": q2_vec,
764         "q4_vector": q4_vec,
765         "lexicographically_not_worse": not_worse,
766         "cancel_total_delta": sum(q4_vec[0:3]) - sum(q2_vec[0:3]),
767         "adjust_total_delta": sum(q4_vec[3:6]) - sum(q2_vec[3:6]),
768         "amplitude_delta": (q4_vec[6] - q2_vec[6]) / 10.0,
769         "q2_table": q2["table"],
770         "q4_table": q4["table"],
771         "q2_horizon_after": q2["horizon_after"],
772         "q4_horizon_after": q4["horizon_after"],
773         "q4_gap_changed": q4["shifts"]["gap"]["count"],
774     }
775     ctx.write_json("comparison.json", comparison)
776     ctx.number("QfourLexNotWorse", PASS if not_worse else FAIL)
777     ctx.number("QfourCancelDelta", comparison["cancel_total_delta"])
778     ctx.number("QfourAdjustDelta", comparison["adjust_total_delta"])
779     ctx.number("QfourAmplitudeDelta", comparison["amplitude_delta"], ".1f")
780     ctx.number("QfourHorizonDelta", q4["horizon_after"] - q2["horizon_after"])
781     return {"lexicographically_not_worse": not_worse, "q2": q2_vec, "q4": q4_vec}
782
783
784 # ----- Q3
785 def _pack_once(
786     ctx: StageContext,
787     existing: list[Plan],
788     horizon_t: int,
789     *,
790     tag: str,
791     time_limit: float,
792     mip_limit: float,
793     lp_limit: float,
794 ) -> dict[str, Any]:
795     import numpy as np
796
797     workers = int(ctx.param("workers", 8))
798     seed = ctx.seed(f"pack-{tag}")
799     grid = occupancy_grid(existing, horizon_t)
800     free_cells = int((~grid).sum())
801     free_by_band = (~grid).sum(axis=1)
802     cells_per_plan = C_TEMPLATE["w"] * C_TEMPLATE["d"] * C_TEMPLATE["n"]
803     t0 = time.perf_counter()
804     cands = candidate_placements(grid, C_TEMPLATE)
805     rows = cell_rows(cands, C_TEMPLATE, horizon_t)
806     build_seconds = time.perf_counter() - t0
807     greedy = greedy_pack(cands, C_TEMPLATE, grid)
808     ctx.log.info(
809         "pack.model", tag=tag, candidates=len(cands), rows=len(rows), greedy=len(greedy), seconds=build_seconds
810     )
811     cp = solve_pack_cpsat(rows, len(cands), hint=greedy, seed=seed, workers=workers, time_limit=time_limit)
812     ctx.log.info(
813         "pack.cpsat", tag=tag, status=cp["status"], value=cp["value"], bound=cp["bound"], seconds=cp["seconds"]
814     )
815     lp = (
816         highs_pack(rows, len(cands), integer=False, time_limit=lp_limit, seed=seed, threads=workers)
817         if lp_limit > 0
818         else None
819     )
820     mip = (
821         highs_pack(rows, len(cands), integer=True, time_limit=mip_limit, seed=seed, threads=workers)
822         if mip_limit > 0
823         else None
824     )
825     bounds: dict[str, float] = {
826         "free_cells": math.floor(free_cells / cells_per_plan),

```

```

827     "per_band": math.floor(
828         sum(int(b) // (C_TEMPLATE["d"] * C_TEMPLATE["n"]) for b in free_by_band) / C_TEMPLATE["w"]
829     ),
830     "cpsat": math.floor(cp["bound"] + 1e-6) if cp["bound"] is not None else math.inf,
831 }
832 if lp is not None and lp["status"] == "Optimal":
833     bounds["lp"] = math.floor(lp["value"] + 1e-6)
834 if mip is not None and mip.get("bound") is not None and math.isfinite(mip["bound"]):
835     bounds["highs_mip"] = math.floor(mip["bound"] + 1e-6)
836 upper = int(min(bounds.values()))
837 best_name, best = "cpsat", cp
838 if mip is not None and mip.get("chosen") and len(mip["chosen"]) > cp["value"]:
839     best_name, best = "highs_mip", mip
840 new_plans = plans_from_choice(cands, best["chosen"], C_TEMPLATE)
841 validation = validators.validate_packing(existing, new_plans, horizon=horizon_t, template=C_TEMPLATE)
842 if not validation["ok"]:
843     raise RuntimeError(f"independent packing validation failed: {validation['errors']}")
844 value = len(new_plans)
845 return {
846     "tag": tag,
847     "horizon": int(horizon_t),
848     "existing": len(existing),
849     "cells_total": int(grid.size),
850     "cells_occupied": int(grid.sum()),
851     "free_cells": free_cells,
852     "candidates": len(cands),
853     "rows": len(rows),
854     "build_seconds": build_seconds,
855     "greedy": len(greedy),
856     "cpsat": {k: v for k, v in cp.items() if k != "chosen"},
857     "lp": lp,
858     "highs_mip": {k: v for k, v in (mip or {}).items() if k != "chosen"} if mip else None,
859     "bounds": bounds,
860     "upper_bound": upper,
861     "value": value,
862     "gap": upper - value,
863     "optimal": upper == value,
864     "source": best_name,
865     "utilisation_before": float(grid.sum() / grid.size),
866     "utilisation_after": float((grid.sum() + value * cells_per_plan) / grid.size),
867     "new_plans": _records(new_plans),
868     "validation": validation,
869     "free_by_band": [int(b) for b in free_by_band],
870     "span": template_span(C_TEMPLATE),
871     "seed": seed,
872     "np_version": str(np.__version__),
873 }
874
875
876 @stage("pack", deps=("detect", "resolve"), description="Q3: maximum number of additional C-class plans (set packing)")
877 def pack(ctx: StageContext) -> dict[str, Any]:
878     existing = _plans(ctx.dep_data("resolve", "resolved_plans.json"))
879     original = _plans(ctx.dep_data("detect", "plans.json"))
880     horizon_t = horizon(existing)
881     result = _pack_once(
882         ctx,
883         existing,
884         horizon_t,
885         tag="main",
886         time_limit=float(ctx.param("time_limit", 900)),
887         mip_limit=float(ctx.param("mip_time_limit", 300)),
888         lp_limit=float(ctx.param("lp_time_limit", 300)),
889     )
890     ctx.write_json("new_plans.json", result["new_plans"])
891     ctx.write_json("validation_report.json", result["validation"])
892     alternatives: dict[str, Any] = {}
893     alt_horizon = horizon(original)
894     if bool(ctx.param("alt_horizon", True)) and alt_horizon != horizon_t:
895         alt = _pack_once(
896             ctx,
897             existing,

```

```

898     alt_horizon,
899     tag="original-horizon",
900     time_limit=float(ctx.param("alt_time_limit", 240)),
901     mip_limit=0,
902     lp_limit=float(ctx.param("lp_time_limit", 600)),
903 )
904     alternatives["original_horizon"] = {k: v for k, v in alt.items() if k not in {"new_plans", "free_by_band"}}
905 # MDR-0010 interpretation B: the existing plans may themselves be shifted by an unrestricted amount.
906 # A first-fit-decreasing re-placement is one feasible layout, so the number it admits is a valid lower
907 # bound on that interpretation's optimum; the free-cell capacity bound is a valid upper bound for both
908 # interpretations, because shifting never changes how many cells a plan occupies. Several construction
909 # orders are tried and the best is kept — each is feasible, so the maximum is still a valid lower bound.
910 if bool(ctx.param("repack", True)):
911     best_rep: dict[str, Any] | None = None
912     rep_trials: list[dict[str, Any]] = []
913     for order, time_first in [
914         (o, tf) for o in ctx.param("repack_orders", ["area", "width", "span", "duty"]) for tf in (False, True)
915     ]:
916         repacked, _ = repack_existing(existing, horizon_t, order=str(order), time_first=bool(time_first))
917         moved = sum(1 for a, b in zip(sorted(existing, key=lambda p: p.pid), repacked) if (a.f, a.s) != (b.f, b.s))
918         params_kept = all(
919             (a.w, a.d, a.g, a.n) == (b.w, b.d, b.g, b.n)
920             for a, b in zip(sorted(existing, key=lambda p: p.pid), repacked)
921         )
922         contained = all(0 <= p.f and p.f_end <= 100 and p.s >= 0 and p.end <= horizon_t for p in repacked)
923         if not params_kept or not contained or len(repacked) != len(existing):
924             raise RuntimeError("repack changed a plan parameter, dropped a plan, or left the resource region")
925         trial = _pack_once(
926             ctx,
927             repacked,
928             horizon_t,
929             tag=f"repack-{order}-{ 't' if time_first else 'f' }",
930             time_limit=float(ctx.param("repack_time_limit", 300)),
931             mip_limit=0,
932             lp_limit=0,
933         )
934         trial["order"], trial["time_first"], trial["existing_moved"] = str(order), bool(time_first), moved
935         trial["params_kept"] = params_kept
936         trial["repacked"] = _records(repacked)
937         rep_trials.append({k: v for k, v in trial.items() if k not in {"new_plans", "free_by_band", "repacked"}})
938         if best_rep is None or trial["value"] > best_rep["value"]:
939             best_rep = trial
940     rep = best_rep
941     assert rep is not None
942     alternatives["repack"] = {
943         k: v for k, v in rep.items() if k not in {"new_plans", "free_by_band", "validation", "repacked"}
944     } | {"trials": rep_trials, "n_trials": len(rep_trials)}
945     ctx.write_json("repack_plans.json", {"existing": rep["repacked"], "new": rep["new_plans"]})
946     ctx.log.info("pack.repack", order=rep["order"], moved=rep["existing_moved"], new_plans=rep["value"])
947     ctx.number("QthreeAltRepackNewPlans", rep["value"])
948     ctx.number("QthreeAltRepackBound", rep["bounds"]["free_cells"])
949     ctx.number("QthreeAltRepackMoved", rep["existing_moved"])
950     ctx.number("QthreeAltRepackStatus", rep["cpsat"]["status"])
951     ctx.number("QthreeAltRepackGreedy", rep["greedy"])
952     ctx.number("QthreeAltRepackTrials", len(rep_trials))
953     ctx.number("QthreeAltRepackWorst", min(t["value"] for t in rep_trials))
954     ctx.number("QthreeAltRepackUtil", 100 * rep["utilisation_after"], ".1f")
955     ctx.number("QthreeAltRepackValidator", PASS)
956 # Layout sensitivity (MDR-0014): the same packing model on each of the four objective schemes'
957 # conflict-free layouts. All four are legitimate answers to Problem 2, so the spread quantifies how
958 # conditional Q3's number is on *which* Problem-2 solution it is built upon.
959 layout_rows: list[dict[str, Any]] = []
960 if bool(ctx.param("layouts", True)):
961     alt_decisions = ctx.dep_data("resolve", "alternative_decisions.json")
962     for name, decs in sorted(alt_decisions.items()):
963         survivors = [plan_from_record(d["plan"]) for d in decs if d.get("plan")]
964         if not survivors:
965             continue
966         h = horizon(survivors)
967         res = _pack_once(
968             ctx,

```

```

969         survivors,
970         h,
971         tag=f"layout-{name}",
972         time_limit=float(ctx.param("layout_time_limit", 120)),
973         mip_limit=0,
974         lp_limit=0,
975     )
976     layout_rows.append(
977         {
978             "scheme": name,
979             "survivors": len(survivors),
980             "cancelled": 150 - len(survivors),
981             "horizon": h,
982             "free_cells": res["free_cells"],
983             "new_plans": res["value"],
984             "bound": res["upper_bound"],
985             "status": res["cpsat"]["status"],
986             "optimal": res["optimal"],
987             "validator_ok": bool(res["validation"]["ok"]),
988         }
989     )
990     ctx.log.info("pack.layout", scheme=name, free=res["free_cells"], new_plans=res["value"])
991     ctx.write_json("layouts.json", layout_rows)
992     if layout_rows:
993         ctx.number("QthreeLayoutCases", len(layout_rows))
994         ctx.number("QthreeLayoutMin", min(r["new_plans"] for r in layout_rows))
995         ctx.number("QthreeLayoutMax", max(r["new_plans"] for r in layout_rows))
996         ctx.number(
997             "QthreeLayoutSpreadPercent",
998             100.0
999             * (max(r["new_plans"] for r in layout_rows) / max(min(r["new_plans"] for r in layout_rows), 1) - 1),
1000             ".0f",
1001         )
1002         ctx.number("QthreeLayoutFreeMin", min(r["free_cells"] for r in layout_rows))
1003         ctx.number("QthreeLayoutFreeMax", max(r["free_cells"] for r in layout_rows))
1004         ctx.number(
1005             "QthreeLayoutFreeSpreadPercent",
1006             100.0
1007             * (max(r["free_cells"] for r in layout_rows) / max(min(r["free_cells"] for r in layout_rows), 1) - 1),
1008             ".1f",
1009         )
1010         ctx.number("QthreeLayoutAllValid", PASS if all(r["validator_ok"] for r in layout_rows) else FAIL)
1011         lo = min(layout_rows, key=lambda r: r["new_plans"])
1012         hi = max(layout_rows, key=lambda r: r["new_plans"])
1013         ctx.number("QthreeLayoutMinScheme", lo["scheme"])
1014         ctx.number("QthreeLayoutMaxScheme", hi["scheme"])
1015         ctx.number("QthreeLayoutMinCancelled", lo["cancelled"])
1016         ctx.number("QthreeLayoutMaxCancelled", hi["cancelled"])
1017         # Is the paper's primary scheme (the only one that never cancels an A/B plan) the worst host
1018         # for future plans? Registered rather than asserted in prose.
1019         ctx.number("QthreePrimaryIsWorstHost", PASS if lo["scheme"] == "P" else FAIL)
1020     # capacity of the empty region (what "no extra resource" could hold at most without any existing plan)
1021     empty_bound = math.floor(100 * horizon_t / (C_TEMPLATE["w"] * C_TEMPLATE["d"] * C_TEMPLATE["n"]))
1022     report = {k: v for k, v in result.items() if k not in {"new_plans"}}
1023     report["alternatives"] = alternatives
1024     report["layouts"] = layout_rows
1025     report["empty_region_bound"] = empty_bound
1026     ctx.write_json("pack_report.json", report)
1027     ctx.number("QthreeNewPlans", result["value"])
1028     ctx.number("QthreeUpperBound", result["upper_bound"])
1029     ctx.number("QthreeGap", result["gap"])
1030     ctx.number("QthreeOptimal", PASS if result["optimal"] else FAIL)
1031     ctx.number("QthreeHorizon", horizon_t)
1032     ctx.number("QthreeFreeCells", result["free_cells"])
1033     ctx.number("QthreeCellsTotal", result["cells_total"])
1034     ctx.number("QthreeCandidates", result["candidates"])
1035     ctx.number("QthreeRows", result["rows"])
1036     ctx.number("QthreeGreedy", result["greedy"])
1037     ctx.number("QthreeCpsatStatus", result["cpsat"]["status"])
1038     ctx.number("QthreeCpsatSeconds", result["cpsat"]["seconds"], ".1f")
1039     ctx.number("QthreeCpsatValue", result["cpsat"]["value"])

```

```

1040     ctx.number("QthreeCpsatBound", result["bounds"]["cpsat"] if math.isfinite(result["bounds"]["cpsat"]) else "--")
1041     ctx.number("QthreeFreeCellBound", result["bounds"]["free_cells"])
1042     ctx.number("QthreePerBandBound", result["bounds"]["per_band"])
1043     ctx.number("QthreeEmptyRegionBound", empty_bound)
1044     if "lp" in result["bounds"]:
1045         ctx.number("QthreeLpBound", result["bounds"]["lp"])
1046         ctx.number("QthreeLpValue", result["lp"]["value"], ".2f")
1047     if result["highs_mip"]:
1048         ctx.number("QthreeHighsStatus", result["highs_mip"]["status"])
1049         ctx.number("QthreeHighsValue", int(result["highs_mip"].get("value", 0)))
1050         if "highs_mip" in result["bounds"]:
1051             ctx.number("QthreeHighsBound", result["bounds"]["highs_mip"])
1052     ctx.number("QthreeUtilBefore", 100 * result["utilisation_before"], ".1f")
1053     ctx.number("QthreeUtilAfter", 100 * result["utilisation_after"], ".1f")
1054     ctx.number("QthreeValidator", PASS)
1055     # length of the time-offset template used by the vectorised candidate scan: d_C * n_C time slots,
1056     # to be distinguished from the w_C*d_C*n_C cells a plan occupies
1057     ctx.number("QthreeOffsets", C_TEMPLATE["d"] * C_TEMPLATE["n"])
1058     ctx.number("QthreeSpan", result["span"])
1059     if "original_horizon" in alternatives:
1060         ctx.number("QthreeAltHorizon", alt_horizon)
1061         ctx.number("QthreeAltNewPlans", alternatives["original_horizon"]["value"])
1062         ctx.number("QthreeAltUpperBound", alternatives["original_horizon"]["upper_bound"])
1063     return {
1064         "new_plans": result["value"],
1065         "upper_bound": result["upper_bound"],
1066         "gap": result["gap"],
1067         "status": result["cpsat"]["status"],
1068         "validator_ok": True,
1069     }
1070
1071
1072 # ----- bench
1073 @stage("bench", deps=("detect",), description="Exhaustive small-instance comparison and scaling benchmark")
1074 def bench(ctx: StageContext) -> dict[str, Any]:
1075     workers = int(ctx.param("workers", 8))
1076     seed = ctx.seed_everything("bench")
1077     tiny_limits = Limits(fmax=2, tmax=1)
1078     tiny_bands = [int(b) for b in ctx.param("tiny_bands", [6, 7, 8, 9])]
1079     tiny_rows = []
1080     agree = 0
1081     deterministic = 0
1082     cancel_instances = 0
1083     for k in range(int(ctx.param("n_tiny", 24))):
1084         n_plans = 4 + k % 3
1085         # a crowded band budget forces cancellations, so the exhaustive comparison exercises the three
1086         # cancellation levels — the levels every optimality claim in the paper rests on (MDR-0007 addendum)
1087         plans = tiny_instance(seed + k, n_plans=n_plans, bands=tiny_bands[k % len(tiny_bands)])
1088         options = [enumerate_options(p, tiny_limits) for p in plans]
1089         model = build_cell_model(plans, options)
1090         t0 = time.perf_counter()
1091         truth = exhaustive_lexicographic(plans, tiny_limits, canonical_levels, model)
1092         t_exh = time.perf_counter() - t0
1093         res = solve_lexicographic_cpsat(model, canonical_levels(model), seed=seed, workers=workers, time_limit=120)
1094         again = solve_lexicographic_cpsat(model, canonical_levels(model), seed=seed, workers=workers, time_limit=120)
1095         same = res["vector"] == truth
1096         agree += int(same)
1097         deterministic += int(again["chosen"] == res["chosen"] and again["vector"] == res["vector"])
1098         cancels = sum(truth[0:3]) if truth and len(truth) == 7 else 0
1099         cancel_instances += int(cancels > 0)
1100         combos = 1
1101         for o in options:
1102             combos *= len(o)
1103         tiny_rows.append(
1104             {
1105                 "instance": k + 1,
1106                 "plans": n_plans,
1107                 "bands": tiny_bands[k % len(tiny_bands)],
1108                 "conflicts": len(detect_pairwise(plans)),
1109                 "combinations": combos,
1110                 "exhaustive_vector": truth,

```

```

1111         "cpsat_vector": res["vector"],
1112         "cancellations": cancels,
1113         "agree": same,
1114         "exhaustive_seconds": t_exh,
1115         "cpsat_seconds": res["seconds"],
1116     }
1117 )
1118 ctx.log.info("bench.tiny", agree=agree, total=len(tiny_rows), with_cancellations=cancel_instances)
1119 ctx.write_json("tiny.json", tiny_rows)
1120
1121 scaling = []
1122 sizes = [int(v) for v in ctx.param("sizes", [150, 300, 600])]
1123 scale_limit = float(ctx.param("scale_time_limit", 120))
1124 repeats = int(ctx.param("timing_repeats", 3))
1125
1126 def _median(xs: list[float]) -> float:
1127     ordered = sorted(xs)
1128     mid = len(ordered) // 2
1129     return ordered[mid] if len(ordered) % 2 else 0.5 * (ordered[mid - 1] + ordered[mid])
1130
1131 for n in sizes:
1132     plans = synthetic_instance(n, seed + n)
1133     # a single timing is noisy, so both detectors are timed `repeats` times and the median reported
1134     pair_times, sweep_times = [], []
1135     pairs = sweep = None
1136     for _ in range(repeats):
1137         t0 = time.perf_counter()
1138         pairs = detect_pairwise(plans)
1139         pair_times.append(time.perf_counter() - t0)
1140         t0 = time.perf_counter()
1141         sweep = detect_bandsweep(plans)
1142         sweep_times.append(time.perf_counter() - t0)
1143     options = [enumerate_options(p, Q2_LIMITS) for p in plans]
1144     t0 = time.perf_counter()
1145     model = build_cell_model(plans, options)
1146     t_build = time.perf_counter() - t0
1147     levels = canonical_levels(model)
1148     res = solve_weighted_cpsat(model, levels, seed=seed, workers=workers, time_limit=scale_limit)
1149     row = {
1150         "n": n,
1151         "horizon": horizon(plans),
1152         "conflicts": len(pairs or []),
1153         "detectors_agree": pairs == sweep,
1154         "repeats": repeats,
1155         "pairwise_seconds": _median(pair_times),
1156         "sweep_seconds": _median(sweep_times),
1157         "pairwise_seconds_max": max(pair_times),
1158         "sweep_seconds_max": max(sweep_times),
1159         "vars": model.n_vars,
1160         "rows": len(model.rows),
1161         "memberships": model.memberships,
1162         "build_seconds": t_build,
1163         "cpsat_status": res["status"],
1164         "cpsat_seconds": res["seconds"],
1165         "vector": res["vector"],
1166     }
1167     scaling.append(row)
1168     ctx.log.info("bench.scale", **row)
1169 ctx.write_json("scaling.json", scaling)
1170 ctx.number("BenchTinyInstances", len(tiny_rows))
1171 ctx.number("BenchTinyAgree", agree)
1172 ctx.number("BenchTinyAllAgree", PASS if agree == len(tiny_rows) else FAIL)
1173 ctx.number("BenchTinyDeterministic", deterministic)
1174 ctx.number("BenchTinyAllDeterministic", PASS if deterministic == len(tiny_rows) else FAIL)
1175 ctx.number("BenchTinyMaxCombinations", max(r["combinations"] for r in tiny_rows))
1176 ctx.number("BenchTinyWithCancellations", cancel_instances)
1177 ctx.number("BenchTinyMaxCancellations", max(r["cancellations"] for r in tiny_rows))
1178 ctx.number("BenchScaleMaxN", max(sizes))
1179 ctx.number("BenchScaleMinN", min(sizes))
1180 ctx.number("BenchScaleRepeats", repeats)
1181 # "Max" over every size, not the value at the largest size

```

```

1182     ctx.number("BenchScaleMaxPairwiseSeconds", max(r["pairwise_seconds"] for r in scaling), ".2f")
1183     ctx.number("BenchScaleMaxSweepSeconds", max(r["sweep_seconds"] for r in scaling), ".2f")
1184     ctx.number("BenchScaleMaxCpsatSeconds", max(r["cpsat_seconds"] for r in scaling), ".1f")
1185     ctx.number("BenchScaleLastPairwiseSeconds", scaling[-1]["pairwise_seconds"], ".2f")
1186     ctx.number("BenchScaleLastSweepSeconds", scaling[-1]["sweep_seconds"], ".2f")
1187     ctx.number("BenchScaleMaxStatus", scaling[-1]["cpsat_status"])
1188     ctx.number("BenchScaleAllOptimal", PASS if all(r["cpsat_status"] == "OPTIMAL" for r in scaling) else FAIL)
1189     ctx.number("BenchScaleDetectorsAgree", PASS if all(r["detectors_agree"] for r in scaling) else FAIL)
1190     return {
1191         "tiny_agree": agree,
1192         "tiny_deterministic": deterministic,
1193         "tiny_total": len(tiny_rows),
1194         "tiny_with_cancellations": cancel_instances,
1195         "scaling": [(r["n"], r["cpsat_status"], round(r["cpsat_seconds"], 1)) for r in scaling],
1196     }
1197
1198
1199 # ----- sensitivity
1200 @stage(
1201     "sensitivity",
1202     deps=("detect", "resolve"),
1203     description="Q2 sensitivity to the maximum shift amplitudes (base case taken from `resolve`, others warm-started)",
1204 )
1205 def sensitivity(ctx: StageContext) -> dict[str, Any]:
1206     workers = int(ctx.param("workers", 8))
1207     seed = ctx.seed_everything("sensitivity")
1208     plans = _plans(ctx.dep_data("detect", "plans.json"))
1209     base_report = ctx.dep_data("resolve", "solver_report.json")
1210     base_decisions = ctx.dep_data("resolve", "decisions.json")
1211     base_limits = (int(base_report["limits"]["fmax"]), int(base_report["limits"]["tmax"]))
1212     cases = ctx.param("cases", [[5, 2], [5, 5], [10, 2], [10, 5], [15, 5], [20, 10]])
1213     level_limits = ctx.param("level_time_limits", None) or float(ctx.param("time_limit", 60))
1214     rows = []
1215     for fmax, tmax in cases:
1216         limits = Limits(fmax=int(fmax), tmax=int(tmax))
1217         options = [enumerate_options(p, limits) for p in plans]
1218         model = build_cell_model(plans, options)
1219         if (limits.fmax, limits.tmax) == base_limits:
1220             # identical model: reuse the primary Q2 solution instead of re-solving with a shorter limit
1221             decisions = base_decisions
1222             vec = list(base_report["vector"])
1223             all_optimal, seconds, source = (
1224                 bool(base_report["all_optimal"]),
1225                 float(base_report["primary"]["seconds"]),
1226                 "resolve",
1227             )
1228         else:
1229             hint = hint_from_decisions(model, base_decisions)
1230             res = solve_lexicographic_cpsat(
1231                 model,
1232                 canonical_levels(model),
1233                 seed=seed,
1234                 workers=workers,
1235                 time_limit=level_limits,
1236                 hint=hint,
1237                 strength=strengthen(model),
1238             )
1239             decisions = decisions_from_choice(model, res["chosen"]) if res["chosen"] else []
1240             vec, all_optimal, seconds, source = res["vector"], res["all_optimal"], res["seconds"], "cp-sat"
1241         valid = (
1242             validators.validate_resolution(plans, decisions, fmax=limits.fmax, tmax=limits.tmax)
1243             if decisions
1244             else {"ok": False}
1245         )
1246         rows.append(
1247             {
1248                 "fmax": int(fmax),
1249                 "tmax": int(tmax),
1250                 "vars": model.n_vars,
1251                 "vector": vec,
1252                 "cancel_total": sum(vec[0:3]) if len(vec) == 7 else None,

```

```

1253         "adjust_total": sum(vec[3:6]) if len(vec) == 7 else None,
1254         "amplitude": vec[6] / 10.0 if len(vec) == 7 else None,
1255         "table": table_from_decisions(decisions) if decisions else None,
1256         "all_optimal": all_optimal,
1257         "seconds": seconds,
1258         "source": source,
1259         "validator_ok": bool(valid["ok"]),
1260         "shifts": _shift_stats(decisions),
1261     }
1262 )
1263 ctx.log.info("sensitivity.case", fmax=fmax, tmax=tmax, vector=vec, seconds=seconds, source=source)
1264 ctx.write_json("cases.json", rows)
1265 base = next((r for r in rows if (r["fmax"], r["tmax"]) == base_limits), rows[0])
1266 # lexicographic monotonicity: a larger action set can never give a worse vector than the base solution
1267 monotone = all(
1268     list(r["vector"]) <= list(base["vector"])
1269     for r in rows
1270     if len(r["vector"]) == 7 and r["fmax"] >= base_limits[0] and r["tmax"] >= base_limits[1]
1271 ) and all(
1272     list(r["vector"]) >= list(base["vector"])
1273     for r in rows
1274     if len(r["vector"]) == 7 and r["fmax"] <= base_limits[0] and r["tmax"] <= base_limits[1]
1275 )
1276 ctx.number("SensCases", len(rows))
1277 ctx.number("SensBaseAdjustTotal", base["adjust_total"])
1278 ctx.number("SensBaseCancelTotal", base["cancel_total"])
1279 ctx.number("SensMinAdjustTotal", min(r["adjust_total"] for r in rows if r["adjust_total"] is not None))
1280 ctx.number("SensMaxAdjustTotal", max(r["adjust_total"] for r in rows if r["adjust_total"] is not None))
1281 ctx.number("SensMaxCancelTotal", max(r["cancel_total"] for r in rows if r["cancel_total"] is not None))
1282 ctx.number("SensMinCancelTotal", min(r["cancel_total"] for r in rows if r["cancel_total"] is not None))
1283 ctx.number("SensAllOptimal", PASS if all(r["all_optimal"] for r in rows) else FAIL)
1284 ctx.number("SensAllValid", PASS if all(r["validator_ok"] for r in rows) else FAIL)
1285 ctx.number("SensMonotone", PASS if monotone else FAIL)
1286 # Per-case macros, so that the paper can attribute a change to one parameter at a time instead of
1287 # quoting the extremum over the whole grid (which varies both limits at once).
1288 words = {
1289     2: "Two",
1290     3: "Three",
1291     4: "Four",
1292     5: "Five",
1293     6: "Six",
1294     9: "Nine",
1295     10: "Ten",
1296     11: "Eleven",
1297     15: "Fifteen",
1298     20: "Twenty",
1299 }
1300 by_case = {(r["fmax"], r["tmax"]): r for r in rows}
1301 for (fmax, tmax), r in by_case.items():
1302     if fmax in words and tmax in words and r["cancel_total"] is not None:
1303         key = f"SensPhi-{words[fmax]}Tau-{words[tmax]}"
1304         ctx.number(f"{key}Cancel", r["cancel_total"])
1305         ctx.number(f"{key}Adjust", r["adjust_total"])
1306
1307 # One-at-a-time elasticities: cancellations avoided per extra unit of each limit, holding the other
1308 # at the value the problem statement gives. Reported instead of the raw "twice as effective" claim,
1309 # which compares a +5 df increment against a +3 dt increment.
1310 def _cancel(fmax: int, tmax: int) -> int | None:
1311     r = by_case.get((fmax, tmax))
1312     return None if r is None else r["cancel_total"]
1313
1314 base_f, base_t = int(base["fmax"]), int(base["tmax"])
1315 lo_f, lo_t = _cancel(5, base_t), _cancel(base_f, 2)
1316 if lo_f is not None and base["cancel_total"] is not None:
1317     ctx.number("SensFreqElasticity", (lo_f - base["cancel_total"]) / (base_f - 5), ".1f")
1318     ctx.number("SensFreqLowCancel", lo_f)
1319 if lo_t is not None and base["cancel_total"] is not None:
1320     ctx.number("SensTimeElasticity", (lo_t - base["cancel_total"]) / (base_t - 2), ".1f")
1321     ctx.number("SensTimeLowCancel", lo_t)
1322 if lo_f is not None and lo_t is not None and base["cancel_total"] is not None:
1323     per_f = (lo_f - base["cancel_total"]) / (base_f - 5)

```

```

1324     per_t = (lo_t - base["cancel_total"]) / (base_t - 2)
1325     ctx.number("SensElasticityRatio", per_f / per_t if per_t else float("nan"), ".2f")
1326     return {"cases": [(r["fmax"], r["tmax"], r["vector"]) for r in rows], "monotone": monotone}
1327
1328
1329 # ----- results
1330 @stage(
1331     "results",
1332     deps=("detect", "resolve", "pack", "resolve_interval"),
1333     description="Fill result1-4.xlsx from the organisers' templates after every validator passed",
1334 )
1335 def results(ctx: StageContext) -> dict[str, Any]:
1336     for name in ("detect", "resolve", "pack", "resolve_interval"):
1337         report = ctx.dep_data(name, "validation_report.json")
1338         if not report.get("ok"):
1339             raise RuntimeError(f"validator of stage {name} did not pass; refusing to write result workbooks")
1340     templates = ctx.templates
1341     written: dict[str, Any] = {}
1342
1343     conflicts = ctx.dep_data("detect", "conflicts.json")
1344     rows1 = [
1345         [k, r["id1"], r["id2"]] for k, r in enumerate(sorted(conflicts, key=lambda r: (r["id1"], r["id2"])), start=1)
1346     ]
1347     written["result1.xlsx"] = write_result(
1348         templates / "result1.xlsx", ctx.out("result1.xlsx"), {"Sheet1": ([("序号", "冲突装备1", "冲突设备2"), rows1])}
1349     )
1350
1351     def adjustment_rows(decisions: list[dict[str, Any]], with_gap: bool) -> list[list[Any]]:
1352         rows: list[list[Any]] = []
1353         for dec in sorted(decisions, key=lambda d: d["pid"]):
1354             if dec["kind"] == "keep":
1355                 continue
1356             plan = plan_from_record(dec["plan"]) if dec["plan"] else None
1357             freq = plan.freq_text() if plan and dec["kind"] == "freq" else None
1358             tim = plan.time_text() if plan and dec["kind"] == "time" else None
1359             gap = plan.g if plan and dec["kind"] == "gap" else None
1360             cancel = "是" if dec["kind"] == "cancel" else None
1361             rows.append([dec["pid"], freq, tim, gap, cancel] if with_gap else [dec["pid"], freq, tim, cancel])
1362         return rows
1363
1364     rows2 = adjustment_rows(ctx.dep_data("resolve", "decisions.json"), with_gap=False)
1365     written["result2.xlsx"] = write_result(
1366         templates / "result2.xlsx",
1367         ctx.out("result2.xlsx"),
1368         {"Sheet1": ([("用频装备编号", "调整后频段区间", "调整后时间区间", "是否撤销用频计划"], rows2))},
1369     )
1370     new_plans = _plans(ctx.dep_data("pack", "new_plans.json"))
1371     rows3 = [[k, p.freq_text(), p.time_text()] for k, p in enumerate(new_plans, start=1)]
1372     written["result3.xlsx"] = write_result(
1373         templates / "result3.xlsx",
1374         ctx.out("result3.xlsx"),
1375         {"Sheet1": ([("新增用频装备序号", "调整后频段区间", "调整后时间区间"], rows3))},
1376     )
1377     rows4 = adjustment_rows(ctx.dep_data("resolve_interval", "decisions.json"), with_gap=True)
1378     written["result4.xlsx"] = write_result(
1379         templates / "result4.xlsx",
1380         ctx.out("result4.xlsx"),
1381         {"Sheet1": ([("用频装备编号", "调整后频段范围", "调整后时间区间", "调整后间隔时长", "是否撤销用频计划"], rows4))},
1382     )
1383     ctx.write_json("results_report.json", written)
1384     ctx.number("ResultOneRows", len(rows1))
1385     ctx.number("ResultTwoRows", len(rows2))
1386     ctx.number("ResultThreeRows", len(rows3))
1387     ctx.number("ResultFourRows", len(rows4))
1388     return {name: info["sheets"]["Sheet1"]["rows"] for name, info in written.items()}
1389
1390
1391 from pipelines.d import report as _report # noqa: E402,F401 (registers figures/tables stages)

```

pipelines/d/synth.py

```
1 """Synthetic instances and an exhaustive reference solver for benchmarking (MDR-0007)."""
2
3 from __future__ import annotations
4
5 import random
6 from itertools import combinations
7 from typing import Any
8
9 from pipelines.d.detect import plans_conflict
10 from pipelines.d.plans import BANDS, Plan
11 from pipelines.d.resolve import CellModel, Level, Limits, Option, enumerate_options
12
13 CATEGORY_PARAMS = {"A": (10, 5, 60, 3), "B": (15, 3, 40, 4), "C": (3, 2, 8, 12)} # (w, d, g, n)
14 CATEGORY_SHARE = {"A": 20, "B": 40, "C": 90}
15
16
17 def synthetic_instance(n: int, seed: int, *, base_n: int = 150, base_start_max: int = 531) -> list[Plan]:
18     """`n` plans with the attachment's category mix and parameters; the time range scales with `n`
19     so that the occupancy density stays comparable to the real instance."""
20     rng = random.Random(seed)
21     start_max = max(1, round(base_start_max * n / base_n))
22     plans: list[Plan] = []
23     counts = {c: round(n * share / base_n) for c, share in CATEGORY_SHARE.items()}
24     counts["C"] += n - sum(counts.values())
25     for cat, count in counts.items():
26         w, d, g, m = CATEGORY_PARAMS[cat]
27         for k in range(count):
28             plans.append(
29                 Plan(f"{cat}-{k + 1:03d}", cat, rng.randint(0, BANDS - w), w, rng.randint(0, start_max), d, g, m)
30             )
31     return plans
32
33
34 def tiny_instance(seed: int, *, n_plans: int = 5, bands: int = 12) -> list[Plan]:
35     """A crowded miniature instance for exhaustive comparison."""
36     rng = random.Random(seed)
37     plans = []
38     for k in range(n_plans):
39         cat = rng.choice("ABC")
40         w = rng.randint(2, 3)
41         plans.append(
42             Plan(
43                 f"{cat}-{k + 1:03d}",
44                 cat,
45                 rng.randint(0, bands - w),
46                 w,
47                 rng.randint(0, 6),
48                 rng.randint(1, 2),
49                 rng.randint(1, 3),
50                 rng.randint(2, 3),
51             )
52         )
53     return plans
54
55
56 def exhaustive_lexicographic(plans: list[Plan], limits: Limits, levels_fn: Any, model: CellModel) -> list[int] | None:
57     """Depth-first enumeration of every action combination; returns the lexicographically smallest level vector.
58
59     ``levels_fn(model)`` gives the levels; conflicts are decided with ``plans_conflict`` on the applied plans,
60     i.e. by interval arithmetic rather than by the cell rows, so the comparison with CP-SAT is meaningful."""
61     levels: list[Level] = levels_fn(model)
62     n = len(plans)
63     applied: list[list[Plan | None]] = [[opt.apply(p) for opt in model.options[i]] for i, p in enumerate(plans)]
64     conflict: dict[tuple[int, int, int, int], bool] = {}
65     for i, j in combinations(range(n), 2):
66         for oi, pi in enumerate(applied[i]):
67             for oj, pj in enumerate(applied[j]):
68                 conflict[(i, oi, j, oj)] = pi is not None and pj is not None and plans_conflict(pi, pj)
```

```

69     best: list[int] | None = None
70     choice: list[int] = []
71
72     def rec(i: int) -> None:
73         nonlocal best
74         if i == n:
75             vec = [sum(terms.get(model.plan_vars[k][choice[k]], 0) for k in range(n)) for _, terms in levels]
76             if best is None or vec < best:
77                 best = vec
78             return
79         for o in range(len(model.options[i])):
80             if all(not conflict[(k, choice[k], i, o)] for k in range(i)):
81                 choice.append(o)
82                 rec(i + 1)
83                 choice.pop()
84
85     rec(0)
86     return best
87
88
89 def options_for(plans: list[Plan], limits: Limits) -> list[list[Option]]:
90     return [enumerate_options(p, limits) for p in plans]

```

pipelines/d/validators.py

```

1  """Independent validators for Problem D (deliberately separate from detect.py / resolve.py / pack.py).
2
3  Every check here re-derives what a plan occupies from first principles — the plan is expanded into an
4  explicit set of (band, time) cells and conflicts are found by set intersection — so that agreement with
5  the interval-arithmetic detectors and the CP-SAT/HiGHS solutions is genuine, not tautological.
6  """
7
8  from __future__ import annotations
9
10 from itertools import combinations
11 from typing import Any
12
13 from pipelines.d.plans import BANDS, CATEGORIES, Plan
14
15 KINDS = ("keep", "freq", "time", "gap", "cancel")
16
17
18 def cell_set(plan: Plan) -> set[tuple[int, int]]:
19     """Every (band, time) cell the plan occupies, from the definition of its uses."""
20     cells: set[tuple[int, int]] = set()
21     for k in range(plan.n):
22         start = plan.s + k * (plan.d + plan.g)
23         for band in range(plan.f, plan.f + plan.w):
24             for t in range(start, start + plan.d):
25                 cells.add((band, t))
26     return cells
27
28
29 def conflicts_by_cells(plans: list[Plan]) -> list[tuple[str, str]]:
30     """All conflicting id pairs (id1 < id2), found by intersecting cell sets."""
31     cells = {p.pid: cell_set(p) for p in plans}
32     out = []
33     for a, b in combinations(plans, 2):
34         if cells[a.pid] & cells[b.pid]:
35             out.append((min(a.pid, b.pid), max(a.pid, b.pid)))
36     return sorted(out)
37
38
39 def plan_bounds_errors(plan: Plan) -> list[str]:
40     errors = []
41     if plan.f < 0 or plan.f + plan.w > BANDS:
42         errors.append(f"{plan.pid}: bands {plan.f}..{plan.f + plan.w} outside [0,{BANDS})")
43     if plan.s < 0:

```

```

44     errors.append(f"{plan.pid}: start {plan.s} < 0")
45 if plan.g < 1 or plan.n < 1 or plan.d < 1 or plan.w < 1:
46     errors.append(f"{plan.pid}: non-positive parameter (w={plan.w}, d={plan.d}, g={plan.g}, n={plan.n})")
47 return errors
48
49
50 def validate_detection(plans: list[Plan], reported: list[tuple[str, str]]) -> dict[str, Any]:
51     """Compare a reported conflict list with the cell-set ground truth."""
52     truth = conflicts_by_cells(plans)
53     rep = sorted((min(a, b), max(a, b)) for a, b in reported)
54     missing = sorted(set(truth) - set(rep))
55     extra = sorted(set(rep) - set(truth))
56     duplicates = len(rep) - len(set(rep))
57     ok = not missing and not extra and duplicates == 0
58     return {
59         "ok": ok,
60         "method": "cell-set intersection",
61         "truth_pairs": len(truth),
62         "reported_pairs": len(rep),
63         "missing": missing[:20],
64         "extra": extra[:20],
65         "duplicates": duplicates,
66     }
67
68
69 def _changed_fields(before: Plan, after: Plan) -> list[str]:
70     return [name for name in ("f", "w", "s", "d", "g", "n") if getattr(before, name) != getattr(after, name)]
71
72
73 def validate_resolution(
74     original: list[Plan],
75     decisions: list[dict[str, Any]],
76     *,
77     fmax: int,
78     tmax: int,
79     gmax: int = 0,
80     gap_categories: tuple[str, ...] = (),
81     horizon_cap: int = 0,
82     reported_table: dict[str, dict[str, int]] | None = None,
83     reported_vector: list[int] | None = None,
84 ) -> dict[str, Any]:
85     """Check a resolution: one legal action per plan, one changed parameter, amplitude limits, bounds,
86     zero conflicts among surviving plans, and (optionally) the reported Table-1 statistics / objective."""
87     errors: list[str] = []
88     by_id = {p.pid: p for p in original}
89     seen: set[str] = set()
90     survivors: list[Plan] = []
91     table = {c: {"keep": 0, "adjust": 0, "cancel": 0} for c in CATEGORIES}
92     vector = {"cancel": dict.fromkeys(CATEGORIES, 0), "adjust": dict.fromkeys(CATEGORIES, 0), "amplitude": 0}
93     for dec in decisions:
94         pid = str(dec["pid"])
95         kind = str(dec["kind"])
96         if pid not in by_id:
97             errors.append(f"unknown plan {pid}")
98             continue
99         if pid in seen:
100             errors.append(f"{pid}: more than one decision")
101             continue
102         seen.add(pid)
103         before = by_id[pid]
104         if kind not in KINDS:
105             errors.append(f"{pid}: unknown action {kind!r}")
106             continue
107         if kind == "cancel":
108             table[before.cat]["cancel"] += 1
109             vector["cancel"][before.cat] += 1
110             continue
111         after = Plan(**dec["plan"]) if dec.get("plan") else before
112         if after.pid != pid or after.cat != before.cat:
113             errors.append(f"{pid}: identity changed in adjusted plan")
114         changed = _changed_fields(before, after)

```

```

115     expected = {"keep": [], "freq": ["f"], "time": ["s"], "gap": ["g"]}[kind]
116     if changed != expected:
117         errors.append(f"{pid}: action {kind} but changed fields {changed}")
118     errors += plan_bounds_errors(after)
119     if kind == "freq" and abs(after.f - before.f) > fmax:
120         errors.append(f"{pid}: band shift {after.f - before.f} exceeds {fmax}")
121     if kind == "time" and abs(after.s - before.s) > tmax:
122         errors.append(f"{pid}: time shift {after.s - before.s} exceeds {tmax}")
123     if kind == "gap":
124         if before.cat not in gap_categories:
125             errors.append(f"{pid}: gap change not allowed for category {before.cat}")
126         if abs(after.g - before.g) > gmax:
127             errors.append(f"{pid}: gap change {after.g - before.g} exceeds {gmax}")
128     survivors.append(after)
129     if kind == "keep":
130         table[before.cat]["keep"] += 1
131     else:
132         table[before.cat]["adjust"] += 1
133         vector["adjust"][before.cat] += 1
134         vector["amplitude"] += {
135             "freq": abs(after.f - before.f),
136             "time": 2 * abs(after.s - before.s),
137             "gap": abs(after.g - before.g),
138         }[kind]
139     undecided = sorted(set(by_id) - seen)
140     if undecided:
141         errors.append(f"{len(undecided)} plans without decision, e.g. {undecided[:5]}")
142     if horizon_cap:
143         late = [p.pid for p in survivors if p.end > horizon_cap]
144         if late:
145             errors.append(f"{len(late)} surviving plans end after the horizon cap {horizon_cap}, e.g. {late[:5]}")
146     remaining = conflicts_by_cells(survivors)
147     if remaining:
148         errors.append(f"{len(remaining)} conflicts remain, e.g. {remaining[:5]}")
149     if reported_table is not None and reported_table != table:
150         errors.append(f"reported table {reported_table} differs from recomputed {table}")
151     recomputed_vector = [vector["cancel"][c] for c in CATEGORIES] + [vector["adjust"][c] for c in CATEGORIES]
152     recomputed_vector.append(int(vector["amplitude"]))
153     if reported_vector is not None and list(reported_vector) != recomputed_vector:
154         errors.append(f"reported objective {reported_vector} differs from recomputed {recomputed_vector}")
155     return {
156         "ok": not errors,
157         "method": "independent per-plan legality checks + cell-set conflict scan",
158         "errors": errors[:30],
159         "n_decisions": len(decisions),
160         "n_survivors": len(survivors),
161         "remaining_conflicts": len(remaining),
162         "table": table,
163         "vector": recomputed_vector,
164         "amplitude_normalised": vector["amplitude"] / 10.0,
165         "horizon": max((p.end for p in survivors), default=0),
166     }
167
168
169 def validate_packing(
170     existing: list[Plan], new_plans: list[Plan], *, horizon: int, template: dict[str, int]
171 ) -> dict[str, Any]:
172     """Check new plans: template parameters, bounds, inside the horizon, no conflicts with anything."""
173     errors: list[str] = []
174     for p in new_plans:
175         if (p.w, p.d, p.g, p.n) != (template["w"], template["d"], template["g"], template["n"]):
176             errors.append(f"{p.pid}: parameters {(p.w, p.d, p.g, p.n)} differ from template")
177         if p.cat != "C":
178             errors.append(f"{p.pid}: category {p.cat} is not C")
179         errors += plan_bounds_errors(p)
180         if p.end > horizon:
181             errors.append(f"{p.pid}: ends at {p.end} > horizon {horizon}")
182     ids = [p.pid for p in new_plans]
183     if len(ids) != len(set(ids)) or set(ids) & {p.pid for p in existing}:
184         errors.append("duplicate or clashing plan ids")
185     remaining = conflicts_by_cells(existing + new_plans)

```

```

186     if remaining:
187         errors.append(f"{len(remaining)} conflicts, e.g. {remaining[:5]}")
188     return {
189         "ok": not errors,
190         "method": "parameter/bound checks + cell-set conflict scan over existing and new plans",
191         "errors": errors[:30],
192         "n_new": len(new_plans),
193         "remaining_conflicts": len(remaining),
194         "cells_new": sum(p.n_cells for p in new_plans),
195     }

```

pyproject.toml

```

1 [project]
2 name = "cumcm2026-d"
3 version = "0.1.0"
4 description = "时频冲突检测与消解 — cloud-only reproducible modelling study (CUMCM 2026 Problem D)"
5 requires-python = ">=3.11"
6 license = "Unlicense"
7
8 [tool.ruff]
9 line-length = 120
10 target-version = "py311"
11 extend-exclude = ["artifacts", "releases", "manuscript"]
12
13 [tool.ruff.lint]
14 select = ["E", "F", "W", "I", "B", "UP", "NPY", "RUF", "S102"]
15 ignore = ["RUF001", "RUF002", "RUF003", "E741", "B905"]
16
17 [tool.ruff.lint.per-file-ignores]
18 "tests/**" = ["S101", "B011"]
19
20 [tool.mypy]
21 python_version = "3.11"
22 ignore_missing_imports = true
23 check_untyped_defs = true
24 warn_unused_ignores = false
25 no_implicit_optional = true
26 exclude = "(artifacts|releases|manuscript)/"
27
28 [tool.pytest.ini_options]
29 testpaths = ["tests"]
30 addopts = "-p no:cacheprovider -q"
31 markers = ["integration: needs the mounted repository inputs (runs in Modal)"]
32
33 [tool.coverage.run]
34 source = ["forge", "pipelines"]

```

tests/__init__.py

tests/conftest.py

```

1 """Shared test configuration: deterministic hypothesis profile, no on-disk example database."""
2
3 from __future__ import annotations
4
5 import os
6 from pathlib import Path
7
8 import pytest

```

```

9 from hypothesis import HealthCheck, settings
10
11 settings.register_profile(
12     "forge", database=None, deadline=None, max_examples=60, suppress_health_check=[HealthCheck.too_slow]
13 )
14 settings.load_profile("forge")
15
16 REPO = Path(os.environ.get("FORGE_REPO", Path(__file__).resolve().parents[1]))
17
18
19 @pytest.fixture(scope="session")
20 def repo() -> Path:
21     return REPO
22
23
24 @pytest.fixture
25 def tmp_repo(tmp_path: Path) -> Path:
26     """A minimal repository layout (configs only) for runtime tests that must not touch real data."""
27     (tmp_path / "configs").mkdir()
28     (tmp_path / "configs" / "default.toml").write_text("[run]\nseed = 7\n[paper]\nsources = []\n", encoding="utf-8")
29     (tmp_path / "configs" / "project.json").write_text(
30         '{"letter": "T", "package": "t", "problem_pdf": "T.pdf", "attachments_dir": "att", '
31         '"templates_dir": "att/tpl", "result_files": []}',
32         encoding="utf-8",
33     )
34     (tmp_path / "att" / "tpl").mkdir(parents=True)
35     return tmp_path

```

tests/integration/__init__.py

tests/integration/test_ingest_validate.py

```

1 """End-to-end ingest + validate against the real attachments (runs inside Modal where /repo exists)."""
2
3 from __future__ import annotations
4
5 import json
6 import os
7 from pathlib import Path
8
9 import pytest
10
11 from forge import runner
12
13 REPO = Path(os.environ.get("FORGE_REPO", "/repo"))
14 pytestmark = pytest.mark.integration
15
16
17 @pytest.mark.skipif(not (REPO / "configs" / "project.json").exists(), reason="repository inputs not mounted")
18 def test_ingest_then_validate_on_real_inputs(tmp_path: Path) -> None:
19     vol = tmp_path / "vol"
20     ingest = runner.execute("ingest", "itest", {"code_ref": {"git_sha": "test"}}, False, REPO, vol)
21     assert ingest["status"] == "completed" and ingest["metrics"]["files"] >= 1
22     inventory = json.loads((vol / "runs" / "itest" / "ingest" / "inventory.json").read_text(encoding="utf-8"))
23     assert any(not f["template"] for f in inventory["files"])
24     validate = runner.execute("validate", "itest", {}, False, REPO, vol)
25     assert validate["status"] == "completed" and validate["metrics"]["ok"]

```

tests/unit/__init__.py

tests/unit/test_config.py

```
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import pytest
6
7 from forge.config import Config, deep_merge, load_config
8
9
10 def test_deep_merge_is_recursive_and_non_destructive() -> None:
11     base = {"a": {"x": 1, "y": 2}, "b": 1}
12     merged = deep_merge(base, {"a": {"y": 3}, "c": 4})
13     assert merged == {"a": {"x": 1, "y": 3}, "b": 1, "c": 4}
14     assert base == {"a": {"x": 1, "y": 2}, "b": 1}
15
16
17 def test_config_dotted_access_and_digest() -> None:
18     cfg = Config({"paper": {"sources": ["q1"], "max_body_pages": 30}})
19     assert cfg.get("paper.max_body_pages") == 30
20     assert cfg.get("paper.missing", "d") == "d"
21     assert cfg["paper.sources"] == ["q1"]
22     with pytest.raises(KeyError):
23         cfg["nope.nope"]
24     assert cfg.digest == Config({"paper": {"max_body_pages": 30, "sources": ["q1"]}}).digest
25
26
27 def test_load_config_with_profile_and_overrides(tmp_repo: Path) -> None:
28     (tmp_repo / "configs" / "fast.toml").write_text("[run]\nseed = 99\n", encoding="utf-8")
29     cfg = load_config(tmp_repo, profile="fast", overrides={"paper": {"sources": ["x"]}})
30     assert cfg.get("run.seed") == 99
31     assert cfg.get("paper.sources") == ["x"]
```

tests/unit/test_contracts.py

```
1 from __future__ import annotations
2
3 import numpy as np
4 import pandas as pd
5 from hypothesis import given
6 from hypothesis import strategies as st
7
8 from forge.contracts import Column, FrameContract, validate_frame
9
10 CONTRACT = FrameContract(
11     name="t",
12     columns=(
13         Column("t", "int", min=0, unique=True, monotonic="increasing", step=60),
14         Column("v", "float", min=0, max=1),
15     ),
16     min_rows=2,
17 )
18
19
20 def test_valid_frame_passes() -> None:
21     df = pd.DataFrame({"t": [0, 60, 120], "v": [0.1, 0.2, 0.3]})
22     report = validate_frame(df, CONTRACT)
23     assert report["ok"], report["errors"]
24
25
```

```

26 def test_violations_are_reported_precisely() -> None:
27     df = pd.DataFrame({"t": [0, 60, 60], "v": [0.1, 1.5, np.nan], "extra": [1, 2, 3]})
28     report = validate_frame(df, CONTRACT)
29     joined = " | ".join(report["errors"])
30     assert not report["ok"]
31     assert "unexpected columns" in joined
32     assert "duplicated" in joined
33     assert "max 1.5 > 1" in joined
34     assert "null" in joined
35     assert "not increasing" in joined
36
37
38 def test_column_names_are_stripped() -> None:
39     df = pd.DataFrame({"t ": [0, 60], " v": [0.5, 0.6]})
40     assert validate_frame(df, CONTRACT)["ok"]
41
42
43 @given(st.lists(st.floats(min_value=0, max_value=1, allow_nan=False), min_size=2, max_size=40))
44 def test_property_ranges_within_bounds_always_pass(values: list[float]) -> None:
45     df = pd.DataFrame({"t": np.arange(len(values)) * 60, "v": values})
46     assert validate_frame(df, CONTRACT)["ok"]
47
48
49 def test_regex_and_custom_checks() -> None:
50     contract = FrameContract(
51         "ids", (Column("id", "str", regex=r"[ABC]\d{3}"),), checks=(lambda df: ["custom"] if len(df) > 1 else [],)
52     )
53     assert validate_frame(pd.DataFrame({"id": ["A001"]}), contract)["ok"]
54     report = validate_frame(pd.DataFrame({"id": ["A001", "Z9"]}), contract)
55     assert any("violate" in e for e in report["errors"]) and "custom" in report["errors"]

```

tests/unit/test_d_hints.py

```

1  """Problem D: warm-start hints mapped from recorded decisions, and the HiGHS per-level cross-check."""
2
3  from __future__ import annotations
4
5  from pipelines.d.plans import Plan
6  from pipelines.d.resolve import (
7      Limits,
8      build_cell_model,
9      canonical_levels,
10     decisions_from_choice,
11     enumerate_options,
12     evaluate,
13     hint_from_decisions,
14     hint_is_feasible,
15     solve_lexicographic_cpsat,
16     solve_lexicographic_highs,
17 )
18 from pipelines.d.synth import tiny_instance
19
20
21 def _model(plans: list[Plan], limits: Limits): # type: ignore[no-untyped-def]
22     return build_cell_model(plans, [enumerate_options(p, limits) for p in plans])
23
24
25 def test_hint_round_trips_and_falls_back_to_keep() -> None:
26     plans = tiny_instance(31, n_plans=5)
27     model = _model(plans, Limits(fmax=2, tmax=1))
28     res = solve_lexicographic_cpsat(model, canonical_levels(model), seed=1, workers=2, time_limit=60)
29     decisions = decisions_from_choice(model, res["chosen"])
30     hint = hint_from_decisions(model, decisions)
31     assert hint == res["chosen"]
32     assert hint_is_feasible(model, hint)
33     assert evaluate(model, hint, canonical_levels(model)) == res["vector"]
34     # unknown plans are ignored, missing plans and actions outside the limits fall back to keep
35     partial = [{"pid": "Z999", "kind": "cancel", "delta": 0}, {"pid": plans[0].pid, "kind": "freq", "delta": 9}]

```

```

36     fallback = hint_from_decisions(model, partial)
37     assert fallback == [vars_of_plan[0] for vars_of_plan in model.plan_vars]
38     assert all(model.option_of(v).kind == "keep" for v in fallback)
39
40
41 def test_hint_infeasibility_is_detected() -> None:
42     a = Plan("A001", "A", 0, 10, 0, 5, 60, 3)
43     b = Plan("B001", "B", 0, 10, 0, 5, 60, 3)
44     model = _model([a, b], Limits(fmax=10, tmax=5))
45     keep_both = [vars_of_plan[0] for vars_of_plan in model.plan_vars]
46     assert not hint_is_feasible(model, keep_both)
47     cancel_b = [model.plan_vars[0][0], next(v for v in model.plan_vars[1] if model.option_of(v).kind == "cancel")]
48     assert hint_is_feasible(model, cancel_b)
49
50
51 def test_highs_levels_are_independent_of_earlier_time_limits() -> None:
52     # every level gets a fresh solver, so a tiny LP limit on one level cannot starve the following ones
53     plans = tiny_instance(32, n_plans=6)
54     model = _model(plans, Limits(fmax=2, tmax=1))
55     levels = canonical_levels(model)
56     res = solve_lexicographic_cpsat(model, levels, seed=1, workers=2, time_limit=60)
57     highs = solve_lexicographic_highs(
58         model, levels, seed=1, threads=2, time_limit=60, reference=res["vector"], hint=res["chosen"], lp_time_limit=60
59     )
60     assert highs["agrees_with_reference"] and highs["all_optimal"] and highs["vector"] == res["vector"]
61     assert all(e.get("lp_status") == "Optimal" for e in highs["levels"])

```

tests/unit/test_d_pack.py

```

1  """Problem D: packing model on analytically solvable regions."""
2
3  from __future__ import annotations
4
5  import numpy as np
6  import pytest
7
8  from pipelines.d.pack import (
9      C_TEMPLATE,
10     candidate_placements,
11     cell_rows,
12     greedy_pack,
13     highs_pack,
14     occupancy_grid,
15     plans_from_choice,
16     repack_existing,
17     solve_pack_cpsat,
18     template_span,
19 )
20 from pipelines.d.plans import Plan
21 from pipelines.d.validators import conflicts_by_cells, validate_packing
22
23
24 def test_empty_strip_packs_five_phases() -> None:
25     # 3 free bands over exactly one span: five plans with starts 0,2,4,6,8 tile the strip (period 10, duration 2)
26     horizon = template_span(C_TEMPLATE) # 112
27     grid = np.ones((100, horizon), dtype=bool)
28     grid[0:3, :] = False
29     cands = candidate_placements(grid, C_TEMPLATE)
30     assert cands.tolist() == [[0, 0]] # only s = 0 fits inside the horizon
31     grid = np.ones((100, horizon + 8), dtype=bool)
32     grid[0:3, :] = False
33     cands = candidate_placements(grid, C_TEMPLATE)
34     assert len(cands) == 9
35     rows = cell_rows(cands, C_TEMPLATE, horizon + 8)
36     res = solve_pack_cpsat(rows, len(cands), hint=None, seed=0, workers=1, time_limit=30)
37     assert res["status"] == "OPTIMAL" and res["value"] == 5
38     lp = highs_pack(rows, len(cands), integer=False, time_limit=30, seed=0, threads=1)
39     assert lp["value"] >= 5 - 1e-6

```

```

40     new_plans = plans_from_choice(cands, res["chosen"], C_TEMPLATE)
41     report = validate_packing([], new_plans, horizon=horizon + 8, template=C_TEMPLATE)
42     assert report["ok"], report["errors"]
43
44
45 def test_existing_plan_blocks_candidates_and_greedy_is_feasible() -> None:
46     existing = [Plan("C001", "C", 0, 3, 0, 2, 8, 12)]
47     horizon = 130
48     grid = occupancy_grid(existing, horizon)
49     assert grid.sum() == 72
50     cands = candidate_placements(grid, C_TEMPLATE)
51     assert not any((f < 3 and s % 10 in (0, 1, 9)) for f, s in cands.tolist() if s <= 110)
52     greedy = greedy_pack(cands, C_TEMPLATE, grid)
53     new_plans = plans_from_choice(cands, greedy, C_TEMPLATE)
54     report = validate_packing(existing, new_plans, horizon=horizon, template=C_TEMPLATE)
55     assert report["ok"] and len(new_plans) == len(greedy) > 0
56
57
58 def test_repack_keeps_parameters_compacts_and_stays_conflict_free() -> None:
59     """MDR-0010 interpretation B: first-fit decreasing may move a plan anywhere but must not change it."""
60     scattered = [
61         Plan("A001", "A", 80, 10, 30, 5, 60, 3),
62         Plan("B001", "B", 40, 15, 7, 3, 40, 4),
63         Plan("B002", "B", 60, 15, 90, 3, 40, 4),
64         Plan("C001", "C", 20, 3, 11, 2, 8, 12),
65         Plan("C002", "C", 97, 3, 40, 2, 8, 12),
66     ]
67     horizon = 300
68     repacked, grid = repack_existing(scattered, horizon)
69     assert [p.pid for p in repacked] == sorted(p.pid for p in scattered)
70     before = {p.pid: p for p in scattered}
71     for p in repacked:
72         q = before[p.pid]
73         assert (p.w, p.d, p.g, p.n, p.cat) == (q.w, q.d, q.g, q.n, q.cat) # only f and s may move
74         assert 0 <= p.f and p.f_end <= 100 and p.s >= 0 and p.end <= horizon
75     assert not conflicts_by_cells(repacked) # the layout itself is conflict-free
76     assert grid.sum() == sum(p.n_cells for p in scattered) # no cell is double-counted
77     assert max(p.f_end for p in repacked) <= max(p.f_end for p in scattered) # compacted towards low bands
78     # the repacked layout admits at least as many new C plans as the scattered one
79     scattered_grid = occupancy_grid(scattered, horizon)
80     assert len(candidate_placements(grid, C_TEMPLATE)) >= len(candidate_placements(scattered_grid, C_TEMPLATE))
81
82
83 def test_repack_fills_a_strip_exactly_and_refuses_an_impossible_region() -> None:
84     # period 10, duration 2: five phases (s = 0,2,4,6,8) tile a 3-band strip, but only once every offset
85     # still ends inside the horizon, i.e. from span + 8 = 120 onwards
86     horizon = template_span(C_TEMPLATE) + 8
87     strip = [Plan(f"C{k:03d}", "C", 0, 3, 2 * k, 2, 8, 12) for k in range(5)]
88     repacked, grid = repack_existing(strip, horizon)
89     assert not conflicts_by_cells(repacked)
90     assert sorted(p.s for p in repacked) == [0, 2, 4, 6, 8] # first-fit reproduces the five phases
91     assert grid[0:3].all() and not grid[3:].any() # and they tile exactly the first three bands
92     with pytest.raises(RuntimeError, match="could not place"):
93         repack_existing([*strip, Plan("C999", "C", 0, 3, 0, 2, 8, 12)], horizon, bands=3)

```

tests/unit/test_d_plans_detect.py

```

1 """Problem D: plan semantics, the two detectors and the independent cell-set validator."""
2
3 from __future__ import annotations
4
5 from hypothesis import given
6 from hypothesis import strategies as st
7
8 from pipelines.d.detect import detect_bandsweep, detect_pairwise, overlapping Uses, plans_conflict
9 from pipelines.d.plans import Plan, fmt_interval, parse_interval, plan_from_row
10 from pipelines.d.synth import synthetic_instance
11 from pipelines.d.validators import cell_set, conflicts_by_cells, validate_detection

```

```

12
13
14 def test_problem_statement_example_a001() -> None:
15     plan = plan_from_row("A001", "[80,90]", "[35,40]", 60, 3)
16     assert plan.cat == "A" and plan.w == 10 and plan.d == 5 and plan.period == 65
17     assert plan.uses() == [(35, 40), (100, 105), (165, 170)]
18     assert plan.end == 170 and plan.n_cells == 150
19     assert parse_interval(" [3, 7) ") == (3, 7) and fmt_interval(3, 7) == "[3,7)"
20
21
22 def test_half_open_intervals_do_not_touch() -> None:
23     a = Plan("A001", "A", 0, 10, 0, 5, 60, 3)
24     b = Plan("B001", "B", 10, 15, 0, 3, 40, 4) # bands [10,25) touch [0,10) without overlap
25     c = Plan("C001", "C", 5, 3, 5, 2, 8, 1) # single use [5,7) touches [0,5)
26     d = Plan("C002", "C", 5, 3, 4, 2, 8, 1) # single use [4,6) overlaps [0,5)
27     e = Plan("C003", "C", 5, 3, 5, 2, 8, 12) # 7th use [65,67) overlaps A's second use [65,70)
28     assert not plans_conflict(a, b) and not plans_conflict(a, c) and plans_conflict(a, d)
29     assert overlapping_uses(a, d) == [(0, 0)] and overlapping_uses(a, e) == [(1, 6)]
30
31
32 def test_detectors_agree_with_cell_sets_on_synthetic_instances() -> None:
33     for seed in (1, 2, 3):
34         plans = synthetic_instance(60, seed)
35         pairs = detect_pairwise(plans)
36         assert pairs == detect_bandsweep(plans)
37         truth = conflicts_by_cells(plans)
38         assert [(plans[i].pid, plans[j].pid) for i, j in pairs] == truth
39         report = validate_detection(plans, truth)
40         assert report["ok"] and report["truth_pairs"] == len(pairs)
41
42
43 def test_validate_detection_flags_missing_and_extra() -> None:
44     plans = synthetic_instance(40, 7)
45     truth = conflicts_by_cells(plans)
46     assert truth, "instance must contain conflicts"
47     report = validate_detection(plans, truth[1:] + [("A001", "A002")])
48     assert not report["ok"] and report["missing"] == truth[:1]
49     assert ("A001", "A002") in report["extra"] or truth[0] == ("A001", "A002")
50
51
52 plan_strategy = st.builds(
53     Plan,
54     pid=st.just("X"),
55     cat=st.just("C"),
56     f=st.integers(0, 12),
57     w=st.integers(1, 4),
58     s=st.integers(0, 15),
59     d=st.integers(1, 3),
60     g=st.integers(1, 6),
61     n=st.integers(1, 4),
62 )
63
64
65 @given(plan_strategy, plan_strategy)
66 def test_property_interval_arithmetic_matches_cell_sets(a: Plan, b: Plan) -> None:
67     assert plans_conflict(a, b) == bool(cell_set(a) & cell_set(b))
68     assert len(cell_set(a)) == a.n_cells

```

tests/unit/test_d_report.py

```

1 """Problem D: report helpers (tables) on hand-made inputs."""
2
3 from __future__ import annotations
4
5 from pipelines.d.plans import Plan
6 from pipelines.d.report import _cancelled_rows, _table1, _vector_text
7 from pipelines.d.resolve import Limits
8

```

```

9
10 def test_cancelled_rows_explain_each_cancelled_plan() -> None:
11     a = Plan("A001", "A", 0, 10, 0, 5, 60, 3)
12     c1 = Plan("C001", "C", 5, 3, 3, 2, 8, 12)
13     c2 = Plan("C002", "C", 95, 3, 3, 2, 8, 12)
14     decisions = [
15         {"pid": "A001", "kind": "keep", "delta": 0, "plan": a.record()},
16         {"pid": "C001", "kind": "cancel", "delta": 0, "plan": None},
17         {"pid": "C002", "kind": "cancel", "delta": 0, "plan": None},
18     ]
19     conflicts = [{"id1": "A001", "id2": "C001"}]
20     rows = _cancelled_rows([a, c1, c2], decisions, conflicts, Limits(fmax=10, tmax=5))
21     assert [r[0] for r in rows] == ["C001", "C002"]
22     assert rows[0][3] == 1 and rows[0][4] == 1 # one neighbour, which is an A plan
23     assert rows[1][3] == 0 and rows[1][4] == 0
24     assert rows[0][5] == 15 + 8 # band start 5: shifts -5..10 minus 0 (15); time start 3: shifts -3..5 minus 0 (8)
25     assert rows[1][5] == 12 + 8 # band start 95: only -10..2 (12 shifts) keep the plan inside [0,100)
26
27
28 def test_table1_and_vector_text() -> None:
29     table = {
30         "A": {"keep": 1, "adjust": 2, "cancel": 0},
31         "B": {"keep": 0, "adjust": 0, "cancel": 1},
32         "C": {"keep": 3, "adjust": 4, "cancel": 5},
33     }
34     rows = _table1(table)
35     assert rows[-1] == ["合计", 4, 6, 6]
36     assert _vector_text([0, 1, 2]) == "(0, 1, 2)" and _vector_text(None) == "-"

```

tests/unit/test_d_resolve.py

```

1 """Problem D: actions, cell model, lexicographic CP-SAT vs exhaustive enumeration, HiGHS agreement, validator."""
2
3 from __future__ import annotations
4
5 import pytest
6
7 from pipelines.d.plans import Plan
8 from pipelines.d.resolve import (
9     Limits,
10     Option,
11     build_cell_model,
12     canonical_levels,
13     decisions_from_choice,
14     enumerate_options,
15     evaluate,
16     hint_is_feasible,
17     level_upper_bounds,
18     level_value,
19     scheme_levels,
20     separated_weights,
21     solve_lexicographic_cpsat,
22     solve_lexicographic_highs,
23     solve_weighted_cpsat,
24     strengthen,
25     table_from_decisions,
26 )
27 from pipelines.d.synth import exhaustive_lexicographic, tiny_instance
28 from pipelines.d.validators import validate_resolution
29
30
31 def test_option_enumeration_respects_bounds_and_single_parameter() -> None:
32     plan = Plan("C001", "C", 95, 3, 2, 2, 8, 12)
33     opts = enumerate_options(plan, Limits(fmax=10, tmax=5))
34     freq = sorted(o.delta for o in opts if o.kind == "freq")
35     time = sorted(o.delta for o in opts if o.kind == "time")
36     assert freq == [*range(-10, 0), 1, 2] # band interval must stay inside [0,100)
37     assert time == [-2, -1, 1, 2, 3, 4, 5] # start must stay >= 0

```

```

38     assert [o.kind for o in opts].count("keep") == 1 and [o.kind for o in opts].count("cancel") == 1
39     assert all(o.kind != "gap" for o in opts)
40     with_gap = enumerate_options(plan, Limits(fmax=10, tmax=5, gmax=10, gap_categories=("C",)))
41     gaps = sorted(o.delta for o in with_gap if o.kind == "gap")
42     assert gaps == [d for d in range(-7, 11) if d != 0] # g' = 8 + delta >= 1
43     assert Option("time", -3).amplitude == 6 and Option("freq", 4).amplitude == 4 and Option("gap", 5).amplitude == 5
44     assert Option("freq", 2).apply(plan) == plan.shifted(df=2)
45     assert Option("cancel").apply(plan) is None
46
47
48 def test_separated_weights_dominate_lower_levels() -> None:
49     bounds = [20, 40, 90, 20, 40, 90, 1500]
50     weights = separated_weights(bounds)
51     for k in range(len(bounds) - 1):
52         assert weights[k] > sum(w * b for w, b in zip(weights[k + 1 :], bounds[k + 1 :]))
53
54
55 @pytest.mark.parametrize("seed", [11, 12, 13, 14])
56 def test_cpsat_lexicographic_matches_exhaustive_and_validator(seed: int) -> None:
57     plans = tiny_instance(seed, n_plans=5)
58     limits = Limits(fmax=2, tmax=1)
59     options = [enumerate_options(p, limits) for p in plans]
60     model = build_cell_model(plans, options)
61     levels = canonical_levels(model)
62     truth = exhaustive_lexicographic(plans, limits, canonical_levels, model)
63     res = solve_lexicographic_cpsat(model, levels, seed=1, workers=2, time_limit=60)
64     assert res["all_optimal"] and res["vector"] == truth
65     decisions = decisions_from_choice(model, res["chosen"])
66     report = validate_resolution(
67         plans, decisions, fmax=2, tmax=1, reported_table=table_from_decisions(decisions), reported_vector=res["vector"]
68     )
69     assert report["ok"], report["errors"]
70     weighted = solve_weighted_cpsat(model, levels, seed=1, workers=2, time_limit=60)
71     assert weighted["vector"] == truth
72     highs = solve_lexicographic_highs(model, levels, seed=1, threads=2, time_limit=60)
73     assert highs["all_optimal"] and highs["vector"] == truth
74     assert evaluate(model, highs["chosen"], levels) == truth
75     assert level_upper_bounds(model, levels)[-1] >= truth[-1]
76
77
78 def test_schemes_are_well_formed_and_forced_cancellation_is_detected() -> None:
79     # two identical plans can never coexist: the optimum must cancel one of them (no shift can separate them
80     # because they are also identical after any common shift? no - shifting one of them resolves it)
81     a = Plan("A001", "A", 0, 10, 0, 5, 60, 3)
82     b = Plan("B001", "B", 0, 10, 0, 5, 60, 3)
83     plans = [a, b]
84     model = build_cell_model(plans, [enumerate_options(p, Limits(fmax=0, tmax=0)) for p in plans])
85     for name in ("P", "T", "S", "W"):
86         assert scheme_levels(name, model)
87     res = solve_lexicographic_cpsat(model, canonical_levels(model), seed=0, workers=1, time_limit=30)
88     assert res["vector"] == [0, 1, 0, 0, 0, 0, 0] # the B plan is cancelled, never the A plan
89     model2 = build_cell_model(plans, [enumerate_options(p, Limits(fmax=10, tmax=5)) for p in plans])
90     res2 = solve_lexicographic_cpsat(model2, canonical_levels(model2), seed=0, workers=1, time_limit=30)
91     assert res2["vector"][:6] == [0, 0, 0, 0, 1, 0] # with shifts allowed: adjust the B plan instead
92
93
94 def test_validator_rejects_illegal_decisions() -> None:
95     a = Plan("A001", "A", 0, 10, 0, 5, 60, 3)
96     b = Plan("C001", "C", 5, 3, 3, 2, 8, 12)
97     bad = [
98         {"pid": "A001", "kind": "keep", "delta": 0, "plan": a.record()},
99         {"pid": "C001", "kind": "freq", "delta": 12, "plan": b.shifted(df=12).record()},
100     ]
101     report = validate_resolution([a, b], bad, fmax=10, tmax=5)
102     assert not report["ok"] and any("exceeds" in e for e in report["errors"])
103     two_params = [
104         {"pid": "A001", "kind": "keep", "delta": 0, "plan": a.record()},
105         {"pid": "C001", "kind": "time", "delta": 1, "plan": b.shifted(df=1, dt=1).record()},
106     ]
107     report = validate_resolution([a, b], two_params, fmax=10, tmax=5)
108     assert not report["ok"] and any("changed fields" in e for e in report["errors"])

```

```

109     unresolved = [
110         {"pid": "A001", "kind": "keep", "delta": 0, "plan": a.record()},
111         {"pid": "C001", "kind": "keep", "delta": 0, "plan": b.record()},
112     ]
113     report = validate_resolution([a, b], unresolved, fmax=10, tmax=5)
114     assert not report["ok"] and report["remaining_conflicts"] == 1
115
116
117 def test_strengthen_detects_forced_cancellation_and_keeps_optimum() -> None:
118     a = Plan("A001", "A", 0, 10, 0, 5, 60, 3)
119     b = Plan("B001", "B", 0, 10, 0, 5, 60, 3)
120     model = build_cell_model([a, b], [enumerate_options(p, Limits(fmax=0, tmax=0)) for p in (a, b)])
121     strength = strengthen(model)
122     assert strength.forced == [(0, 1)] and strength.interacting_pairs == 1 and strength.groups
123     for seed in (21, 22):
124         plans = tiny_instance(seed, n_plans=5)
125         limits = Limits(fmax=2, tmax=1)
126         model = build_cell_model(plans, [enumerate_options(p, limits) for p in plans])
127         truth = exhaustive_lexicographic(plans, limits, canonical_levels, model)
128         res = solve_lexicographic_cpsat(
129             model, canonical_levels(model), seed=1, workers=2, time_limit=60, strength=strengthen(model)
130         )
131         assert res["all_optimal"] and res["vector"] == truth
132         highs = solve_lexicographic_highs(
133             model,
134             canonical_levels(model),
135             seed=1,
136             threads=2,
137             time_limit=60,
138             reference=truth,
139             strength=strengthen(model),
140         )
141         assert highs["agrees_with_reference"] and highs["vector"] == truth
142
143
144 @pytest.mark.parametrize("seed", [31, 32, 33])
145 def test_incumbent_cut_preserves_optimum_and_never_worsens_the_hint(seed: int) -> None:
146     """MDR-0009: a feasible hint bounds every level from above without removing any optimum.
147
148     The lexicographic vector must equal the exhaustive optimum whether or not a hint is supplied, and it
149     must never be lexicographically worse than the hint it started from."""
150     plans = tiny_instance(seed, n_plans=5)
151     limits = Limits(fmax=2, tmax=1)
152     model = build_cell_model(plans, [enumerate_options(p, limits) for p in plans])
153     levels = canonical_levels(model)
154     truth = exhaustive_lexicographic(plans, limits, canonical_levels, model)
155     cold = solve_lexicographic_cpsat(model, levels, seed=1, workers=2, time_limit=60)
156     assert cold["all_optimal"] and cold["vector"] == truth
157     # "cancel everything" is always feasible and (except in degenerate instances) far from optimal
158     cancel_all = [next(v for v in vs if model.option_of(v).kind == "cancel") for vs in model.plan_vars]
159     assert hint_is_feasible(model, cancel_all)
160     warm = solve_lexicographic_cpsat(model, levels, seed=1, workers=2, time_limit=60, hint=cancel_all)
161     assert warm["all_optimal"] and warm["vector"] == truth # the cut preserved every level's optimum
162     assert warm["vector"] <= evaluate(model, cancel_all, levels) # and never returned a worse vector
163     cuts = [lv["cut"] for lv in warm["levels"]]
164     assert cuts[0] == evaluate(model, cancel_all, levels)[0] # first cut comes straight from the hint
165     assert all(c is not None and v <= c for c, v in zip(cuts, warm["vector"])) # every level respects its cut
166
167
168 def test_lexicographic_never_worse_than_a_good_hint() -> None:
169     """A one-second budget is far too short for the search, but the incumbent must still be returned."""
170     plans = tiny_instance(41, n_plans=6)
171     limits = Limits(fmax=2, tmax=1)
172     model = build_cell_model(plans, [enumerate_options(p, limits) for p in plans])
173     levels = canonical_levels(model)
174     truth = exhaustive_lexicographic(plans, limits, canonical_levels, model)
175     optimal = solve_lexicographic_cpsat(model, levels, seed=1, workers=2, time_limit=60)
176     starved = solve_lexicographic_cpsat(
177         model, levels, seed=1, workers=1, time_limit=0.001, hint=optimal["chosen"], strength=strengthen(model)
178     )
179     assert starved["vector"] == truth

```

```

180     assert all(lv["status"] in {"OPTIMAL", "FEASIBLE", "INCUMBENT"} for lv in starved["levels"])
181
182
183 def test_level_value_matches_evaluate() -> None:
184     plans = tiny_instance(51, n_plans=4)
185     model = build_cell_model(plans, [enumerate_options(p, Limits(fmax=1, tmax=1)) for p in plans])
186     levels = canonical_levels(model)
187     chosen = [vs[0] for vs in model.plan_vars]
188     assert [level_value(terms, chosen) for _, terms in levels] == evaluate(model, chosen, levels)

```

tests/unit/test_hashing.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from hypothesis import given
6 from hypothesis import strategies as st
7
8 from forge.hashing import hash_tree, sha256_bytes, sha256_json, tree_digest
9
10
11 @given(st.binary())
12 def test_sha256_bytes_is_deterministic(data: bytes) -> None:
13     assert sha256_bytes(data) == sha256_bytes(data)
14     assert len(sha256_bytes(data)) == 64
15
16
17 @given(
18     st.dictionaries(
19         st.text(min_size=1, max_size=8), st.integers() | st.floats(allow_nan=False) | st.text(max_size=5), max_size=6
20     )
21 )
22 def test_sha256_json_ignores_key_order(mapping: dict) -> None:
23     reversed_mapping = dict(reversed(list(mapping.items())))
24     assert sha256_json(mapping) == sha256_json(reversed_mapping)
25
26
27 def test_hash_tree_ignores_records_and_is_sorted(tmp_path: Path) -> None:
28     (tmp_path / "b.txt").write_text("b")
29     (tmp_path / "a.txt").write_text("a")
30     (tmp_path / "manifest.json").write_text("{}")
31     (tmp_path / "__pycache__").mkdir()
32     (tmp_path / "__pycache__" / "x.pyc").write_bytes(b"\x00")
33     tree = hash_tree(tmp_path, ignore=("manifest.json", "__pycache__"))
34     assert list(tree) == ["a.txt", "b.txt"]
35     digest = tree_digest(tree)
36     (tmp_path / "manifest.json").write_text('{"changed": true}')
37     assert tree_digest(hash_tree(tmp_path, ignore=("manifest.json", "__pycache__"))) == digest

```

tests/unit/test_manifest.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from forge.manifest import Manifest
6
7
8 def test_manifest_lifecycle_and_verification(tmp_path: Path) -> None:
9     stage_dir = tmp_path / "stage"
10     stage_dir.mkdir()
11     (stage_dir / "out.json").write_text("{}")
12     (stage_dir / "events.jsonl").write_text("{}\n")
13     manifest = Manifest.start("run1", "stage", {"p": 1}, {"git_sha": "abc"}, "cfg", {"ingest": "d"}, {})

```

```

14     assert manifest.status == "running" and manifest.runtime["python"]
15     manifest.complete(stage_dir, {"m": 1.5})
16     assert manifest.status == "completed"
17     assert list(manifest.outputs) == ["out.json"]
18     manifest.write(stage_dir / "manifest.json")
19     loaded = Manifest.read(stage_dir / "manifest.json")
20     assert loaded.outputs_digest == manifest.outputs_digest
21     assert loaded.verify_outputs(stage_dir) == []
22     (stage_dir / "out.json").write_text('{ "x": 1 }')
23     assert loaded.verify_outputs(stage_dir) == ["sha256 mismatch: out.json"]
24
25
26 def test_manifest_failure_records_traceback() -> None:
27     manifest = Manifest.start("run1", "stage", {}, {}, "cfg", {}, {})
28     try:
29         raise ValueError("boom")
30     except ValueError as exc:
31         manifest.fail(exc)
32     assert manifest.status == "failed"
33     assert manifest.error and manifest.error["type"] == "ValueError"
34     assert manifest.duration_s is not None

```

tests/unit/test_packaging_tex.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 from forge.packaging import build_zip, code_members, run_members
6 from forge.tex import aux_page, summarize_log, tex_escape
7
8
9 def test_zip_is_deterministic(tmp_path: Path) -> None:
10     src = tmp_path / "src"
11     src.mkdir()
12     (src / "b.py").write_text("print(2)")
13     (src / "a.py").write_text("print(1)")
14     (src / "__pycache__").mkdir()
15     (src / "__pycache__" / "a.pyc").write_bytes(b"x")
16     members = code_members(src, ["**/*.py"])
17     assert [m[0] for m in members] == ["a.py", "b.py"]
18     one = build_zip(members, tmp_path / "one.zip")
19     two = build_zip(list(reversed(members)), tmp_path / "two.zip")
20     assert one["sha256"] == two["sha256"] and one["members"] == 2
21
22
23 def test_run_members_prefix(tmp_path: Path) -> None:
24     run = tmp_path / "runs" / "r1"
25     (run / "q1").mkdir(parents=True)
26     (run / "q1" / "out.json").write_text("{}")
27     assert run_members(run, ["q1", "missing"]) == [(run / "q1" / "out.json"), (run / "q1" / "out.json")]
28
29
30 def test_log_summary_and_aux_page(tmp_path: Path) -> None:
31     log = (
32         "LaTeX Warning: Reference `fig:x' on page 3 undefined on input line 10.\n"
33         "Overfull \hbox (12.5pt too wide) in paragraph\n"
34         "Missing character: There is no  in font Latin Modern!\n"
35         "!\n Undefined control sequence.\n"
36         "Output written on main.pdf (7 pages).\n"
37     )
38     summary = summarize_log(log)
39     assert summary["undefined_references"] == ["fig:x"]
40     assert summary["overfull_max_pt"] == 12.5 and summary["missing_characters"] == 1
41     assert summary["errors"] == ["Undefined control sequence."] and summary["pages_from_log"] == 7
42     aux = tmp_path / "main.aux"
43     aux.write_text("\newlabel{page:body-end}{{27}}{page:body-end}}\n\newlabel{sec:a}{{2.1}{5}}\n")
44     assert aux_page(aux, "page:body-end") == 27 and aux_page(aux, "sec:a") == 5 and aux_page(aux, "nope") is None

```

```

45
46
47 def test_tex_escape() -> None:
48     assert tex_escape("a_b & 100% #1") == r"a\_b \& 100\% \#1"

```

tests/unit/test_runner.py

```

1  from __future__ import annotations
2
3  import json
4  from pathlib import Path
5
6  import pytest
7
8  from forge import runner
9  from forge.context import StageContext
10
11
12 @pytest.fixture(autouse=True)
13 def isolated_registry(monkeypatch: pytest.MonkeyPatch) -> None:
14     monkeypatch.setattr(runner, "STAGES", {})
15     monkeypatch.setattr(runner, "discover", lambda repo: None)
16
17
18 def test_execute_records_manifest_skips_reruns_and_enforces_deps(tmp_repo: Path, tmp_path: Path) -> None:
19     vol = tmp_path / "vol"
20
21     @runner.stage("first", description="first stage")
22     def first(ctx: StageContext) -> dict:
23         ctx.out("a.txt").write_text("hello")
24         ctx.number("Answer", 42)
25         return {"n": 1}
26
27     @runner.stage("second", deps=("first",))
28     def second(ctx: StageContext) -> dict:
29         assert (ctx.dep("first") / "a.txt").read_text() == "hello"
30         return {"ok": True}
31
32     with pytest.raises(RuntimeError):
33         runner.execute("second", "r1", {}, False, tmp_repo, vol)
34     result = runner.execute("first", "r1", {"code_ref": {"git_sha": "x"}}, False, tmp_repo, vol)
35     assert result["status"] == "completed" and result["metrics"] == {"n": 1}
36     manifest = json.loads((vol / "runs" / "r1" / "first" / "manifest.json").read_text())
37     assert (
38         manifest["status"] == "completed" and "a.txt" in manifest["outputs"] and "numbers.json" in manifest["outputs"]
39     )
40     assert json.loads((vol / "runs" / "r1" / "first" / "numbers.json").read_text()) == {"Answer": "42"}
41     assert runner.execute("first", "r1", {}, False, tmp_repo, vol)["status"] == "skipped"
42     assert runner.execute("second", "r1", {}, False, tmp_repo, vol)["status"] == "completed"
43     combined = runner.execute("all", "r1", {"stages": ["first", "second"]}, False, tmp_repo, vol)
44     assert combined["status"] == "completed" and [r["status"] for r in combined["results"]] == ["skipped", "skipped"]
45
46
47 def test_failed_stage_writes_failed_manifest(tmp_repo: Path, tmp_path: Path) -> None:
48     vol = tmp_path / "vol"
49
50     @runner.stage("bad")
51     def bad(ctx: StageContext) -> dict:
52         raise ValueError("boom")
53
54     with pytest.raises(ValueError):
55         runner.execute("bad", "r1", {}, False, tmp_repo, vol)
56     manifest = json.loads((vol / "runs" / "r1" / "bad" / "manifest.json").read_text())
57     assert manifest["status"] == "failed" and manifest["error"]["message"] == "boom"
58
59
60 def test_from_run_copies_dependency_outputs(tmp_repo: Path, tmp_path: Path) -> None:
61     vol = tmp_path / "vol"

```

```

62
63 @runner.stage("base")
64 def base(ctx: StageContext) -> dict:
65     ctx.out("x.txt").write_text("x")
66     return {}
67
68 @runner.stage("child", deps=("base",))
69 def child(ctx: StageContext) -> dict:
70     return {"seen": (ctx.dep("base") / "x.txt").read_text()}
71
72 runner.execute("base", "old", {}, False, tmp_repo, vol)
73 result = runner.execute("child", "new", {"from_run": "old"}, False, tmp_repo, vol)
74 assert result["metrics"] == {"seen": "x"}

```

tests/unit/test_xlsx.py

```

1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import openpyxl
6 import pytest
7
8 from forge.xlsx import SheetContract, WorkbookContract, check_workbook, read_sheets, sheet_headers, write_result
9
10
11 def _template(path: Path) -> Path:
12     wb = openpyxl.Workbook()
13     ws = wb.active
14     ws.title = "温度"
15     ws.append(["时间\\到药材中心的距离", 0, 0.1, "...", 2])
16     ws.append([1, None, None, None, None])
17     ws2 = wb.create_sheet("水分浓度")
18     ws2.append(["时间\\到药材中心的距离", 0, 0.1, "...", 2])
19     wb.save(path)
20     return path
21
22
23 def test_write_result_keeps_all_rounds_and_passes_contract(tmp_path: Path) -> None:
24     template = _template(tmp_path / "tpl.xlsx")
25     header = ["ignored", 0.0, 0.1, 0.2]
26     rows = [[t, 28.123456, 28.2, 28.3] for t in range(1, 6)]
27     info = write_result(template, tmp_path / "out.xlsx", {"温度": (header, rows), "水分浓度": (header, rows)})
28     assert info["sheets"]["温度"]["rows"] == 5
29     headers = sheet_headers(tmp_path / "out.xlsx")
30     assert headers["温度"][0] == "时间\\到药材中心的距离"
31     assert headers["温度"][1:] == [0.0, 0.1, 0.2]
32     frames = read_sheets(tmp_path / "out.xlsx")
33     assert float(frames["温度"].iloc[0, 1]) == pytest.approx(28.1235)
34     contract = WorkbookContract(
35         "out.xlsx",
36         "tpl.xlsx",
37         (
38             SheetContract("温度", min_rows=5, max_rows=5, header_len=4, numeric_from_col=0),
39             SheetContract("水分浓度", min_rows=5, header_len=4, numeric_from_col=0),
40         ),
41     )
42     report = check_workbook(tmp_path / "out.xlsx", contract, template)
43     assert report["ok"], report["errors"]
44
45
46 def test_check_workbook_flags_blank_and_text_cells(tmp_path: Path) -> None:
47     template = _template(tmp_path / "tpl.xlsx")
48     rows = [[1, 1.0, None, "x"], [2, 2.0, 2.0, 2.0]]
49     write_result(template, tmp_path / "out.xlsx", {"温度": ([ "h", 0, 0.1, 0.2 ], rows)})
50     contract = WorkbookContract("out.xlsx", "tpl.xlsx", (SheetContract("温度", min_rows=2, numeric_from_col=0),))
51     report = check_workbook(tmp_path / "out.xlsx", contract, template)
52     joined = " ".join(report["errors"])

```

```

53     assert not report["ok"] and "blank" in joined and "non-numeric" in joined
54
55
56 def test_write_result_rejects_unknown_sheet(tmp_path: Path) -> None:
57     template = _template(tmp_path / "tpl.xlsx")
58     with pytest.raises(KeyError):
59         write_result(template, tmp_path / "out.xlsx", {"nope": ([ "h" ], [ ])})
60
61
62 def test_write_only_path_for_large_sheets(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) -> None:
63     import forge.xlsx as mod
64
65     monkeypatch.setattr(mod, "BIG_SHEET_CELLS", 10)
66     template = _template(tmp_path / "tpl.xlsx")
67     rows = [[t, 1.0, 2.0] for t in range(1, 8)]
68     info = write_result(template, tmp_path / "big.xlsx", {"温度": ([ "h" ], 0, 0.1], rows)})
69     assert info["write_only"]
70     headers = sheet_headers(tmp_path / "big.xlsx")
71     assert list(headers) == ["温度", "水分浓度"]
72     assert headers["温度"][0] == "时间\\到药材中心的距离"

```

tools/__init__.py

tools/cli.py

```

1  #!/usr/bin/env python3
2  """Forge control plane (local). Never computes: submits cloud jobs, downloads, verifies hashes.
3
4  Examples
5      python3 tools/cli.py run ingest                                # new run id, small container
6      python3 tools/cli.py run validate --run-id <id>              # continue the same run
7      python3 tools/cli.py run q1,q2 --size medium                 # sequential stages in one container
8      python3 tools/cli.py run q3 --size large --spawn             # detached; poll with `wait`/`status`
9      python3 tools/cli.py exec --code 'print(sorted(p.name for p in RUN_DIR.iterdir()))'
10     python3 tools/cli.py status                                    # manifests of the last run
11     python3 tools/cli.py download --stage results                 # into artifacts/, verified against manifests
12     python3 tools/cli.py release --version v1.0.0                 # deliverables into releases/v1.0.0/
13 """
14
15 from __future__ import annotations
16
17 import argparse
18 import hashlib
19 import json
20 import os
21 import shutil
22 import subprocess
23 import sys
24 from datetime import datetime
25 from pathlib import Path
26
27 REPO = Path(__file__).resolve().parents[1]
28 PROJECT = json.loads((REPO / "configs" / "project.json").read_text(encoding="utf-8"))
29 APP_FILE = REPO / "forge" / "modal_app.py"
30 STATE_DIR = REPO / ".forge"
31 LAST_RUN = STATE_DIR / "last_run"
32 MODAL = shutil.which("modal") or str(Path.home() / ".local" / "bin" / "modal")
33
34
35 def _sh(cmd: list[str], **kw) -> subprocess.CompletedProcess:
36     print("$ " + " ".join(cmd), flush=True)
37     return subprocess.run(cmd, cwd=REPO, check=False, **kw)
38
39

```

```

40 def git_ref() -> dict:
41     def out(*args: str) -> str:
42         try:
43             return subprocess.run(["git", *args], cwd=REPO, capture_output=True, text=True, check=False).stdout.strip()
44         except OSError:
45             return ""
46
47     sha = out("rev-parse", "HEAD") or "nogit"
48     return {
49         "git_sha": sha,
50         "branch": out("rev-parse", "--abbrev-ref", "HEAD"),
51         "dirty": bool(out("status", "--porcelain")),
52     }
53
54
55 def new_run_id() -> str:
56     return datetime.now().strftime("%Y%m%d-%H%M%S") + "-" + git_ref()["git_sha"][:7]
57
58
59 def remember(run_id: str) -> None:
60     STATE_DIR.mkdir(exist_ok=True)
61     LAST_RUN.write_text(run_id, encoding="utf-8")
62
63
64 def resolve_run_id(explicit: str | None, create: bool) -> str:
65     if explicit:
66         remember(explicit)
67         return explicit
68     if create or not LAST_RUN.exists():
69         run_id = new_run_id()
70         remember(run_id)
71         return run_id
72     return LAST_RUN.read_text(encoding="utf-8").strip()
73
74
75 def parse_params(items: list[str] | None) -> dict:
76     params: dict = {}
77     for item in items or []:
78         key, _, raw = item.partition("=")
79         try:
80             params[key] = json.loads(raw)
81         except json.JSONDecodeError:
82             params[key] = raw
83     return params
84
85
86 def modal_run(args: list[str], detach: bool = False) -> int:
87     cmd = [MODAL, "run"] + (["--detach"] if detach else []) + [str(APP_FILE)] + args
88     return _sh(cmd).returncode
89
90
91 def cmd_run(ns: argparse.Namespace) -> int:
92     stages = [s for s in ns.stage.split(",") if s]
93     run_id = resolve_run_id(ns.run_id, create=ns.new_run)
94     params = parse_params(ns.param)
95     params.update(
96         {"code_ref": git_ref(), "size": ns.size, "submitted_at": datetime.now().isoformat(timespec="seconds")}
97     )
98     if ns.from_run:
99         params["from_run"] = ns.from_run
100     if ns.profile:
101         params["profile"] = ns.profile
102     stage = stages[0] if len(stages) == 1 else "all"
103     if stage == "all":
104         params["stages"] = stages
105     args = ["--stage", stage, "--run-id", run_id, "--params", json.dumps(params, ensure_ascii=False), "--size", ns.size]
106     if ns.force:
107         args.append("--force")
108     if ns.spawn:
109         args.append("--spawn")

```

```

110     print(f"run_id={run_id} stages={stages} size={ns.size} spawn={ns.spawn}")
111     return modal_run(args, detach=ns.spawn)
112
113
114 def cmd_exec(ns: argparse.Namespace) -> int:
115     code = Path(ns.file).read_text(encoding="utf-8") if ns.file else ns.code
116     if not code:
117         print("provide --code or --file", file=sys.stderr)
118         return 2
119     run_id = resolve_run_id(ns.run_id, create=False)
120     return modal_run(
121         ["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": code}, ensure_ascii=False)]
122     )
123
124
125 def cmd_provision(_: argparse.Namespace) -> int:
126     return modal_run(["--stage", "provision"])
127
128
129 STATUS_SNIPPET = r"""
130 import json
131 from pathlib import Path
132 run = RUN_DIR
133 if not run.exists():
134     print("run directory missing:", run)
135 else:
136     rows = []
137     for m in sorted(run.glob("*/manifest.json")):
138         d = json.loads(m.read_text())
139         err = (d.get("error") or {}).get("message", "")[:80]
140         digest = d.get("outputs_digest", "")[:12]
141         rows.append((d["stage"], d["status"], d.get("duration_s"), digest, len(d.get("outputs", {})), err))
142     print(f'{"stage":14s} {"status":10s} {"seconds":>9s} {"digest":12s} {"files":>5s} error')
143     for r in rows:
144         print(f'{"r[0]:14s} {"r[1]:10s} {"str(r[2]):>9s} {"r[3]:12s} {"r[4]:5d} {"r[5]}"')
145     print("run_dir:", run)
146 """
147
148
149 def cmd_status(ns: argparse.Namespace) -> int:
150     run_id = resolve_run_id(ns.run_id, create=False)
151     return modal_run(["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": STATUS_SNIPPET})])
152
153
154 def cmd_runs(_: argparse.Namespace) -> int:
155     return _sh([MODAL, "volume", "ls", PROJECT["volume"], "runs"]).returncode
156
157
158 def cmd_logs(ns: argparse.Namespace) -> int:
159     run_id = resolve_run_id(ns.run_id, create=False)
160     code = (
161         "from pathlib import Path\n"
162         f"p = RUN_DIR / {ns.stage!r} / 'events.jsonl'\n"
163         "print(p.read_text()[-%d:] if p.exists() else 'no events for stage')\n" % ns.tail
164     )
165     return modal_run(["--stage", "exec", "--run-id", run_id, "--params", json.dumps({"code": code})])
166
167
168 def sha256(path: Path) -> str:
169     h = hashlib.sha256()
170     with path.open("rb") as fh:
171         for chunk in iter(lambda: fh.read(1 << 20), b""):
172             h.update(chunk)
173     return h.hexdigest()
174
175
176 def verify_dir(root: Path) -> int:
177     """Verify every downloaded stage directory against its manifest (transport check only)."""
178     problems = 0
179     manifests = sorted(root.rglob("manifest.json"))
180     if not manifests:

```

```

181     print(f"no manifests under {root}")
182     return 1
183 for manifest in manifests:
184     data = json.loads(manifest.read_text(encoding="utf-8"))
185     stage_dir = manifest.parent
186     bad = []
187     for rel, meta in data.get("outputs", {}).items():
188         file = stage_dir / rel
189         if not file.exists():
190             bad.append(f"missing {rel}")
191         elif sha256(file) != meta["sha256"]:
192             bad.append(f"sha256 mismatch {rel}")
193     status = "OK " if not bad else "BAD"
194     print(
195         f"[{status}] {data.get('stage'):14s} {data.get('status'):10s} "
196         f"files={len(data.get('outputs', {}))} {stage_dir}"
197     )
198     for b in bad:
199         print("      " + b)
200     problems += len(bad)
201 return 0 if problems == 0 else 1
202
203
204 def volume_get(remote: str, local_parent: Path) -> int:
205     """Download a remote directory into ``local_parent/<basename>`` (replacing any previous copy)."""
206     target = local_parent / Path(remote).name
207     shutil.rmtree(target, ignore_errors=True)
208     local_parent.mkdir(parents=True, exist_ok=True)
209     return _sh([MODAL, "volume", "get", "--force", PROJECT["volume"], remote, str(local_parent) + "/"]).returncode
210
211
212 def cmd_download(ns: argparse.Namespace) -> int:
213     run_id = resolve_run_id(ns.run_id, create=False)
214     if ns.stage:
215         remote, parent = f"runs/{run_id}/{ns.stage}", REPO / ns.dest / "runs" / run_id
216     else:
217         remote, parent = f"runs/{run_id}", REPO / ns.dest / "runs"
218     rc = volume_get(remote, parent)
219     if rc != 0:
220         return rc
221     return verify_dir(parent / Path(remote).name)
222
223
224 def cmd_verify(ns: argparse.Namespace) -> int:
225     return verify_dir(Path(ns.path))
226
227
228 def cmd_release(ns: argparse.Namespace) -> int:
229     run_id = resolve_run_id(ns.run_id, create=False)
230     dest = REPO / "releases" / ns.version
231     if dest.exists() and not ns.force:
232         print(f"{dest} exists; pass --force to overwrite", file=sys.stderr)
233         return 2
234     staging = REPO / "artifacts" / "release_staging" / run_id
235     rc = volume_get(f"runs/{run_id}/release", staging)
236     if rc != 0:
237         return rc
238     shutil.rmtree(dest, ignore_errors=True)
239     shutil.copytree(staging / "release", dest)
240     manifest = json.loads((dest / "release_manifest.json").read_text(encoding="utf-8"))
241     bad = 0
242     for name, meta in manifest["files"].items():
243         file = dest / name
244         ok = file.exists() and sha256(file) == meta["sha256"]
245         bad += 0 if ok else 1
246         print(f"[{'OK ' if ok else 'BAD'}] {name} {meta['bytes']} bytes sha256={meta['sha256'][:16]}")
247     print(f"release {ns.version} from run {run_id}: {'verified' if not bad else f'{bad} problems'} -> {dest}")
248     return 0 if not bad else 1
249
250
251 def cmd_wait(ns: argparse.Namespace) -> int:

```

```

252 interpreter = (
253     Path(MODAL).read_text(encoding="utf-8", errors="ignore").splitlines()[0].lstrip("#!").strip()
254     if Path(MODAL).exists()
255     else sys.executable
256 )
257 code = (
258     "import json, modal, sys\n"
259     f"call = modal.FunctionCall.from_id({ns.call_id!r})\n"
260     "print('STAGE_RESULT ' + json.dumps(call.get(), ensure_ascii=False, default=str))\n"
261 )
262 return _sh([interpreter, "-c", code]).returncode
263
264
265 def cmd_fmt(ns: argparse.Namespace) -> int:
266     """Format in the cloud, then copy the exported files back into the working tree."""
267     run_id = resolve_run_id(ns.run_id, create=False)
268     params = {"code_ref": git_ref(), "size": "small"}
269     rc = modal_run(["--stage", "fmt", "--run-id", run_id, "--params", json.dumps(params), "--force"])
270     if rc != 0:
271         return rc
272     dest = REPO / "artifacts" / "runs" / run_id / "fmt"
273     rc = volume_get(f"runs/{run_id}/fmt", dest.parent)
274     if rc != 0:
275         return rc
276     files = dest / "files"
277     changed = sorted(p for p in files.rglob("*") if p.is_file()) if files.exists() else []
278     for src in changed:
279         rel = src.relative_to(files)
280         target = REPO / rel
281         target.parent.mkdir(parents=True, exist_ok=True)
282         shutil.copy2(src, target)
283         print(f"updated {rel}")
284     report = json.loads((dest / "fmt_report.json").read_text(encoding="utf-8"))
285     print(f"{len(changed)} files formatted; remaining ruff findings: {report.get('remaining')}")
286     return 0
287
288
289 def cmd_new_run(_: argparse.Namespace) -> int:
290     run_id = new_run_id()
291     remember(run_id)
292     print(run_id)
293     return 0
294
295
296 def build_parser() -> argparse.ArgumentParser:
297     p = argparse.ArgumentParser(prog="forge", description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter
298     ↪ )
299     sub = p.add_subparsers(dest="command", required=True)
300
301     r = sub.add_parser("run", help="run one stage or a comma-separated sequence in one container")
302     r.add_argument("stage")
303     r.add_argument("--run-id")
304     r.add_argument("--new-run", action="store_true", help="start a fresh run id")
305     r.add_argument("--size", default="small", choices=["small", "medium", "large"])
306     r.add_argument("--force", action="store_true", help="re-run even if the stage already completed")
307     r.add_argument("--spawn", action="store_true", help="detach: return immediately, poll with status/wait")
308     r.add_argument("--from-run", help="copy missing dependency outputs from this earlier run")
309     r.add_argument("--profile", help="overlay configs/<profile>.toml")
310     r.add_argument("--param", action="append", help="key=value (JSON-parsed) forwarded to the stage")
311     r.set_defaults(fn=cmd_run)
312
313     e = sub.add_parser("exec", help="run an ad-hoc Python snippet in the cloud against the run directory")
314     e.add_argument("--code")
315     e.add_argument("--file")
316     e.add_argument("--run-id")
317     e.set_defaults(fn=cmd_exec)
318
319     sub.add_parser("provision", help="verify the cloud toolchain").set_defaults(fn=cmd_provision)
320
321     s = sub.add_parser("status", help="print manifests of a run")
322     s.add_argument("--run-id")

```

```

322     s.set_defaults(fn=cmd_status)
323
324     lg = sub.add_parser("logs", help="tail the event log of a stage")
325     lg.add_argument("stage")
326     lg.add_argument("--run-id")
327     lg.add_argument("--tail", type=int, default=6000)
328     lg.set_defaults(fn=cmd_logs)
329
330     sub.add_parser("runs", help="list runs on the volume").set_defaults(fn=cmd_runs)
331
332     d = sub.add_parser("download", help="download a run (or one stage) into artifacts/ and verify hashes")
333     d.add_argument("--run-id")
334     d.add_argument("--stage")
335     d.add_argument("--dest", default="artifacts")
336     d.set_defaults(fn=cmd_download)
337
338     v = sub.add_parser("verify", help="verify a downloaded directory against its manifests")
339     v.add_argument("path")
340     v.set_defaults(fn=cmd_verify)
341
342     rel = sub.add_parser("release", help="download the release stage into releases/<version>/ and verify")
343     rel.add_argument("--version", required=True)
344     rel.add_argument("--run-id")
345     rel.add_argument("--force", action="store_true")
346     rel.set_defaults(fn=cmd_release)
347
348     w = sub.add_parser("wait", help="block on a spawned function call id and print its result")
349     w.add_argument("call_id")
350     w.set_defaults(fn=cmd_wait)
351
352     f = sub.add_parser("fmt", help="ruff --fix + format in the cloud and copy the results back")
353     f.add_argument("--run-id")
354     f.set_defaults(fn=cmd_fmt)
355
356     sub.add_parser("new-run", help="mint a run id").set_defaults(fn=cmd_new_run)
357     return p
358
359
360 def main(argv: list[str] | None = None) -> int:
361     os.chdir(REPO)
362     ns = build_parser().parse_args(argv)
363     return int(ns.fn(ns))
364
365
366 if __name__ == "__main__":
367     sys.exit(main())

```

tools/run_local.py

```

1  #!/usr/bin/env python3
2  """Run the Problem-D pipeline on this machine, without Modal.
3
4  The reference execution of this repository is in the cloud (`tools/cli.py`), but an evaluator has no
5  account there. This script is the local equivalent: it calls exactly the same stage functions, in the
6  same order, through the same runner, writing into a local directory instead of a cloud volume.
7
8      python3 tools/run_local.py                # ingest -> ... -> tables, into ./local_runs/<id>
9      python3 tools/run_local.py --stages detect,resolve --out /tmp/d
10     python3 tools/run_local.py --quick          # small time limits: minutes rather than hours
11
12  Requirements (pip install): pandas, numpy, openpyxl, pyarrow, networkx, matplotlib, ortools, highspy.
13  The organisers' attachments must be present in ``附件/`` exactly as delivered; nothing else is needed.
14  Typesetting stages (`paper`/`qa`/`package`/`release`) need a TeX Live installation with XeLaTeX
15  and the CJK fonts, so they are not part of the default list here — the scientific stages are.
16  """
17
18  from __future__ import annotations
19

```

```

20 import argparse
21 import json
22 import sys
23 import time
24 from datetime import UTC, datetime
25 from pathlib import Path
26
27 REPO = Path(__file__).resolve().parents[1]
28 DEFAULT_STAGES = "ingest,validate,detect,resolve,pack,resolve_interval,compare,bench,sensitivity,results,figures,tables
    ↳ "
29 # Small budgets so that the whole pipeline finishes in minutes. The published numbers were produced with
30 # the budgets recorded in docs/RUNBOOK.md section 7; with --quick the lexicographic levels may stop at a
31 # feasible (not proven optimal) value, which the stage reports honestly in solver_report.json.
32 QUICK = {
33     "resolve": {
34         "level_time_limits": [5, 5, 60, 30, 30, 30, 30],
35         "weighted_time_limit": 30,
36         "highs_time_limit": 10,
37         "lp_time_limit": 10,
38         "alt_time_limit": 10,
39         "conditional_time_limits": [60, 20, 20, 20],
40         "workers": 4,
41     },
42     "resolve_interval": {
43         "level_time_limits": [5, 5, 60, 30, 30, 30, 30],
44         "weighted_time_limit": 30,
45         "highs_time_limit": 10,
46         "lp_time_limit": 10,
47         "alt_time_limit": 10,
48         "conditional_time_limits": [30, 10, 10, 10],
49         "variant_time_limits": [5, 5, 30, 20, 20, 20, 20],
50         "workers": 4,
51     },
52     "pack": {
53         "time_limit": 120,
54         "mip_time_limit": 60,
55         "lp_time_limit": 60,
56         "repack_time_limit": 60,
57         "layout_time_limit": 30,
58         "workers": 4,
59     },
60     "bench": {"n_tiny": 8, "sizes": [150], "scale_time_limit": 30, "workers": 4},
61     "sensitivity": {"level_time_limits": [5, 5, 30, 20, 20, 20, 20], "workers": 4},
62 }
63
64
65 def main() -> int:
66     ap = argparse.ArgumentParser(description=__doc__, formatter_class=argparse.RawDescriptionHelpFormatter)
67     ap.add_argument("--stages", default=DEFAULT_STAGES, help=f"comma-separated (default: {DEFAULT_STAGES})")
68     ap.add_argument("--out", default=str(REPO / "local_runs"), help="directory that plays the role of the volume")
69     ap.add_argument("--run-id", default=None, help="reuse an existing run id instead of creating one")
70     ap.add_argument("--quick", action="store_true", help="small solver budgets (minutes instead of hours)")
71     ap.add_argument("--force", action="store_true", help="re-run stages that already completed")
72     ap.add_argument("--param", action="append", default=[], help="stage param, e.g. workers=4 or all:workers=4")
73     ns = ap.parse_args()
74
75     sys.path.insert(0, str(REPO))
76     import pipelines.d.stages # noqa: F401 - registers the stages
77     from forge.runner import execute
78
79     vol = Path(ns.out).resolve()
80     run_id = ns.run_id or datetime.now(UTC).strftime("%Y%m%d-%H%M%S-local")
81     (vol / "runs" / run_id).mkdir(parents=True, exist_ok=True)
82     extra: dict[str, dict[str, object]] = {}
83     for item in ns.param:
84         scope, _, rest = item.partition(":") if ":" in item else ("all", "", item)
85         key, _, value = rest.partition("=")
86         try:
87             parsed = json.loads(value)
88         except json.JSONDecodeError:
89             parsed = value

```

```

90     extra.setdefault(scope, {})[key] = parsed
91
92     print(f"repo={REPO}\nvolume={vol}\nrun_id={run_id}")
93     for stage in [s.strip() for s in ns.stages.split(",") if s.strip()]:
94         params: dict[str, object] = {"size": "local"}
95         if ns.quick:
96             params.update(QUICK.get(stage, {}))
97         params.update(extra.get("all", {}))
98         params.update(extra.get(stage, {}))
99         t0 = time.monotonic()
100         result = execute(stage_name=stage, run_id=run_id, params=params, force=ns.force, repo=REPO, vol=vol)
101         status = result.get("status")
102         print(f"[{time.monotonic() - t0:8.1f}s] {stage:18s} {status}")
103         if status == "failed":
104             print(json.dumps(result.get("error", {}), ensure_ascii=False, indent=2))
105             return 1
106     print(f"\nresults: {vol / 'runs' / run_id / 'results'} (result1-4.xlsx)")
107     print(f"figures: {vol / 'runs' / run_id / 'figures' / 'figures'}")
108     print(f"tables: {vol / 'runs' / run_id / 'tables' / 'tables'}")
109     return 0
110
111
112 if __name__ == "__main__":
113     raise SystemExit(main())

```